A COMPLETE TEXTBOOK

System Design From First Principles to Production

Networking, data, caching, queues, reliability, security and operations — taught step by step, with worked designs, object-oriented exercises and interview practice.



every design is a version of this path

Built from the “System Design Primer — Complete Handwritten Study Notes” (itself an original study guide following the open-source System Design Primer by Donne Martin, CC BY 4.0). Explanations, diagrams and exercises in this book are written and drawn for this textbook.

About this book

This textbook takes a reader with no system-design background to a working understanding of production architecture. Each chapter explains a concept from a simple definition, shows why it exists, walks through how it works, and finishes with failure modes, trade-offs, interview questions and exercises.

How the book is built

The syllabus comes from the 51-page handwritten notebook *System Design Primer — Complete Handwritten Study Notes*. Every chapter opening states which part is **derived from that notebook** and which material was **added** to reach professional depth. Added material inside a chapter is boxed as *Supplemental material*. The final appendix maps every notebook topic to its chapter.

Reading the boxes

Box	Meaning
Learning objectives	What you can do after the chapter
Everyday analogy	A non-software picture of the idea
Worked example	Numbers or code worked through step by step
Common mistake	A frequent error and why it hurts
Clarification	A point where the original notes were compressed or imprecise
Interview questions	Typical questions, with explained answers
Supplemental material	Added beyond the notebook

A note on numbers

Capacity figures in worked designs are **assumptions chosen for teaching**, not measurements. Always replace them with measurements from your own system. Latency figures are rough orders of magnitude and change with hardware.

Attribution

The notebook, and therefore this book's syllabus, follows the open-source *System Design Primer* by Donne Martin (CC BY 4.0). All prose, diagrams and code here are original to this book.

Contents

Part I — Foundations 8

1 What System Design Is 9
- 1.1 What system design is 10
- 1.2 Requirements, constraints and scope 12
- 1.3 Performance versus scalability 13
- 1.4 Latency, throughput and percentiles 14
- 1.5 Scaling up and scaling out 15
- 1.6 Trade-offs: the central skill 16
- 1.7 The design loop 17

2 Processes, Threads and Concurrency 20
- 2.1 Programs, processes and threads 21
- 2.2 Concurrency and parallelism 21
- 2.3 Race conditions 22
- 2.4 Locks and other remedies 23
- 2.5 Deadlock, livelock and starvation 24
- 2.6 When the threads are machines 25

3 Data Structures and Algorithms for System Design 28
- 3.1 Why data structures matter in system design 29
- 3.2 Hash tables 29
- 3.3 Ordered structures: trees, B-trees and indexes 30
- 3.4 Log-structured merge trees 31
- 3.5 Heaps, queues and ranking 32
- 3.6 Tries 32
- 3.7 Bloom filters and probabilistic structures 32
- 3.8 Choosing the structure 33

4 Estimation and Capacity Planning 36
- 4.1 Why we estimate 37
- 4.2 Powers of two and units 37
- 4.3 Latency numbers worth knowing 37
- 4.4 Turning traffic into numbers 39
- 4.5 Availability arithmetic 40
- 4.6 Sensitivity and headroom 41
- 4.7 Worked example: a URL shortener 41

Part II — Networking and the Entry Layer 44

5 Networking and the Request Lifecycle 45
- 5.1 Layers in one minute 46
- 5.2 DNS: names to addresses 46
- 5.3 TCP: a reliable stream 48
- 5.4 UDP and QUIC 49
- 5.5 TLS: encryption and identity 50
- 5.6 HTTP: the language of the web 50
- 5.7 The full lifecycle of a cold request 52

6 APIs: RPC, REST and Contracts 56
- 6.1 Why APIs are contracts 57
- 6.2 RPC 57
- 6.3 REST 58
- 6.4 Designing endpoints, errors and status codes 59
- 6.5 Idempotency keys and safe retries 60
- 6.6 Pagination 61
- 6.7 Versioning and compatibility 61
- 6.8 Webhooks and polling 61

7 The Entry Layer: CDN, Load Balancers and Proxies 64
- 7.1 Why an entry layer 65
- 7.2 Content delivery networks 65
- 7.3 Load balancers 66
- 7.4 Reverse proxies and gateways 68
- 7.5 Rate limiting 69
- 7.6 Putting it together: sizing the entry layer 70

Part III — Data 73

8 Relational Databases and SQL 74
8.1 The relational model 75
8.2 Indexes 76
8.3 Transactions and ACID 77
8.4 Changing a live schema safely 79
8.5 Alternatives and trade-offs 80

9 Scaling Relational Databases 83
9.1 The scaling ladder 84
9.2 Replication 85
9.3 Federation and denormalisation 87
9.4 Sharding 87
9.5 SQL tuning checklist 89
9.6 Backups, replicas and failover 89

10 NoSQL and Choosing a Database 93
10.1 What NoSQL is, and why it appeared 94
10.2 The four families 94
10.3 Modelling for NoSQL: queries first 96
10.4 SQL or NoSQL? 97

11 Consistency, Availability and Partitioning Data 101
11.1 Why this chapter exists 102
11.2 The CAP theorem 102
11.3 Consistency models 103
11.4 Quorums 104
11.5 Conflicts and repair 105
11.6 Availability patterns 106
11.7 Partitioning data with consistent hashing 107

Part IV — Performance and Asynchrony 110

12 Caching 111
12.1 Why caching works 112
12.2 Where caches live 112
12.3 Update strategies 113
12.4 Eviction policies 116
12.5 Failure patterns and remedies 117

13 Asynchronous Processing 121
13.1 Why work asynchronously 122
13.2 Message queues 122
13.3 Publish/subscribe and logs 124
13.4 Retries, dead letters and idempotent consumers 125
13.5 Back pressure 125
13.6 Consistency across services: outbox and sagas 126

Part V — Reliability in Distributed Systems 130

14 Time, Coordination and Consensus 131
14.1 Why time is hard 132
14.2 Failure detection and the uncertain timeout 133
14.3 Leader election, leases and fencing 134
14.4 Consensus 135
14.5 Atomic commit across systems 137
14.6 Duplicate and out-of-order events 137

15 Resilience Patterns 140
15.1 How failures spread 141
15.2 Timeouts 141
15.3 Retries done right 142
15.4 Circuit breakers 142
15.5 Bulkheads and isolation 143
15.6 Load shedding and graceful degradation 143
15.7 Designing failover 144

16 Application Architecture and Microservices 147
16.1 Layers and tiers 148
16.2 Monolith, modular monolith, microservices 148

16.3 Drawing the boundaries 150
16.4 Communicating between services 151
16.5 Service discovery 151
16.6 Migrating: the strangler pattern 152
16.7 Recording architecture 153

Part VI — Security and Operations 158

17 Security and Privacy 159
17.1 Thinking like an attacker 160
17.2 Authentication 160
17.3 Authorization 162
17.4 Injection and common web attacks 162
17.5 Encryption and key management 163
17.6 Abuse prevention 164
17.7 Privacy by design 164

18 Observability and Reliability Engineering 167
18.1 Seeing inside a running system 168
18.2 Logs 168
18.3 Metrics 168
18.4 Traces 169
18.5 Alerting 169
18.6 SLIs, SLOs, SLAs and error budgets 170
18.7 Incident response 170
18.8 Disaster recovery: RPO, RTO and strategies 171

19 Deployment, Cloud Infrastructure, Cost and Capacity 174
19.1 From servers to platforms 175
19.2 Shipping changes safely 175
19.3 Autoscaling 177
19.4 A reference path: scaling a system up 177
19.5 Cost: estimation and habits 178
19.6 Capacity planning 178
19.7 Operational ownership 179

Part VII — Design Interviews and Worked Systems 182

20 The System Design Interview Method 183
20.1 What the interview is testing 183
20.2 The four steps and the 45-minute plan 184
20.3 The 20-point checklist 185
20.4 Communicating well 186
20.5 What interviewers expect at different levels 186
20.6 Mistakes to avoid 187
20.7 A reusable answer template 187
20.8 Practising effectively 188

21 Case Study: Pastebin and URL Shortener 190
21.1 Problem and requirements 190
21.2 Scale estimate 191
21.3 API 191
21.4 Data model 192
21.5 Generating keys 192
21.6 Architecture 194
21.7 Caching, expiry and cleanup 196
21.8 Scaling and bottlenecks 196
21.9 Failure handling, security and operations 197
21.10 Alternatives and trade-offs 197

22 Case Study: A Social Timeline and Search 200
22.1 Problem and requirements 200
22.2 Scale estimate 201
22.3 Data model and API 201
22.4 Architecture 202
22.5 The core decision: fan-out strategy 203
22.6 Supporting services 205
22.7 Scaling, bottlenecks and failure handling 205
22.8 Alternatives and trade-offs 206

23 Case Study: A Web Crawler and Search Engine 209
23.1 Problem and requirements 209
23.2 Scale estimate 210

23.3 Architecture overview 210
23.4 The URL frontier 211
23.5 Fetching politely and robustly 213
23.6 Duplicates, traps and freshness 213
23.7 From documents to search results 214
23.8 Scaling, failures and operations 215
23.9 Alternatives and trade-offs 216

24 Case Study: A Personal Finance Aggregator 218
24.1 Problem and requirements 218
24.2 Scale estimate 219
24.3 Architecture overview 220
24.4 API contracts 221
24.5 Data model 221
24.6 Ingestion, step by step 222
24.7 Categories, summaries and budgets 223
24.8 Security, privacy and compliance 224
24.9 Scaling, failures and operations 224
24.10 Alternatives and trade-offs 224

25 Case Study: Social Graph and Shortest Path 227
25.1 Problem and requirements 227
25.2 Scale estimate 228
25.3 Why naive BFS fails 228
25.4 Architecture 229
25.5 Algorithm in detail 230
25.6 Caching, hot nodes and limits 231
25.7 Privacy, consistency and operations 231
25.8 Alternatives and trade-offs 232

26 Case Study: A Query Cache with LRU 234
26.1 Problem and requirements 234
26.2 Scale and sizing 235
26.3 LRU: the data structure 235
26.4 The query path 237
26.5 Distributed cache design 237
26.6 Freshness, stampedes and hot keys 238
26.7 Operations and failure modes 238
26.8 Alternatives and trade-offs 239

27 Case Study: Sales Rank by Category 241
27.1 Problem and requirements 241
27.2 Scale estimate 242
27.3 Architecture overview 242
27.4 Batch computation step by step 242
27.5 Correctness: duplicates, late data, ties 243
27.6 Streaming alternative 244
27.7 Serving ranks 244
27.8 Failure handling and operations 245
27.9 Alternatives and trade-offs 245

28 Case Study: Scaling from One User to Millions 247
28.1 Problem and approach 247
28.2 Stage 1: one box (users: 1 to 1,000) 248
28.3 Stage 2: separate tiers and vertical scaling (to about 10,000) 248
28.4 Stage 3: load balancing and a stateless web tier (to about 100,000) 249
28.5 Stage 4: caching, read replicas and asynchronism (to about 1 million) 250
28.6 Stage 5: scaling the data tier (millions of users) 250
28.7 Capacity and cost along the way 251
28.8 Vendor-neutral map 251
28.9 Failure thinking at every stage 252

29 End-to-End Exercises: Payments and Notifications 254
29.1 Exercise 1: A payment service 254
29.2 Exercise 2: A notification system 257

Part VIII — Object-Oriented Design 262

30 Object-Oriented Design Exercises 263
30.1 Method 263
30.2 Design 1: Hash map 264
30.3 Design 2: LRU cache 265
30.4 Design 3: Call center 268
30.5 Design 4: Deck of cards and blackjack 270

30.6 Design 5: Parking lot ... 272
30.7 Design 6: Online chat ... 274
30.8 Exercise from the notebook: a circular array ... 276
30.9 Tests and what they prove ... 277

Part IX — Practice and Real-World Architectures ... 279

31 Practice Bank: Twenty-Three More Design Questions ... 280
31.1 How to use the bank ... 280
31.2 The questions ... 281
31.3 A study plan ... 283

32 Reading Real-World Architectures ... 285
32.1 A method for reading ... 285
32.2 The seventeen systems in the notebook's table ... 286
32.3 Company architectures: what to look for ... 287
32.4 Engineering blogs worth following ... 288

Back Matter — Appendices ... 290

Appendix A Revision Sheets and Formulas ... 291
A.1 Symptom to first fix ... 291
A.2 Numbers and formulas ... 292
A.3 One-line trade-offs ... 293

Appendix B Coverage Matrix: Notebook to Book ... 294
B.1 Topics added beyond the notebook ... 296
B.2 Source content that could not be used ... 297

Appendix C Glossary ... 298

Appendix D References and Further Reading ... 301
D.1 Books ... 301
D.2 Papers ... 301
D.3 Standards and specifications ... 302
D.4 Other resources ... 303

PART I

Foundations

The vocabulary, the machine-level background and the arithmetic that every later chapter relies on.

CHAPTER 1

What System Design Is

Derived from your notes: Notebook chapter 1 (Foundations: the life of a request, performance vs scalability, latency vs throughput, scaling up vs out) and the "How to use this notebook" page. **Added in this book:** requirements framework, percentiles, bottleneck thinking, the design loop.

LEARNING OBJECTIVES

After this chapter you can:

- define *system design* and describe the path a request takes through a typical web system;
- separate functional requirements, non-functional requirements and constraints;
- tell a **performance** problem from a **scalability** problem and say why the difference changes the fix;
- explain latency, throughput and percentiles, and why averages hide the experience of unlucky users;
- compare vertical and horizontal scaling and explain why *stateless* servers make horizontal scaling easy;
- state the design loop used throughout this book: requirements, numbers, boxes, bottlenecks, trade-offs.

PREREQUISITE KNOWLEDGE

None. You should be comfortable with the idea of a program that talks to another program over a network, and you should have used a website or a mobile app. Everything else is introduced here.

KEY TERMINOLOGY

Server
A computer (or a program on one) that waits for requests and answers them.

Client
The program that sends requests: a browser, a mobile app, or another server.

Request / response
One message asking for work, and the message returned with the result.

Latency
The time one operation takes from start to finish, for example 120 milliseconds (ms) for a page request.

Throughput
How many operations the system completes per unit of time, for example 5,000 requests per second (RPS).

Scalability
The ability to handle more load by adding resources, with performance improving roughly in proportion.

Bottleneck
The single resource that limits the whole system right now (CPU, memory, disk, network, a lock, a database).

Stateless / stateful
A stateless server keeps nothing between requests that another server could not recreate; a stateful one remembers things locally.

Trade-off
A choice where improving one property (say, speed) costs another (say, freshness of data or money).

1.1 What system design is

System design is the activity of deciding which components a software system needs, what each one is responsible for, how they communicate, where data lives, and what happens when parts fail or when traffic grows. It is not about writing the code inside one function. It is about the shape of the whole.

Why does this need to be done *on purpose*? A program that serves ten users from one laptop needs almost no design: one process, one file or database, done. As usage grows, four pressures appear, and each forces a design decision:

1. **Load.** One machine can only do so much work per second. Beyond that you need several machines, and something must decide which machine handles which request.
2. **Data.** Data can outgrow one disk, and many users reading and writing at once can overwhelm one database. You must decide how to copy, split and cache data.
3. **Failure.** Disks die, networks drop packets, software has bugs, and deployments go wrong. With many machines, *something* is always failing. The design must keep working anyway.
4. **Change.** Teams grow and features multiply. The structure of the system determines whether ten engineers can work independently or trip over each other.

A useful way to see all of this at once is to follow a request.

The path of a request

Suppose a user opens a photo-sharing site and loads their timeline. Roughly, the following happens (every step is taught in a later chapter):

1. The browser asks the **Domain Name System** (DNS) to turn `photos.example` into an Internet Protocol (IP) address (Section 5.2).
2. Static files such as images, scripts and style sheets may be answered by a **content delivery network** (CDN) located near the user (Section 7.2).
3. Dynamic requests reach a **load balancer**, which picks one of many application servers (Section 7.3).

4. A **web server or reverse proxy** terminates the secure connection and forwards the request (Section 7.4).
5. An **application server** runs the business logic. It first asks a **cache** for ready-made answers (Chapter 12), and on a miss reads a **database** (Chapter 8).
6. Slow or non-urgent work (resizing a new photo, sending a notification) is placed on a **queue** and done later by **workers** (Chapter 13).
7. Large files such as the photos themselves live in an **object store**.

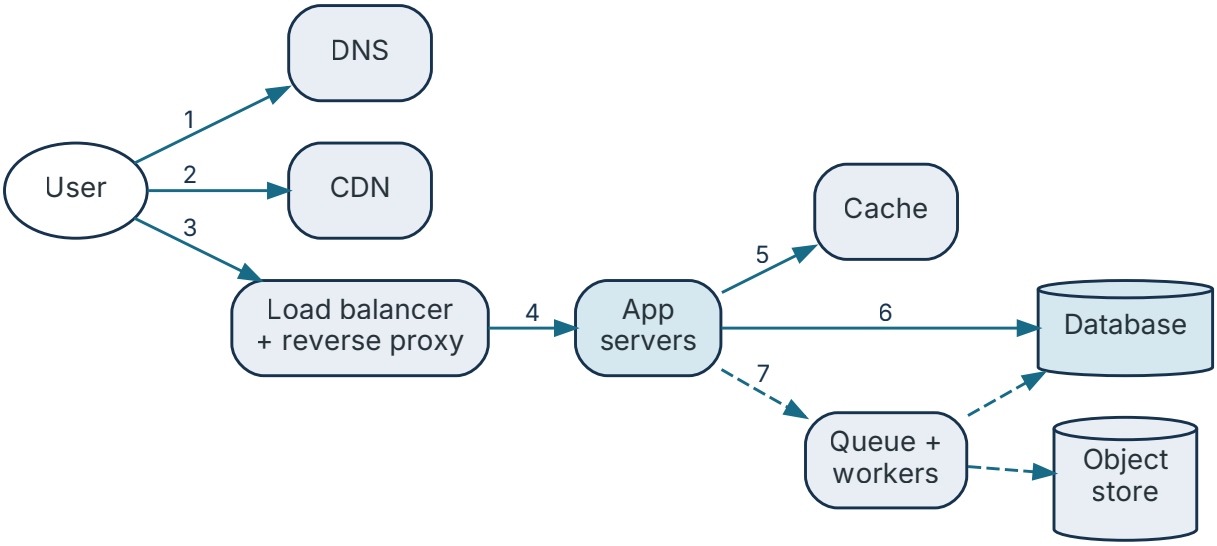


Figure 1.1 The life of a request. Solid arrows are the synchronous path the user waits for; dashed arrows are background work after the response. The numbers follow the steps above, with the load balancer and reverse proxy (steps 3 and 4) drawn as one box. Every later chapter zooms into one box.

Read Figure 1.1 left to right. The **solid arrows** show what the user waits for: name lookup, then (for static content) the CDN, otherwise the load balancer and proxy, the application server, and finally cache or database. The **dashed arrows** show work the application server hands to a queue so the user does not wait: a worker picks the job up later and may write results to the object store or database. The cache sits *beside* the database because the application checks it first.

> **EVERYDAY ANALOGY**
>
> **A restaurant.** DNS is the phone book that gives you the address. The CDN is a take-away counter that hands out popular dishes without troubling the kitchen. The load balancer is the host who seats you at a free table. The application server is the waiter, the cache is the pass where finished dishes wait, and the database is the pantry. A queue is the order rail: the waiter clips a ticket for a slow dish and moves on, instead of standing in the kitchen.

Note the pattern: nearly every box exists to do one of three jobs — make the path **faster** (CDN, cache), make it **bigger** (load balancer, more servers, sharding) or make it **safer** (replication, queues, retries). When you meet a new component later, ask which of the three it serves and what it costs.

1.2 Requirements, constraints and scope

No design is good or bad in the abstract; it is good or bad *for a set of requirements*. A system serving a classroom of thirty can use one database server happily, while a payments network cannot. Therefore every design starts by writing down what must be true.

Functional requirements describe *what the system does*: "a user can post a message", "a user can follow another user", "the system shortens a URL and redirects". They are features, usually phrased as verbs.

Non-functional requirements (also called *quality attributes*) describe *how well* it does them. The ones that most often drive architecture are:

Table 1.1 Common non-functional requirements and the questions that sharpen them.

Quality	What it means	Sharpening question
Performance	Time per operation	What is acceptable for the 95th or 99th percentile?
Scalability	Handling growth	How many users, requests per second, and how fast will they grow?
Availability	Fraction of time the service works	Is 99.9% enough, or is every minute of downtime costly?
Durability	Data is not lost once acknowledged	What if the last second of writes disappears?
Consistency	Everyone sees the same data	Can a reader see slightly stale data? For how long?
Security & privacy	Only the right people do the right things	Who are the attackers? What data is regulated?
Cost	Money and operational effort	What is the budget, and who runs it at 3 a.m.?

Constraints are fixed facts that limit the solution: "must run on the company's existing cloud", "must store data in one country", "team of three engineers", "launch in six weeks". Constraints are not goals; they are walls.

Scope is the explicit list of what is *not* being designed. Stating scope out loud ("search ranking is out of scope; we only need keyword matching") prevents an open-ended problem from consuming all your time, in an interview or in real work.

WORKED EXAMPLE

Turning a vague idea into requirements. The request: "*Build a service like a text-sharing site where you paste text and get a link.*"

- *Functional:* create a paste and receive a short URL; open the URL to view the text; optionally let pastes expire.
- *Non-functional:* reads far outnumber writes (assume 10 reads per write); a viewer should see a paste within about 200 ms at the 95th percentile; stored pastes must not be lost (durability); 99.9% availability.
- *Constraints:* a team of two; use managed cloud services.
- *Scope:* user accounts, editing and syntax highlighting are out of scope.

Notice how the non-functional line "reads outnumber writes 10 to 1" already suggests caching, and "must not be lost" already rules out keeping pastes only in memory. Requirements *are* the first design decisions. This exact problem is worked through in full in Chapter 21.

1.3 Performance versus scalability

These two words are used loosely but mean different things, and mixing them up leads to expensive mistakes.

- A **performance problem** means the system is slow *even for a single user*. A badly written query that scans a ten-million-row table takes seconds no matter how quiet the system is.
- A **scalability problem** means the system is fast for one user but slows down (or fails) as load grows. A well-written query is fast; but if every request queues on one shared lock, ten users each wait for nine others.

The diagnostic test is simple: *measure one request on an otherwise idle system*. If it is already slow, you have a performance problem; profile and fix the code, query, or algorithm. If it is fast when alone and slow under load, you have a scalability problem; look for a shared resource that serialises work.

A system is **scalable** when adding resources improves capacity in rough proportion: doubling the servers lets you serve close to double the traffic. Load can grow in two ways: *more units of work* (more users, more requests) or *larger units of work* (a dataset that keeps growing, so each query touches more data). Different techniques address each; sharding, for example, helps when data grows, while load balancing helps when requests multiply.

COMMON MISTAKE

Buying servers for a performance problem. If one request takes ten seconds because of a missing database index, ten servers give you ten slow requests at the same time, not a fast one. Always find out which kind of problem you have *before* spending money.

1.4 Latency, throughput and percentiles

Latency is how long one operation takes. **Throughput** is how many operations complete per unit of time. They are related but not the same, and improving one does not automatically improve the other.

> **EVERYDAY ANALOGY**
>
> A highway. Latency is the time *one car* needs to travel from A to B. Throughput is the number of cars per hour that pass a point. Adding lanes raises throughput without making any single car arrive sooner. Raising the speed limit lowers latency but, if cars are spaced farther apart for safety, may not raise throughput much.

Near capacity, the two interact badly. Requests arrive, wait in a queue for a free worker, and *then* run. As utilisation approaches 100%, queue waiting time explodes, so latency climbs steeply while throughput flattens at its maximum (see Figure 1.2). This is why running a system at 95% utilisation is dangerous: a small bump in traffic turns into very long waits. Many teams plan to stay well below saturation (often 50% to 70% at peak) to leave room for bursts.

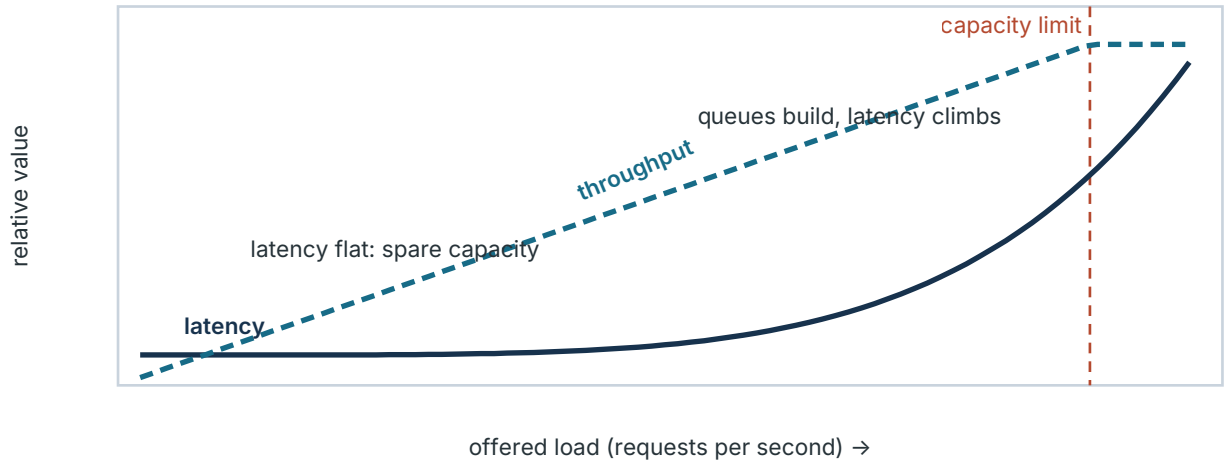


Figure 1.2 Latency stays flat while the system has spare capacity, then rises steeply near saturation. Throughput stops growing at the capacity limit; extra offered load only lengthens queues.

Why averages mislead

If 99 requests take 100 ms and one takes 10 seconds, the **average** is about 199 ms: a number that describes *nobody's* experience. Engineers therefore report **percentiles**. The *p50* (median) is the value half of requests beat. The *p95* is the value 95% of requests beat; the *p99* beats 99%. The slowest few per cent are called the **tail**.

Tail latency matters more than it first appears. A single page view may trigger dozens of internal calls. If each call has a 1% chance of being slow, a page that makes 100 calls will hit at least one slow call about 63% of the time ($1 - 0.99^{100} \approx 0.63$). The user experiences the *worst* of the many calls, so internal p99 becomes external median. We return to this in Chapter 15.

Table 1.2 A tiny dataset showing why the mean is not enough (ten requests, in milliseconds).

Request	1	2	3	4	5	6	7	8	9	10
Latency (ms)	90	95	98	100	102	105	110	115	120	2,400

The mean is 333.5 ms, yet nine of ten users saw 120 ms or better. The median is 103.5 ms and the maximum is 2,400 ms; reporting only the mean would misrepresent both the typical and the unlucky user.

1.5 Scaling up and scaling out

Once one server is not enough, there are two directions.

Vertical scaling (scale up) means buying a bigger machine: more CPU cores, more memory, faster disks. It is simple because the software does not change. Its limits are that machines have a maximum size, the price rises faster than the capacity at the top end, and a single machine remains a **single point of failure** (SPOF): if it dies, everything stops.

Horizontal scaling (scale out) means adding more ordinary machines and spreading the work among them. It has no hard ceiling and tolerates the loss of individual machines, but it demands software that can run in many copies, and it introduces coordination problems: where does shared data live, how are requests spread, and what happens when two machines disagree?

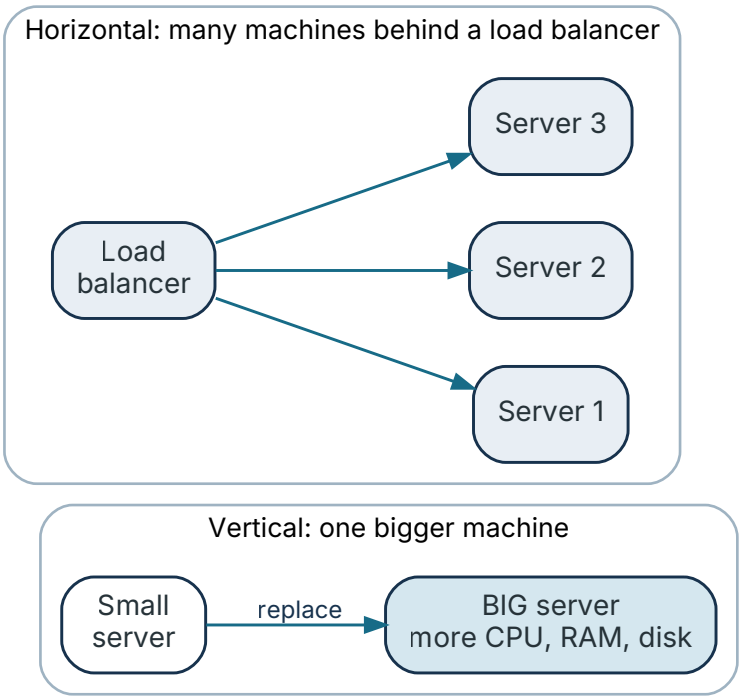


Figure 1.3 Vertical scaling replaces one machine with a larger one; horizontal scaling adds machines behind a load balancer. The second approach can grow further and survive failures, at the price of coordination.

Table 1.3 Vertical versus horizontal scaling.

	Vertical (scale up)	Horizontal (scale out)
Change required	Usually none	Software must run as many copies
Cost curve	Rises steeply at the high end	Roughly linear with commodity machines
Failure	One machine = one SPOF	Loss of one machine is tolerated
Upper limit	Largest machine available	Limited by coordination, not hardware
Typical first use	Databases, early startups	Web and application tiers

Why stateless servers scale out easily

The easiest thing to scale out is a **stateless** server. If a server keeps no per-user data in its own memory or disk, then *any* server can handle *any* request, and a load balancer can send each request anywhere. If a server keeps the user's login session in local memory, the user must always return to *that* server (a design called *sticky sessions*), and losing the server logs the user out.

The fix is to move state out of the server into a shared place: a database, a distributed cache, or a signed token held by the client. After that, servers become interchangeable and can be added or removed at will. We use this idea repeatedly: *keep the application tier stateless and push state into specialised stores that are designed to hold it.*

SUPPLEMENTAL MATERIAL

Amdahl's law: why scaling out has limits. If a fraction s of the work is inherently serial (it cannot be split among machines), then no number of machines gives a speed-up greater than $1/s$. If 5% of each request goes through one lock or one database row, the maximum speed-up is $1/0.05 = 20\times$, however many servers you add. Practical lesson: when scaling stops working, look for the serial section, which is usually shared state. The *Universal Scalability Law* adds a second effect, coordination cost, which can make throughput *fall* when too many nodes talk to each other.

1.6 Trade-offs: the central skill

Almost every technique in this book buys something by spending something else. Learning to name both sides is the single most valuable habit in system design, and interviewers listen for it more than for any specific technology.

Table 1.4 Familiar techniques and what each one costs.

Technique	You gain	You pay
Cache	Lower latency, less database load	Stale data, invalidation complexity, extra component
Replication	Availability, read capacity	Replication lag; complexity of failover
Sharding	Write capacity, data size	Cross-shard queries, rebalancing, operational load

Technique	You gain	You pay
Queue	Absorbs bursts, decouples services	Delay, duplicate or reordered messages, harder debugging
Microservices	Independent teams and deployment	Network calls, distributed failure, data consistency problems
Denormalisation	Fast reads	Duplicated data, harder writes
Strong consistency	Simple reasoning	Higher latency, lower availability under partitions

CLARIFICATION

Clarification on "this improves scalability". The original notes sometimes say a technique "improves scalability". That is shorthand. More precisely, a technique improves scalability *under a stated condition*: a cache improves read scalability **if the same data is read many times between changes** (a high hit ratio); it does nothing for write scalability and can worsen consistency. Whenever you write "X improves Y", add "when Z" and "at the cost of W".

1.7 The design loop

The remainder of this book repeats one loop. Learn it now and apply it to every problem, from the worked case studies to the exercises.

1. **Clarify requirements.** Functional, non-functional, constraints, scope.
2. **Estimate.** Turn "millions of users" into requests per second, bytes stored and bandwidth (Chapter 4).
3. **Sketch the simplest design that could work.** Often one server and one database.
4. **Find the bottleneck.** Measure or reason about which resource saturates first.
5. **Relieve the bottleneck with the cheapest suitable technique**, naming its cost.
6. **Repeat from step 4** until the requirements are met, then examine failure modes and security.
7. **Record the decisions and trade-offs** so others can understand and revise them.

The discipline of step 3 is often skipped. Start simple and let measured or estimated pressure justify each added component; every box you add must be built, monitored, secured, paid for and debugged.

FAILURE MODES

Typical failure patterns at the foundation level. (1) The single server is a SPOF and dies. (2) A hidden shared resource (one database, one lock, one log file) serialises everything. (3) A slow dependency makes every request slow because requests wait on it. (4) A traffic burst arrives faster than capacity can be added. Later chapters give a specific remedy for each: replication (Chapter 9), partitioning (Chapter 11), timeouts and load shedding (Chapter 15), and queues with back-pressure (Chapter 13).

COMMON MISCONCEPTION

- "*Scalable means fast.*" No. A scalable system keeps performance acceptable as load grows; a fast single-user system may collapse under load.
- "*Horizontal scaling is always better.*" It is necessary beyond a point, but vertical scaling is simpler and often correct for a database or an early-stage product.
- "*More servers always means more availability.*" Only if the servers are independent and nothing shared (database, network, configuration) is a SPOF.
- "*Good design means using many technologies.*" Good design means the fewest components that meet the requirements.

INTERVIEW QUESTIONS

Q1. What is the difference between a performance problem and a scalability problem? A performance problem is slow for one user; a scalability problem is fast alone but degrades under load. Test with a single request on an idle system. The fix differs: optimise code or queries for the first, remove shared bottlenecks and add capacity for the second.

Q2. Why do engineers quote p99 latency rather than the average? Averages hide the tail. A few very slow requests barely move the mean but make real users unhappy, and in systems where one user action fans out into many internal calls, tail latency dominates the user's experience.

Q3. Why are stateless servers easier to scale out? Because any server can handle any request, a load balancer can distribute freely and servers can be added, removed or replaced without losing user data. State is moved to shared stores built for it.

Q4. When would you prefer scaling up? When the component is hard to distribute (a single relational database early on), when the load is modest, or when simplicity and development time matter more than headroom. Know the ceiling and have a plan for when you reach it.

EXERCISES

1. A page loads in 4 s for one user on an idle test machine. A colleague proposes adding five servers. Is this a good first step? Why or why not?
2. Ten request latencies are 80, 85, 90, 90, 95, 100, 100, 105, 110, 900 ms. Compute the mean and the median. Which better describes the typical user?
3. A page needs 50 internal calls, each of which exceeds 200 ms with probability 2%. What is the probability that at least one is slow?
4. Write functional requirements, non-functional requirements, constraints and scope for "a service that sends one-time passwords by SMS".
5. List three places a web application might hold per-user state, and for each say whether it prevents scaling out.

Answers.

1. No. It is a performance problem (slow even alone). Profile and fix the slow step first; extra servers would give five slow requests.
2. Mean = 1,755 / 10 = 175.5 ms (sum 80+85+90+90+95+100+100+105+110+900 = 1,755). Median = (95+100)/2 = 97.5 ms. The median describes the typical user; the mean is dragged up by one outlier.

3. $1 - 0.98^{50} = 1 - 0.364 =$ about 0.636, i.e. roughly 64%.
4. Sample: *functional* send an OTP to a phone number, verify a code, expire codes; *non-functional* delivery within a few seconds at p95, 99.95% availability, rate-limit abuse; *constraints* use an SMS provider's API, comply with local telecom rules; *scope* excludes marketing messages and user accounts.
5. In server memory (blocks scaling out unless sessions are sticky), in a shared cache or database (does not block), in a signed token held by the client (does not block, but the token cannot be revoked easily).

CHAPTER SUMMARY

- System design is deciding components, responsibilities, communication, data placement and failure behaviour for stated requirements.
- A request path (DNS, CDN, load balancer, proxy, application, cache, database, queue, object store) is the map of the whole book.
- Requirements come first: functional, non-functional, constraints and scope.
- Performance is slowness for one user; scalability is how performance holds as load grows. Diagnose before fixing.
- Latency is time per operation, throughput is operations per time; use percentiles, not averages.
- Scale up is simple but bounded and a SPOF; scale out is unbounded but needs stateless servers and coordination.
- Every technique has a price. Start simple, find the bottleneck, relieve it, name the cost.

REVISION CHECKLIST

- [ ] I can draw the request path from memory and say what each box is for.
- [ ] I can list functional and non-functional requirements for a new feature.
- [ ] I can explain p50, p95, p99 and the tail.
- [ ] I can say why stateless servers scale out and where the state goes instead.
- [ ] For any technique I can state what it gains and what it costs.

SUPPLEMENTAL MATERIAL

Further reading. Martin Kleppmann, *Designing Data-Intensive Applications* (O'Reilly), chapters 1 and 2 for reliability, scalability and maintainability. Jeff Dean and Luiz André Barroso, "The Tail at Scale", *Communications of the ACM*, 2013, for why tail latency dominates fan-out systems. Neil Gunther's writing on the Universal Scalability Law for the mathematics of coordination cost.

CHAPTER 2

Processes, Threads and Concurrency

Derived from your notes: Notebook chapter 1 (scaling out, stateless servers) and the object-oriented chapter's LRU and hash-map exercises, which rely on these ideas. **Added in this book:** the whole chapter: processes, threads, concurrency, race conditions, locking and deadlocks.

LEARNING OBJECTIVES

After this chapter you can:

- explain the difference between a program, a process and a thread, and why threads are cheaper but riskier;
- distinguish concurrency from parallelism, and I/O-bound from CPU-bound work;
- recognise a race condition (check-then-act, lost update) and fix it with a lock, an atomic operation or optimistic concurrency;
- list the four conditions for deadlock and name practical ways to avoid it;
- explain why these problems come back, in harder form, when the "threads" are separate machines.

PREREQUISITE KNOWLEDGE

Chapter 1. A little programming experience (variables, functions, a loop) is enough; the code samples are short Python.

KEY TERMINOLOGY

Process
A running program with its own private memory and resources, isolated from other processes by the operating system (OS).

Thread
A path of execution inside a process. Threads of one process share its memory.

Context switch
The OS pausing one thread and resuming another on a CPU core; it costs time because state must be saved and restored.

Concurrency
Making progress on several tasks during overlapping time periods, possibly on one core by interleaving.

Parallelism
Literally executing several tasks at the same instant on different cores or machines.

Race condition
A bug whose outcome depends on the unpredictable timing of two or more threads.

Critical section
A piece of code that must not be executed by two threads at once.

Mutex (lock)
A mechanism allowing only one thread at a time into a critical section.

Deadlock
Two or more threads each waiting for something another holds, so none can proceed.

2.1 Programs, processes and threads

A **program** is a file of instructions. When the OS runs it, it creates a **process**: a private address space (its own memory), open files and network connections, and at least one **thread** of execution. Because processes do not share memory, one process crashing normally does not corrupt another. That isolation is valuable and expensive: starting a process takes noticeable time and memory, and processes must talk to each other through explicit channels such as sockets or pipes.

A process may start more **threads**. All threads of a process see the same memory, so they can cooperate cheaply, for example by updating a shared in-memory cache. The price is that they can also corrupt each other's data. A bug in one thread can crash the whole process.

Table 2.1 Processes versus threads.

	Process	Thread
Memory	Private	Shared with sibling threads
Creation cost	Higher	Lower
Communication	Sockets, pipes, files (explicit)	Shared variables (implicit, dangerous)
Failure isolation	Strong	Weak: one thread can take down all
Typical use	Separate services, worker pools	Handling many requests inside one server

A CPU core executes one thread at a time. With more threads than cores, the OS rapidly switches between them (a **context switch**), giving the *illusion* of simultaneity. Each switch costs microseconds and disturbs CPU caches, so thousands of busy threads can spend more time switching than working. This is one reason web servers use thread pools of bounded size rather than one thread per connection without limit.

2.2 Concurrency and parallelism

Concurrency is about *structure*: dealing with many things at once. Parallelism is about *execution*: doing many things at once. A single-core machine can be concurrent (interleaving) but not parallel. A server answering a thousand clients is concurrent; whether it is also parallel depends on how many cores or machines it has.

Choose the technique by what the work waits for.

- **I/O-bound** work spends most time waiting for something external: a disk, a network call, a database. While one request waits, the CPU is idle and can serve another. Concurrency gives big gains here, often with a single thread using an **event loop** (the thread starts an operation, registers a callback, and handles other events until it completes; used by Node.js, Python's `asyncio`, and many proxies).
- **CPU-bound** work (image resizing, encryption, compression) keeps a core busy. Only more cores, i.e. real parallelism, speed it up. More threads than cores just add switching cost.

> **WORKED EXAMPLE**
>
> **How many threads?** A server handles requests that each spend 5 ms computing and 45 ms waiting for a database. One thread can finish 20 requests per second (each takes 50 ms). If the machine has 4 cores, how many threads keep them busy? During the 50 ms a thread is blocked for 45 ms and runs for 5 ms, so one core can interleave about 50 / 5 = 10 threads before saturating. With 4 cores, roughly 40 threads fully use the CPU, giving about 4 × 200 = 800 requests per second (each core can compute 1000 / 5 = 200 requests per second). Beyond about 40 threads you gain nothing; you only queue longer. The rule of thumb: threads ≈ cores × (1 + wait time / compute time). These are illustrative numbers; real systems are measured, not computed.

2.3 Race conditions

A **race condition** appears when the result depends on the order in which threads happen to run. The classic example is two threads incrementing a shared counter. The statement `count = count + 1` looks atomic (indivisible) but is really three steps: read the value, add one, write it back.

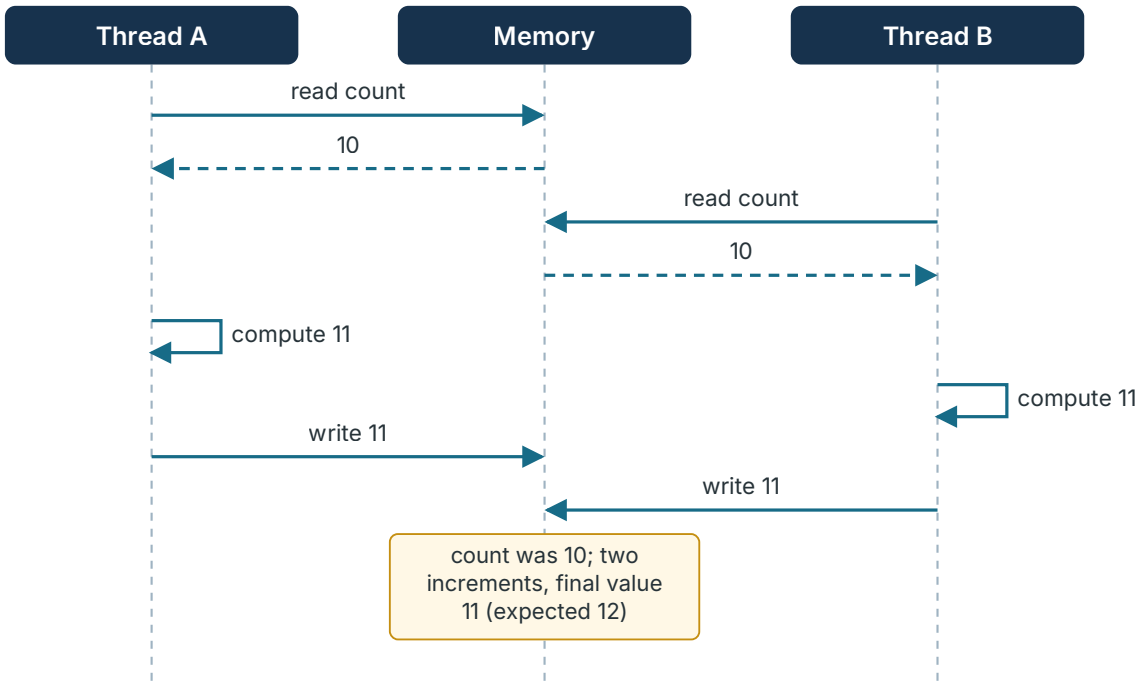


Figure 2.1 A lost update. Both threads read 10, both compute 11, both write 11. Two increments happened, but the counter rose by only one.

In Figure 2.1, thread A and thread B each read 10 before either writes. Both compute 11 and write 11. One update is **lost**, with no error message and no crash. Races like this are rare in testing (the timing window is tiny) and appear under production load, which makes them especially nasty.

The second common shape is **check-then-act**: you check a condition and then act on it, but the condition changes in between.

```
# BROKEN: two requests can both pass the check
def withdraw(account, amount):
    if account.balance >= amount:       # check
        account.balance -= amount       # act (balance may have changed!)
        return True
    return False
```

If the balance is 100 and two threads each withdraw 80, both can pass the check before either subtracts, leaving a balance of -60. The same shape appears in "if the username is free, create it", "if stock > 0, sell one", and "if the cache is empty, load it" (the last causes the *cache stampede* in Chapter 12).

2.4 Locks and other remedies

There are three families of fix.

1. Mutual exclusion (locks). Wrap the critical section so only one thread at a time enters it.

```
import threading
lock = threading.Lock()

def withdraw(account, amount):
    with lock:                      # only one thread inside at a time
        if account.balance >= amount:
            account.balance -= amount
            return True
        return False
```

Locks are correct but have costs: threads wait (lower throughput), the lock can become the serial bottleneck (recall Amdahl's law, Chapter 1), and misuse causes deadlock. Keep critical sections *short*, and never perform slow I/O while holding a lock. A **read-write lock** allows many simultaneous readers but only one writer, which suits read-heavy data.

2. Atomic operations. Modern CPUs offer indivisible instructions such as *compare-and-swap* (CAS): "set this variable to *new* only if it currently equals *expected*; tell me whether it worked."

```
# pseudocode: lock-free increment using compare-and-swap
def increment(counter):
    while True:
        old = counter.load()
        if counter.compare_and_swap(old, old + 1):   # succeeds only if unchanged
            return                                   # otherwise retry
```

3. Optimistic concurrency. Assume conflicts are rare; do the work without locking, and at the end check that nothing changed, retrying if it did. Databases use this with a **version column**: `UPDATE accounts SET balance = ?, version = version + 1 WHERE id = ? AND version = ?`. If zero rows are updated, someone else got there first; re-read and retry. **Pessimistic** locking (lock first) wins when conflicts are frequent; optimistic wins when they are rare, because nobody waits.

> **COMMON MISTAKE**
>
> **Common mistakes.** (1) Locking only the *write* and not the *check*, which leaves the check-then-act gap. (2) Using a lock that protects *one process* to protect data shared across *several servers*; it does nothing there. (3) Holding a lock while calling a remote service that may hang. (4) Assuming that "it passed my tests" means "it has no race".

2.5 Deadlock, livelock and starvation

A **deadlock** occurs when threads wait for each other forever. Suppose thread 1 holds lock A and wants lock B, while thread 2 holds lock B and wants lock A. Neither can proceed (Figure 2.2).

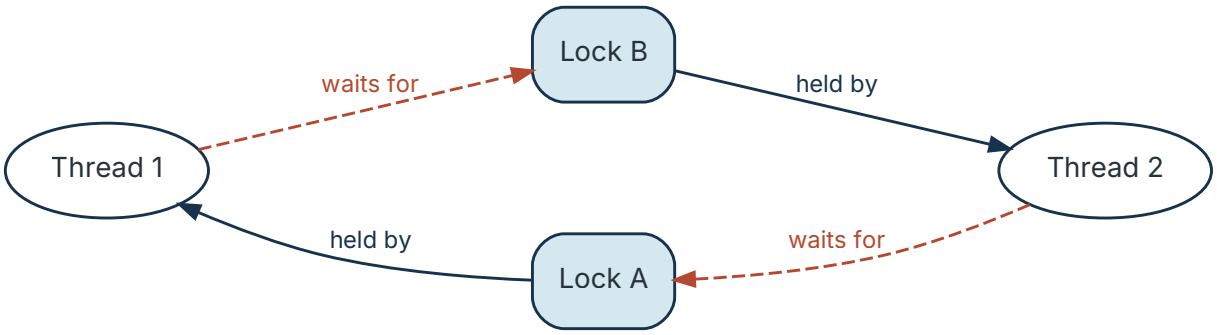


Figure 2.2 A circular wait. Thread 1 holds lock A and waits for lock B; thread 2 holds B and waits for A.

Deadlock needs all four **Coffman conditions** at once: (1) *mutual exclusion*: a resource is held by at most one thread; (2) *hold and wait*: threads hold resources while waiting for others; (3) *no preemption*: resources cannot be forcibly taken; (4) *circular wait*: a cycle of waiting threads exists. Break any one and deadlock is impossible. The most practical way is to break circular wait by imposing a **global lock order**: every thread acquires locks in the same order (say alphabetically by name), so no cycle can form. Other defences are *lock timeouts* (give up and retry, with random delay), acquiring all needed locks at once, and detecting cycles and aborting one victim, which databases do automatically.

Two relatives: **livelock**, where threads keep reacting to each other and make no progress (two people in a corridor stepping aside in the same direction forever), and **starvation**, where a thread never gets the resource because others always go first.

2.6 When the threads are machines

SUPPLEMENTAL MATERIAL

Everything above assumed shared memory on one computer. A web application on ten servers has the same problems, but a process-local lock is useless between machines, and a "pause" can last seconds and may not be noticed. The distributed equivalents are: **database transactions and row locks** (Chapter 8), **optimistic version checks** (as above), **idempotency keys** so retries do not duplicate work (Chapter 13), and **distributed locks and leases** coordinated through a consensus system (Chapter 14). Distributed locks are subtle: a lock holder can pause (a long garbage-collection pause, a stalled network) and still believe it holds a lock that has expired, so well-designed systems also use *fencing tokens*, increasing numbers that the protected resource uses to reject stale holders.

FAILURE MODES

Failure modes. *Lost update* and *double spend* from check-then-act; *deadlock* freezing a service until restart; *thread-pool exhaustion* when all threads are blocked on a slow dependency (new requests queue and time out); *priority inversion*, where a low-priority thread holding a lock delays a high-priority one; *memory visibility* bugs, where one thread does not see another's write without proper synchronisation. Recovery usually means timeouts on every wait, bounded thread pools and queues, and metrics showing lock wait time and pool utilisation (Chapter 18).

COMMON MISCONCEPTION

- "*Python code is safe because of the global interpreter lock.*" The GIL protects the interpreter's internals, not your multi-step logic; `count += 1` can still lose updates across threads.
- "*More threads means faster.*" Only until cores or the shared resource saturate.
- "*A race is a rare bug I can ignore.*" Rare per request becomes frequent at millions of requests per day.

INTERVIEW QUESTIONS

Q1. What is a race condition? Give an example and a fix. A bug whose result depends on timing. Two concurrent withdrawals both pass a balance check and overdraw. Fix by making check and update atomic: a lock, a database transaction with a row lock, or a conditional update (`WHERE balance >= amount`).

Q2. Threads versus processes? Threads share memory (cheap, fast communication, risky); processes are isolated (safer, heavier). Services often use processes for fault isolation and threads or event loops inside each for request concurrency.

Q3. How do you prevent deadlock? Acquire locks in a fixed global order, hold as few locks as possible for as short as possible, use timeouts with retry and jitter, or let the database detect and abort a victim.

Q4. When would you choose optimistic over pessimistic locking? When conflicts are rare and retrying is cheap, such as editing different records. Pessimistic locking suits high-conflict hot rows, where repeated retries would waste work.

EXERCISES

1. A thread-pool server has 8 cores; each request computes for 2 ms and waits 18 ms on I/O. Estimate the thread count that saturates the CPU and the resulting request rate.
2. Rewrite `if stock > 0: stock -= 1` so that two buyers cannot both buy the last item, first with a lock, then with a single SQL statement.
3. Two services each lock a customer row and an order row, but in opposite orders. Explain how a deadlock arises and fix it.
4. Why does a `threading.Lock` not protect a counter shared by two web servers?

Answers.

1. Threads ≈ 8 × (1 + 18/2) = 80. Each core computes 500 requests/second (1000 ms / 2 ms), so about 8 × 500 = 4,000 requests per second at saturation (illustrative).
2. With a lock: `with lock: if stock > 0: stock -= 1`. In SQL: `UPDATE items SET stock = stock - 1 WHERE id = ? AND stock > 0;` then check that one row changed.
3. Service A holds the customer lock and waits for the order lock; B holds the order lock and waits for the customer lock: a cycle. Fix by always locking customer before order.
4. The lock exists only in one process's memory; the other server has its own lock object, so both can enter the critical section. Use a database row lock, a conditional update, or a distributed lock with fencing.

CHAPTER SUMMARY

- Processes are isolated; threads share memory and are cheaper but riskier.
- Concurrency is structure, parallelism is simultaneous execution; I/O-bound work benefits from concurrency, CPU-bound work from more cores.
- Race conditions (lost update, check-then-act) are timing bugs fixed by atomicity: locks, atomic instructions or optimistic version checks.
- Deadlock needs four conditions; a global lock order breaks circular wait.
- Across machines the same problems return; use database transactions, version checks, idempotency and fenced distributed locks.

REVISION CHECKLIST

- [ ] I can explain a lost update with a timeline.
- [ ] I can fix a check-then-act bug three ways.
- [ ] I can list the four deadlock conditions and one prevention.
- [ ] I can say why a local lock fails across servers.

SUPPLEMENTAL MATERIAL

Further reading. Maurice Herlihy and Nir Shavit, *The Art of Multiprocessor Programming*; Brian Goetz et al., *Java Concurrency in Practice* (concepts apply widely); Martin Kleppmann, "How to do distributed locking" (2016) on fencing tokens.

CHAPTER 3

Data Structures and Algorithms for System Design

Derived from your notes: Notebook chapter 13 (hash map and LRU cache designs), chapter 12.6 (query cache) and the sharding and caching chapters, which assume these structures. **Added in this book:** the whole chapter: hash tables, trees, B-trees, LSM trees, heaps, bloom filters, tries, skip lists, probabilistic counting, complexity.

LEARNING OBJECTIVES

After this chapter you can:

- read Big-O notation and use it to compare operations on large data;
- explain hash tables, balanced trees, B-trees and log-structured merge (LSM) trees and say which databases use which;
- use heaps, queues, tries, bloom filters and counting sketches where they fit;
- explain how these structures reappear inside caches, indexes, queues and rankings in later chapters.

PREREQUISITE KNOWLEDGE

Chapter 1 and basic programming. No algorithms course is assumed.

KEY TERMINOLOGY

Big-O notation
: A way of describing how the cost of an operation grows with input size n, ignoring constant factors. O(1) is constant, O(log n) grows slowly, O(n) is linear, O(n log n) is typical of sorting, O(n^2) quickly becomes unusable.

Hash function
: A function turning a key of any size into a fixed-size number, intended to spread keys evenly.

Collision
: Two different keys mapped to the same hash bucket.

Index
: An extra structure that lets you find rows by a key without scanning everything.

Write amplification
: The ratio of bytes physically written to disk to bytes the application asked to write.

False positive
: A structure answering "yes, present" when the item is in fact absent. Some structures allow these in exchange for tiny memory use.

3.1 Why data structures matter in system design

System design diagrams show boxes: a cache, a database index, a queue, a leaderboard. Inside each box is a data structure chosen to make one operation cheap, and every choice makes some other operation expensive. Knowing the structure lets you predict behaviour you would otherwise discover during an outage: why a database slows when an index no longer fits in memory, why a cache needs a second structure to evict, why a counter of unique visitors can be 1% wrong and use 12 KB instead of gigabytes.

Reading Big-O

Big-O describes growth, not speed. Suppose you search a list of one billion items. An $O(n)$ scan examines up to 10^9 items. An $O(\log n)$ search in a sorted structure examines about $\log_2(10^9) \approx 30$. An $O(1)$ hash lookup examines one bucket (on average). The *constants* matter too: reading memory is roughly 100 ns, a random disk seek on a spinning disk about 10 ms, so a 30-step search over disk can still be slower than a 1,000-step scan in memory. Therefore ask two questions: how does the cost *grow*, and *where* does each step happen (CPU cache, memory, SSD, disk, network)?

Table 3.1 Growth of common complexities for n = 1,000,000.

Complexity	Steps for n = 10^6	Typical example
O(1)	1	Hash lookup by key
O(log n)	about 20	Binary search, balanced tree, B-tree level count
O(n)	1,000,000	Full scan of a table
O(n log n)	about 20,000,000	Sorting
$O(n^2)$	10^{12}	Comparing every pair; avoid on large data

3.2 Hash tables

A **hash table** stores key-value pairs in an array of *buckets*. To find a key, compute `hash(key) mod bucket_count` to pick a bucket, then look inside it. With a good hash function and enough buckets, each bucket holds about one item, so lookup, insert and delete are O(1) on average.

Collisions are unavoidable (many more possible keys than buckets). Two standard strategies:

- **Chaining**: each bucket holds a linked list (or small array) of the entries that landed there. Lookup walks the short list.
- **Open addressing**: if the bucket is taken, probe the next buckets by a fixed rule until a free one is found.

The **load factor** is items divided by buckets. As it grows, collisions multiply and operations slow, so the table is *resized* (usually doubled) when the load factor crosses a threshold such as 0.75, and every entry is *rehashed* into the new array. Resizing is expens-

ive (O(n)) and happens rarely, so the *amortised* cost per insert stays O(1), but a single insert may pause. Distributed systems face the same problem at larger scale, where adding a server changes `hash mod N` for almost every key. Chapter 11 introduces *consistent hashing* to avoid that.

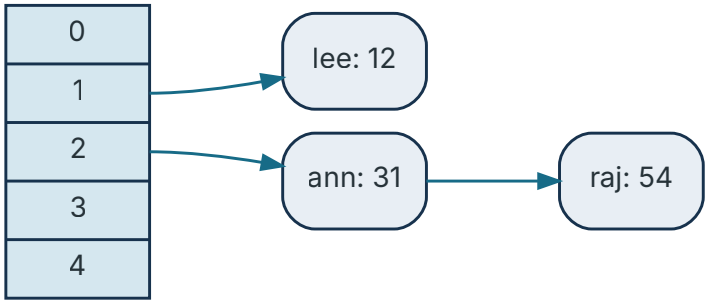


Figure 3.1 A hash table with chaining. Keys "ann" and "raj" collide in bucket 2 and are kept in a short list.

Strengths: very fast point lookups. **Weaknesses:** no ordering, so range queries ("all keys between a and f") need a full scan; unpredictable resize pauses; poor hash functions create hot buckets. In practice: in-memory caches, dictionaries, deduplication sets, and the key-value family of databases (Chapter 10).

3.3 Ordered structures: trees, B-trees and indexes

When you need keys *in order*, for range queries and sorted results, use a tree. A **binary search tree** keeps smaller keys on the left and larger on the right; a *balanced* one (red-black, AVL) guarantees height O(log n) so search, insert and delete are all O(log n).

Databases and file systems almost never use binary trees directly, because each tree step is a separate random read, and disk reads are slow. A **B-tree** (in practice the **B+ tree** variant) fixes this by making each node *wide*: one node is sized like a disk page (commonly 4 to 16 KB) and holds hundreds of keys. The tree is therefore very shallow.

> **WORKED EXAMPLE**
>
> **How deep is a B-tree?** Suppose each node holds up to 500 child pointers. Level 1 (the root) reaches 500 children; level 2 reaches 500 × 500 = 250,000; level 3 reaches 125,000,000; level 4 reaches 62.5 billion. So a table with 100 million rows can be indexed by a tree of only about **3 levels**, and finding any row costs about 3 page reads, often fewer because the upper levels stay cached in memory. (These are illustrative numbers; real fan-out depends on key size.) In a B+ tree, leaf pages are also linked in order, so a range scan walks along the leaves without going back to the top.

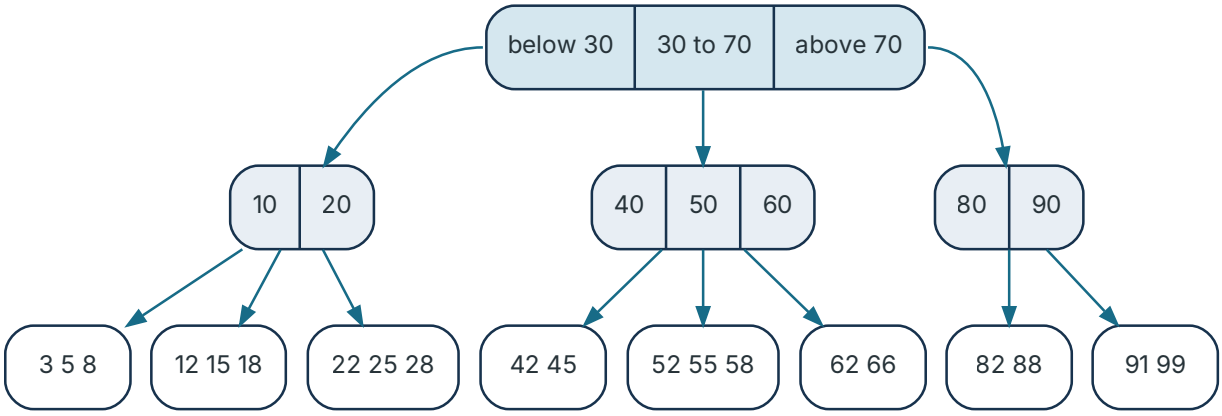


Figure 3.2 A three-level B+ tree index. The root and middle pages guide the search; leaf pages hold keys (with row pointers) in sorted order and are linked for range scans.

Each level narrows the search (Figure 3.2). A lookup of key 42 reads the root, follows the pointer for the range containing 42, reads the middle page, then the leaf. B-trees are the default index in most relational databases (Chapter 8). Their cost is on writes: inserting into a full page *splits* it, and updates modify pages in place, which can cause random writes and, under heavy write load, more disk work.

3.4 Log-structured merge trees

An **LSM tree** takes the opposite approach to writes: never update in place; **append** instead. Incoming writes go first to a **write-ahead log** on disk (for durability) and into a sorted in-memory structure (the *memtable*). When the memtable fills, it is written to disk as an immutable sorted file (an *SSTable*, sorted string table). Over time many files accumulate, so a background process called **compaction** merges them, discarding overwritten and deleted entries. To avoid checking every file on a read, each file has a **bloom filter** (below) that quickly says "definitely not here".

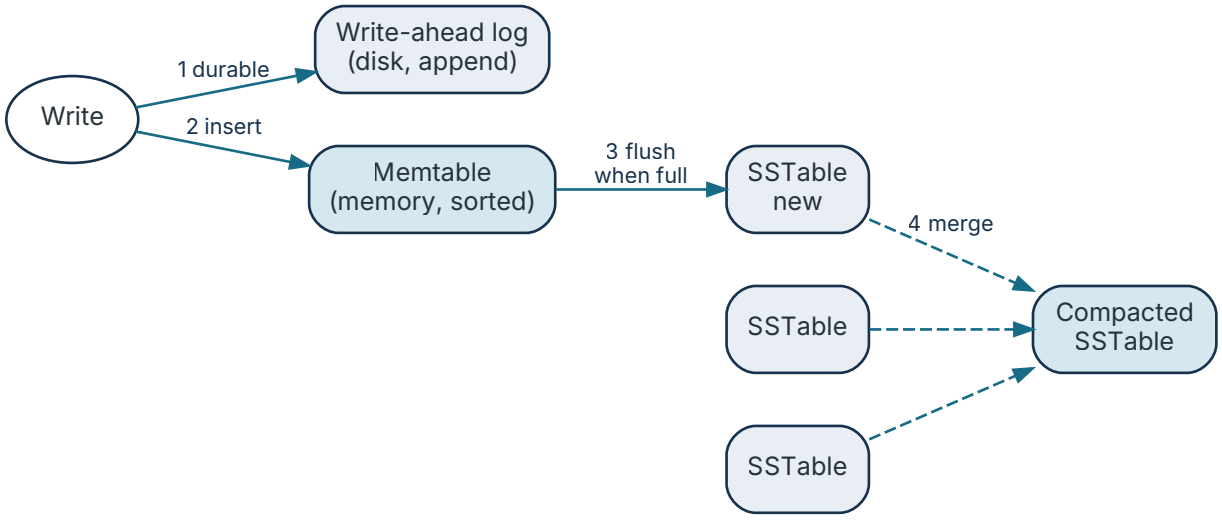


Figure 3.3 Write path of an LSM tree: append to a log, update the in-memory table, flush to sorted immutable files, then compact them in the background.

Table 3.2 B-tree versus LSM tree.

	B-tree	LSM tree
Write pattern	In-place page updates (random writes)	Sequential appends and batched merges
Write throughput	Good	Usually higher, especially on disks
Read cost	Few page reads, predictable	May check several files (mitigated by bloom filters)
Space	Some wasted space in pages	Compaction needs temporary extra space
Write amplification	Moderate	Can be high (data rewritten during compaction)
Typical users	PostgreSQL, MySQL (InnoDB), most relational databases	Cassandra, HBase, RocksDB, LevelDB; many NoSQL stores

3.5 Heaps, queues and ranking

A **queue** returns items in the order added (first in, first out, FIFO); a **stack** in reverse (last in, first out). A **priority queue** returns the item with the highest priority, and is usually built as a **heap**: a tree stored in an array where each parent is smaller (min-heap) than its children. Insertion and removal of the minimum cost O(log n); peeking at the minimum is O(1).

Heaps solve *top-k* problems elegantly. To find the 10 highest-scoring items in a stream of a billion, keep a min-heap of size 10: for each new item, if it beats the heap's minimum, replace it. Memory is O(10), time O(n log 10). This idea underlies leaderboards, "trending" lists, scheduling (which job runs next), and the crawler's frontier (Chapter 23). When you need a *live* ranking you can update and query by position, as in a game leaderboard, the usual answer is a **sorted set** (a skip list or tree with scores), as offered by Redis.

3.6 Tries

A **trie** (prefix tree) stores strings one character per edge, so all strings sharing a prefix share a path. Looking up a word of length L costs O(L), independent of how many words are stored, and "all words starting with `sys`" is simply the subtree below that path. Tries power autocomplete and spell-checkers, and routing tables for IP prefixes. Their weakness is memory use for sparse data, reduced by compressed variants (radix trees).

3.7 Bloom filters and probabilistic structures

Sometimes an exact answer costs too much memory and an approximate one is acceptable. A **bloom filter** answers "have I seen this item?" using a bit array and several hash functions. To add an item, set the bits at its k hash positions. To test an item, check those

k bits: if any is 0, the item is **definitely absent**; if all are 1, it is **probably present**. It never gives false negatives but may give false positives; the rate falls as you spend more bits per item (about 10 bits per item gives a false-positive rate near 1%).

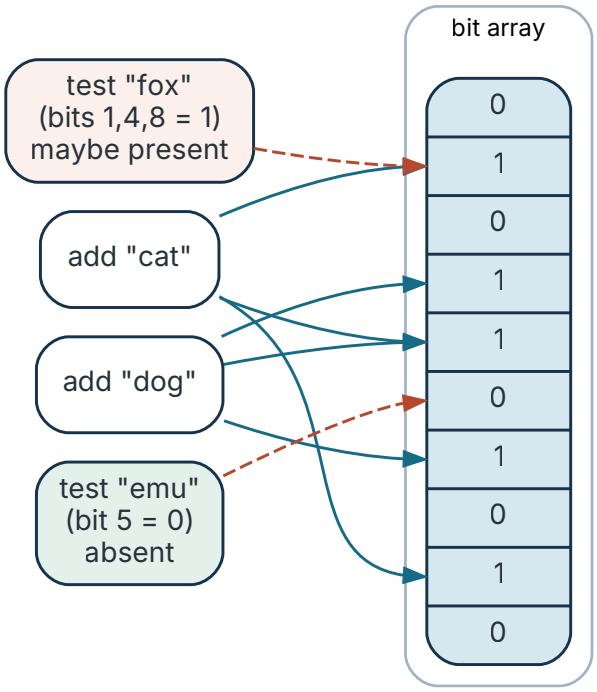


Figure 3.4 A bloom filter with three hash functions. "cat" and "dog" were added. Testing "emu" finds a zero bit, so it is definitely absent; testing "fox" finds all ones by coincidence, a false positive.

Uses: LSM trees skip files that cannot hold a key; a web crawler checks "have I already queued this URL?" without storing every URL; a cache or database avoids expensive lookups for keys that do not exist (protecting against *cache penetration*, Chapter 12). Limitations: standard bloom filters cannot delete items (counting variants can), and the size must be planned in advance.

Two related tools: **HyperLogLog** estimates the number of *distinct* items (unique visitors) with a typical error around 1% using only a few kilobytes, and a **count-min sketch** estimates how often each item occurred (frequent search terms) using a small table, never under-counting but possibly over-counting.

3.8 Choosing the structure

Table 3.3 Which structure for which need.

Need	Structure	Why
Exact lookup by key	Hash table	O(1) average
Range queries, sorted order	B-tree or LSM	Ordered keys
Heavy write rate	LSM tree	Sequential appends

Need	Structure	Why
Top-k, scheduling	Heap / priority queue	O(log n) updates
Prefix search	Trie	Shared prefixes
"Definitely not present" cheaply	Bloom filter	Tiny memory, no false negatives
Distinct count at scale	HyperLogLog	~1% error, kilobytes
Evict least-recently-used	Hash map + doubly linked list	O(1) for both operations (see Chapter 30)

FAILURE MODES

Failure modes. A hash table resize pausing a latency-sensitive path; a hot bucket or hot shard from skewed keys; a B-tree index larger than memory so that every lookup becomes a disk read; compaction falling behind writes in an LSM store, so reads slow and disk fills; a bloom filter sized for 1 million items now holding 100 million, whose false-positive rate approaches 100% and makes it useless.

COMMON MISCONCEPTION

- "*O(1) means fast.*" It means the cost does not grow with n; the constant might be a network round trip.
- "*An index always speeds things up.*" Each index speeds some reads and slows every write, and uses space.
- "*A bloom filter can say an item is present.*" It can only say *probably* present; "absent" is the reliable answer.

INTERVIEW QUESTIONS

Q1. Why do databases use B-trees rather than binary trees? Disk and page reads dominate cost. A B-tree node holds hundreds of keys, so the tree is only a few levels deep, minimising reads; binary trees would need many more, scattered reads.

Q2. When is an LSM tree better than a B-tree? For write-heavy workloads: LSM turns random writes into sequential appends. The trade-off is compaction work and reads that may touch several files.

Q3. How would you implement an LRU cache with O(1) operations? A hash map from key to a node in a doubly linked list ordered by recency; on access move the node to the head; on insert beyond capacity remove the tail node and its map entry.

Q4. How can you count unique visitors across billions of events without storing them? HyperLogLog: about 1% error with kilobytes of memory, and sketches from different servers can be merged.

EXERCISES

1. A B-tree has fan-out 200. How many levels are needed to index 8 million rows? Then 1.6 billion?
2. A bloom filter has 10 bits per item and holds 1 million items. How many bytes is the bit array? What happens if you insert 10 million items without resizing?

3. Design a data structure to return the top 5 most frequent search terms of the last hour (state your assumptions).
4. Why can a hash table not answer "all keys starting with `sys`" quickly, and what would you use?

Answers.

1. $200^1 = 200$; $200^2 = 40{,}000$; $200^3 = 8{,}000{,}000$, so 3 levels for 8 million; $200^4 = 1.6$ billion, so 4 levels.
2. 10 bits × 1,000,000 = 10,000,000 bits = 1.25 MB. With 10 million items the array is nearly all ones and false positives approach 100%: it must be resized or rebuilt.
3. One reasonable design: per-minute count-min sketches or exact hash maps of counts, summed over 60 buckets (drop the oldest minute each minute), plus a min-heap of size 5 over the summed counts. Exact counts are possible if the vocabulary fits in memory; otherwise use the sketch.
4. A hash destroys ordering, so keys with a shared prefix are scattered; use a trie (or an ordered structure such as a B-tree or sorted set with range scan).

CHAPTER SUMMARY

- Big-O shows growth; also ask where each step runs (memory, disk, network).
- Hash tables: O(1) point lookups, no order. B-trees: shallow, ordered, read-friendly. LSM trees: sequential writes, compaction cost.
- Heaps give top-k and scheduling; tries give prefix search; bloom filters give cheap "definitely not"; HyperLogLog and count-min sketch trade exactness for tiny memory.
- Choose by the dominant operation, and remember every structure has a price.

REVISION CHECKLIST

- [ ] I can explain chaining, load factor and resizing.
- [ ] I can compute B-tree depth from fan-out.
- [ ] I can describe the LSM write path and compaction.
- [ ] I can say what a bloom filter can and cannot guarantee.
- [ ] I can build an O(1) LRU cache on paper.

SUPPLEMENTAL MATERIAL

Further reading. Kleppmann, *Designing Data-Intensive Applications*, chapter 3 (storage engines: B-trees and LSM trees); Burton Bloom's 1970 paper on hash coding with allowable errors; Flajolet et al. on HyperLogLog (2007); Cormode and Muthukrishnan on the count-min sketch (2005).

CHAPTER 4

Estimation and Capacity Planning

Derived from your notes: Notebook chapter 10 (Capacity and back-of-the-envelope: powers of two, latency numbers, turning traffic into numbers, the URL-shortener worked example) and the "numbers and formulas to remember" sheet. **Added in this book:** availability and cost arithmetic, bandwidth and storage planning, headroom, and sensitivity analysis.

LEARNING OBJECTIVES

After this chapter you can:

- produce order-of-magnitude estimates of requests per second, storage, bandwidth and memory in a few minutes;
- use powers of two and the standard latency numbers to reason about where time goes;
- convert availability percentages into allowed downtime and combine components in series and in parallel;
- state assumptions and units explicitly, and test how sensitive an answer is to them.

PREREQUISITE KNOWLEDGE

Chapter 1 (requests, latency, throughput). Arithmetic with powers of ten is enough.

KEY TERMINOLOGY

Back-of-the-envelope estimate
A rough calculation done quickly, accurate to within a factor of about two to ten, used to choose between designs.

RPS / QPS
Requests (or queries) per second.

Peak factor
How much higher the busiest moment is than the average; commonly 2 to 10 times.

Headroom
Spare capacity kept above expected peak so bursts and failures do not cause overload.

Nines
Shorthand for availability: "three nines" means 99.9%.

4.1 Why we estimate

Design choices depend on scale, and scale is rarely given as the number you need. "A million users" does not tell you whether to use one database or fifty. An estimate converts a vague number into **requests per second, bytes stored and bytes moved**, and those decide whether a single machine suffices, whether caching is essential, and whether the data fits in memory.

Estimates are for *choosing between designs and spotting impossible ones*, not for predicting a bill. Aim for the right power of ten. State every assumption aloud, label every unit, and say that the numbers are assumptions, not measurements. If one assumption dominates the answer, say so: that is the number to go and measure.

4.2 Powers of two and units

Storage and memory sizes are powers of two. You only need a handful by heart.

Table 4.1 Powers of two worth memorising.

Power	Exact value	Approximate	Name
2^{10}	1,024	one thousand	1 KB (kilobyte)
2^{20}	1,048,576	one million	1 MB (megabyte)
2^{30}	1,073,741,824	one billion	1 GB (gigabyte)
2^{40}	1,099,511,627,776	one trillion	1 TB (terabyte)
2^{8} / 2^{16} / 2^{32}	256 / 65,536 / 4,294,967,296		one byte / two bytes / 32-bit limit

Each step of ten bits multiplies by about a thousand. Two rules help: **1 character is about 1 byte** in plain English text (more in some encodings), and **a day has about 86,400 seconds, roughly 10^5**. A month has about 2.5 million seconds (2.5×10^6).

4.3 Latency numbers worth knowing

Where does time go? The numbers below are rough orders of magnitude for typical hardware; absolute values change with every hardware generation, but the *ratios* have stayed remarkably stable and are what matter.

Table 4.2 Approximate latencies (orders of magnitude; follow the notebook's table).

Operation	Time	Feels like
L1 cache reference	0.5 ns	one heartbeat of a CPU
Main-memory reference	100 ns	200 × L1
Compress 1 KB	10 µs	
Send 1 KB over 1 Gbps network	10 µs	

Operation	Time	Feels like
Read 4 KB randomly from SSD	150 µs	1,500 × memory
Read 1 MB sequentially from memory	250 µs	
Round trip within the same data centre	500 µs	
Read 1 MB sequentially from SSD	1 ms	4 × memory
Disk seek (spinning disk)	10 ms	20 × data-centre round trip
Read 1 MB sequentially from disk	30 ms	120 × memory
Send packet California to Netherlands and back	150 ms	

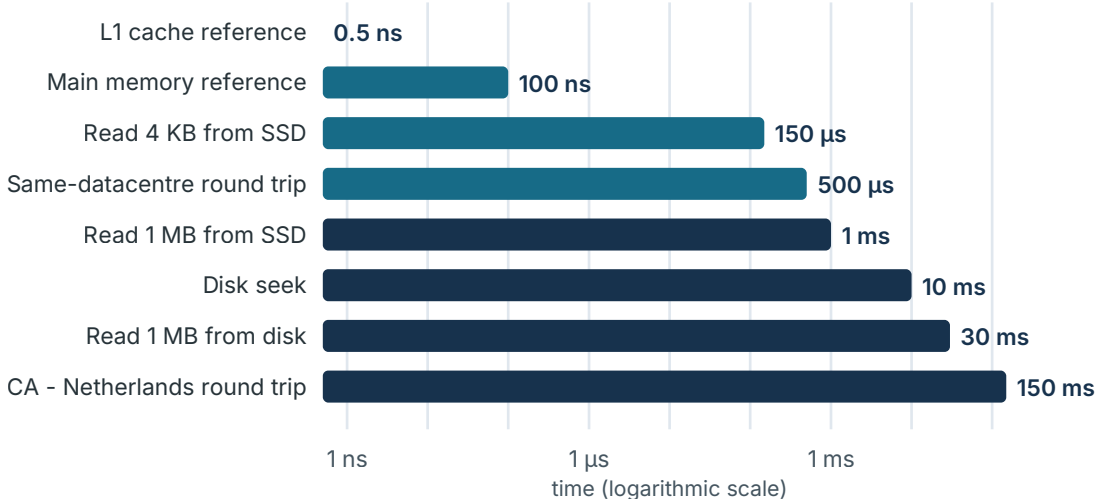


Figure 4.1 The latency ladder on a logarithmic scale. Each tick is ten times the previous one; the whole chart spans eight orders of magnitude, from a CPU cache hit to an intercontinental round trip.

From these numbers come the most useful rules of thumb:

- Memory is fast, disk seeks are slow: avoid random disk reads on the request path; cache hot data in RAM.
- Sequential reads are far cheaper than random ones: append-only logs and sorted files beat scattered updates (Chapter 3).
- Compress data before sending it over a long-distance link when CPU is cheaper than bandwidth.
- Network round trips dominate: a design that makes ten sequential calls across data centres costs more than 1.5 seconds of pure propagation. Prefer fewer, larger calls and keep related data close together.
- Cross-continent calls cost about 150 ms: replicate data or use a CDN to serve users locally.

4.4 Turning traffic into numbers

Requests per second

Start from users and behaviour. **Daily active users (DAU)** times *actions per user per day* gives requests per day; divide by 86,400 for the average per second; then multiply by a peak factor.

WORKED EXAMPLE

Estimate a messaging feature. Assume 10 million daily active users, each sending 20 messages and reading 200 per day.

- Writes per day = $10 \times 10^6 \times 20 = 2 \times 10^8$. Per second = $2 \times 10^8 / 10^5 \approx 2{,}000$ writes per second on average.
- Reads per day = $10 \times 10^6 \times 200 = 2 \times 10^9$, about 20,000 reads per second on average.
- Assume peak is 5 × average: about 10,000 writes/s and 100,000 reads/s at peak.
- Read : write ratio is 10 : 1, so caching reads will help a lot.

One database node typically handles low thousands to tens of thousands of simple operations per second (depending on hardware and queries, so measure it). So 100,000 reads/s at peak needs caching and replicas, and 10,000 writes/s needs a careful look at the write path. The estimate has already told us the shape of the design.

Storage

Storage = (items per day) × (bytes per item) × (days retained), multiplied by a **replication factor** (commonly 3 copies) and a safety margin for indexes and overhead (often 1.5 to 2 times).

WORKED EXAMPLE

Storage for the same messages. 2×10^8 messages per day at 200 bytes each = 4×10^{10} bytes = 40 GB per day. Over five years (1,825 days) = 73 TB. Three replicas = 219 TB. Add 50% for indexes and overhead: about 330 TB. That does not fit in one machine; it implies sharding (Chapter 9) or a store built for large volumes (Chapter 10). Also ask whether old messages can move to cheaper storage.

Bandwidth and memory

Bandwidth = requests per second × bytes per response. At 100,000 reads per second and 1 KB per response, that is 100 MB/s = 800 Mbps outbound: heavy for one network link (1 Gbps), so you need many servers or a CDN. Convert carefully between **bytes** (B) and **bits** (b); network speeds are quoted in bits per second, sizes in bytes. A useful constant: **1 Gbps ≈ 125 MB/s.**

Memory for a cache: if 20% of items serve 80% of reads (the *80/20 rule*, a heuristic, not a law), cache the hot 20%. If the data worth caching is 200 GB, that is a handful of large-memory servers, not one.

Single-machine capacity

Rough orders for one modern server (to calibrate, then measure): a web server doing simple work handles thousands of requests per second; a database handles low thousands to tens of thousands of simple queries per second; a distributed cache node handles on the order of 100,000 operations per second; an SSD handles tens to hundreds of thousands of random reads per second. If your estimate is far below these, one machine may suffice; far above, you need a distributed design.

4.5 Availability arithmetic

Availability = uptime / (uptime + downtime). Expressed in nines, it translates into allowed downtime:

Table 4.3 Allowed downtime for each availability level.

Availability	Down per year	Down per month	Down per week	Down per day
99% (two nines)	3.65 days	7.3 h	1.68 h	14.4 min
99.9% (three nines)	8.77 h	43.8 min	10.1 min	1.44 min
99.99% (four nines)	52.6 min	4.4 min	1.01 min	8.6 s
99.999% (five nines)	5.26 min	26.3 s	6.05 s	0.86 s

In series (a request needs *all* components): multiply availabilities. Two components at 99.9% give $0.999 \times 0.999 = 99.8\%$. Every extra dependency on the critical path lowers availability. **In parallel** (any one component suffices, independent failures): unavailability multiplies. Two replicas at 99.9% each give $1 - 0.001 \times 0.001 = 99.9999\%$. This assumes failures are *independent*; two replicas on the same power supply or sharing one buggy deployment fail together, so the real figure is lower.

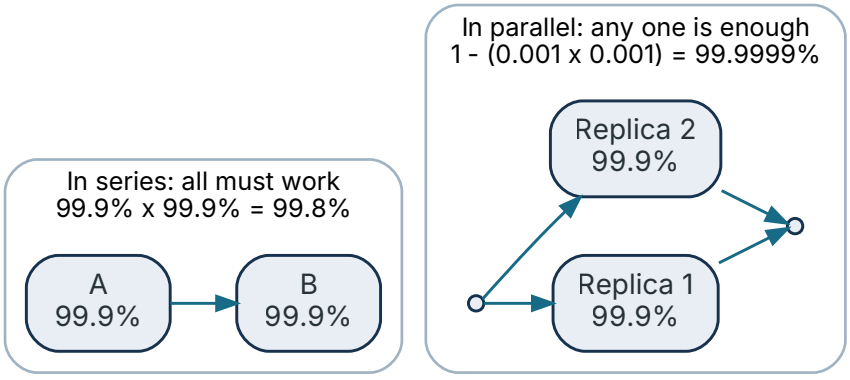


Figure 4.2 Availability in series (all components required: multiply) and in parallel (any one suffices: multiply the failure probabilities).

COMMON MISTAKE

Common mistakes. Mixing bits and bytes (a factor of eight). Forgetting the peak factor and sizing for the average. Forgetting replication when estimating storage. Treating a derived "five nines" as real when the design has a single SPOF. Presenting an estimate with false precision ("218.4 TB") instead of "a few hundred TB".

4.6 Sensitivity and headroom

A good estimator asks "what if the main assumption is off by 10×?" If read volume were ten times larger, would the design survive? If writes were ten times smaller, would it be over-built? Keep a list: *assumption, value, how to measure, impact if wrong.* Then plan for **headroom**: run at no more than about 50% to 70% of peak capacity so one failed node, a traffic burst or a slow dependency does not push the system into overload (Chapter 1, latency curve).

SUPPLEMENTAL MATERIAL

Cost estimation. Convert capacity to money: servers or instances × price per hour × hours, plus storage per GB-month, plus network egress per GB (often the surprise). Use the cloud provider's current price calculator rather than remembering prices; they change. A rough habit: storage is cheap, data transfer out of the cloud is expensive, and managed services trade money for operational effort. Chapter 19 returns to cost and capacity planning.

4.7 Worked example: a URL shortener

This is the notebook's own worked example, expanded. Assume 100 million new URLs per month and a 100 : 1 read-to-write ratio.

WORKED EXAMPLE

1. **Writes per second.** 100×10^6 / (2.5×10^6 seconds per month) = 40 writes per second on average. A peak factor of 10 would be about 400 per second.
2. **Reads per second.** 100 × 40 = 4,000 per second on average; peak about 10,000 (the notebook rounds peak at 10,000).
3. **Storage per URL.** Assume 500 bytes (long URL plus metadata). $100 \times 10^6 \times 500$ B = 50 GB per month; × 12 × 5 years = 3 TB over five years (at 3 copies, about 9 TB).
4. **Bandwidth.** Incoming: 40 writes/s × 500 B = 20 KB/s. Outgoing: 4,000 reads/s × 500 B = 2 MB/s. A trivial amount; bandwidth is not the constraint.
5. **Cache.** Hottest 20% of daily reads: 4,000 × 86,400 = 345 million reads/day; 20% × 345 million × 500 B ≈ 35 GB: fits in the memory of one or two cache servers.
6. **Conclusion.** The load is small (a few hundred writes/s, ~10,000 reads/s); the cache and a replicated database comfortably serve it. The hard questions are not scale but **unique key generation** and **durability**, so spend interview time there.

INTERVIEW QUESTIONS

Q1. How would you estimate the storage for a photo-sharing service? Photos per day × average size (after compression and multiple resolutions) × retention × replication, plus metadata. State assumptions, give an order of magnitude, and note that originals and thumbnails dominate while metadata is small.

Q2. Convert 99.95% availability into downtime per month. 0.05% of 43,800 minutes (about 730 hours) ≈ 21.9 minutes per month.

Q3. Two services in series each have 99.9% availability. What is the combined availability? About 99.8%. Series dependencies multiply; this is why long synchronous chains reduce availability.

Q4. Why does the interviewer care about your numbers if they are rough anyway? They show whether you can turn vague scale into design decisions (cache, shard, queue or single database) and whether you notice what dominates.

EXERCISES

1. A site has 50 million daily active users (DAU), each making 30 requests per day. Estimate average and peak RPS (peak factor 4).
2. Each request returns 20 KB. Estimate peak outbound bandwidth in Gbps.
3. A service stores 300 million records per year, 1 KB each, replicated 3 times, kept 4 years. How many terabytes?
4. A request touches three services at 99.95%, 99.9% and 99.99%. What is the combined availability and monthly downtime?
5. A user in London calls a service in California that makes four sequential calls to a database in the same data centre and two to a service in Singapore (assume 170 ms per round trip). Estimate the minimum latency.

Answers.

1. $50 \times 10^6 \times 30 = 1.5 \times 10^9$ per day; / 86,400 ≈ 17,400 RPS average; peak ≈ 70,000 RPS.
2. 70,000 × 20 KB = 1.4 GB/s = 11.2 Gbps (× 8 bits). A single 10 Gbps link would saturate; use a CDN and several servers.
3. $300 \times 10^6 \times 1$ KB = 300 GB per year; × 4 = 1.2 TB; × 3 = 3.6 TB (more with index overhead).
4. 0.9995 × 0.999 × 0.9999 ≈ 0.9984, i.e. 99.84%; downtime ≈ 0.16% of 730 h ≈ 1.2 hours per month.
5. London to California is about 150 ms round trip (use 150 ms), four in-data-centre calls at 0.5 ms = 2 ms, and two Singapore calls at 170 ms = 340 ms: roughly 490 ms minimum, dominated by the long-distance calls. Parallelising the Singapore calls would save 170 ms; replicating data nearer would save most of it.

CHAPTER SUMMARY

- Estimate to choose designs: convert users into RPS, storage and bandwidth; state assumptions and units.
- Memorise powers of two and the latency ladder; ratios matter more than absolute values.
- Peak, replication, indexes and headroom change the answer; bits are not bytes.
- Availability multiplies in series and improves in parallel only if failures are independent.
- Test sensitivity of the dominant assumption, then go and measure it.

REVISION CHECKLIST

- [ ] 1 day ≈ 10^5 seconds; 1 month ≈ 2.5×10^6 seconds.
- [ ] RPS = DAU × actions / 86,400, then × peak factor.
- [ ] Storage = items × size × retention × replicas × overhead.
- [ ] Three nines ≈ 8.8 hours a year; four nines ≈ 53 minutes a year.
- [ ] Series multiplies availability; parallel multiplies unavailability.

SUPPLEMENTAL MATERIAL

Further reading. Jeff Dean's talk "Numbers Everyone Should Know" and the interactive "Latency numbers every programmer should know" web pages (numbers there are updated by their authors; check the date); Google's *Site Reliability Engineering* book for availability targets.

PART II

Networking and the Entry Layer

How requests find a server, travel to it, and are spread across many servers.

CHAPTER 5

Networking and the Request Lifecycle

Derived from your notes: Notebook chapter 3 (layers in one minute, HTTP, TCP and UDP) and the DNS section of chapter 4. **Added in this book:** TLS, QUIC and HTTP/3, the connection-setup latency budget, and the complete request lifecycle.

LEARNING OBJECTIVES

After this chapter you can:

- describe the layers a message passes through and name a protocol at each layer;
- trace a DNS lookup from the browser to an authoritative server and explain TTLs and caching;
- explain the TCP handshake, reliability, flow control and congestion control, and when UDP is the better choice;
- explain what TLS adds and what it costs in round trips;
- distinguish HTTP/1.1, HTTP/2 and HTTP/3, including that HTTP/3 runs over QUIC on UDP while HTTP/2 normally runs over TCP;
- count the round trips of a cold request and predict its minimum latency.

PREREQUISITE KNOWLEDGE

Chapter 1 (request path, latency) and Chapter 4 (latency numbers).

KEY TERMINOLOGY

IP address
A number identifying a network interface, for example `203.0.113.7` (IPv4) or `2001:db8::7` (IPv6).

Port
A number (0 to 65535) selecting a program on a machine; HTTPS conventionally uses 443.

Packet
A small chunk of data, with addressing information, that is routed independently across the network.

Protocol
An agreed set of rules for exchanging messages.

Round trip time (RTT)
The time for a message to reach the other side and a reply to return.

Domain Name System (DNS)
The distributed directory that maps names such as `example.com` to IP addresses.

Transmission Control Protocol (TCP) / User Datagram Protocol (UDP)
The two main transport protocols: TCP provides a reliable ordered stream; UDP sends independent datagrams with no guarantees.

Transport Layer Security (TLS)
The protocol that encrypts and authenticates a connection (the "S" in HTTPS).

5.1 Layers in one minute

Networks are built in layers, each using the layer below and hiding its details. You do not need every layer, but you need to know which one a problem belongs to.

Table 5.1 The layers we use and example protocols.

Layer	Job	Examples	Addressed by
Application	What the two programs say to each other	HTTP, DNS, SMTP (email), gRPC	URL, name
Transport	Delivery between programs; reliability	TCP, UDP, QUIC	Port
Network (internet)	Routing packets between machines	IP	IP address
Link / physical	Moving bits across one hop	Ethernet, Wi-Fi, fibre	MAC address

As a request is sent, each layer wraps the data from the layer above in its own header (**encapsulation**); the receiver unwraps in reverse order (Figure 5.1). Load balancers are often described as "layer 4" (they look at IP addresses and ports, treating traffic as opaque) or "layer 7" (they read HTTP and can route by URL or header); see Chapter 7.

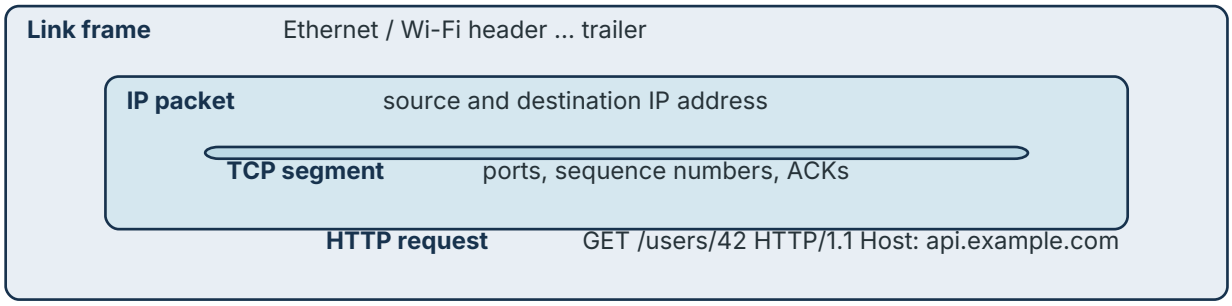


Figure 5.1 Encapsulation. The HTTP request is wrapped in a TCP segment (adding ports and sequencing), which is wrapped in an IP packet (adding addresses), which is carried in a link-layer frame.

5.2 DNS: names to addresses

People remember names; routers need numbers. **DNS** is a worldwide, hierarchical, cached directory. A name is read right to left: in `www.example.com`, the dot at the end is the **root**, `com` is the **top-level domain** (TLD), `example` is the domain registered by a company, and `www` is a host chosen by that company.

Three kinds of servers cooperate:

- A **recursive resolver** (run by your internet provider, your company, or a public service) does the legwork for you and caches results.
- **Root and TLD servers** do not know the answer but know who does.
- The **authoritative name server** holds the real records for a domain and gives the final answer.

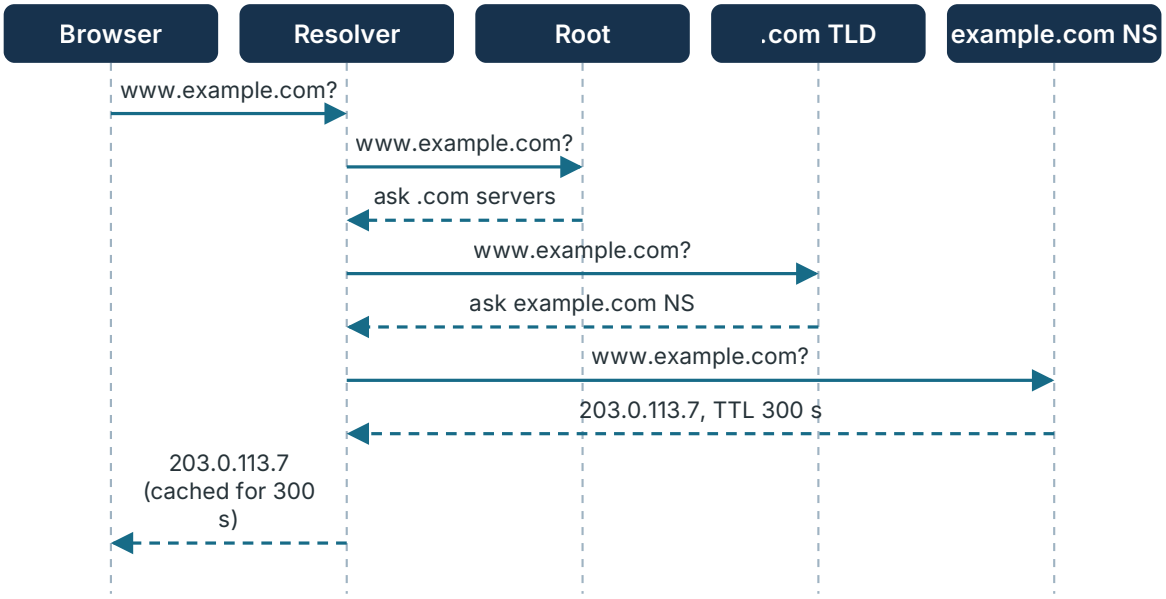


Figure 5.2 Resolving www.example.com on a cold cache. The resolver asks the root, then the TLD servers, then the authoritative server; each step narrows the search. The answer is cached for its time to live.

The steps in Figure 5.2: the browser asks its local resolver; the resolver, with nothing cached, asks a root server, which refers it to the servers for `.com`; those refer it to the authoritative servers for `example.com`; and those return the address. The resolver returns the answer and *caches* it. Typically a lookup that hits a warm cache takes about a millisecond; a cold lookup may take tens to hundreds of milliseconds.

Record types and TTL

Table 5.2 Common DNS record types.

Type	Meaning	Example
A	Name to IPv4 address	`www A 203.0.113.7`
AAAA	Name to IPv6 address	`www AAAA 2001:db8::7`
CNAME	Alias to another name	`static CNAME cdn.provider.net`
NS	Which servers are authoritative	`example.com NS ns1.provider.net`
MX	Where email goes	`example.com MX 10 mail.example.com`
TXT	Free text (verification, email policy)	`"v=spf1 ..."`

Every answer carries a **time to live** (TTL), the number of seconds caches may keep it. A long TTL (hours) reduces load and speeds lookups; a short TTL (seconds or minutes) lets you move traffic quickly. The trade-off is explicit: *caching makes DNS fast, but changes propagate slowly*. Before a planned migration, lower the TTL a day ahead.

Routing tricks DNS can do

Because the authoritative server chooses the answer, it can steer traffic: **round robin** (rotate among several addresses), **weighted** (send 10% to a new data centre), **latency-based** (answer with the region fastest for the resolver) and **geographic** (answer by user country, useful for data-residency rules). It can also withhold unhealthy addresses if a health checker tells it which servers are down.

> COMMON MISTAKE
>
> **Common mistakes.** Assuming DNS failover is instant: clients and resolvers ignore low TTLs sometimes, so recovery can take minutes. Running your own authoritative DNS without redundancy (a single DNS provider outage can take your whole site offline even though your servers are healthy; many organisations use a managed, anycast DNS service or two providers). Putting a CNAME at the zone apex, which standard DNS disallows.

5.3 TCP: a reliable stream

IP delivers packets on a best-effort basis: they can be lost, duplicated, reordered or delayed. **TCP** builds a dependable ordered byte stream on top. It does this with:

- a **handshake** to establish a connection;
- **sequence numbers** so the receiver can reorder data and detect missing pieces;
- **acknowledgements** and **retransmission** of anything unacknowledged;
- **checksums** to detect corruption;
- **flow control**, which stops a fast sender from overwhelming a slow receiver (the receiver advertises how much more it can accept);
- **congestion control**, which slows the sender when the network seems overloaded (packet loss is taken as the signal), starting slowly and speeding up (*slow start*).

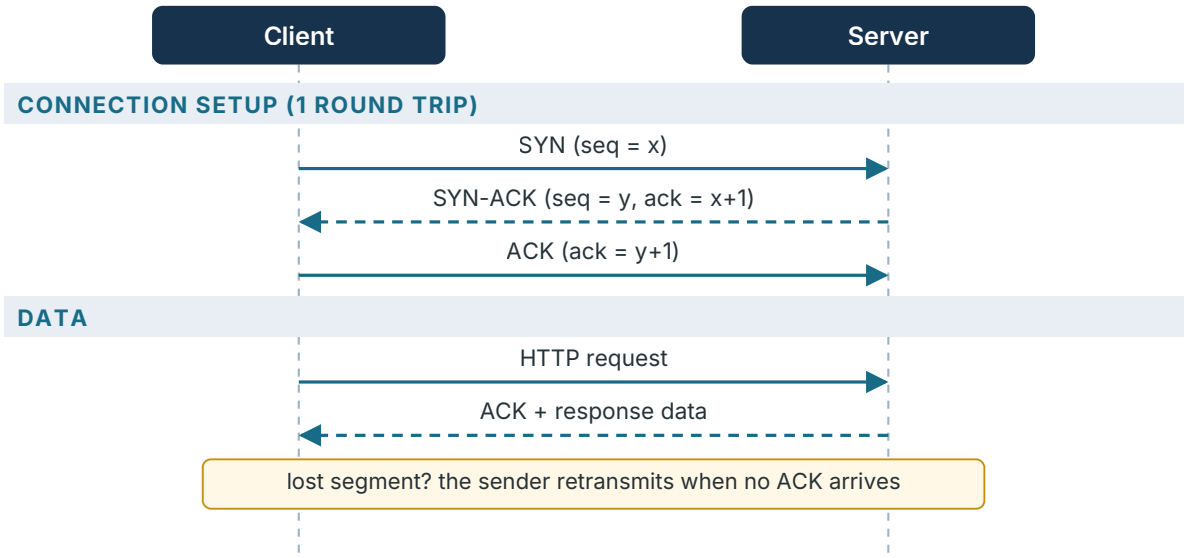


Figure 5.3 The TCP three-way handshake, then data. The connection costs one round trip before the first byte of application data can be sent.

The handshake (Figure 5.3) is SYN (client says hello and proposes a starting sequence number), SYN-ACK (server acknowledges and proposes its own), ACK (client acknowledges). After that, both sides can send data. The setup costs **one RTT** before the request can leave. That is why servers and clients **reuse connections** (keep-alive, connection pools) instead of opening a new one for each request, and why a new connection starts slowly because of slow start. A TCP stream also suffers **head-of-line blocking**: if one packet is lost, later data already received must wait until the missing piece is retransmitted.

5.4 UDP and QUIC

UDP sends independent datagrams with just a port and a checksum: no handshake, no ordering, no retransmission, no congestion control. It is therefore fast to start and light, but the application must cope with loss and reordering itself.

Table 5.3 TCP versus UDP.

	TCP	UDP
Connection	Handshake required, stateful	None
Delivery	Reliable, ordered	Best effort; loss and reordering possible
Overhead	Higher (headers, state, retransmission)	Lower
Speed to first byte	One extra RTT	Immediate
Good for	Web pages, APIs, file transfer, email, databases	Live voice and video, games, DNS queries, telemetry

Use TCP when you need every byte, in order (a bank transfer, a web page). Use UDP when stale data is worthless and speed matters more than completeness (a video call: a lost frame is better skipped than waited for). UDP also supports broadcast and multicast, useful for service discovery on a local network.

QUIC (standardised in 2021) is a modern transport protocol built *on top of UDP*. It reimplements reliability, ordering, congestion control and encryption in user space, as many independent streams over one connection. Because streams are independent, one lost packet delays only its own stream, removing TCP's head-of-line blocking. It also combines the transport and TLS handshakes, saving round trips, and a connection can survive a client changing networks (such as moving from Wi-Fi to mobile data), because it is identified by a connection ID rather than by addresses.

5.5 TLS: encryption and identity

Plain HTTP can be read and altered by anyone on the path. **TLS** provides three properties: **confidentiality** (outsiders cannot read the data), **integrity** (tampering is detected) and **authentication** (the server proves it is who it claims to be).

The server presents a **certificate**: a document binding its name to a public key, signed by a **certificate authority** (CA) that browsers trust. In the handshake the client verifies the certificate chain, the client and server agree on session keys using public-key cryptography (a key exchange), and the rest of the connection is encrypted with fast symmetric keys. With TLS 1.3 the handshake takes **one RTT** (zero for a resumed connection in some cases, with replay-attack caveats), against two in TLS 1.2.

> CLARIFICATION
>
> **Clarification on terminology.** People say "SSL" but the old SSL protocols are obsolete; what is used today is TLS. "SSL termination" in load-balancer language means ending the TLS connection at the balancer, which then forwards decrypted (or re-encrypted) traffic to the servers behind it; see Chapter 7.

5.6 HTTP: the language of the web

HTTP (HyperText Transfer Protocol) is the application protocol of the web and most public APIs. It is **request/response** and stateless: each request carries everything the server needs, and the server remembers nothing between them (state, such as a login, is carried explicitly by cookies or tokens).

A request has a **method**, a **path** (and query), **headers** and an optional **body**; a response has a **status code**, headers and a body.

```
GET /users/42 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer <token>

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=60

{"id": 42, "name": "Asha"}
```

Table 5.4 Common methods. *Safe* methods do not change server state; *idempotent* methods have the same effect whether sent once or many times.

Method	Meaning	Safe	Idempotent
GET	Read a resource	Yes	Yes
HEAD	Like GET without the body	Yes	Yes
POST	Create a resource or trigger processing	No	**No**
PUT	Create or replace a resource	No	Yes
PATCH	Partially update	No	Not guaranteed
DELETE	Remove	No	Yes

Table 5.5 Status code families.

Family	Meaning	Examples
1xx	Informational	101 Switching Protocols
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirect	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

Idempotency matters because the network is unreliable: if a client sends a request and the connection drops before the reply arrives, it does not know whether the server acted. Retrying a PUT or DELETE is safe; retrying a POST may duplicate the action unless the API provides an *idempotency key* (Chapter 6).

HTTP versions

Table 5.6 How HTTP versions differ.

Version	Transport	Key idea	Limitation
HTTP/1.1	TCP	Keep-alive reuses connections; text format	One request at a time per connection; browsers open several connections

Version	Transport	Key idea	Limitation
HTTP/2	TCP (normally, with TLS)	Binary framing; many requests multiplexed on one connection; header compression	TCP head-of-line blocking: one lost packet stalls all streams
HTTP/3	**QUIC over UDP**	Same ideas as HTTP/2 over independent QUIC streams	UDP may be blocked or throttled on some networks; clients fall back to HTTP/2

CLARIFICATION

Clarification. The notebook pairs HTTP with TCP, which is accurate for HTTP/1.1 and HTTP/2. HTTP/3 does not use TCP at all: it uses QUIC, which runs over UDP. This does not make HTTP/3 unreliable; QUIC supplies reliability itself. The practical rule: HTTP/1.1 and HTTP/2 need a TCP (and usually TLS) handshake; HTTP/3 needs one combined QUIC handshake.

5.7 The full lifecycle of a cold request

Putting it together: what must happen before the first byte of a web page arrives at a user who has never visited the site?

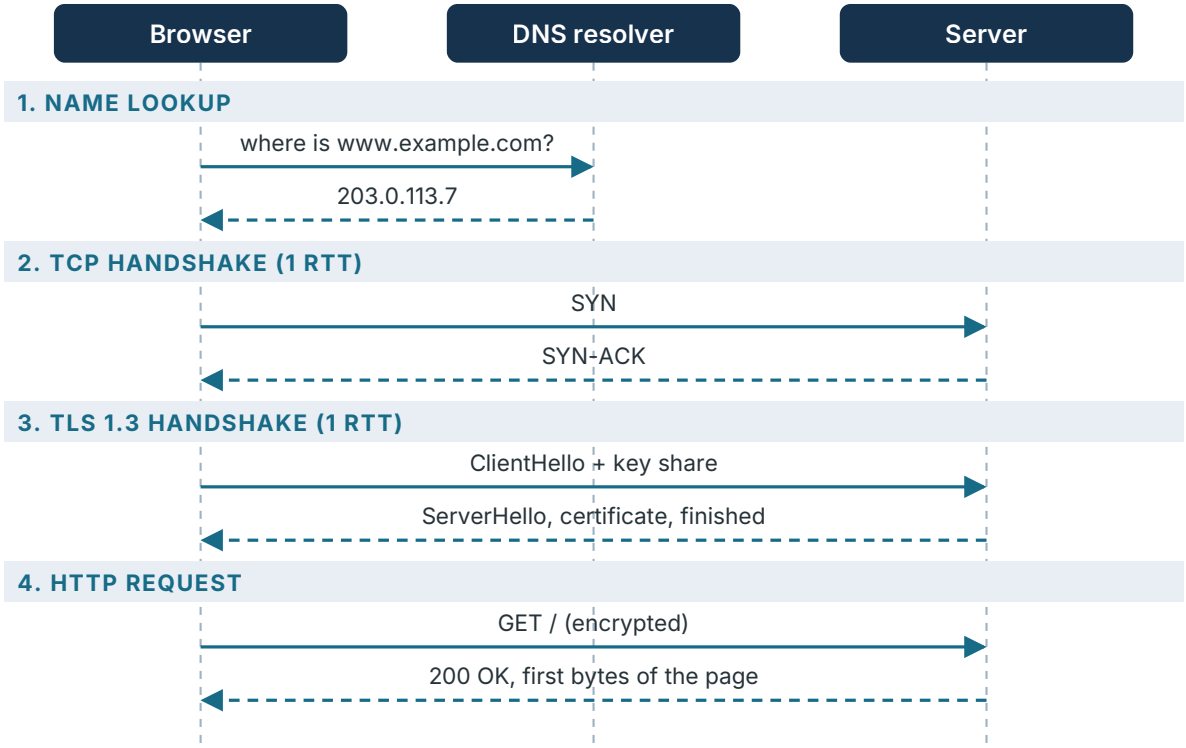


Figure 5.4 Round trips of a first request to https://www.example.com over HTTP/2 with TLS 1.3. Each step must complete before the next begins, so latency adds up.

WORKED EXAMPLE

Minimum latency of a cold request. Suppose the user is 40 ms RTT from the server, the DNS resolver is 10 ms away, and the authoritative lookup is cached at the resolver.

- DNS (cached at resolver): 1 RTT to resolver = 10 ms.
- TCP handshake: 1 RTT = 40 ms.
- TLS 1.3 handshake: 1 RTT = 40 ms.
- HTTP request and first byte of response: 1 RTT plus server work = 40 ms + say 20 ms = 60 ms.
- **Total ≈ 150 ms** before the first byte, of which only 20 ms is the server doing real work.

If the server is on another continent (RTT 150 ms) the same sequence costs about 10 + 150 + 150 + 170 = 480 ms. Remedies follow directly: terminate connections near the user (a CDN or edge proxy), reuse connections, use TLS 1.3 and session resumption, and use HTTP/3 where possible so that transport and encryption share one handshake. Without any change to application code the user sees roughly half the delay.

FAILURE MODES

Failure modes. DNS outages or poisoned caches; expired TLS certificates (a very common cause of outages, prevented by automated renewal and monitoring); TCP connection exhaustion when many short-lived connections leave the server with thousands in a waiting state; packet loss triggering retransmission delays and congestion backoff; a firewall blocking UDP so HTTP/3 must fall back; MTU (maximum packet size) mismatches causing mysterious hangs for large responses.

COMMON MISCONCEPTION

- "*HTTPS is slow.*" The handshake costs round trips, but connection reuse and TLS 1.3 make the steady-state cost small.
- "*UDP is unreliable so protocols built on it are unreliable.*" QUIC is reliable; reliability just moves into the protocol above UDP.
- "*DNS changes are instant.*" Caches keep old answers until their TTL expires, and some ignore low TTLs.
- "*HTTP keeps a session.*" HTTP is stateless; sessions are built from cookies or tokens.

INTERVIEW QUESTIONS

Q1. What happens when you type a URL and press Enter? Browser cache check; DNS resolution (resolver, root, TLD, authoritative); TCP (or QUIC) connection; TLS handshake; HTTP request; server processing, possibly via CDN, load balancer and cache; response; the browser parses, requests sub-resources, renders. Mention connection reuse and caching for extra credit.

Q2. TCP versus UDP, and when would you choose each? TCP: reliable, ordered, connection-based; use for correctness-critical transfers. UDP: no guarantees, low overhead; use for real-time media, games, DNS, or when the application adds its own recovery.

Q3. How does HTTP/3 differ from HTTP/2? HTTP/3 runs over QUIC on UDP rather than TCP: independent streams avoid head-of-line blocking, the transport and TLS handshakes are combined, and connections survive network changes.

Q4. Why is a POST not safe to retry blindly? It may create a second resource; use an idempotency key so the server recognises the repeated request.

EXERCISES

1. A user is 80 ms RTT from the server. Estimate time to first byte for a brand-new HTTPS connection (HTTP/2, TLS 1.3), ignoring DNS and server work. How much does reusing the connection save?
2. A DNS record has a TTL of 86,400 s. You must move to a new IP address in one hour. What should you have done earlier?
3. Name one case each for choosing TCP, UDP and HTTP/3.
4. Explain how head-of-line blocking can make an HTTP/2 page load slow on a lossy network.

Answers.

1. TCP 80 ms + TLS 80 ms + HTTP 80 ms = 240 ms. Reusing the connection leaves only the HTTP round trip: 80 ms, saving 160 ms.
2. Lower the TTL to minutes at least a day beforehand (longer than the old TTL), then switch, then raise the TTL again.
3. TCP: file or database transfer; UDP: live video call or DNS query; HTTP/3: a mobile app on varying networks that benefits from fast setup and connection migration.
4. HTTP/2 multiplexes many streams over one TCP connection; a lost packet stalls all later data on that connection until it is retransmitted, so every stream waits even if their own data arrived.

CHAPTER SUMMARY

- Layers hide detail: application (HTTP, DNS), transport (TCP, UDP, QUIC), network (IP), link.
- DNS is a cached hierarchy; TTL balances speed against agility; DNS can steer traffic.
- TCP gives reliable ordered streams at the cost of handshakes, head-of-line blocking and slow start; UDP trades guarantees for speed; QUIC rebuilds reliability over UDP.
- TLS adds confidentiality, integrity and authentication for one extra round trip (TLS 1.3).
- HTTP is stateless request/response; methods differ in safety and idempotency; HTTP/3 uses QUIC over UDP.
- Cold-request latency is mostly round trips: reduce them with edge termination, reuse and modern protocols.

REVISION CHECKLIST

- [ ] I can trace a DNS lookup and say what the TTL does.
- [ ] I can draw the TCP handshake and list TCP's guarantees.
- [ ] I can say which HTTP version uses which transport.
- [ ] I can count round trips for a cold request.
- [ ] I can say which HTTP methods are idempotent.

SUPPLEMENTAL MATERIAL

Further reading. IETF RFC 9000 (QUIC) and RFC 9114 (HTTP/3), RFC 8446 (TLS 1.3), RFC 9110 (HTTP semantics); Ilya Grigorik, *High Performance Browser Networking* (free online).

CHAPTER 6

APIs: RPC, REST and Contracts

Derived from your notes: Notebook chapter 3 (RPC and REST sections, the RPC-versus-REST operations table) and the API sketches inside the design problems. **Added in this book:** idempotency keys, pagination, versioning, error design, contracts and compatibility, webhooks versus polling.

LEARNING OBJECTIVES

After this chapter you can:

- explain what an API is and why it is a *contract* between teams;
- compare RPC and REST, and choose between them for public and internal interfaces;
- design resource-oriented endpoints, status codes and error responses;
- implement safe retries with idempotency keys;
- page through large result sets with offset and cursor pagination and explain the trade-off;
- evolve an API without breaking existing clients.

PREREQUISITE KNOWLEDGE

Chapter 5 (HTTP methods, status codes, idempotency).

KEY TERMINOLOGY

API (application programming interface)
The defined way one program asks another to do something.

Endpoint
One addressable operation or resource of an API, for example `POST /pastes`.

RPC (remote procedure call)
A style where the client calls a named action on a server as if it were a local function.

REST (representational state transfer)
A style where clients operate on *resources*, identified by URLs, using standard HTTP methods.

Idempotent
Doing it many times has the same effect as doing it once.

Idempotency key
A unique value the client attaches to a request so the server can recognise and ignore duplicates.

Contract
The agreed shape of requests, responses and behaviour that clients depend on.

6.1 Why APIs are contracts

Every boundary in a system, between a browser and a server, between two internal services, between your company and a partner, is an API. Once other programs depend on it, an API stops being an implementation detail and becomes a **contract**. Changing it carelessly breaks code you cannot see or redeploy. So API design is partly the design of *what can change later without breaking anyone*.

Two ideas shape most API decisions: the **style** (action-oriented RPC or resource-oriented REST) and the **failure behaviour** (what happens when a request is retried, partially fails, or arrives twice).

6.2 RPC

In **Remote Procedure Call** the client calls a named action with parameters, and the framework turns it into a network message. The call looks like a local function call, which is both its appeal and its danger: a local call does not fail with a timeout, but a remote one does, and the caller cannot always know whether the remote side ran it.

```
POST /rpc/getUser           {"userId": 42}
POST /rpc/addItemToUserList {"userId": 42, "item": "book"}
```

Modern RPC frameworks (gRPC, Thrift, Avro) define the contract in a schema file, generate client and server code in many languages, and encode messages in a compact binary format over HTTP/2. They are fast, strongly typed and well suited to **internal** service-to-service calls where you control both ends.

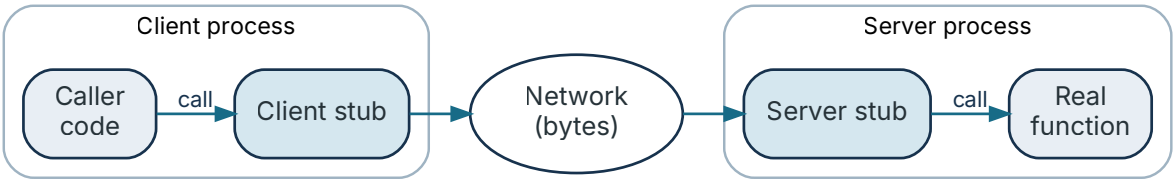


Figure 6.1 How an RPC call crosses the network. The client calls a generated stub, which marshals the arguments into bytes; the server skeleton unmarshals them and calls the real function. The reply travels back the same way.

Strengths: compact and fast, strict schemas, generated clients, streaming support. **Weaknesses:** clients are tightly coupled to the server's operations (a new operation means a new client release); binary formats are harder to debug by hand; browsers and third parties cannot call them as easily; caching at the HTTP layer is difficult because everything is typically a POST.

6.3 REST

REST treats the server as a collection of **resources**: nouns such as users, orders and pastes, each with a URL. Clients use standard HTTP methods to operate on them, and the server is **stateless** between requests. Responses are *representations* of a resource, commonly JSON, and often contain links to related resources (this last property, called HATEOAS, is rarely fully implemented).

Table 6.1 The same operations expressed in RPC and REST (from the notebook, extended).

Operation	RPC style	REST style
Sign up	`POST /signup`	`POST /persons`
Resign	`POST /resign` with `{"personid": 1234}`	`DELETE /persons/1234`
Read a person	`GET /readPerson?personid=1234`	`GET /persons/1234`
Read a person's items	`GET /readUsersItemsList?personid=1234`	`GET /persons/1234/items`
Add an item	`POST /addItemToUsersItemsList`	`POST /persons/1234/items`
Update an item	`POST /modifyItem`	`PUT /persons/1234/items/456`
Delete an item	`POST /removeItem`	`DELETE /persons/1234/items/456`

REST rules of thumb: use **nouns, plural, hierarchical** URLs; let the **method** express the action; return the right **status code**; make `GET` safe and cacheable; make `PUT` and `DELETE` idempotent; and keep servers stateless so any instance can answer.

Strengths: easy to understand and debug (curl, a browser), works with HTTP caching and proxies, natural fit for public APIs and web clients. **Weaknesses:** can need several round trips to assemble one screen (the "chatty API" problem, which gateways or query-style APIs such as GraphQL address); not every action maps to a noun ("send the password-reset email"); versioning needs discipline; text formats are larger than binary.

> **COMMON MISTAKE**
>
> **Common mistakes.** Using GET for actions that change state (crawlers and prefetchers will trigger them). Returning 200 with an error message in the body, which breaks generic clients and monitoring. Exposing database identifiers and internal structure directly as the public contract, so every schema change breaks clients.

Table 6.2 Choosing between RPC and REST.

Situation	Prefer	Why
Public or third-party API	REST (JSON over HTTP)	Universal tooling, cacheable, human-readable
Internal calls between your own services	RPC (e.g. gRPC)	Speed, schemas, generated code
Mobile app wanting one call per screen	REST with aggregation, or a query-style API	Fewer round trips

Situation	Prefer	Why
Streaming or very high call volume	RPC with streaming	Efficient framing

6.4 Designing endpoints, errors and status codes

A well-designed endpoint tells a client three things: what to send, what it will get, and what to do when something fails.

WORKED EXAMPLE

A paste API. Suppose we build the text-sharing service from Chapter 1.

```
POST /v1/pastes
Content-Type: application/json
Idempotency-Key: 6f1c0b52-...

{"content": "hello world", "expires_in_seconds": 86400}

HTTP/1.1 201 Created
Location: /v1/pastes/7Hq2Zx
Content-Type: application/json

{"id": "7Hq2Zx", "url": "https://paste.example/7Hq2Zx",
 "created_at": "2026-10-12T09:30:00Z", "expires_at": "2026-10-13T09:30:00Z"}
```

```
GET /v1/pastes/7Hq2Zx          ->  200 OK  (or 404 Not Found, or 410 Gone if expired)
DELETE /v1/pastes/7Hq2Zx       ->  204 No Content  (idempotent: second call may return
404 or 204)
```

Observe: the version is in the path (`/v1`); the create call returns **201** with a `Location` header; an expired paste returns **410 Gone**, distinguishing "never existed" from "existed and was removed"; and the create call accepts an idempotency key (next section).

Error responses should be consistent and machine-readable: a stable error *code* clients can switch on, a human message, and where relevant a field name and a request identifier for support.

```
{"error": {"code": "paste_too_large", "message": "Content exceeds 512 KB.",
           "field": "content", "request_id": "req_91f3"}}
```

Choose status codes carefully. **400** for malformed requests, **401** for missing or invalid credentials (authentication), **403** for valid credentials without permission (authorization), **404** for unknown resources, **409** for conflicts such as a version mismatch, **422** for well-formed but semantically invalid input, **429** for rate limiting (with a `Retry-After` header), **503** when overloaded. The 4xx/5xx split tells clients whether *retrying unchanged* can help: 4xx errors normally cannot be fixed by retrying; 5xx and timeouts may be.

6.5 Idempotency keys and safe retries

Networks fail in a way that leaves the client ignorant: it sent a request, and the connection dropped before the response. Did the server process it? *The client cannot know.* If the operation is "charge the card" or "send the email", blind retry may do it twice; giving up may do it zero times.

The standard solution is the **idempotency key**. The client generates a unique value (a random UUID) for each *logical* operation and sends it with every attempt. The server records, for each key, the result of the first execution. A retry carrying the same key does not execute again; the server returns the stored result.

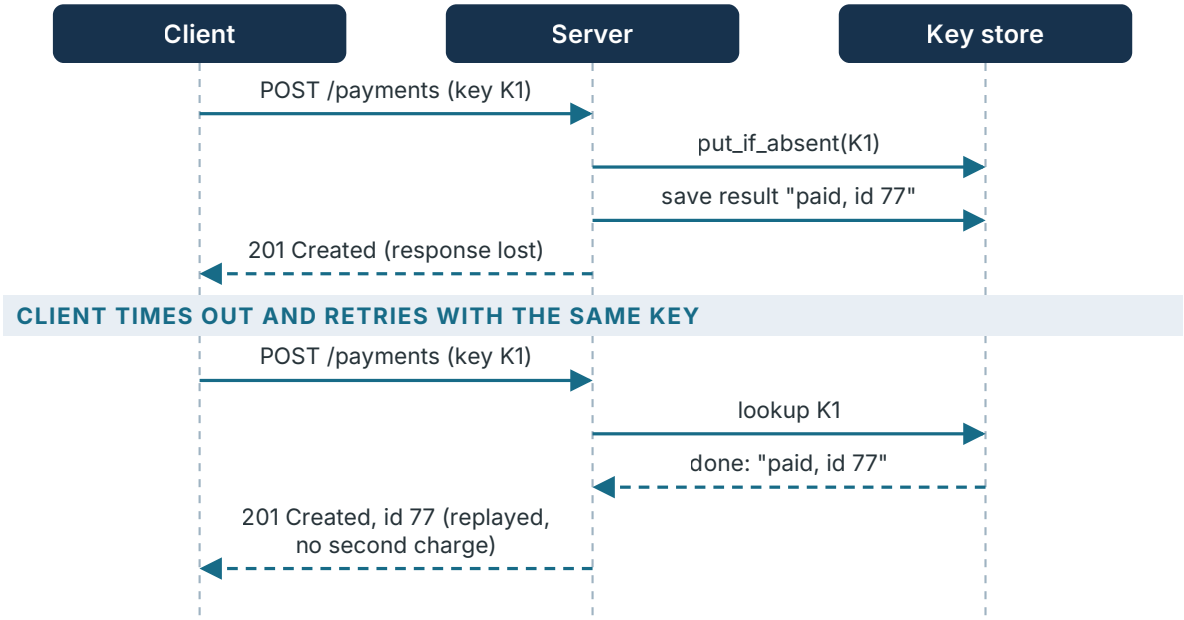


Figure 6.2 A retry with an idempotency key. The first response is lost, the client retries with the same key, and the server returns the stored result instead of charging twice.

Implementation sketch for the server:

```
def handle_create_payment(request):
    key = request.headers["Idempotency-Key"]
    record = store.get(key)                      # shared store, not local memory
    if record and record.fingerprint != hash(request.body):
        return error(422, "key reused with a different request")
    if record and record.status == "done":
        return record.response                   # replay the saved answer
    if record and record.status == "in_progress":
        return error(409, "request in progress")
    store.put_if_absent(key, status="in_progress", fingerprint=hash(request.body))
    response = do_payment(request)               # may itself need a transaction
    store.update(key, status="done", response=response)
    return response
```

Details that matter: store the key with the *request fingerprint* so a reused key with different content is rejected; make `put_if_absent` atomic (otherwise two simultaneous retries both run); keep keys for a bounded time (for example 24 hours); and put the key check

and the business effect in the *same transaction* where possible. An idempotency key makes the *client-visible* result stable; it does not make arbitrary side effects (an email already sent) disappear, so the downstream actions must be idempotent too.

6.6 Pagination

Returning a million rows in one response is slow and fragile. APIs return results in **pages**.

Offset pagination: `GET /pastes?limit=20&offset=40`. Simple and allows jumping to page 3. But the database must walk past the first 40 rows to return the next 20, so deep pages are slow (offset 1,000,000 scans a million rows), and if items are inserted or deleted while the client pages, rows are skipped or repeated.

Cursor (keyset) pagination: `GET /pastes?limit=20&after=7Hq2Zx`. The client sends the *last item it saw* (an opaque token encoding the sort key), and the server queries `WHERE (created_at, id) < (cursor values) ORDER BY created_at DESC, id DESC LIMIT 20`. With an index on the sort key this is fast at any depth and stable under inserts. The costs: no random jump to page 50, and the sort key must be unique and indexed.

Table 6.3 Offset versus cursor pagination.

	Offset	Cursor (keyset)
Jump to page *n*	Yes	No
Deep pages	Slow (cost grows with offset)	Fast (index seek)
Stable if data changes	No (skips and duplicates)	Yes
Typical use	Small admin lists	Feeds, timelines, large datasets

6.7 Versioning and compatibility

Clients cannot all upgrade at once, so evolve APIs **compatibly**. Safe changes: adding a new optional field, a new endpoint, or a new optional parameter. Breaking changes: removing or renaming a field, changing a field's type or meaning, making an optional input required, changing status codes. Follow **Postel's principle** carefully: clients should *ignore unknown fields*, so servers can add them without a new version.

When a break is unavoidable, introduce a new version (`/v2/...` or a header), run both for a deprecation period, measure who still uses the old one, announce a date, then remove. For schema-based RPC such as Protocol Buffers, the rules are explicit: never reuse a field number, never change a field's type, only add optional fields.

6.8 Webhooks and polling

How does a client learn that something finished? **Polling:** repeatedly ask "is it done?" Simple, but wasteful, and latency is bounded by the poll interval. **Webhooks:** the server calls a URL the client registered when the event happens. Efficient and timely, but the

server must handle slow or failing receivers (retries with backoff, signatures so receivers can verify the sender, and idempotent event delivery because retries cause duplicates). For high-volume streams, **message queues or streaming APIs** replace both (Chapter 13).

SUPPLEMENTAL MATERIAL

Authentication versus authorization at the API. *Authentication* establishes **who** is calling (verified by a password, an API key, or a signed token such as a JWT or an OAuth 2.0 access token). *Authorization* decides **what that caller may do**. A 401 means "I do not know who you are"; a 403 means "I know who you are and the answer is no". Never trust data supplied by the client for authorization decisions: check ownership and permissions on the server for every request. Chapter 17 treats this in depth.

FAILURE MODES

Failure modes. Timeouts leaving outcomes unknown (use idempotency keys and, for important work, a status endpoint the client can query); retry storms when many clients retry at once against a struggling server (use exponential backoff with jitter, Chapter 15); breaking changes shipped silently; unbounded page sizes or unfiltered list endpoints that allow one request to read the whole database; inconsistent error formats that make client code brittle.

COMMON MISCONCEPTION

- "*REST means JSON over HTTP.*" REST is a set of constraints (resources, uniform interface, statelessness); many "REST" APIs follow only some of them, which is usually fine.
- "*RPC is old and REST is modern.*" They solve different problems; large companies use gRPC heavily internally and REST publicly.
- "*A retry is always safe if the server is correct.*" Only if the operation is idempotent or protected by an idempotency key.
- "*HTTP 200 with an error field is friendlier.*" It breaks caches, monitoring and generic clients.

INTERVIEW QUESTIONS

Q1. When would you pick gRPC over REST? For internal, high-volume service-to-service calls where both ends are under your control: schemas, speed and generated clients win. For public or browser-facing APIs choose REST for ubiquity and cacheability.

Q2. How do you make a payment endpoint safe to retry? Require an idempotency key; store the key with a request fingerprint and result atomically; replay the stored result on retry; reject reuse with a different body; expire keys after a window.

Q3. Why is cursor pagination better for a feed? It is stable when new items arrive and performs the same for any depth because it uses an index seek, while offset pagination slows with depth and skips or duplicates items under change.

Q4. How do you evolve an API without breaking clients? Make only additive changes within a version, tolerate unknown fields, version breaking changes, support the old version through a deprecation period, and watch usage metrics before removal.

EXERCISES

1. Design REST endpoints for a library: books, members, and loans (a member borrows a book). Give method, path and success status for creating a loan, listing a member's loans, and returning a book.
2. A client calls `POST /orders` and the connection times out. Describe, step by step, how an idempotency key lets the client recover safely.
3. A feed has 10 million rows. Compare the cost of fetching rows 9,999,980 to 10,000,000 with offset versus cursor pagination.
4. Which of these are breaking changes: adding an optional response field; renaming `userId` to `user_id`; making `email` a required request field; adding a new endpoint?

Answers.

1. `POST /members/{id}/loans` body `{bookId}` → 201 with a Location header; `GET /members/{id}/loans?limit=20` → 200; returning: `DELETE /loans/{loanId}` → 204, or `PATCH /loans/{loanId}` setting `returned_at` → 200 (a state change on the same resource).
2. The client generated a key before the first attempt. It retries with the *same* key. If the first attempt succeeded, the server finds the stored result and returns it; if it never arrived, the server executes now. Either way exactly one order exists.
3. Offset must walk past ~10 million rows (cost proportional to the offset); cursor seeks directly in the index and reads 20 rows (cost about the same as page one).
4. Breaking: the rename and the new required field. Not breaking: the optional response field and the new endpoint.

CHAPTER SUMMARY

- An API is a contract; design it to change compatibly.
- RPC is action-oriented, fast and tightly coupled: ideal internally. REST is resource-oriented, cacheable and universal: ideal publicly.
- Use correct methods and status codes; make errors machine-readable.
- Idempotency keys make retries safe when outcomes are uncertain.
- Cursor pagination scales and is stable; offset is simple but degrades.
- Webhooks push events; polling pulls; both need retry and idempotency handling.

REVISION CHECKLIST

- [ ] I can convert an RPC call list into REST resources.
- [ ] I can write the server logic for an idempotency key.
- [ ] I can choose and defend a pagination style.
- [ ] I can list safe and breaking API changes.

SUPPLEMENTAL MATERIAL

Further reading. Roy Fielding's dissertation, chapter 5 (the REST constraints); Google's API Improvement Proposals (aip.dev) for resource-oriented design; Stripe's public API documentation on idempotent requests; the Protocol Buffers language guide on updating message types.

CHAPTER 7

The Entry Layer: CDN, Load Balancers and Proxies

Derived from your notes: Notebook chapter 4 (CDN, push and pull CDNs, load balancers, layer 4 and layer 7, reverse proxy, the comparison tables). **Added in this book:** rate limiting, API gateways, health checking and graceful draining, global traffic management, and capacity worked examples.

LEARNING OBJECTIVES

After this chapter you can:

- explain what a content delivery network does, when to use push or pull, and how to control and invalidate its caches;
- describe load balancer algorithms, layer 4 versus layer 7 balancing, health checks and the SPOF problem of the balancer itself;
- explain what a reverse proxy does and how it differs from a forward proxy and a load balancer;
- explain an API gateway and implement a token-bucket rate limiter;
- size an entry layer for a given traffic estimate.

PREREQUISITE KNOWLEDGE

Chapter 5 (DNS, TCP, TLS, HTTP) and Chapter 4 (traffic estimation).

KEY TERMINOLOGY

Edge / point of presence (PoP)
A CDN data centre located close to users.

Origin
The real servers that hold the authoritative content.

Cache hit / miss
The requested item was found (hit) or not found (miss) in a cache.

Health check
A periodic probe that decides whether a server may receive traffic.

TLS termination
Ending the encrypted connection at an intermediate server, which then forwards the request.

Rate limiting
Rejecting or delaying requests beyond an allowed rate per client.

7.1 Why an entry layer

Between users and application servers sits a layer of components that absorb, filter, spread and speed up traffic *before* it reaches code you wrote. Each does a job you would otherwise have to build into every application: answer static content from nearby (Section 7.2), spread load and hide failures (Section 7.3), terminate encryption and compress (Section 7.4), and protect against abuse (Section 7.5). Putting these jobs at the edge keeps application servers simple and stateless.

7.2 Content delivery networks

A **content delivery network (CDN)** is a globally distributed network of caching servers. A user's request for an image, script or video is answered by an edge server close to them instead of travelling to the origin across an ocean. This cuts latency (distance dominates: Chapter 4), reduces load and bandwidth on the origin, and helps absorb traffic spikes and some attacks.

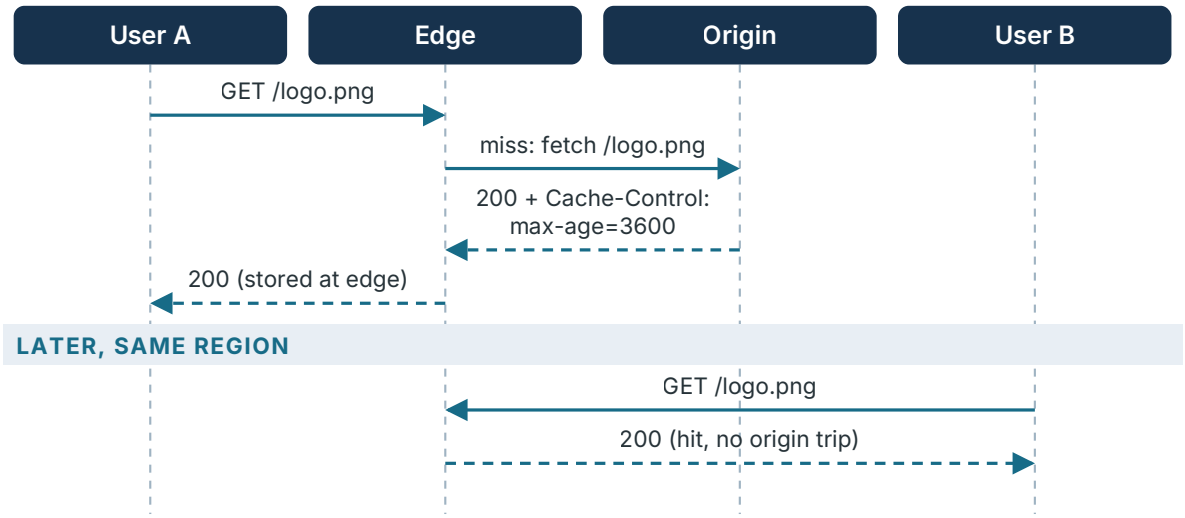


Figure 7.1 A pull CDN. On the first request the edge misses and fetches from the origin, then stores the object. Later users near that edge get it directly until the object expires.

Push and pull

Pull CDN. The CDN fetches content from your origin the first time someone asks, then caches it for a time to live. You change nothing in your upload process: you point URLs at the CDN. The first user in each region sees a slower request (a miss); popular content is soon cached everywhere. Cost: origin traffic on first requests and on expiry; risk of stale content until TTL ends.

Push CDN. *You* upload content to the CDN when it changes, and the CDN stores it until you replace or remove it. Nothing expires on its own, so there is no miss penalty and no unnecessary origin traffic, but you must manage uploads and storage, and a forgotten update leaves old content live.

Table 7.1 Push versus pull CDN.

	Pull	Push
Who loads content	CDN, on first request	You, on change
Origin traffic	Misses and expiry	Only your uploads
Effort	Minimal	You manage uploads and removals
First request	Slower (miss)	Fast (already there)
Best for	Sites with much content that changes often or unpredictably	Small, rarely changing content: installers, firmware, big media

Controlling caches

The origin tells caches what to do with HTTP headers: `Cache-Control: max-age=86400` (cache for a day), `s-maxage` (a CDN-specific TTL), `no-store` (never cache; for private data), `private` (browser may cache, shared caches may not), and validators `ETag` and `Last-Modified` so a cache can ask "is my copy still current?" and receive `304 Not Modified` instead of the full body.

Invalidation is the hard part. Options: wait for the TTL; send a **purge** request (slow or costly at scale, and not instant everywhere); or best, use **versioned, content-hashed URLs** (`app.3f9c2a.js`). A changed file gets a new URL, so old copies are simply never requested again, and the files can be cached for a year safely. Only the HTML that references them needs a short TTL.

> COMMON MISTAKE
>
> **Common mistakes.** Caching personalised or private responses at the CDN (leaking one user's page to another; always vary or exclude by `Authorization` or cookies). Short TTLs on static files that never change. No cache key discipline: query strings that differ per user make every request a miss. Forgetting that a CDN is a cache: stale content is the price of speed.

7.3 Load balancers

A **load balancer** receives requests and distributes them across a pool of servers. It provides **capacity** (many servers share traffic), **availability** (it stops sending traffic to failed servers), and **flexibility** (servers can be added, removed or upgraded without clients noticing).

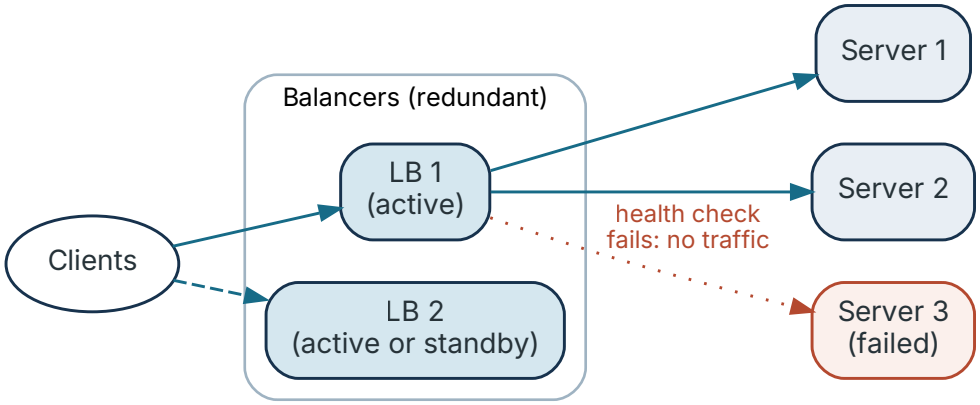


Figure 7.2 A load balancer spreads requests over a pool. Health checks (dashed) remove failed servers from rotation; the balancer is itself duplicated to avoid being a single point of failure.

Algorithms

Table 7.2 Load-balancing algorithms.

Algorithm	How it chooses	Good when	Watch out
Round robin	Next server in turn	Similar servers, similar requests	Ignores current load
Weighted round robin	Round robin with weights	Servers of different sizes	Weights must be maintained
Least connections	Server with fewest open connections	Long or uneven requests	Needs connection tracking
Least response time	Fastest recent responses	Heterogeneous backends	Measurement noise
IP / key hash	Hash of client IP or key	Same client to the same server (cache locality)	Skewed keys; reshuffles when pool changes (consistent hashing helps)
Random of two	Pick two at random, take the less loaded	Large pools, little coordination	None serious; surprisingly effective

Layer 4 and layer 7

A **layer 4 balancer** works on transport information (IP address and port), forwarding TCP or UDP streams without reading them. It is very fast and protocol-agnostic. A **layer 7 balancer** understands HTTP: it can route by URL path or header (`/api` to one pool, `/images` to another), terminate TLS, compress, add or remove headers, and retry on another server. It is more flexible and more expensive per request.

Table 7.3 Layer 4 versus layer 7 balancing.

	Layer 4	Layer 7
Looks at	IP, port	Full HTTP request
Routing by path or header	No	Yes

	Layer 4	Layer 7
TLS termination	Passes through (or terminates in some products)	Typically terminates
Throughput per machine	Higher	Lower
Typical use	Front door for huge traffic, non-HTTP protocols	Web applications, microservice routing

Health checks, stickiness, draining

Health checks probe each server (a TCP connect, or an HTTP `GET /healthz`). After several consecutive failures the balancer stops using the server; after successes it returns. A good health check tests the server's *ability to serve* (for example, can it reach its database?) but not so deeply that one shared dependency failing marks *every* server unhealthy and takes the whole site down. **Session persistence ("sticky sessions")** pins a client to one server, a workaround for servers that keep state in memory (Chapter 1); it makes load uneven and loses sessions when the server dies, so prefer stateless servers. **Connection draining** lets in-flight requests finish before a server is removed during a deployment.

The balancer as a SPOF

A single balancer is a single point of failure. Typical remedies: **active-passive** pairs (a standby takes over the same address if the active fails), **active-active** pairs or clusters (both serve traffic), or **DNS** and **anycast** routing to several balancers in different locations. Cloud balancers are already distributed and managed for you; the trade-off is cost and less control.

7.4 Reverse proxies and gateways

A **forward proxy** sits in front of *clients* (a corporate proxy that filters outbound traffic). A **reverse proxy** sits in front of *servers*: clients talk to it as if it were the server, and it forwards requests to the real servers. Examples are NGINX, HAProxy and Envoy.

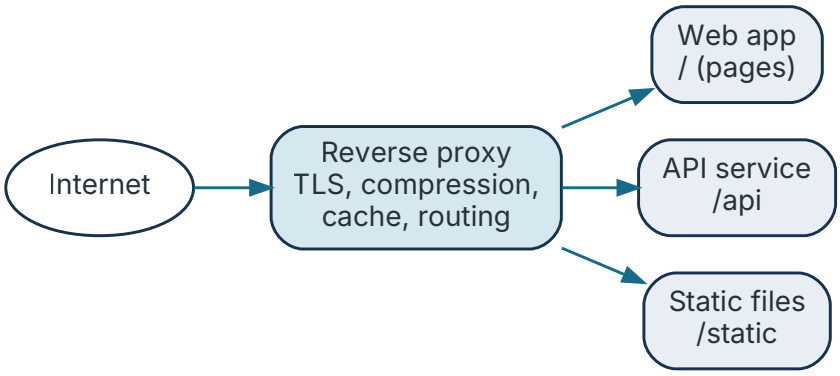


Figure 7.3 A reverse proxy is the public face of the servers. It terminates TLS, compresses, caches, and routes by path to different back-end services, which are never exposed to the internet.

Benefits: **security** (back ends have no public addresses; it can block bad requests and hide topology), **TLS termination** (certificates and encryption work in one place), **compression and caching** of responses, **static file serving**, **request routing** and **buffering slow clients** so application servers are not tied up waiting for them. Costs: another hop (a little latency), another component to operate, and it can become a bottleneck or SPOF if not replicated.

> **CLARIFICATION**
>
> **Load balancer versus reverse proxy.** The terms overlap. A load balancer's defining job is *spreading load across several servers*; a reverse proxy's defining job is *standing in for the servers* (security, TLS, caching, routing), and it is useful even with a single back end. Many products do both: NGINX is commonly used as a reverse proxy *and* a layer 7 balancer. In diagrams, name the *role* each box plays.

An **API gateway** is a reverse proxy specialised for APIs. It adds authentication, **rate limiting**, request and response transformation, API-key management, versioned routing, and usage metrics, at one entry point for many back-end services. It centralises cross-cutting policy but must be kept thin; business logic placed in the gateway becomes a bottleneck and a hidden monolith.

7.5 Rate limiting

Rate limiting protects a service from overload and abuse by capping how many requests a client may make in a time window; excess requests receive `429 Too Many Requests` (with `Retry-After`). It also enforces fairness and billing tiers.

The **token bucket** algorithm is the usual choice. A bucket holds up to B tokens and refills at rate r tokens per second. Each request takes one token; if none is available the request is rejected. The result: sustained rate r, with bursts up to B allowed.

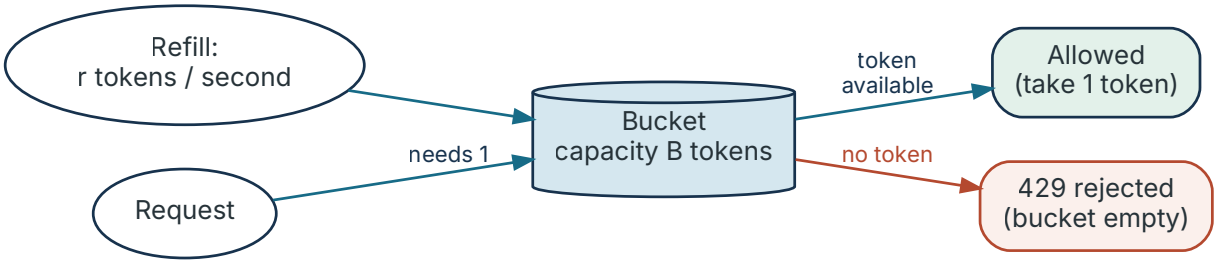


Figure 7.4 Token bucket: tokens drip in at a constant rate up to a maximum; each request consumes one. A full bucket permits a burst; an empty bucket rejects.

```
class TokenBucket:
    def __init__(self, rate, capacity):
        self.rate, self.capacity = rate, capacity      # tokens/sec, max tokens
        self.tokens, self.last = capacity, now()
    def allow(self):
        t = now()
        self.tokens = min(self.capacity, self.tokens + (t - self.last) * self.rate)
        self.last = t
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False
```

Alternatives: **fixed window** counters (simple, but allow double bursts at window edges), **sliding window log** (exact, memory-heavy) and **sliding window counter** (an approximation using two adjacent windows). In a distributed deployment the counters live in a shared fast store (for example a distributed cache with atomic increment and expiry); the trade-off is an extra network call per request versus the accuracy of limits when each server counts independently.

7.6 Putting it together: sizing the entry layer

WORKED EXAMPLE

Sizing for the messaging example of Chapter 4. Peak: 100,000 reads per second and 10,000 writes per second, response 1 KB, of which perhaps 70% could be served by a CDN or edge cache if static.

- Traffic reaching the origin: 30% × 110,000 ≈ 33,000 requests per second; outbound ≈ 33 MB/s ≈ 264 Mbps.
- If one application server handles about 2,000 requests per second comfortably (assumed, to be measured), we need 33,000 / 2,000 ≈ 17 servers; with headroom for peak and one failed zone (run at ≤ 60% load, tolerate losing one of three zones), plan about 17 / 0.6 × 1.5 ≈ 43, rounded to about 45 servers across three zones.
- Two layer-4 balancers (active-active) comfortably handle this traffic; a layer-7 tier or gateway adds routing and rate limiting.
- The CDN absorbs 77,000 requests per second that never touch the origin: that is where most of the saving comes from.

FAILURE MODES

Failure modes. CDN serving stale or wrong content after a bad deployment (versioned URLs and purge procedures help); thundering herd on the origin when a popular object expires everywhere at once (*request collapsing* and stale-while-revalidate headers help); a health check that is too sensitive flapping servers in and out; a balancer misconfigured to send everything to one server; the gateway becoming the bottleneck; rate limiters failing closed and rejecting all traffic when their counter store is down (decide deliberately whether to fail open or closed); sticky sessions pinning load to one overloaded server.

COMMON MISCONCEPTION

- "*A CDN is only for images.*" It can cache API responses, accelerate dynamic requests through optimised routes, terminate TLS near users, and absorb attacks.
- "*Round robin balances load.*" It balances request *counts*; if requests differ widely in cost, load is still uneven.
- "*A load balancer makes the system highly available.*" Only if the balancer is itself redundant and servers do not share a hidden SPOF.
- "*Rate limiting is only for security.*" It also protects capacity and enforces fairness.

INTERVIEW QUESTIONS

Q1. Push versus pull CDN? Pull fetches on demand and caches with a TTL (easy, first request is slow, may be stale); push has you upload content (no misses or surprise origin traffic, more management). Use pull for most sites, push for small, rarely changing, large files.

Q2. Layer 4 versus layer 7 load balancing? Layer 4 forwards by IP and port: fast, protocol-agnostic. Layer 7 reads HTTP: routes by path or header, terminates TLS, retries; slower but flexible.

Q3. How do you avoid the load balancer being a single point of failure? Run several, active-active or active-passive, behind DNS or anycast, or use a managed cloud balancer; test failover.

Q4. Design a rate limiter. Token bucket per client key stored in a shared fast store with atomic operations; return 429 and `Retry-After`; decide fail-open or fail-closed; apply at the gateway; consider per-user, per-IP and per-endpoint limits.

EXERCISES

1. A pull CDN has a 90% hit ratio. Origin capacity is 2,000 requests per second. What total request rate can the site serve?
2. A bucket has capacity 20 and refill rate 5 tokens/s. A client sends 30 requests at once at time 0, then one per 100 ms. How many of the first 30 are accepted? What sustained rate is allowed?
3. Choose a load-balancing algorithm for (a) identical stateless web servers, (b) a pool where requests vary from 10 ms to 30 s, (c) a cache tier where the same key should hit the same node.
4. Why can a health check that calls the database cause an outage?

Answers.

1. Only 10% of requests miss: total ≈ 2,000 / 0.10 = 20,000 requests per second (if the CDN itself is unconstrained).
2. 20 accepted (the full bucket), 10 rejected. Sustained rate 5 per second (1 per 200 ms), so the client's one-per-100 ms stream (10/s) would see about half rejected.
3. (a) Round robin or random-of-two; (b) least connections; (c) hash by key, preferably consistent hashing.

4. If the database slows or fails, every server's health check fails at once, so the balancer removes all servers and the service goes fully down even though the web servers were healthy; shallow checks or "fail static" behaviour avoid this.

CHAPTER SUMMARY

- The entry layer handles caching, spreading, filtering and protecting traffic before application code.
- CDNs cut latency and origin load; pull is easy, push gives control; use versioned URLs to avoid invalidation pain.
- Load balancers spread load and hide failures; choose the algorithm by request shape; layer 7 is flexible, layer 4 is fast; replicate the balancer.
- Reverse proxies and gateways centralise TLS, caching, routing, authentication and rate limiting.
- Token buckets allow bursts at a bounded sustained rate; decide fail-open or fail-closed.

REVISION CHECKLIST

- [ ] I can explain a cache hit and miss at the CDN and how to purge or version.
- [ ] I can name four balancing algorithms and when each fits.
- [ ] I can explain L4 versus L7 and the balancer SPOF.
- [ ] I can write a token bucket.

SUPPLEMENTAL MATERIAL

Further reading. The HTTP caching specification (RFC 9111); Google's "The Site Reliability Workbook" chapters on load balancing; Mitzenmacher's analysis of "the power of two choices"; documentation for NGINX, HAProxy and Envoy for concrete configuration.

PART III

Data

How data is stored, indexed, scaled, partitioned and kept consistent.

CHAPTER 8

Relational Databases and SQL

Derived from your notes: Notebook chapter 6 (RDBMS and ACID, SQL tuning) and the "SQL or NoSQL" comparison. **Added in this book:** joins and normalisation, composite and covering indexes, query plans, isolation levels and anomalies, MVCC, and safe schema migrations.

LEARNING OBJECTIVES

After this chapter you can:

- model data as tables with keys and relationships, and explain normalisation;
- write basic SQL (select, join, group, update) and read what it asks the database to do;
- explain how indexes work, design single and composite indexes, and read a query plan;
- explain the four ACID properties, the isolation levels and the anomalies each prevents;
- change a live schema without downtime using the expand-and-contract pattern.

PREREQUISITE KNOWLEDGE

Chapter 3 (hash tables, B-trees) and Chapter 2 (race conditions, locking).

KEY TERMINOLOGY

Relational database (RDBMS)
A database that stores data in tables of rows and columns and is queried with SQL (for example PostgreSQL, MySQL, Oracle, SQL Server).

Primary key
A column (or set) that uniquely identifies each row.

Foreign key
A column that refers to the primary key of another table, expressing a relationship.

Transaction
A group of operations that the database treats as one unit: all succeed or none do.

Index
A separate structure, usually a B-tree, that speeds up lookups on chosen columns.

Query plan
The steps the database chooses to execute a query (which index, which join method).

8.1 The relational model

A **relational database** stores data in **tables**. Each table has typed **columns** and holds many **rows**. Rows in different tables are connected by **keys**, not by physical pointers. The language for asking questions is **SQL** (Structured Query Language), which is *declarative*: you say *what* you want, and the database decides *how* to get it.

```sql
CREATE TABLE users (
  id          BIGINT PRIMARY KEY,
  email       VARCHAR(255) NOT NULL UNIQUE,
  created_at  TIMESTAMP    NOT NULL
);

CREATE TABLE pastes (
  id          BIGINT PRIMARY KEY,
  user_id     BIGINT REFERENCES users(id),     -- foreign key
  short_code  CHAR(7)  NOT NULL UNIQUE,
  content_key VARCHAR(255) NOT NULL,           -- where the text is stored
  created_at  TIMESTAMP NOT NULL,
  expires_at  TIMESTAMP
);
```

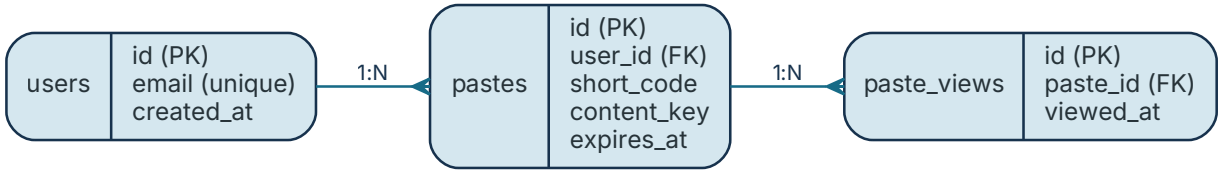


Figure 8.1 An entity-relationship sketch. One user owns many pastes (the foreign key lives on the "many" side); each paste has many views.

Why do relational databases remain the default choice? Because they offer **a flexible query language** (new questions without new code), **integrity constraints** (a paste cannot point to a missing user), **transactions**, and decades of tooling. Their traditional limit is *scaling writes beyond one machine*, discussed in Chapter 9.

Joins, grouping and normalisation

A **join** combines rows from two tables by matching keys.

```sql
-- How many pastes does each user have?
SELECT u.email, COUNT(*) AS paste_count
FROM users u
JOIN pastes p ON p.user_id = u.id
WHERE p.created_at >= '2026-01-01'
GROUP BY u.email
ORDER BY paste_count DESC
LIMIT 10;
```

Normalisation means organising tables so each fact is stored once. If every paste row stored the owner's email, changing an email would need many updates, and a missed row would create contradictory data. Putting the email in `users` and referencing it by key re-

moves the duplication (the "update anomaly"). Normalisation gives integrity and smaller writes, at the cost of **joins** on reads. Later (Chapter 9) we meet **denormalisation**: deliberately duplicating data to make reads fast when joins become the bottleneck.

8.2 Indexes

Without an index, finding `WHERE email = 'a@x.com'` means a **full table scan**: reading every row. An **index** is a separate structure, usually a B+ tree (Chapter 3), that maps column values to row locations. With it, the database finds the row in a handful of page reads.

```
CREATE INDEX idx_pastes_user_created ON pastes (user_id, created_at);
```

Indexes are not free: every insert, update and delete must also update each index (slower writes), and indexes consume disk and memory. The craft is to index what queries actually need.

Composite indexes cover several columns and obey the **leftmost-prefix rule**. An index on `(user_id, created_at)` serves queries filtering by `user_id`, or by `user_id` and `created_at` (and sorted output by `created_at` for one user), but *not* a query filtering by `created_at` alone, just as a phone book sorted by (surname, first name) cannot find everyone named "Asha" quickly. Put equality-tested columns first, then range or sort columns.

A **covering index** contains every column a query needs, so the database answers from the index alone without visiting the table. **Selectivity** matters: an index on a column with two values (a boolean) rarely helps, because it narrows nothing; an index on a nearly unique column narrows to one row.

> COMMON MISTAKE
>
> **Common mistakes.** Indexing every column "just in case" (writes slow, memory fills). Wrapping an indexed column in a function (`WHERE LOWER(email) = ...` or `WHERE created_at + 1 > ...`), which prevents index use unless an expression index exists. Using a leading wildcard (`LIKE '%abc'`), which cannot use a normal B-tree. Missing the index on a foreign-key column, making joins and cascading deletes scan whole tables.

Reading a query plan

Databases expose their chosen strategy with `EXPLAIN`. Learn to read three things: **how rows are accessed** (sequential scan or index scan), **how tables are joined** (nested loop, hash join, merge join), and **estimated versus actual row counts** (wild differences mean stale statistics).

WORKED EXAMPLE

Before and after an index. Table `pastes` holds 50 million rows.

```
EXPLAIN SELECT * FROM pastes WHERE user_id = 42 ORDER BY created_at DESC LIMIT 20;

-- no suitable index:
Limit
  -> Sort  (cost=...  rows=48000)
       -> Seq Scan on pastes  Filter: (user_id = 42)   -- reads all 50,000,000 rows

-- after CREATE INDEX idx_pastes_user_created ON pastes (user_id, created_at):
Limit
  -> Index Scan Backward using idx_pastes_user_created
       Index Cond: (user_id = 42)                      -- reads about 20 index entries
```

The first plan reads every row and sorts the matches; the second walks the index backwards from the newest entry for user 42 and stops after 20, because the index is already in the order requested. Time falls from seconds to about a millisecond. The lesson is to design the index to match *both* the filter and the order.

8.3 Transactions and ACID

A **transaction** groups operations into one unit. The classic example: transferring 100 from account A to account B is two updates; the system must never apply only one.

```sql
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 'A' AND balance >= 100;
UPDATE accounts SET balance = balance + 100 WHERE id = 'B';
COMMIT;   -- or ROLLBACK to undo everything since BEGIN
```

ACID names four properties:

- **Atomicity**: all operations take effect or none do. A crash mid-transfer leaves no half-applied change.
- **Consistency**: a transaction moves the database from one valid state to another, respecting constraints (balances never negative, foreign keys valid). Note: here "consistency" means *integrity rules*, a different notion from the "consistency" of replicas in the CAP theorem (Chapter 11).
- **Isolation**: concurrent transactions do not see each other's partial work; the result is as if they ran one after another (to a degree set by the isolation level).
- **Durability**: once committed, the data survives crashes. Databases achieve this by writing to a **write-ahead log (WAL)** on disk *before* confirming the commit; after a crash the log is replayed.

Isolation levels and anomalies

Perfect isolation (**serializable**) is expensive, so databases offer weaker levels that allow certain *anomalies*.

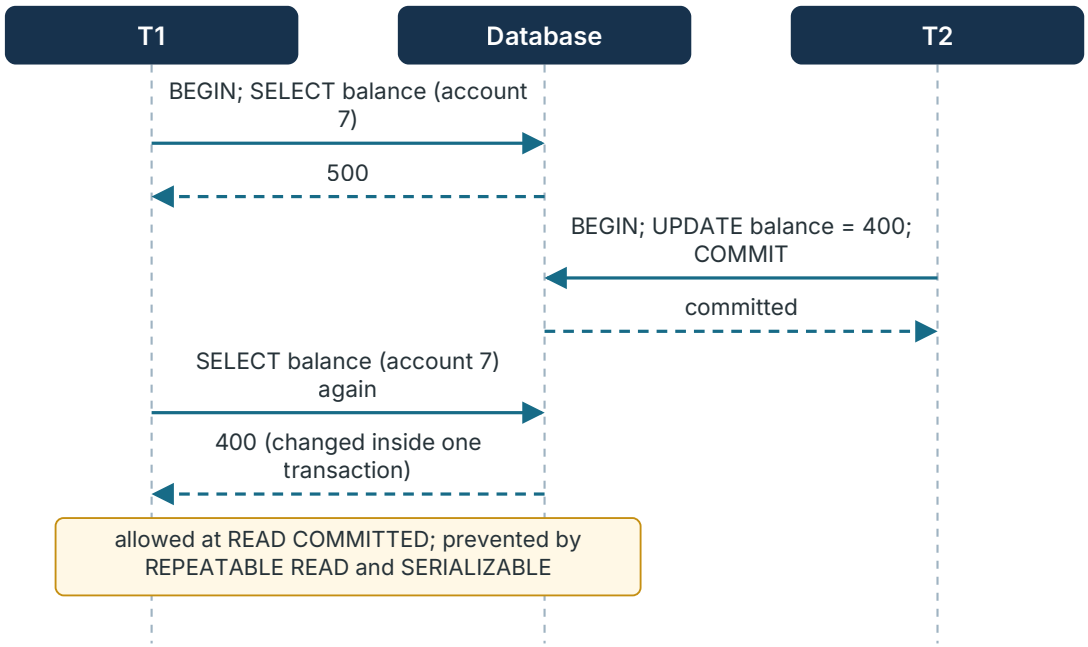


Figure 8.2 A non-repeatable read. Transaction T1 reads a balance twice; transaction T2 commits a change in between, so T1 sees two different values within one transaction.

Table 8.1 Isolation levels (SQL standard names) and which anomalies each permits. "Possible" means the anomaly can occur.

Level	Dirty read	Non-repeatable read	Phantom read	Notes
Read uncommitted	Possible	Possible	Possible	Rarely used
Read committed	Prevented	Possible	Possible	Default in PostgreSQL and many others
Repeatable read	Prevented	Prevented	Possible (prevented in some engines, e.g. MySQL InnoDB, PostgreSQL)	Default in MySQL InnoDB
Serializable	Prevented	Prevented	Prevented	Strongest; may abort transactions that conflict

Definitions: a **dirty read** sees another transaction's *uncommitted* change; a **non-repeatable read** sees a row change between two reads in the same transaction; a **phantom** sees new rows appear in a repeated range query. There is a further anomaly the standard's table does not show: **write skew**, where two transactions each read overlapping data, make decisions based on it, and write *different* rows, jointly breaking an invariant (two doctors both go off call because each saw the other still on call). Snapshot-based "repeatable read" allows write skew; only serializable isolation or explicit locking prevents it.

Most databases implement isolation with **MVCC** (multi-version concurrency control): each write creates a new *version* of a row, and a transaction reads a consistent **snapshot** as of its start, so readers do not block writers and writers do not block readers. Old versions are cleaned up later (vacuum or purge), which is a maintenance task whose neglect causes bloat.

For correctness at the application level: use the **strongest isolation you can afford** for money and inventory; use `SELECT ... FOR UPDATE` to lock rows you intend to modify; use **conditional updates** (`... WHERE balance >= 100`) and check the affected-row count; and keep transactions **short** because long ones hold locks and block cleanup. As in Chapter 2, databases can deadlock; they detect cycles and abort one transaction, so your code must be ready to **retry** on a deadlock or serialization failure.

CLARIFICATION

Clarification: a database transaction is not a business workflow. A transaction is atomic only *inside one database*. If a workflow spans a payment provider, an email service and your database, no single `COMMIT` covers it. Chapter 13 covers how to coordinate such workflows (outbox, sagas) and why "exactly once" claims need care.

8.4 Changing a live schema safely

Schema changes are among the riskiest operations: a change that locks a large table can stop the application, and code and schema are never deployed at the same instant. The safe approach is **expand and contract** (also called parallel change), done in small reversible steps.

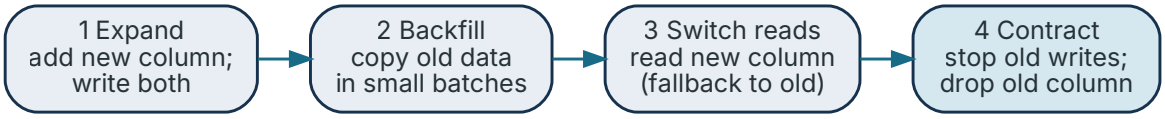


Figure 8.3 Expand and contract. The schema is widened first, data is copied, readers and writers are switched step by step, and only then is the old column removed. At every stage both old and new application versions work.

WORKED EXAMPLE

Renaming a column `content_key` to `blob_key` without downtime. 1. *Expand:* add the new nullable column `blob_key` (a quick change). Deploy code that writes **both** columns. 2. *Backfill:* copy old values in small batches (for example 10,000 rows at a time with pauses) so you never hold long locks or saturate the database. 3. *Switch reads:* deploy code that reads `blob_key`, falling back to `content_key` if null; verify with metrics that no fallback occurs. 4. *Contract:* stop writing `content_key`, wait one release, then drop the column. Each step can be rolled back by deploying the previous code. A "one-step rename" would have broken every server still running old code.

General rules: add columns as nullable (or with cheap defaults) first; create indexes **concurrently** where the database supports it; never combine a risky schema change with a risky code change; test migrations on a copy of production-sized data; and keep every mi-

gration backward-compatible with the *previous* application version. **Backups** are your final safety net, but a backup you have never *restored* is only a hope, so restoration is drilled and timed (Chapter 18).

8.5 Alternatives and trade-offs

Table 8.2 Relational databases: when they fit.

Strength	Weakness
Flexible queries and joins	Scaling writes across machines is hard
ACID transactions and constraints	Schema changes need care
Mature tooling, strong ecosystem	Joins across large tables can be slow
Strong consistency on one node	Replicas lag; sharding breaks cross-shard transactions

A relational database is the **right default**: begin there unless a requirement (enormous write volume, flexible documents, graph traversal, extreme latency) clearly rules it out, as discussed in Chapter 10.

FAILURE MODES

Failure modes. A missing index turning a fast query into a table scan as data grows (the system worked in testing and collapses in production); a long transaction blocking vacuum and bloating tables; lock contention on a hot row (a global counter updated by every request); connection exhaustion (each connection costs memory, so applications use a **connection pool** and the database a limit); replication lag after a failover causing lost recent commits; a migration locking a table; disk full stopping all writes; an accidental `DELETE` without a `WHERE` (mitigated by point-in-time recovery, which replays the log to a chosen moment).

COMMON MISCONCEPTION

- "*ACID consistency is the same as CAP consistency.*" No: ACID consistency is about integrity constraints within a database; CAP consistency is about all replicas returning the latest write.
- "*More indexes are always faster.*" They speed reads and slow writes.
- "*Transactions make my whole workflow atomic.*" Only operations inside one database transaction.
- "*SQL databases cannot scale.*" They scale far (read replicas, careful design, sharding); the difficulty is scaling *writes* and cross-shard transactions.

INTERVIEW QUESTIONS

Q1. Explain ACID. Atomicity (all or nothing), Consistency (valid state to valid state under constraints), Isolation (concurrent transactions do not interfere, to a configurable degree), Durability (committed data survives crashes via write-ahead logging).

Q2. A query on a 100-million-row table became slow. How do you investigate? Run `EXPLAIN`, look for sequential scans and bad row estimates; check indexes against the filter and sort columns; check statistics are current; look at lock waits and I/O; consider a composite or covering index; verify the fix on realistic data.

Q3. What is a phantom read, and what prevents it? New rows matching a range query appear between two executions in the same transaction; serializable isolation (or range locks / predicate locking) prevents it.

Q4. How do you rename a column with zero downtime? Expand and contract: add new column, dual-write, backfill in batches, switch reads, stop old writes, then drop the old column.

EXERCISES

1. Table `orders(id, customer_id, status, created_at)` has 20 million rows. Design indexes for (a) "latest 10 orders of a customer" and (b) "all orders with status = 'pending' created in the last hour".
2. Transaction T1 runs `SELECT SUM(balance)` twice; between the reads T2 deposits 50 into one account and commits. At which isolation levels can T1 see two different totals?
3. Two doctors each check that at least two are on call and then each marks themselves off call. Explain write skew and give two ways to prevent it.
4. Write the SQL to sell the last unit of an item so two buyers cannot both succeed.

Answers.

1. (a) `(customer_id, created_at DESC)`; (b) `(status, created_at)` (status first for equality, then the time range). A partial index only on pending rows is even smaller if the database supports it.
2. At read uncommitted and read committed: the second read sees T2's committed deposit (a non-repeatable read, and for a sum over rows a phantom-style effect). At repeatable read and serializable, T1 keeps reading from its snapshot and sees the same total both times.
3. Each transaction reads the other doctor's state from its snapshot, sees "two on call", and writes only its own row, so the invariant "at least one on call" breaks. Prevent by serializable isolation, by locking the rows read (`SELECT ... FOR UPDATE`), or by a constraint enforced in a single row or table.
4. `UPDATE items SET stock = stock - 1 WHERE id = ? AND stock > 0;` then check the affected-row count is 1; only that buyer succeeds.

CHAPTER SUMMARY

- Relational databases store tables of rows linked by keys, queried with declarative SQL; normalisation keeps facts in one place at the cost of joins.
- Indexes (B+ trees) trade write cost and space for read speed; composite indexes follow the leftmost-prefix rule; read plans with EXPLAIN.
- ACID: atomicity, consistency (integrity), isolation (levels and anomalies, MVCC), durability (write-ahead log).
- Use strong isolation or explicit locks for money and stock; retry on deadlock or serialization failure.
- Change schemas with expand and contract; always keep backward compatibility; drill restores.

REVISION CHECKLIST

- [ ] I can write a join and group query.
- [ ] I can design a composite index and explain the leftmost prefix.
- [ ] I can list the isolation levels and anomalies.
- [ ] I can explain write-ahead logging and MVCC in two sentences each.
- [ ] I can run an expand-and-contract migration step by step.

SUPPLEMENTAL MATERIAL

Further reading. The PostgreSQL documentation chapters on indexes, EXPLAIN and transaction isolation; Markus Winand, *Use the Index, Luke* (free online); Kleppmann, *Designing Data-Intensive Applications*, chapter 7 (transactions).

CHAPTER 9

Scaling Relational Databases

Derived from your notes: Notebook chapter 6 (replication, master-slave and master-master, federation, sharding, denormalization, SQL tuning) and chapter 12.5 / 12.8 uses of these techniques. **Added in this book:** replication lag and read-your-writes, failover and split brain, shard-key selection and resharding, hot partitions, connection pooling, backups versus replicas, point-in-time recovery.

LEARNING OBJECTIVES

After this chapter you can:

- explain, in order, the tools for scaling a relational database: tuning, caching, replicas, federation, denormalisation and sharding;
- describe leader-follower and multi-leader replication, replication lag, and the anomalies lag causes;
- explain failover and how split-brain happens and is prevented;
- choose a shard key, compare range, hash and directory sharding, and explain resharding and hot shards;
- distinguish backups, replicas and failover, and explain point-in-time recovery.

PREREQUISITE KNOWLEDGE

Chapter 8 (indexes, transactions) and Chapter 4 (capacity estimation).

KEY TERMINOLOGY

Leader (primary, master)
The node that accepts writes; its changes are copied to followers.

Follower (replica, standby, slave)
A node that copies the leader's changes and can serve reads.

Replication lag
The delay between a write on the leader and its appearance on a follower.

Shard
One horizontal slice of a table's rows stored on its own database server.

Shard key
The column whose value decides which shard holds a row.

Failover
Promoting a follower to leader when the leader fails.

Split brain
Two nodes both believing they are the leader.

9.1 The scaling ladder

A single database server handles a surprising amount, often thousands of queries per second and terabytes of data. Scale it in this order, cheapest and least risky first:

1. **Tune**: add missing indexes, fix slow queries, right-size the schema (Chapter 8). Often a 10× to 100× improvement and costs nothing.
2. **Scale up**: more memory (so the working set fits), faster disks, more cores.
3. **Cache**: keep hot results in memory so reads never reach the database (Chapter 12).
4. **Replicate**: add read-only copies to multiply read capacity and improve availability.
5. **Split by function (federation)**: separate databases for separate features.
6. **Denormalise**: duplicate data to avoid expensive joins.
7. **Shard**: split one table across many servers to scale writes and data size.

Each step adds complexity. Measure first; many systems never need step 7.

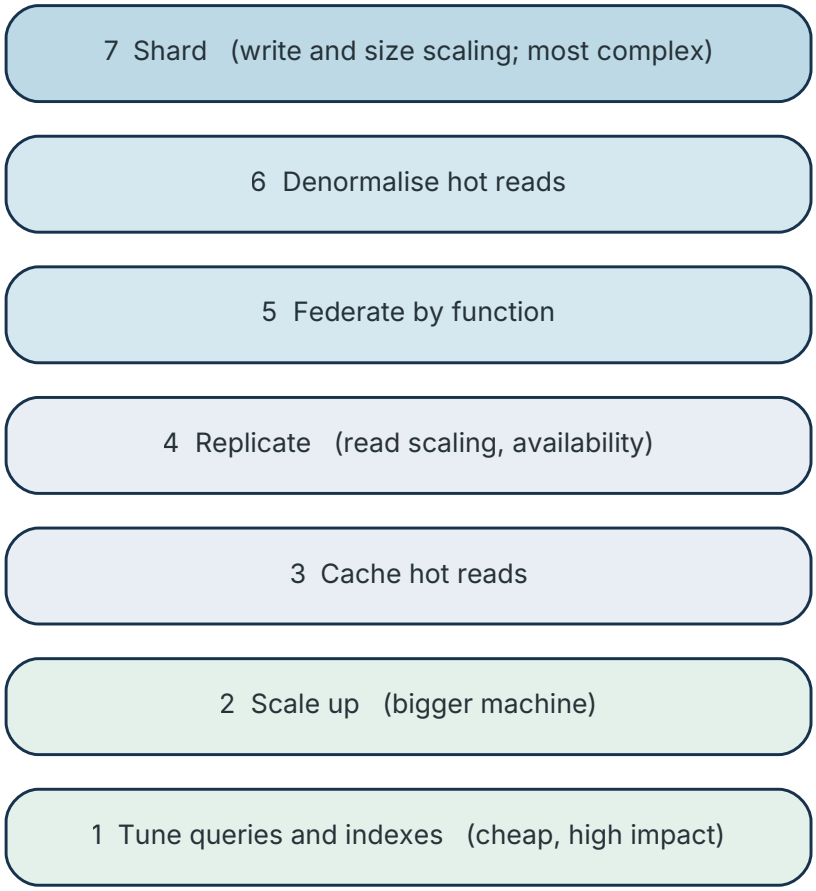


Figure 9.1 The scaling ladder for a relational database. Each rung is more powerful and more complex than the one below.

9.2 Replication

Replication keeps copies of the same data on several servers. It serves two goals that must not be confused: **availability** (another copy survives a failure) and **read scaling** (followers answer read queries). It does *not* by itself scale writes: every write still goes through the leader (in a single-leader design).

Leader-follower

All writes go to the **leader**, which records each change in a replication log (the same write-ahead log used for durability). **Followers** stream and apply that log. Reads can go to any follower.

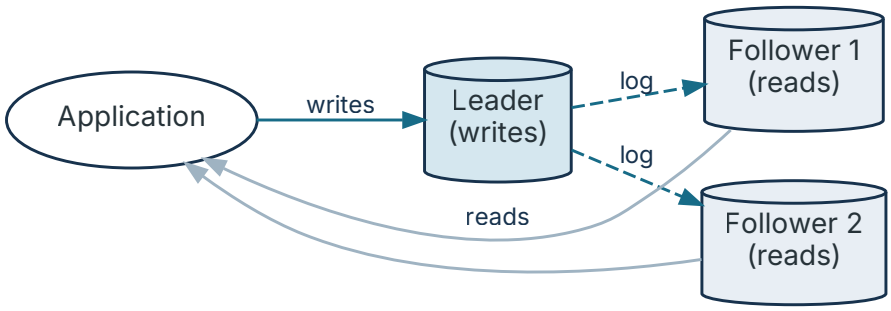


Figure 9.2 Leader-follower replication. The application writes to the leader; changes stream to followers, which serve reads. If the leader fails, a follower can be promoted.

Synchronous replication makes the leader wait for a follower to confirm before acknowledging the write: no acknowledged write is lost if the leader dies, but latency rises and an unavailable follower can block writes. **Asynchronous replication** acknowledges immediately and ships changes in the background: faster and more available, but the most recent writes may be lost if the leader fails before they reach a follower. A common compromise is *semi-synchronous*: one follower is synchronous, the rest asynchronous.

Replication lag and its anomalies

With asynchronous replication, followers are slightly behind: **replication lag**, usually milliseconds, but seconds or more under load. This causes visible anomalies, collectively an *eventual consistency* effect:

- **Read-your-writes violation**: a user posts a comment (written to the leader), reloads, and the read goes to a lagging follower, so the comment is missing.
- **Monotonic reads violation**: a user sees newer data, then on the next request (another follower) sees *older* data, as if time went backwards.
- **Causality violation**: a reply appears before the question it answers, because the two were replicated from different partitions at different speeds.

Remedies: route a user's reads to the **leader for a short time after they write**, or track the log position of the user's last write and only read from a follower that has caught up; **pin a user to one follower** (for monotonic reads); and for rare critical reads, always read from the leader. Cheap in code, but they must be designed in, not discovered in production.

> COMMON MISTAKE
>
> **Common mistake.** Treating replicas as a backup. A replica faithfully copies a bad `DELETE` or a corrupted row within moments. Replication protects against *hardware failure*, not against *human or software error*; only backups with point-in-time recovery protect against that (see below).

Multi-leader replication

In **multi-leader** (master-master) replication, two or more nodes accept writes and replicate to each other. This helps write availability across data centres (each region writes locally) and tolerates the loss of a leader without failover. The cost is **write conflicts**: two users edit the same row on two leaders at once. Resolution strategies include *last-write-wins* (simple, but silently discards one update), merging, custom application logic, or conflict-free replicated data types. Because conflict handling is hard, avoid multi-leader unless you need it, and require that most writes for a given record go to one leader. Also be careful with auto-increment keys and constraints: they cannot be enforced globally without coordination.

Table 9.1 Replication styles compared.

	Leader-follower	Multi-leader
Writes accepted at	One node	Several nodes
Read scaling	Yes (followers)	Yes
Write scaling	No	Partly (by region), with conflicts
Conflicts	None	Must be detected and resolved
Complexity	Moderate	High
Typical use	Most applications	Multi-region writes, offline-first apps

Failover and split brain

When the leader fails, the system must **fail over**: detect the failure (heartbeats, timeouts), choose the most up-to-date follower, promote it, and redirect writes and the other followers to it. This is risky:

- With asynchronous replication, the promoted follower may lack the old leader's last writes (**data loss**), and if the old leader returns, its extra writes conflict.
- A *slow* leader may be wrongly declared dead; if it later wakes up and still accepts writes while a new leader exists, you have **split brain**: two leaders, diverging data.

- Timeouts that are too short cause needless failovers during a brief pause; too long extends the outage.

Defences: use a **majority vote** (a consensus system, Chapter 14) to elect a single leader and require a quorum; **fence** the old leader (cut its network or storage access, or use leases and fencing tokens so a stale leader's writes are rejected); and practise failover regularly.

9.3 Federation and denormalisation

Federation (functional partitioning) splits the database *by feature*: one database for users, one for products, one for forums. Each is smaller, fits more of its working set in memory, and can be scaled and tuned independently; write load is spread across servers. The costs: application code must know which database holds what, **cross-database joins** become application-level work, and **transactions across databases** are no longer simple.

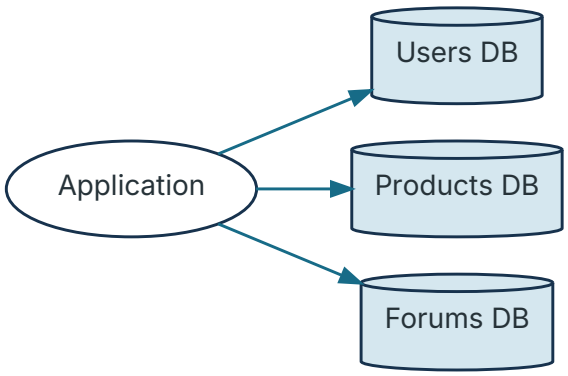


Figure 9.3 Federation. The application talks to separate databases for users, products and forums instead of one large one. A query that needs data from two of them must be assembled in code.

Denormalisation trades write simplicity for read speed by storing redundant data so frequent queries avoid joins. For example, store the author's display name on each comment, or keep a precomputed `comment_count` on each post. Reads become one lookup; writes must update every copy, and copies can drift apart if an update is missed. Denormalise only for measured hot paths. **Materialised views** (stored query results the database refreshes) are a managed version of the same idea.

9.4 Sharding

Sharding (horizontal partitioning) splits *one table's rows* across several servers, each holding a **shard**: users A to H on one, I to P on another. Unlike replication, each shard holds *different* data, so both **writes** and **total data size** scale roughly with the number of shards.

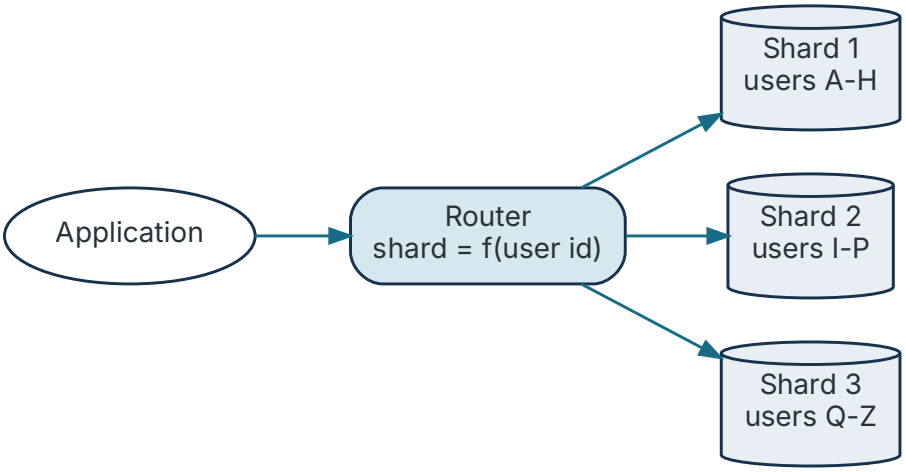


Figure 9.4 Sharding by user id. A routing layer (in the application or a proxy) maps each user to one shard. Each shard is itself usually replicated for availability.

Choosing the shard key

The shard key decides everything. A good key:

- is **present in most queries**, so each query touches one shard instead of all of them (a scatter-gather query across all shards is slow and fragile);
- has **high cardinality** and spreads data and traffic **evenly**;
- is **stable**, because changing a row's key moves it between shards.

Table 9.2 Sharding strategies.

Strategy	How	Strengths	Weaknesses
Range	Key ranges per shard (A-H, I-P)	Efficient range queries	Hot spots (new users all land on the last shard); uneven sizes
Hash	`hash(key) mod N`	Even spread	Range queries scatter; changing N moves most keys
Consistent hash	Keys and nodes on a ring	Adding a node moves only a fraction of keys	More complex (Chapter 11)
Directory (lookup)	A table maps key to shard	Maximum flexibility; easy rebalancing	The directory is a dependency and potential bottleneck; must itself be fast and replicated

What sharding costs

- **Cross-shard queries** and **joins** are expensive or unsupported; design so common operations stay within one shard.
- **Cross-shard transactions** lose simple ACID; you need patterns such as sagas (Chapter 13) or two-phase commit (slow, blocking).
- **Hot shards**: a celebrity user or a popular product concentrates traffic on one shard even if data is evenly distributed. Mitigate with finer keys (split a hot key into several sub-keys), caching, or dedicated shards for the heaviest tenants.

- **Resharding** (changing the number of shards) is a major operation: data must move while the system keeps serving. Plan for it: start with many **logical shards** (say 1,024) mapped to few physical servers, so growth means moving whole logical shards to new servers rather than re-splitting data.
- **Global uniqueness**: auto-increment IDs conflict across shards; use UUIDs, a central ID service, or time-ordered ID schemes.
- **Operational load**: backups, schema changes, monitoring and failover multiply by the shard count.

> **WORKED EXAMPLE**
>
> **How many shards?** The messaging example of Chapter 4 needed about 330 TB and 10,000 writes per second at peak. Suppose one database server comfortably holds 4 TB and handles 2,000 writes per second. Capacity needs: 330 / 4 ≈ 83 servers; writes: 10,000 / 2,000 = 5 servers. Data volume, not write rate, drives the count: about 83 shards (each with replicas). That prompts questions that could shrink it: can old messages be archived to cheaper storage? Is 200 bytes per message accurate? With a hot set of recent messages only a fraction of the data needs fast storage. Estimates guide, then measurement decides.

9.5 SQL tuning checklist

Before adding machines, apply the notebook's tuning list in this order.

- **Benchmark and profile** (load tests, slow-query logs) to find the real bottleneck; fix the biggest first.
- **Tighten the schema**: use the smallest suitable types (`INT` instead of `BIGINT` when it fits; `DECIMAL` for money, never floating point; `CHAR` for fixed-length, `VARCHAR` for variable); `NOT NULL` where possible; avoid storing large blobs in rows (store them in an object store and keep a reference).
- **Use good indexes** on columns used in `WHERE`, `ORDER BY`, `GROUP BY`, `JOIN`; remember the cost to writes.
- **Avoid expensive joins**: denormalise where the profile shows a hot join.
- **Partition tables** so hot recent data is separate from old data; **archive** old rows.
- **Tune the query cache and buffer pool**: ensure the working set fits memory.
- **Use a connection pool** so thousands of application threads share a few dozen database connections; each connection costs memory and a thread on the server.

9.6 Backups, replicas and failover

These three are often confused. They protect against *different* failures.

Table 9.3 Backups, replicas and failover are different tools.

	Backup	Replica	Failover
What it is	A separate copy at a point in time	A live, continuously updated copy	The act of switching to another node
Protects against	Deletion, corruption, bugs, ransomware, disasters	Hardware or node failure; read load	Leader outage
Does *not* protect against	Downtime (restore takes time)	A bad write or delete (copied instantly)	Data errors; also may lose recent writes
Recovery time (RTO)	Minutes to hours or longer	Seconds to minutes	Seconds to minutes
Data loss (RPO)	Since last backup, or near zero with log archiving	Near zero (sync) or small (async)	Depends on replication mode

Point-in-time recovery (PITR) combines a periodic full backup with continuously archived write-ahead log. To undo an accidental `DROP TABLE` at 14:03, restore the last full backup, then replay the log up to 14:02:59. Define two targets before an incident: the **recovery point objective** (RPO: how much data loss is tolerable, measured in time) and the **recovery time objective** (RTO: how long recovery may take). Keep backups in a *different* account or region, protect them from deletion, and **test restores regularly**: an untested backup is not a backup.

FAILURE MODES

Failure modes. Lag so large that read replicas serve minutes-old data or fall permanently behind; split brain after a bad failover; a hot shard overloaded while others are idle; a shard key that cannot support the dominant query; resharding that stalls under live traffic; a migration run on every shard inconsistently; replicas backing up the same corrupt data; backups that were never restorable; connection exhaustion in a failover storm when every client reconnects at once.

COMMON MISCONCEPTION

- "*Replication scales writes.*" In leader-follower it does not; only sharding (or multi-leader with conflicts) does.
- "*We have replicas, so we have backups.*" Replicas copy mistakes instantly.
- "*Sharding is the first thing to do when the database is slow.*" Tuning, caching and replicas come first.
- "*Hash sharding avoids hot spots.*" It evens out *keys*, not *traffic*; a single hot key is still hot.

INTERVIEW QUESTIONS

Q1. How would you scale a read-heavy relational database? Tune and index; cache hot reads; add read replicas with asynchronous replication, handling lag (read-your-writes by routing recent writers to the leader). Only then consider sharding.

Q2. What is replication lag and how does it affect users? Followers apply changes after the leader, so reads may be stale; users may not see their own writes or may see time go backwards. Mitigate with leader reads after writes and session pinning.

Q3. How do you choose a shard key? High cardinality, even distribution, present in most queries so they hit one shard, and stable. Evaluate hot-key risk and resharding cost; consider logical shards.

Q4. What is split brain and how do you prevent it? Two nodes both acting as leader. Prevent with quorum-based election, fencing of the old leader (leases, fencing tokens, cutting access), and conservative failover.

EXERCISES

1. A database has 50,000 reads/s and 2,000 writes/s at peak; one node handles 15,000 reads/s and 5,000 writes/s. Design with replicas and state the minimum number of nodes.
2. After posting, a user's page does not show the post. Explain why and give two fixes.
3. You shard `orders` by `customer_id` with hash mod 8. Which queries are cheap? Which are expensive? What changes when you go to 16 shards?
4. Order these by recovery speed and data protection: nightly backup, asynchronous replica, continuously archived log with a weekly backup.

Answers.

1. Reads: 50,000 / 15,000 → 4 nodes; writes 2,000 fit on the leader alone. Use one leader plus 3 followers (4 nodes), preferably 4 followers for headroom and failover; one follower can take over as leader. Still watch replication lag.
2. The read hit a follower that had not yet applied the write. Fixes: read from the leader for a short window after the user writes; or track the write's log position and read only from a sufficiently caught-up follower.
3. Cheap: anything filtered by `customer_id` (one shard). Expensive: "all orders from today" or by product (every shard queried and merged). Moving from 8 to 16 shards with plain mod moves about half of all keys, so use consistent hashing or logical shards to move less.
4. Fastest recovery: asynchronous replica (seconds, but copies mistakes); nightly backup restore is slowest and loses up to a day; log-archived PITR restores in minutes-hours with near-zero data loss and protects against mistakes. Best protection overall: PITR plus replicas.

CHAPTER SUMMARY

- Scale in order: tune, scale up, cache, replicate, federate, denormalise, shard.
- Replication scales reads and availability, not writes; lag causes read-your-writes and monotonic-read anomalies.
- Failover needs consensus and fencing to avoid split brain and data loss.
- Sharding scales writes and size but breaks cross-shard queries and transactions; the shard key is the critical decision; plan resharding and hot-key handling.
- Backups, replicas and failover solve different problems; restore drills and PITR are essential.

REVISION CHECKLIST

- [ ] I can list the scaling ladder in order and justify it.
- [ ] I can describe lag anomalies and their remedies.
- [ ] I can explain split brain and fencing.
- [ ] I can pick a shard key and name its risks.
- [ ] I can say how backups, replicas and failover differ and define RPO and RTO.

SUPPLEMENTAL MATERIAL

Further reading. Kleppmann, *Designing Data-Intensive Applications*, chapters 5 (replication) and 6 (partitioning); the replication and PITR chapters of the PostgreSQL and MySQL manuals; engineering write-ups on sharding from large social companies (search for the sharding of their main MySQL or PostgreSQL clusters).

CHAPTER 10

NoSQL and Choosing a Database

Derived from your notes: Notebook chapter 7 (what NoSQL is, BASE, the four families, data that fits NoSQL, SQL or NoSQL tables). **Added in this book:** query-first data modelling, embedding versus referencing, secondary indexes, tunable consistency, polyglot persistence, search and time-series stores.

LEARNING OBJECTIVES

After this chapter you can:

- explain what "NoSQL" covers and why such stores appeared;
- describe the four main families (key-value, document, wide-column, graph) with data model, strengths and typical uses;
- model data starting from the queries rather than from entities, and decide between embedding and referencing;
- explain BASE and tunable consistency without oversimplifying SQL versus NoSQL;
- choose a database for a given workload and justify the choice.

PREREQUISITE KNOWLEDGE

Chapter 8 and Chapter 9; Chapter 3 for LSM trees.

KEY TERMINOLOGY

NoSQL
"Not only SQL": a broad label for databases that do not use the relational table model or full SQL, usually trading some relational features for scale, flexibility or speed.

BASE
Basically Available, Soft state, Eventual consistency: a design stance contrasted with ACID.

Document
A self-contained, nested record (commonly JSON) stored and retrieved as a unit.

Wide-column store
A store whose rows hold sparse, dynamically named columns grouped into column families, partitioned by row key.

Access pattern
A specific query your application will make repeatedly; NoSQL modelling begins here.

Eventual consistency
If no new writes arrive, all replicas will eventually hold the same value.

10.1 What NoSQL is, and why it appeared

By the late 2000s, companies running very large web services hit limits of single-node relational databases: write volumes too high for one machine, data too large to join, schemas that changed weekly, and global users demanding local latency. Several systems were built to scale out first and offer fewer features: simple data models, no or limited joins, relaxed consistency, and automatic distribution across many commodity machines. "NoSQL" is the umbrella name. It is not one technology: a key-value cache, a document store and a graph database share little except that they are not relational.

The right question is never "SQL or NoSQL?" but "**what are my access patterns, my scale, my consistency needs and my team's skills?**" Many NoSQL systems now offer transactions and SQL-like languages, and many relational systems offer JSON columns and horizontal scaling, so the lines blur.

> **CLARIFICATION**
>
> **Clarification on BASE.** The notebook summarises NoSQL as BASE: *basically available* (the system answers even during partial failure), *soft state* (data may change over time even without input, as replicas converge), *eventual consistency*. This is a useful design stance, not a rule that every NoSQL database is inconsistent: many offer strong consistency options, and some (for example single-leader document stores) are strongly consistent by default. Conversely, a replicated relational database with lagging followers also gives eventual consistency on reads. Treat "consistency model" as a property to check per system and per operation, not per category.

10.2 The four families

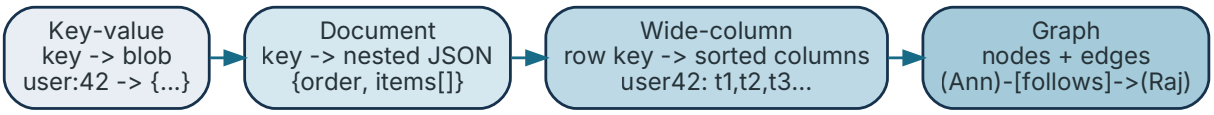


Figure 10.1 The four main NoSQL families, from simplest to richest relationships: key-value, document, wide-column and graph.

Key-value stores

The simplest model: a giant dictionary. Operations are `get(key)`, `put(key, value)` and `delete(key)`; the value is an opaque blob. Because the system never looks inside values, it can partition by key and replicate very simply, giving high throughput and low latency, often in memory (Redis, Memcached) or on SSD (DynamoDB in its basic mode, Riak). Typical uses: caches, sessions, shopping carts, feature flags, counters, rate limiting. Limits: no queries by value, no relationships; you must know the key.

Document stores

A document store holds self-describing, nested records, usually JSON or similar (MongoDB, CouchDB, Firestore, DynamoDB in document mode). Each document can have a different shape. The database understands the structure, so it can index inside documents and query fields.

```
{ "_id": "order-9131",
  "customer": {"id": 42, "name": "Asha"},
  "items": [ {"sku": "A1", "qty": 2, "price": 9.99},
             {"sku": "B7", "qty": 1, "price": 24.00} ],
  "status": "shipped" }
```

Strengths: a whole business object is read or written in one operation (locality), flexible schema, natural fit for object-oriented code, easy evolution. Limits: joins across documents are limited or manual; duplicated data must be kept consistent by the application; large unbounded documents (an ever-growing `items` list) cause problems. Uses: content management, catalogues, user profiles, event and order records.

Wide-column stores

Rows are identified by a **row key**, which determines the partition; within a row, data is grouped into **column families**, and different rows may have different columns (Cassandra, HBase, Bigtable). Data within a partition is stored **sorted by a clustering key**, so range scans within a partition are fast. They are built on LSM trees (Chapter 3) for high write throughput.

Think of it as a *map of sorted maps*: `rowkey -> (column -> value)`, optimised for enormous volumes distributed over many nodes. Strengths: very high write rates, linear scale-out, tunable consistency, good for time-ordered data. Limits: you can efficiently query only by the partition key (and clustering order), so each query shape needs its own table; no joins; limited ad-hoc queries; careful modelling essential. Uses: time series, event logs, messaging, activity feeds, sensor data, analytics ingestion.

Graph databases

Data is **nodes** (entities) and **edges** (relationships), each with properties (Neo4j, Amazon Neptune). The key advantage: following relationships is *cheap and does not slow down as the total dataset grows*, because the engine hops along stored edges instead of performing a join that searches an index at every hop. Questions like "friends of friends who like this film" or "shortest path" are natural; in SQL they need many self-joins whose cost multiplies with depth. Limits: harder to shard (a graph is highly connected, so cutting it across machines cuts many edges), smaller ecosystems, not suited to bulk aggregations. Uses: social networks, recommendations, fraud rings, knowledge graphs, network topology.

Table 10.1 The four families compared.

Family	Data model	Strengths	Weak spots	Examples
Key-value	key to opaque value	Speed, simplicity, easy sharding	No queries by value or relationships	Redis, Memcached, DynamoDB, Riak
Document	key to nested document	Flexible schema, object locality, field indexes	Weak cross-document joins; duplication	MongoDB, CouchDB, Firestore
Wide-column	row key, sorted columns	Huge write throughput, scale-out	Query-first modelling; no joins	Cassandra, HBase, Bigtable
Graph	nodes, edges, properties	Cheap relationship traversal	Hard to shard; niche	Neo4j, Neptune

10.3 Modelling for NoSQL: queries first

Relational modelling starts from *entities and normalisation*, then queries are derived by joins. NoSQL modelling reverses the order: **list the access patterns first**, then shape the data so each is answered by one lookup in one partition.

Embedding versus referencing

In a document store you choose whether related data lives *inside* the parent document (embedding) or in a separate document reached by id (referencing).

Table 10.2 Embed or reference?

Embed when	Reference when
The child is always read with the parent	The child is read or updated independently
The child list is small and bounded	The child list is large or unbounded
Children belong to one parent only	Children are shared by many parents
Atomic update of both is needed	The child data changes often and would require updating many copies

WORKED EXAMPLE

Modelling a timeline for a wide-column store. Access pattern: "show the 50 most recent posts from the accounts I follow". In a relational model you would join `follows` and `posts` at read time, which is slow at scale. In a wide-column model you instead **precompute** a table keyed for the read:

```
table home_timeline
  partition key : user_id          -- one partition per reader
  clustering key: post_time DESC   -- newest first inside the partition
  columns       : post_id, author_id, snippet

read:   SELECT * FROM home_timeline WHERE user_id = 42 LIMIT 50;   -- one partition
write:  for each follower f of author a: INSERT INTO home_timeline (f, now, post_id, a,
snippet)
```

Reading is one partition lookup, fast at any scale. The cost moved to *write time* (one insert per follower, called fan-out) and to **duplication** (the snippet is stored many times). This is the central NoSQL trade: *pay on write, duplicate data, so reads are trivial.* Chapter 22 develops this design.

Secondary indexes and consistency knobs

Querying by something other than the primary key (find users by email) needs a **secondary index**. In distributed stores these are costly: a **local** index (each shard indexes its own rows) requires a scatter-gather query across all shards; a **global** index (partitioned by the indexed value) makes reads cheap but writes touch two partitions and often update asynchronously, so the index can briefly lag. Many designs simply maintain a second table keyed by the alternate attribute.

Many distributed stores let you tune consistency **per operation** using **quorums**: with N replicas, writes wait for W acknowledgements and reads query R replicas; if $\mathbf{R + W > N}$, every read overlaps the latest write and sees it (assuming no failures at the wrong moment). Chapter 11 explains this carefully, including why it is *not* a guarantee of linearizability.

10.4 SQL or NoSQL?

Table 10.3 Reasons to prefer each (from the notebook, with qualifications).

Prefer a relational database when	Prefer NoSQL when
Data is structured and relationships matter	Data is semi-structured, nested or schema changes often
You need ACID transactions across rows or tables	You can accept eventual consistency or single-document atomicity
You need flexible ad-hoc queries and joins	Access patterns are known and few
Scaling needs are moderate, or can be met with replicas and sharding	You need very high write throughput or global distribution

Prefer a relational database when	Prefer NoSQL when
The team knows SQL and the ecosystem matters	Data is naturally key-based, time-ordered or graph-shaped

Examples of data that fits NoSQL particularly well (notebook): rapid ingestion of clickstream and logs; leaderboard and scoring data; temporary data such as a shopping cart; frequently accessed "hot" tables; metadata and lookup tables. Real systems use **polyglot persistence**: several stores, each for what it does best (relational for orders and payments, key-value cache for sessions, search index for text, wide-column for the event firehose). The cost is operational: each store must be run, secured, backed up and kept in sync (often through events, Chapter 13).

SUPPLEMENTAL MATERIAL

Two more store types worth knowing. *Search engines* (Elasticsearch, OpenSearch, Solr) build an **inverted index** (word → list of documents) for full-text search, ranking and faceting; use them as a *derived* copy of data whose source of truth lives elsewhere. *Time-series databases* (InfluxDB, TimescaleDB, Prometheus) optimise for append-only timestamped measurements with compression, downsampling and retention policies. *NewSQL* systems (for example Spanner, CockroachDB) aim to offer SQL and ACID transactions with horizontal scale, at the cost of cross-node coordination latency.

COMMON MISTAKE

Common mistakes. Choosing NoSQL "because it scales" without a scale problem. Choosing a schemaless store and then never writing down the schema (it still exists, in your code, undocumented). Modelling a NoSQL store like a relational one, then discovering that queries require scans. Unbounded arrays inside documents. Using a cache or key-value store as the only copy of important data.

FAILURE MODES

Failure modes. Hot partitions from skewed keys (one celebrity key receives most traffic); partitions that grow without bound (a time-series partition per device per forever); read repair and compaction storms degrading latency; stale reads from relaxed consistency interpreted as bugs; lost updates from last-write-wins conflict resolution; schema drift producing documents that old code cannot read; backups of eventually consistent stores that capture an inconsistent snapshot.

COMMON MISCONCEPTION

- "*NoSQL is faster than SQL.*" It is faster at what it was designed for, with the features it dropped; a well-indexed relational query can beat a badly modelled NoSQL scan.
- "*NoSQL has no schema.*" It has an implicit schema enforced by application code; "schema-on-read" moves the discipline, it does not remove it.
- "*NoSQL means no transactions and no consistency.*" Many offer tunable or strong consistency and limited transactions.
- "*You must pick one database for the whole system.*" Polyglot persistence is normal.

INTERVIEW QUESTIONS

Q1. When would you choose a document store over a relational database? When the data is naturally nested and read as a unit, the schema evolves quickly, and cross-record joins and multi-record transactions are rare; also when horizontal scaling by document key is needed.

Q2. Why is Cassandra good at writes? It uses an LSM-tree design: writes append to a commit log and an in-memory table and flush as sequential immutable files, avoiding random disk writes; data is partitioned by key across many nodes with no single leader.

Q3. How do you model "recent posts of people I follow" in NoSQL? Precompute a per-reader timeline keyed by user and ordered by time (fan-out on write), accept duplication, and handle very popular authors by pulling their posts at read time instead.

Q4. SQL versus NoSQL: how do you decide? By access patterns, scale, consistency requirements, flexibility and team skills; default to relational and move specific workloads to NoSQL when a concrete limit requires it.

EXERCISES

1. Model an e-commerce order as a document, and decide whether product details should be embedded in each order item or referenced. Justify your decision.
2. You need to answer "all events for device D between two times" and "latest reading for every device". Propose a wide-column table design for each query.
3. A social app needs "mutual friends of A and B" and "shortest path between A and B". Which family and why?
4. Replicas = 5. What do R and W need to be so that reads see the latest write with R + W > N, and what trade-off does choosing W = 5 bring?

Answers.

1. Embed a *snapshot* of product name and price at purchase time (it should not change when the catalogue changes), plus the `sku` reference for lookup: embedding the snapshot is correct because an order is a historical record read as one unit.
2. Query 1: partition key `device_id` (plus a time bucket such as the day to bound partition size), clustering key `timestamp`. Query 2: a separate table `latest_by_device` keyed by `device_id` containing one row updated on each reading.
3. A graph database: relationship traversals (common neighbours, shortest path) are natural and fast without multi-way joins.
4. For example W = 3, R = 3 (3 + 3 > 5). W = 5 makes writes wait for all replicas: reads can use R = 1, but any single replica failure blocks writes, reducing write availability.

CHAPTER SUMMARY

- NoSQL is a family of scale-oriented stores with simpler models; judge each by access patterns, consistency options and operations, not by label.
- Key-value: speed and simplicity. Document: nested records and flexible schema. Wide-column: massive writes, query-first modelling. Graph: cheap relationship traversal.
- Model from the queries; embed what is read together and bounded; duplicate on write so reads are one lookup.
- Tunable consistency uses quorums ($R + W > N$); secondary indexes cost extra.
- Use relational by default; add specialised stores for specific needs, accepting operational cost.

REVISION CHECKLIST

- [ ] I can describe each family and give an example and a use.
- [ ] I can model a query-first table or document.
- [ ] I can explain embed versus reference.
- [ ] I can state the quorum rule $R + W > N$.
- [ ] I can justify SQL or NoSQL for a workload.

SUPPLEMENTAL MATERIAL

Further reading. Kleppmann, *Designing Data-Intensive Applications*, chapters 2 and 3; the DynamoDB and Cassandra data-modelling guides from their vendors; Pat Helland, "Life beyond Distributed Transactions"; the original Bigtable (2006) and Dynamo (2007) papers.

CHAPTER 11

Consistency, Availability and Partitioning Data

Derived from your notes: Notebook chapter 2 (CAP theorem, consistency patterns, availability patterns, availability in numbers). **Added in this book:** PACELC, formal consistency models and session guarantees, quorums, consistent hashing, rebalancing, hot partitions, anti-entropy and conflict resolution.

LEARNING OBJECTIVES

After this chapter you can:

- state the CAP theorem correctly, as a statement about behaviour *during a network partition*;
- explain linearizability, sequential, causal and eventual consistency, and the session guarantees;
- apply quorum rules and explain why R + W > N is necessary but not sufficient for strong guarantees;
- explain consistent hashing, virtual nodes, rebalancing and hot-partition mitigation;
- describe availability patterns (active-passive, active-active) and combine availability numbers.

PREREQUISITE KNOWLEDGE

Chapter 8 and Chapter 9 (replication, sharding) and Chapter 10 (families, quorums), and Chapter 4 (availability arithmetic).

KEY TERMINOLOGY

Network partition
A failure in which some nodes cannot communicate with others, though each may still be running.

Linearizability
A strong guarantee: every operation appears to take effect instantly at some point between its start and end, and all clients see one single, up-to-date copy.

Quorum
A minimum number of nodes that must agree for an operation to count (usually a majority).

Consistent hashing
A way of assigning keys to nodes so that adding or removing a node moves only a small fraction of keys.

Hot partition
One partition receiving far more traffic than the others.

Anti-entropy
Background processes that find and repair differences between replicas.

11.1 Why this chapter exists

When data lives on **more than one machine**, three things become unavoidable. Messages between machines can be delayed or lost. Machines can fail independently. And copies can disagree. Everything in this chapter is about choosing, deliberately, what the system does when those facts collide with user requests. There are no free answers: each choice buys one property by spending another.

11.2 The CAP theorem

A distributed data store can offer three desirable properties:

- **Consistency (C)**: every read receives the most recent write or an error (in CAP's precise sense, linearizability).
- **Availability (A)**: every request to a non-failing node receives a (non-error) response, though it may not contain the most recent write.
- **Partition tolerance (P)**: the system continues to operate despite messages between nodes being dropped or delayed.

The theorem (Gilbert and Lynch's proof of Brewer's conjecture) says: **when a network partition occurs, a system cannot be both consistent and available.** It must choose.

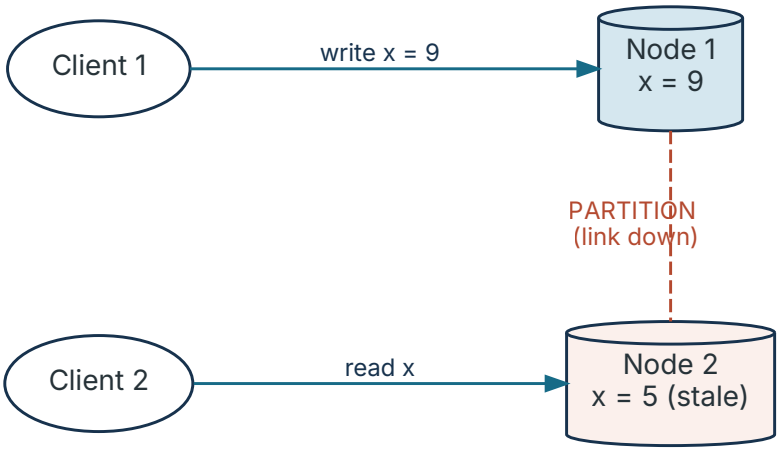


Figure 11.1 A partition separates two replicas. A write of x = 9 reaches node 1 only. A read at node 2 must either refuse (consistent, not available) or answer with the old value x = 5 (available, not consistent).

Look at Figure 11.1. Node 1 accepted the write x = 9. The link to node 2 is cut. A client now reads x at node 2. If node 2 answers "5", it is **available** but **inconsistent** (stale). If node 2 refuses until it can confirm the latest value, it is **consistent** but **unavailable**. The system cannot do both.

CLARIFICATION

Clarification: "pick two of three" is misleading. The notebook draws CAP as a triangle with a choice of two. That picture suggests you can choose to give up partitions, but **partitions are not optional**: networks fail, so any distributed system must tolerate P. The real decision is what to do *during a partition*: **CP** (stay consistent, refuse some requests) or **AP** (stay available, accept possible staleness). When the network is healthy you can have both. Also, CAP's "availability" and "consistency" are narrow technical definitions, so labelling an entire product "CP" or "AP" oversimplifies; many products let you choose per operation.

PACELC extends the picture: *if there is a Partition, choose Availability or Consistency; Else (normal operation), choose Latency or Consistency.* Even without failures, strongly consistent replication requires waiting for coordination between nodes, which costs latency. This explains why some systems offer weaker consistency even when everything is healthy.

Table 11.1 CP and AP behaviour in practice.

	CP (consistency over availability)	AP (availability over consistency)
During a partition	Minority side refuses writes or reads	Both sides keep accepting requests
After healing	Nothing to reconcile	Divergent data must be merged
Good for	Money, inventory, locks, coordination data	Feeds, shopping carts, counters, DNS, social data
Examples (tendencies)	Consensus-backed stores, single-leader databases with quorum writes	Dynamo-style stores with eventual consistency

11.3 Consistency models

"Consistency" in a replicated system is about *what a read may return after a write*. Models run from strongest to weakest:

- **Strong consistency (linearizability)**: after a write completes, every subsequent read, anywhere, sees it. The system behaves as if there were one copy. Achieved with synchronous replication or consensus; costs latency and availability under partition. Suits bank balances, uniqueness checks, leader election.
- **Sequential consistency**: all clients see operations in the same order, but that order need not match real time.
- **Causal consistency**: operations that are causally related (a reply to a question) are seen in that order by everyone; unrelated operations may be seen in different orders. Cheaper than linearizable and often "good enough".
- **Eventual consistency**: if writes stop, all replicas converge. Until then reads may be stale or disordered. Cheap, fast, highly available; suits counters, likes, feeds, DNS.

Table 11.2 The notebook's three patterns, with the models behind them.

Pattern	After a write, a read...	How it is achieved	Typical systems
Weak	May or may not see it; the system makes best effort	Asynchronous copy, no repair guarantees	Memcached-style caches, real-time voice and video
Eventual	Will see it soon (typically milliseconds to seconds)	Asynchronous replication with convergence	DNS, email, Dynamo-style stores
Strong	Always sees it	Synchronous replication, consensus	File systems, relational databases with synchronous replicas

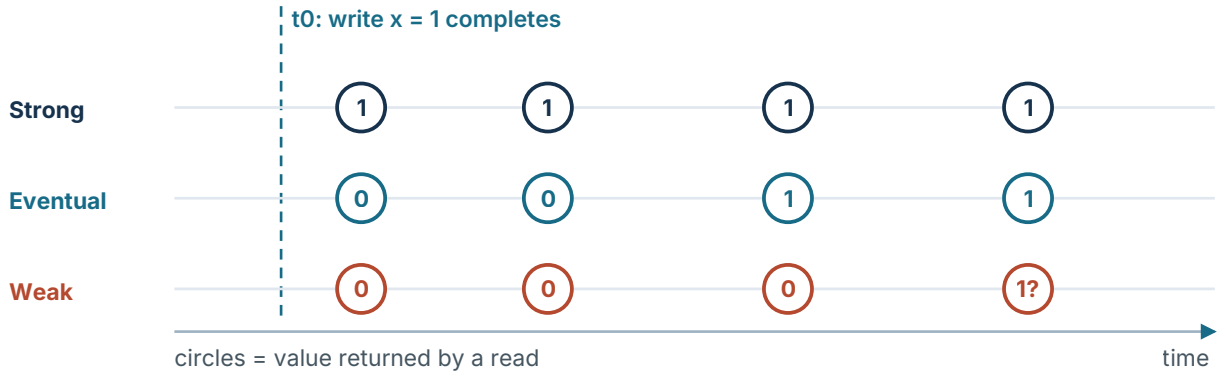


Figure 11.2 The same write seen under three models. The writer sets x = 1 at time t0. Strong: every read after t0 returns 1. Eventual: reads return 0 for a while, then 1. Weak: reads may keep returning 0 and the write may even be lost.

Session guarantees

Between "eventual" and "strong" sit practical promises you can give a *single user session* cheaply:

- **Read-your-writes**: a user always sees their own updates.
- **Monotonic reads**: once you see a value, you never see an older one.
- **Monotonic writes**: your writes are applied in the order you made them.
- **Consistent prefix**: you never see an effect before its cause.

These are often enough for user-facing apps and are implemented by routing a session to the same replica or tracking version numbers (Chapter 9).

11.4 Quorums

Suppose a value is stored on **N** replicas. A write succeeds when **W** replicas acknowledge it; a read queries **R** replicas and takes the newest version. If **R + W > N**, the set of replicas read must overlap the set written in at least one node, so a read will include the latest acknowledged write.

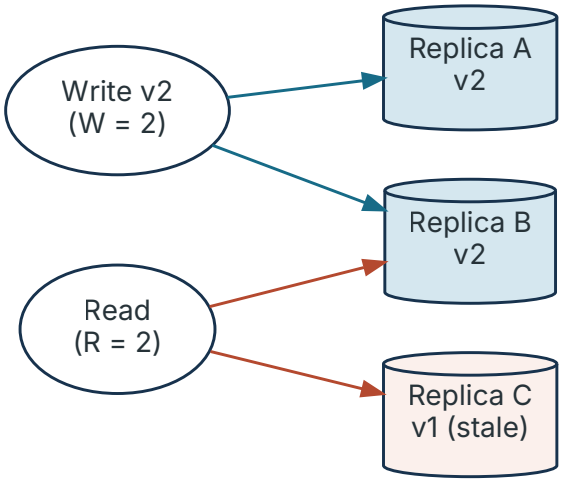


Figure 11.3 N = 3 replicas, W = 2, R = 2. The write reached A and B; a read of B and C overlaps at B, which holds the newest version, so the read returns the latest value.

Typical settings for N = 3: **W = 2, R = 2** (balanced; tolerates one failure for both); **W = 3, R = 1** (fast reads, but writes fail if any replica is down); **W = 1, R = 3** (fast writes, slow reads, risk of loss). Setting both low (R = W = 1) favours latency and availability at the price of stale reads.

> **COMMON MISTAKE**
>
> **R + W > N is not magic.** It does *not* by itself give linearizability. Pitfalls: *sloppy quorums* (during failures, writes go to stand-in nodes outside the usual N, so the overlap guarantee disappears until the data is handed back); concurrent writes, where "newest" is ambiguous when clocks disagree; a write that succeeded on fewer than W nodes is not rolled back and may later appear; and a replica that restored from an old backup can reintroduce stale data. Use consensus (Chapter 14) when you truly need linearizable behaviour.

11.5 Conflicts and repair

AP systems accept concurrent writes on different replicas, so copies diverge and must be reconciled.

- **Last-write-wins (LWW)**: keep the version with the latest timestamp. Simple, but discards a concurrent update silently and depends on clocks (Chapter 14).
- **Version vectors**: each replica tracks counters per node, so the system can tell whether one version *descends* from another or whether the two are truly *concurrent*; concurrent versions are kept as "siblings" and merged by the application.
- **CRDTs (conflict-free replicated data types)**: data structures (counters, sets) designed so that merging any two copies, in any order, gives the same result. A grow-only counter merges by taking the maximum per node.

Replicas also repair themselves in the background (**anti-entropy**): **read repair** (a read that sees differing versions writes the newest back to stale replicas); **hinted handoff** (a node holds writes meant for a temporarily unavailable node and delivers them later); and **Merkle trees** (hash trees that let two replicas compare large datasets by exchanging a few hashes and drilling into the parts that differ).

11.6 Availability patterns

Consistency is only half the story; the other half is staying up when parts fail. Two families of patterns, usually combined.

Fail-over keeps a spare ready.

- **Active-passive** ("master-slave" fail-over): the active node serves traffic and sends *heartbeats* to a passive standby. If heartbeats stop, the standby takes over the active's address (or the router is updated). A *hot* standby is already running and in sync; a *cold* standby must be started. Downtime is the detection-plus-switch time. Risk: data loss if replication to the standby was asynchronous, and the passive node's capacity sits idle.
- **Active-active** ("master-master"): both nodes serve traffic and share the load, so loss of one halves capacity but does not require a switch. The application or DNS must know both addresses; with data, both accept writes, so conflicts must be handled.

Replication makes failure survivable by keeping copies (Chapter 9). Its disadvantages (notebook): potential data loss if the active fails before replicating, and extra hardware and complexity; with multiple write nodes, conflict resolution.

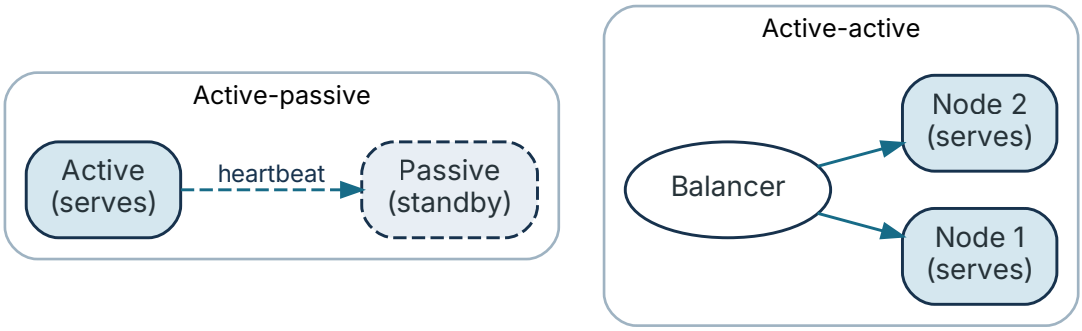


Figure 11.4 Active-passive fail-over (left): the standby watches heartbeats and takes over. Active-active (right): both nodes serve traffic behind a balancer.

Availability in numbers

Recall the arithmetic of Chapter 4: components **in series** multiply (more dependencies, less availability), **in parallel** (independent replicas) multiply the *unavailability*. If the web tier is 99.9% available, the database 99.9% and a third-party API 99.9%, a request needing all three gets about 99.7%, roughly 26 hours of downtime a year. Redundancy only

helps if failures are independent: two replicas in the same rack or running the same buggy release fail together. Spread across **availability zones** or regions, and roll out changes gradually.

11.7 Partitioning data with consistent hashing

Chapter 9 introduced hash sharding: `node = hash(key) mod N`. Its flaw: changing N changes the result for almost every key, so adding one server forces nearly all data to move (and a cache would lose almost all entries at once).

Consistent hashing fixes this. Place both **nodes** and **keys** on a conceptual ring using the same hash function. Each key belongs to the **first node clockwise from its position**. When a node is added, it takes over only the keys between itself and its predecessor; when a node leaves, only its keys move to the next node.

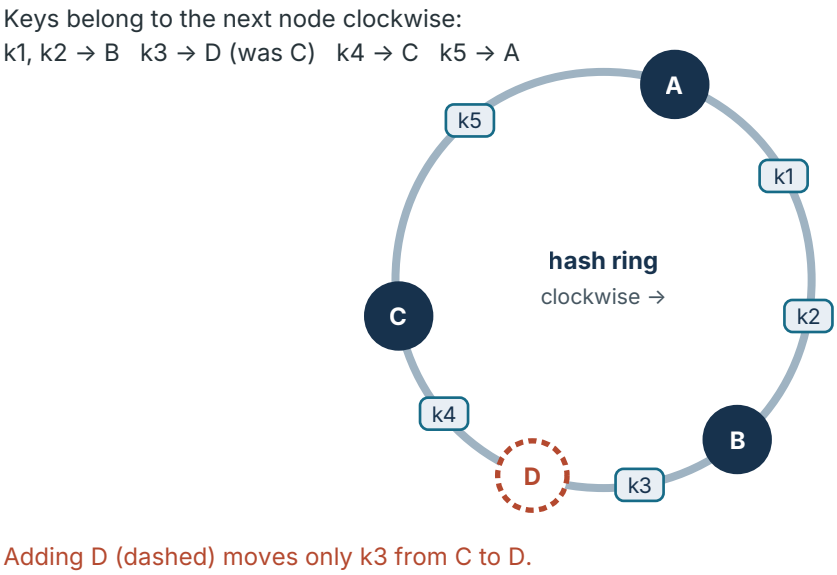


Figure 11.5 A consistent-hash ring with three nodes. Each key is stored on the first node clockwise from it. Adding node D between B and C moves only the keys that fall between B and D.

WORKED EXAMPLE

How much moves? With plain `mod N` hashing and 10 nodes, going to 11 nodes changes the owner of about 10/11, roughly 91%, of keys. With consistent hashing the new node takes about 1/11, roughly 9%, of the keys (assuming an even spread). For a 10 TB store that is about 0.9 TB instead of 9.1 TB of data movement: the difference between a routine operation and an outage.

Plain consistent hashing spreads unevenly with few nodes (random positions leave unequal arcs). The fix is **virtual nodes (vnodes)**: each physical server owns many (for example 100 to 256) positions on the ring. Load evens out statistically, a new server takes

small slices from many others, and a more powerful server can be given more vnodes. Replication fits naturally: store each key on its owner and the next $N - 1$ distinct servers clockwise.

Rebalancing and hot partitions

Moving data must not stop service. Practical approaches: move **whole vnodes or logical shards** (units fixed in advance), copy in the background, then switch ownership with a short cutover; throttle the transfer to protect live traffic; and verify before deleting the old copy.

Even a perfect hash cannot fix **hot keys**: a viral post or a celebrity account gets millions of reads while others get few. Mitigations: **cache** the hot item in front of the store; **replicate** the hot key to more nodes and spread reads; **split** the key with a random suffix (write `counter#0..#9`, read all and sum); give heavy tenants **dedicated capacity**; **rate limit** (Chapter 7); and detect hot keys with metrics so you can react.

FAILURE MODES

Failure modes. A partition causing both sides of a multi-leader system to accept conflicting writes; clock skew making last-write-wins pick the wrong version; a majority partition that elects a new leader while the old leader, isolated, still serves stale reads; a rebalancing storm that saturates the network when many nodes join at once; a hot partition overwhelmed while the cluster average looks healthy; sloppy quorum data lost when a stand-in node dies before handing data back; correlated failures defeating "independent" replicas.

COMMON MISCONCEPTION

- "*CAP means choose two of three.*" Partitions happen; the choice is C or A *during a partition*.
- "*AP systems are inconsistent forever.*" They converge when the partition heals (eventual consistency).
- "*$R + W > N$ guarantees strong consistency.*" Not with sloppy quorums, concurrent writes or clock problems.
- "*Replication across three zones makes us 'five nines'.*" Only if failures are independent and the software, configuration and deployment process are not shared SPOFs.

INTERVIEW QUESTIONS

Q1. Explain CAP in your own words. Under a network partition a replicated store must either reject some requests to avoid returning stale data (consistency) or keep answering and risk staleness (availability). Without a partition it can offer both; partitions cannot be ruled out, so the real choice is CP versus AP during failure.

Q2. What is eventual consistency and when is it acceptable? Replicas converge if updates stop; reads may be stale meanwhile. Acceptable for likes, feeds, counters, profile views; not for balances or inventory without extra controls.

Q3. Why consistent hashing? It minimises key movement when nodes change, avoiding mass reshuffles and cache wipe-outs; virtual nodes smooth the load.

Q4. A user posts and immediately does not see the post. Which guarantee is violated and how do you fix it? Read-your-writes; route their reads to the leader or an up-to-date replica for a short time, or carry a version token.

EXERCISES

1. N = 5. Choose R and W for (a) fast reads, (b) fast writes, (c) balanced, all with R + W > N, and state how many node failures each tolerates for reads and for writes.
2. A 10-node cluster uses `hash mod N`. Compute the fraction of keys that move going to 11 nodes if the hash is uniform. Compare with consistent hashing.
3. Classify each as CP or AP-leaning and justify: (a) account balance, (b) number of likes, (c) distributed lock, (d) shopping cart.
4. Three services in series have availability 99.95%, 99.9% and 99.99%; the middle one is made redundant with two independent replicas. What is the new overall figure?

Answers.

1. (a) R = 1, W = 5: reads tolerate 4 failures, writes tolerate none. (b) R = 5, W = 1 (writes tolerate 4 failures; reads none; risk of loss). (c) R = W = 3: both tolerate 2 failures.
2. A key keeps its node only if hash mod 10 equals hash mod 11, which happens for about 1 in 11 keys (a little more by small-number effects), so roughly 90% move. Consistent hashing moves about 1/11 ≈ 9%.
3. (a) CP: wrong balances are unacceptable. (b) AP: slightly stale counts are fine. (c) CP: two holders of a lock is the failure. (d) AP: a cart can merge copies (Dynamo's classic example).
4. The middle pair: $1 - 0.001^2 = 0.999999$. Overall ≈ 0.9995 × 0.999999 × 0.9999 ≈ 0.9994, i.e. 99.94% (from 99.84% before).

CHAPTER SUMMARY

- During a partition choose consistency or availability; partitions cannot be avoided; PACELC adds the latency-versus-consistency choice in normal operation.
- Models: linearizable (single copy illusion), causal, eventual; session guarantees give cheap, user-visible promises.
- Quorums (R + W > N) give overlap but not a full guarantee; AP systems need conflict handling (LWW, version vectors, CRDTs) and anti-entropy.
- Fail-over (active-passive, active-active) and replication provide availability; numbers multiply in series; independence is essential.
- Consistent hashing with virtual nodes limits data movement; hot partitions need caching, replication or key splitting.

REVISION CHECKLIST

- [ ] I can explain CAP with a partition diagram.
- [ ] I can define strong, causal and eventual consistency and the session guarantees.
- [ ] I can pick R and W for N = 3 and 5 and explain the limits of R + W > N.
- [ ] I can draw a consistent-hash ring with vnodes.
- [ ] I can explain active-passive versus active-active.

PART IV

Performance and Asynchrony

Making reads fast with caches and making slow work invisible with queues.

CHAPTER 12

Caching

Derived from your notes: Notebook chapter 8 (why cache, caching levels, what to cache, the four update strategies with their drawbacks, cache-aside and write-through and write-behind and refresh-ahead diagrams). **Added in this book:** hit-ratio mathematics, eviction policies, cache stampede, penetration and avalanche remedies, hot keys, stale-read analysis, and sizing.

LEARNING OBJECTIVES

After this chapter you can:

- compute the effect of a cache's hit ratio on average latency and on backend load;
- name the layers at which data can be cached and what each is good for;
- implement and compare cache-aside, write-through, write-behind and refresh-ahead;
- choose an eviction policy and a TTL, and explain why caching never removes stale reads;
- prevent cache stampede, penetration and avalanche, and handle hot keys.

PREREQUISITE KNOWLEDGE

Chapter 3 (hash maps, LRU idea), Chapter 4 (latency numbers), Chapter 8 (database reads).

KEY TERMINOLOGY

Cache
A fast, usually smaller store that keeps copies of data so future requests are served faster than from the original source.

Hit ratio
The fraction of requests answered from the cache: hits / (hits + misses).

TTL (time to live)
How long an entry may stay in the cache before it is considered expired.

Eviction
Removing entries to make room when the cache is full.

Invalidation
Removing or updating cached data because the source has changed.

Cache stampede (thundering herd)
Many requests simultaneously miss on the same key and all hit the backend.

12.1 Why caching works

Reading from memory takes about 100 nanoseconds; a database query over the network takes milliseconds, a difference of ten thousand times or more (Chapter 4). Caching exploits a property of real workloads: **locality**. A small fraction of items receives most requests (popular videos, the current news), and recently used items are likely to be used again. Keep those in fast memory and most requests never touch the slow store.

The benefit is quantified by the **hit ratio** h. If a hit costs t_c and a miss costs $t_c + t_s$ (check the cache, then the source), the average latency is

$$\text{average} = t_c + (1 - h) \times t_s.$$

WORKED EXAMPLE

What a cache buys. Cache hit: 1 ms (network call to a cache server). Database read: 20 ms. Peak traffic: 10,000 reads per second.

- No cache: every read costs 20 ms and the database sees 10,000 reads/s.
- h = 90%: average = 1 + 0.10 × 20 = 3 ms; database load = 1,000 reads/s.
- h = 99%: average = 1 + 0.01 × 20 = 1.2 ms; database load = 100 reads/s.

Going from 90% to 99% (a modest-sounding 9 points) cuts database load **tenfold** and makes the system far more tolerant of a database slowdown. Conversely, a cache with a 50% hit ratio saves only half the load and adds a component to operate: small hit-ratio changes at the high end matter enormously. That is why teams track the hit ratio as a first-class metric.

Caching helps only when data is **read much more often than it changes**, requests **repeat**, and **some staleness is acceptable**. It does not help write-heavy workloads, unique one-off queries, or data that must always be perfectly current.

12.2 Where caches live



Figure 12.1 Caching layers between a user and the database. A request is answered at the first layer that holds a valid copy; each layer further left is faster and closer to the user, but less shared.

Table 12.1 Cache layers, from the user inward.

Layer	What it holds	Strength	Limits
Client (browser, app)	Files, API responses (via HTTP cache headers)	Zero network cost	Per user; hard to invalidate
CDN	Static and cacheable responses near users	Offloads origin, low latency globally	Staleness, purge cost (Chapter 7)

Layer	What it holds	Strength	Limits
Reverse proxy / web server	Rendered pages, static files	Protects application servers	Per-server unless shared
Application (in-process)	Objects in the server's memory	Fastest (no network)	Duplicated per server; lost on restart; inconsistent across servers
Distributed cache (Redis, Memcached)	Shared objects and query results	Shared by all servers, large	Network hop; another system to run
Database buffers	Hot pages and query plans	Automatic	Not under your control

A useful rule: cache **as close to the user as correctness allows**, and use a **shared distributed cache** for data that many application servers need, because in-process caches multiply memory use and make servers disagree.

What to cache

- **Query-level**: hash the SQL (plus parameters) as the key and store the result set. Simple, but hard to invalidate: when one row changes you must know which cached queries it affects, so keys often rely on short TTLs.
- **Object-level**: assemble the data into the application's own structure (a user profile, a rendered fragment) and cache *that*. Easier to invalidate (update the object by id) and lets several servers share the same assembled object. Usually preferable.

Good candidates: user sessions, fully rendered pages and fragments, activity streams, user graph data, product catalogue entries, results of expensive computations. Poor candidates: highly personalised, rapidly changing, or security-sensitive data unless keyed carefully.

12.3 Update strategies

The hard question is *when and how the cache gets its data and stays in step with the source*. There are four standard strategies.

Cache-aside (lazy loading)

The application manages the cache. On a read it checks the cache; on a **miss** it reads the database, stores the result in the cache, and returns it. On a write it updates the database and **deletes** (invalidates) the cache entry so the next read reloads it.

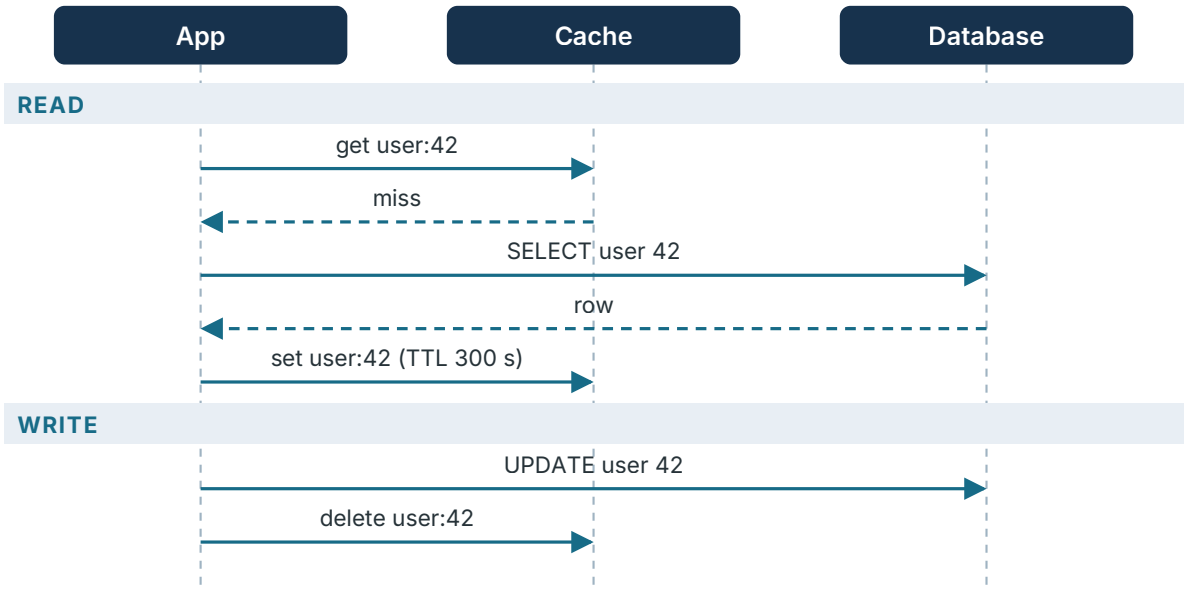


Figure 12.2 Cache-aside. A hit returns immediately; a miss reads the database, fills the cache, then returns. A write goes to the database and invalidates the cached entry.

```
def get_user(user_id):
    key = f"user:{user_id}"
    value = cache.get(key)
    if value is None:                       # miss
        value = db.query_user(user_id)      # slow path
        if value is not None:
            cache.set(key, value, ttl=300) # fill; TTL bounds staleness
    return value

def update_user(user_id, fields):
    db.update_user(user_id, fields)         # source of truth first
    cache.delete(f"user:{user_id}")         # invalidate; next read reloads
```

Advantages: only requested data is cached; a cache failure degrades speed, not correctness (the database still answers). **Disadvantages:** the first request is slow (a *cold start*, so caches may be *warmed* in advance); stale data is possible between a database change and expiry or invalidation; and the invalidate-after-write has a **race**: a reader that missed just before the write may refill the cache with the *old* value after the delete, leaving stale data until the TTL. A short TTL bounds the damage.

Write-through

The application writes to the **cache**, and the cache synchronously writes to the database. Data in the cache is always current.

Advantages: reads are fast and fresh; simple reasoning. **Disadvantages:** every write has the latency of *both* stores; most written data may never be read, wasting cache space (mitigate with a TTL); a new node starts with an empty cache.

Write-behind (write-back)

The application writes to the cache, which acknowledges immediately; the cache **asynchronously** writes to the database later, often batching and coalescing many writes to the same key.

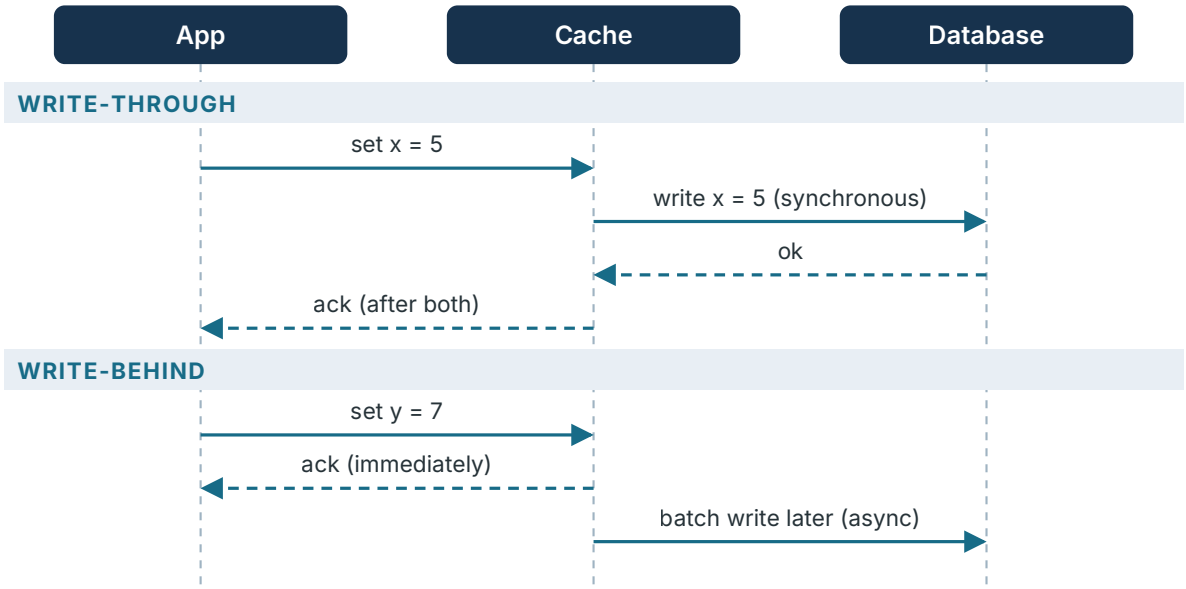


Figure 12.3 Write-through (left) writes to cache and database before acknowledging. Write-behind (right) acknowledges after the cache and flushes to the database later.

Advantages: very fast writes, fewer database writes, absorbs bursts. **Disadvantages: data loss** if the cache fails before flushing (the only copy of recent writes is in memory); complex to implement correctly (ordering, retries, failure handling); reads from the database directly may see stale data.

Refresh-ahead

The cache **proactively reloads** entries that are likely to be requested again, shortly *before* they expire, so popular items never cause a miss. **Advantages:** lower latency than lazy loading for hot data. **Disadvantages:** needs a good prediction of what will be requested; wrong guesses waste reads and cache space.

Table 12.2 Update strategies compared.

Strategy	Read latency	Write latency	Staleness	Data-loss risk	Complexity
Cache-aside	Miss: slow; hit: fast	Fast (DB only)	Until TTL or invalidation	None	Low
Write-through	Fast	Slow (both)	None	None	Medium
Write-behind	Fast	Fastest	Database lags the cache	**Yes**	High
Refresh-ahead	Fastest for hot data	n/a	Bounded	None	High

CLARIFICATION

Clarification: caching does not remove stale reads. The notebook lists the *drawbacks* of each strategy; the common thread deserves stating plainly. Unless every read goes through a single linearizable path, a cache holds a *copy*, and a copy can lag the original. Cache-aside is stale until invalidation or TTL; write-through is current only for writes that go through it; any second writer (another service, a batch job, a manual fix) makes it stale. Design for the staleness you can tolerate: choose TTLs deliberately, version keys, and never cache data whose staleness causes harm unless you also check the source of truth at decision points (for example, re-read the balance from the database before charging a card).

12.4 Eviction policies

A cache is smaller than the data, so it must decide what to throw away.

Table 12.3 Eviction policies.

Policy	Evicts	Good for	Weak spot
LRU (least recently used)	The entry unused for longest	General workloads with temporal locality	A one-time large scan evicts the hot set
LFU (least frequently used)	The entry used fewest times	Stable popularity	Old popular items linger; needs ageing
FIFO	The oldest entry	Simplicity	Ignores usage
Random	A random entry	Cheap; surprisingly decent	Unpredictable
TTL	Entries past their lifetime	Bounding staleness	Not a space policy by itself

LRU is the usual default: a hash map plus a doubly linked list gives O(1) lookup, update and eviction (a design exercise in Chapter 30). Real systems add refinements such as **segmented LRU** or **TinyLFU** admission to resist scans. Combine eviction (for space) with **TTL** (for freshness).

12.5 Failure patterns and remedies

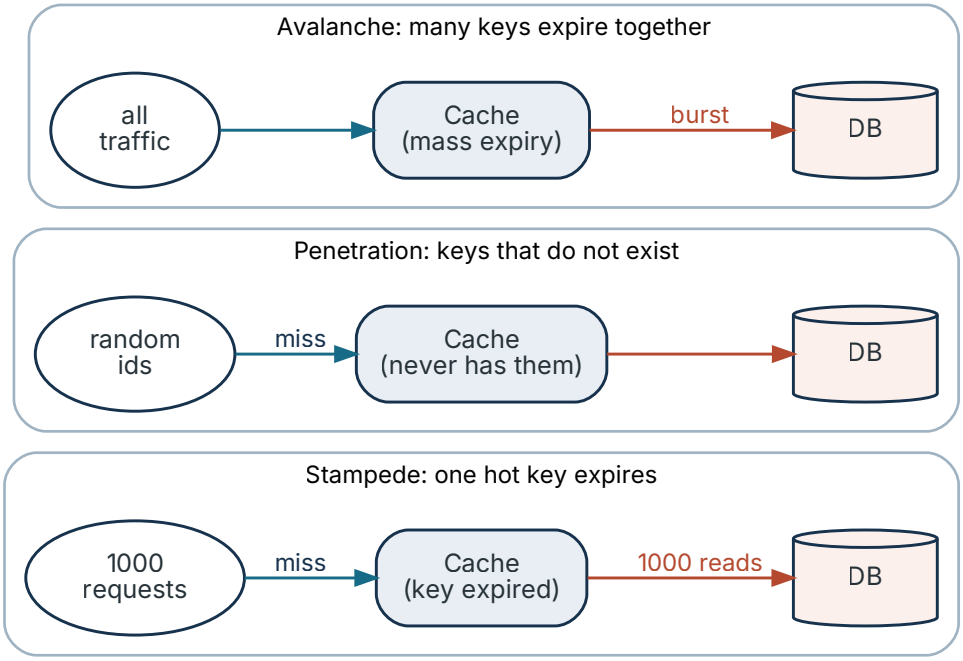


Figure 12.4 Three ways a cache can hurt the backend: a stampede on one expired hot key, penetration by keys that do not exist, and an avalanche when many keys expire together.

Cache stampede (thundering herd). A hot key expires; hundreds of concurrent requests all miss and all query the database for the same data, possibly overwhelming it. Remedies: **single-flight / request coalescing** (the first miss takes a lock or marker and loads; others wait for its result); **stale-while-revalidate** (serve the expired value briefly while one request refreshes it); **probabilistic early refresh** (each request near expiry refreshes with a small random chance, so refresh is spread out); and **refresh-ahead** for hot keys.

Cache penetration. Requests for keys that *do not exist* (random ids, or an attack) miss every time and always reach the database. Remedies: **negative caching** (cache the "not found" result for a short TTL), a **bloom filter** of valid keys in front of the cache (Chapter 3), and input validation and rate limiting.

Cache avalanche. Many keys expire at the same moment (all given the same TTL at startup), or the cache cluster restarts empty, sending the full load to the database at once. Remedies: **add random jitter** to TTLs (for example 300 s ± 10%), warm the cache gradually, keep cache clusters redundant, and protect the database with rate limits and load shedding (Chapter 15).

Hot key. One key gets so much traffic that its single cache node saturates. Remedies: a small in-process **local cache** in front of the distributed cache with a very short TTL; **replicate** the key under several names (`key#1..#8`) and read a random one.

```
# single-flight sketch: one loader per key, others wait for its result
inflight = {}                       # key -> future (per process)
def get_with_coalescing(key, loader, ttl):
    v = cache.get(key)
    if v is not None: return v
    fut = inflight.get(key)
    if fut is None:
        fut = inflight.setdefault(key, Future())
        if fut.owner_is_me():       # first caller loads
            try:
                v = loader(); cache.set(key, v, ttl=ttl + jitter())
                fut.set_result(v)
            finally:
                inflight.pop(key, None)
    return fut.result(timeout=2)
```

SUPPLEMENTAL MATERIAL

Sizing a cache. Use the 80/20 heuristic as a first guess: if 20% of items cause 80% of reads, size the cache to hold that 20%. Better, measure the hit ratio at several sizes (a *miss-ratio curve*): memory is usually wasted beyond the knee of the curve. Account for per-entry overhead (keys, metadata can double the memory for small values), replication (a cache replica doubles memory), and headroom for peaks. Consider **serialisation cost** and entry size: caching a 5 MB object per request wastes bandwidth even on hits.

COMMON MISTAKE

Common mistakes. Treating the cache as the source of truth (it can be flushed at any time). Caching without a TTL and relying on perfect invalidation. Using the same TTL everywhere. Caching errors ("database timeout") as if they were values. Putting per-user data under a shared key (leaking private data between users). Forgetting that serialisation formats change between deployments, so old cached bytes break new code (include a version in the key).

FAILURE MODES

Failure modes. Cache node loss: hit ratio collapses and the database is hit by the full load (design the database for the *miss* load, or have a fallback); network partition between application and cache (decide whether to fail open, reading the database directly, or fail closed); eviction storms when memory is tight; inconsistency between two cache layers (browser, CDN and Redis holding different versions); stale data after a missed invalidation event; thundering herd on restart.

COMMON MISCONCEPTION

- "*A cache makes the system consistent faster.*" It adds another copy that can be stale.
- "*Write-behind is just write-through but faster.*" It can lose acknowledged data.
- "*Bigger cache always means better.*" Beyond the knee the hit ratio barely moves.
- "*Invalidate on write, so stale reads are impossible.*" The refill race and multiple writers still allow them.

INTERVIEW QUESTIONS

Q1. Explain cache-aside and its drawbacks. Application checks the cache, loads from the database on a miss and fills the cache; writes go to the database and delete the entry. Drawbacks: cold-start misses, staleness between change and invalidation, and a refill race; mitigate with TTLs and warming.

Q2. What is a cache stampede and how do you prevent it? Many simultaneous misses on one expired hot key overload the database. Use request coalescing, stale-while-revalidate, jittered TTLs or early probabilistic refresh.

Q3. Cache-aside versus write-through versus write-behind? Cache-aside: simple and safe, possibly stale; write-through: fresh, slower writes; write-behind: fastest writes, risk of loss and complexity.

Q4. How do you keep cache and database consistent? You cannot make them perfectly consistent cheaply; bound staleness with TTLs and invalidation, write the database first and then delete the cache entry, version keys, and re-check the database for critical decisions.

EXERCISES

1. Cache hit costs 2 ms, database 30 ms, hit ratio 95%. Compute average latency and the database load at 8,000 requests per second.
2. 100,000 keys are loaded at startup with a 600-second TTL. What problem appears at 10 minutes and what is the fix?
3. Give a timeline showing the cache-aside refill race, then one mitigation.
4. For each datum say which caching layer(s) and update strategy you would use: (a) company logo, (b) a user's profile page, (c) a bank balance, (d) a leaderboard updated 5,000 times a second.

Answers.

1. Average = 2 + 0.05 × 30 = 3.5 ms. Database load = 5% × 8,000 = 400 per second.
2. An avalanche: all keys expire together and the database receives a burst of 100,000 loads. Add random jitter to TTLs and warm gradually.
3. Reader R1 misses and reads old value v1 from the database; writer W updates the database to v2 and deletes the cache key; R1 then writes v1 into the cache. The cache holds stale v1 until its TTL. Mitigations: short TTLs, writing with a version check (set only if the version is newer), or deferred double-delete.
4. (a) Browser and CDN with a long TTL and versioned URL. (b) Distributed cache with cache-aside and a short TTL (plus maybe a CDN for the public parts). (c) Do not cache for decisions; read the database (possibly cache for display only, with a very short TTL). (d) Write-behind into an in-memory sorted structure with periodic persistence, accepting a small loss window, or cache-aside for reads with batched database updates.

CHAPTER SUMMARY

- Caching exploits locality; hit ratio drives latency and backend load dramatically at the high end.
- Cache at multiple layers; prefer object-level entries in a shared distributed cache for application data.
- Cache-aside is the safe default; write-through gives fresh reads with slower writes; write-behind is fast but can lose data; refresh-ahead helps hot keys.
- Evict with LRU or similar; bound freshness with TTL and jitter.
- Defend against stampede, penetration, avalanche and hot keys; never assume the cache is the source of truth, and never assume it is perfectly fresh.

REVISION CHECKLIST

- [ ] I can compute average latency from a hit ratio.
- [ ] I can write cache-aside code and name its race.
- [ ] I can compare the four update strategies in a table from memory.
- [ ] I can name three cache failure patterns and a remedy for each.

SUPPLEMENTAL MATERIAL

Further reading. The Redis and Memcached documentation on eviction and expiry; Facebook's "Scaling Memcache at Facebook" (NSDI 2013) for leases and stampede control; the Caffeine library's TinyLFU design notes; RFC 5861 for stale-while-revalidate.

CHAPTER 13

Asynchronous Processing

Derived from your notes: Notebook chapter 9 (asynchronous workflows, message queues, task queues, back pressure, the broker comparison table). **Added in this book:** delivery guarantees and the limits of exactly-once, publish/subscribe and consumer groups, retries and dead-letter queues, idempotent consumers, the transactional outbox and sagas.

LEARNING OBJECTIVES

After this chapter you can:

- decide which work belongs on the request path and which should be done later;
- explain queues, topics, consumer groups and acknowledgements, and the difference between a queue and a publish/subscribe log;
- explain at-most-once, at-least-once and "exactly-once" delivery, and why the last claim needs care;
- design retries with backoff, jitter, dead-letter queues and idempotent consumers;
- apply back pressure, the transactional outbox and sagas.

PREREQUISITE KNOWLEDGE

Chapter 6 (idempotency keys), Chapter 8 (transactions), Chapter 11 (consistency).

KEY TERMINOLOGY

Message
A small, self-contained piece of data describing work to do or an event that happened.

Producer / consumer
The program that puts messages on a queue, and the program that takes them off and processes them.

Broker
The server that stores messages durably and hands them to consumers (RabbitMQ, SQS, Kafka, and others).

Acknowledgement (ack)
A consumer's signal to the broker that a message was processed and can be removed.

Dead-letter queue (DLQ)
A holding queue for messages that repeatedly fail, so they do not block others.

Back pressure
Slowing or refusing producers when consumers cannot keep up, instead of letting work pile up without limit.

13.1 Why work asynchronously

A **synchronous** operation makes the caller wait for completion. An **asynchronous** one accepts the request, promises to do it, and lets the caller continue. Asynchrony helps in three ways:

1. **Response time.** Do only what the user must see immediately; do the rest later. Posting a photo returns in 100 ms; thumbnails, notifications and indexing happen in the next seconds.
2. **Smoothing bursts.** A queue absorbs a spike so servers work at a steady pace instead of being overwhelmed (a *buffer* between producers and consumers).
3. **Decoupling and resilience.** If the email service is down, messages wait in the queue instead of failing the user's request; producers and consumers can be deployed and scaled independently.

The notebook names two flavours. **Precompute** work that can be done ahead of time (nightly aggregation, pre-rendered pages, a user's timeline); and **defer** slow work behind a queue. The price is real: results are no longer instantly visible (*eventual consistency*), the user may need to be told "we are working on it", failures happen where nobody is watching, and debugging spans several components.

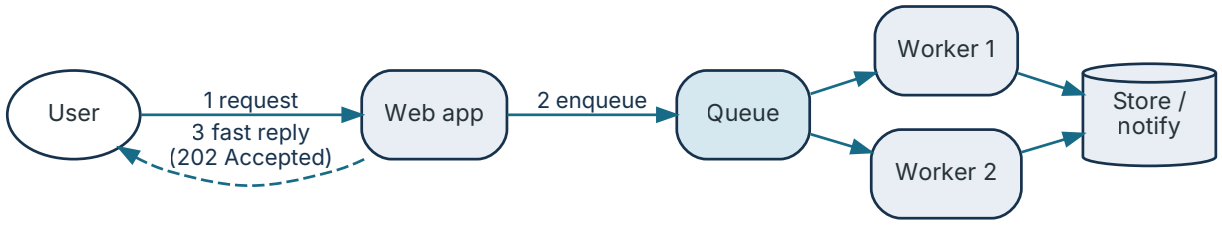


Figure 13.1 A request returns quickly after placing a message on a queue. Workers process messages later at their own pace; the user learns the result by polling, push notification or a refreshed page.

13.2 Message queues

A **message queue** stores messages between a producer and a consumer. The producer sends and moves on. The broker persists the message. A consumer receives it, does the work, and **acknowledges**; only then does the broker delete it. If the consumer crashes before acknowledging, the broker makes the message available again after a **visibility timeout**, so another consumer retries it. Several consumers can read from one queue, sharing the work (**competing consumers**); that is the basis of horizontal scaling of workers.

```
# producer (web request)
queue.send({"type": "resize_image", "photo_id": 981, "request_id": "r-77"})
return 202  # Accepted: work will happen later

# consumer (worker)
while True:
    msg = queue.receive(wait_seconds=20)       # long polling
    if msg is None: continue
    try:
        resize(msg.body["photo_id"])
        queue.ack(msg)                          # delete only after success
    except TemporaryError:
        pass                                    # no ack: it reappears later
```

Ordering. A single queue with one consumer processes messages in order; with several consumers it does not. When order matters per entity (all events for order 17 must stay in sequence), partition messages by a key so that all messages with one key go to one consumer.

Delivery guarantees

Because networks and processes fail, a broker cannot always know whether a consumer finished.

- **At-most-once**: the message is delivered zero or one times. Acknowledge *before* processing; if the worker crashes, the message is lost. Fine for metrics sampling, not for payments.
- **At-least-once**: delivered one or more times. Acknowledge *after* processing; if the worker crashes after the work but before the ack, the message is redelivered and the work repeats. This is the usual default.
- **Exactly-once**: processed precisely once.

CLARIFICATION

Clarification: the limits of "exactly-once". Exactly-once *delivery* between independent systems over an unreliable network is impossible to guarantee in general: when an acknowledgement is lost, the sender cannot tell whether the receiver acted. What products call "exactly-once" usually means **exactly-once *effect* within a bounded scope**: at-least-once delivery combined with **deduplication** or **idempotent processing**, or atomic transactions that commit a consumer's offset and its output together inside the same system (as some streaming platforms support). The guarantee stops at the system's boundary: if the consumer also sends an email or calls a payment provider, that external action can still happen twice. The practical rule: **assume at-least-once delivery and make every consumer idempotent.**

13.3 Publish/subscribe and logs

A queue gives each message to *one* consumer. In **publish/subscribe (pub/sub)**, a producer publishes an event to a **topic**, and **every subscriber** receives a copy. A single "order placed" event can notify billing, shipping, analytics and email independently, and the producer does not know who listens (**decoupling**).

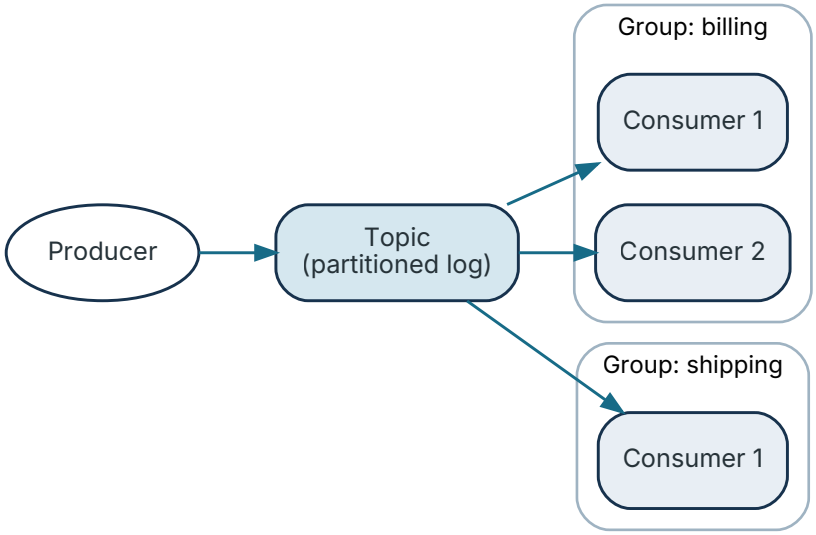


Figure 13.2 A queue delivers each message to one consumer in a group; pub/sub delivers each message to every subscribing group. Within a group, consumers share the work.

Systems such as Kafka combine both with **consumer groups** over a **partitioned, append-only log**. Messages are appended to partitions and kept for a retention period. Each consumer group tracks its own **offset** (position) per partition; consumers within a group split the partitions among themselves; a different group reads independently, and can even **replay** history by resetting its offset. Ordering is guaranteed *within a partition*, so the partition key (for instance the user id) determines ordering and parallelism.

Table 13.1 Types of messaging systems (from the notebook, extended).

Kind	Examples	Model	Strength	Watch out
Simple queue service	Amazon SQS, Redis lists	Queue, competing consumers	Easy, managed	Limited ordering; at-least-once, duplicates
Feature-rich broker	RabbitMQ, ActiveMQ	Queues and exchanges, routing, acks	Flexible routing, priorities	Throughput limits; operations
Distributed log	Kafka, Pulsar, Kinesis	Partitioned append-only log, groups, replay	High throughput, replay, ordering per partition	More concepts; partition design
Task queue frameworks	Celery, Sidekiq, Bull	Job queue with scheduling and retries	Developer-friendly	Built on a broker; check its guarantees

Task queues (the notebook's term) are queues specialised for *jobs*: they receive a task and its data, run it, and report the result. They add scheduling (run at 03:00), priorities, retries, rate limiting and result tracking, and suit CPU-heavy or slow work such as video encoding or report generation.

13.4 Retries, dead letters and idempotent consumers

Failures come in two kinds. *Transient* (a timeout, a briefly unavailable database): retrying will probably succeed. *Permanent* or **poison** messages (malformed data, a bug): retrying will never succeed and can block the queue forever. Handle both.

- **Retry with exponential backoff and jitter**: wait 1 s, then 2, 4, 8, … up to a maximum, each multiplied by a random factor, so a thousand failed jobs do not all retry in the same instant (*retry storm*).
- **Cap the attempts** (for example 5), then move the message to a **dead-letter queue** with the error, so that engineers can inspect it, fix the bug and **redrive** it. Alert on DLQ depth: a growing DLQ means something is wrong.
- **Make consumers idempotent**, because at-least-once delivery *will* produce duplicates. Techniques: a natural idempotency (setting a field to a value, not incrementing it); a **processed-message table** with a unique key on the message id, inserted in the *same transaction* as the work; **conditional writes** (apply only if the version matches); or deduplication windows at the broker.

```
def handle(msg):
    with db.transaction():
        if not db.insert_if_absent("processed", id=msg.id):   # unique constraint
            return                                             # duplicate: ignore
        apply_business_change(msg)                             # same transaction
    queue.ack(msg)                                             # ack last
```

Note the order: the *dedup record and the business effect commit together*, and the ack comes last. If the worker dies after commit but before ack, the redelivery finds the record and does nothing.

13.5 Back pressure

If producers add work faster than consumers finish it, an unbounded queue grows until memory or disk is exhausted, and the wait in the queue grows with it: by the time work is done, nobody wants the answer. **Back pressure** pushes the problem back to the source. Bound the queue; when it is full, **refuse new work** quickly (for example HTTP `503 Service Unavailable` with `Retry-After`) and let clients retry with backoff; or slow producers down (blocking sends, credit-based protocols); or **shed** low-priority work first.

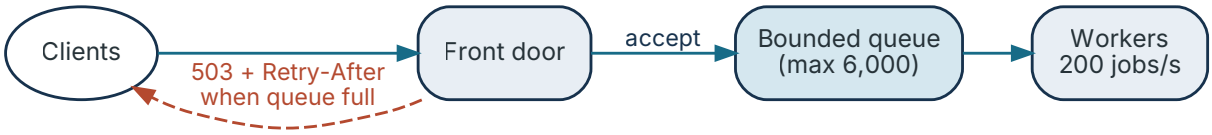


Figure 13.3 A bounded queue with back pressure. When the queue is full the front door rejects new requests quickly, keeping the accepted ones healthy, instead of accepting everything and failing slowly.

WORKED EXAMPLE

How big a queue? Workers complete 200 jobs per second. Users should wait no longer than 30 seconds. A queue longer than 200 × 30 = 6,000 jobs would make the *last* job wait over 30 seconds, so a bound of about 6,000 is the maximum useful size. If a burst offers 1,000 jobs per second for 20 s (20,000 jobs) while workers do 200 per second, the queue fills in 6,000 / (1,000 − 200) = 7.5 s; after that, producers get 503s. The remedies, in order: add workers (autoscale on queue age), shed low-priority jobs, or accept that some users are asked to retry. An unbounded queue would instead have grown to 16,000 and delayed everyone by 80 seconds. Use **queue age (time the oldest message has waited)** as the key metric, more telling than depth.

Back pressure trades **availability for some users** for **latency and survival for the rest**. A rejected request that can be retried is usually better than an accepted one that times out.

13.6 Consistency across services: outbox and sagas

SUPPLEMENTAL MATERIAL

A very common bug: the service updates its database *and* publishes an event, as two separate steps. If it crashes between them, the database says "order placed" but no event goes out, or an event is published for an order that was rolled back. A database transaction cannot span the database and the broker.

The transactional outbox pattern fixes this. In **one local transaction** the service writes the business change *and* a row in an `outbox` table describing the event. A separate **relay** process reads unpublished outbox rows and publishes them to the broker, marking them sent. Delivery is at-least-once (the relay may publish, crash before marking, and publish again), so consumers deduplicate, as above. A variant uses **change data capture (CDC)**: tailing the database's write-ahead log to publish changes without a polling relay.

A **saga** coordinates a business workflow that spans several services without a distributed transaction. It is a sequence of local transactions, each with a **compensating action** that undoes it if a later step fails. For an order: reserve stock, charge payment, schedule shipping; if payment fails, release the stock. Sagas can be **choreographed** (each service reacts to the previous event) or **orchestrated** (a central coordinator tells each service what to do, which makes the flow easier to see and debug). Sagas give *eventual* consistency, not isolation: other users may briefly see intermediate states, so design the business process to tolerate it ("order pending"). Compensations must themselves be idempotent and retried until they succeed.

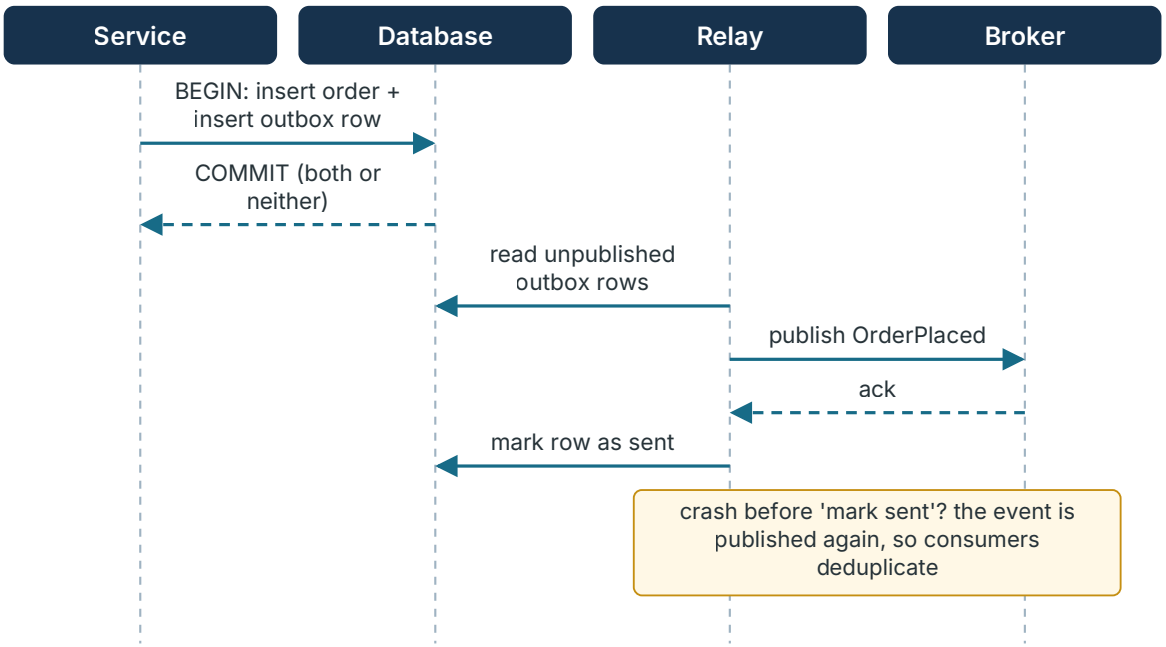


Figure 13.4 Transactional outbox. The business row and the outbox row commit atomically; a relay publishes the outbox rows to the broker with at-least-once delivery.

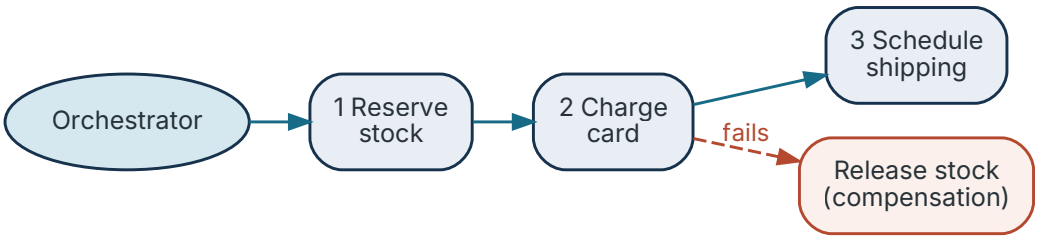


Figure 13.5 An orchestrated saga for an order. If charging the card fails, the orchestrator runs the compensating action for the stock reservation.

FAILURE MODES

Failure modes. A poison message blocking a queue; duplicate processing because acknowledgements were lost; lost messages because the consumer acknowledged before finishing; a growing queue (consumers slower than producers) that hides outages until the delay is hours; retry storms amplifying an outage; out-of-order processing corrupting state; a DLQ nobody watches; broker disk full or broker outage (decide whether producers buffer, fail, or fall back); a rebalance of consumers causing duplicate processing; workers that crash on startup in a loop.

COMMON MISTAKE

Common mistakes. Treating a queue as a database. Acknowledging before the work is done. Assuming messages arrive once or in order. Unbounded retries or unbounded queues. Publishing events inside a database transaction without an outbox. Putting large payloads in messages (store the file in an object store and send its key).

COMMON MISCONCEPTION

- "*A queue makes the system reliable.*" It makes *slow or failing consumers* tolerable; it adds duplicates, ordering questions and a new component.
- "*Kafka (or any broker) gives exactly-once, so I need no idempotency.*" Only within narrow boundaries; external side effects still need idempotency.
- "*Async is always faster.*" Latency to *completion* is usually longer; only the user-visible response time improves.
- "*Dead-letter queues are a place to forget failures.*" They are alarms; someone must own them.

INTERVIEW QUESTIONS

Q1. When would you use a queue? For work the user need not wait for, to absorb bursts, to decouple a slow or unreliable dependency, and to scale workers independently. Not when the user needs the result immediately and the work is fast.

Q2. How do you handle duplicate messages? Assume at-least-once delivery; make consumers idempotent using a processed-ids table committed in the same transaction as the effect, conditional updates, or naturally idempotent operations.

Q3. What is back pressure? A mechanism that refuses or slows producers when consumers cannot keep up, bounding queues and latency: 503 with Retry-After, blocking producers, or shedding low-priority work.

Q4. How do you update the database and publish an event reliably? Transactional outbox (or CDC): write business data and an outbox row in one transaction, then a relay publishes it at least once; consumers deduplicate.

EXERCISES

1. A queue receives 500 messages/s; each worker handles 50 messages/s. How many workers keep up, and how many with 40% headroom?
2. Design the retry policy for a job that calls a flaky email API: delays, attempts, jitter, DLQ.
3. Explain precisely how a lost acknowledgement leads to a duplicate and show the deduptable fix.
4. Sketch a saga for "book flight, book hotel, charge card" with compensations.

Answers.

1. 500 / 50 = 10 workers just to keep up; with 40% headroom, 10 / 0.6 ≈ 17 workers.
2. Example: up to 6 attempts with delays of about 2, 4, 8, 16, 32 s, each multiplied by a random factor between 0.5 and 1.5; classify 4xx responses as permanent (no retry); after the last attempt send to the DLQ with the error and alert; redrive after the API is fixed.
3. The worker finishes the effect and sends the ack; the ack is lost (network failure). The broker's visibility timeout expires and it redelivers the message. With a unique `processed(message_id)` row inserted in the same transaction as the effect, the second attempt's insert fails or finds the row, so the effect is skipped and the message is acknowledged.

4. Steps: book flight, book hotel, charge card. Compensations: cancel flight; cancel hotel; refund card. If the charge fails, run "cancel hotel" then "cancel flight"; each compensation is idempotent and retried until it succeeds.

CHAPTER SUMMARY

- Asynchrony shortens user-visible latency, smooths bursts and decouples services, at the cost of eventual results and more moving parts.
- Queues hand each message to one consumer; pub/sub and logs deliver to every subscribing group; ordering is per partition.
- Assume at-least-once delivery; exactly-once effects come from idempotent consumers; external side effects remain a boundary.
- Retry with backoff and jitter, cap attempts, use and monitor dead-letter queues.
- Bound queues and apply back pressure; watch queue age.
- Use the outbox for atomic state-plus-event and sagas for multi-service workflows.

REVISION CHECKLIST

- [ ] I can explain at-most-once, at-least-once and the limit of exactly-once.
- [ ] I can write an idempotent consumer.
- [ ] I can compute a sensible queue bound.
- [ ] I can draw the outbox and a saga with compensations.

SUPPLEMENTAL MATERIAL

Further reading. Kleppmann, *Designing Data-Intensive Applications*, chapters 11 and 12; Chris Richardson, *Microservices Patterns* (outbox, sagas); the Kafka documentation on delivery semantics; Marc Brooker's articles on timeouts, retries and backoff.

PART V

Reliability in Distributed Systems

Agreeing on facts without a shared clock, and surviving failure on purpose.

CHAPTER 14

Time, Coordination and Consensus

Derived from your notes: Notebook chapter 2 (availability patterns: heartbeats, fail-over) and chapter 6 (master election in replication), and the "under development" topics the primer lists (consistent hashing, scatter-gather). **Added in this book:** the whole chapter: clocks, failure detection, uncertainty after timeouts, leader election, leases and fencing tokens, consensus, two-phase commit.

LEARNING OBJECTIVES

After this chapter you can:

- explain why clocks on different machines disagree and what to use instead of wall-clock time for ordering;
- explain Lamport and vector clocks at an intuitive level;
- explain why a timeout leaves an operation's outcome uncertain and how to design for that;
- describe leader election, leases, split brain and fencing tokens;
- explain, at the level of an interview, how consensus (Raft-style) works, what quorum sizes tolerate, and what it costs;
- compare two-phase commit with sagas.

PREREQUISITE KNOWLEDGE

Chapter 2, Chapter 9 (failover, split brain) and Chapter 11 (quorums, CAP).

KEY TERMINOLOGY

Clock skew / drift
Difference between two machines' clocks (skew) and the rate at which a clock gains or loses time (drift).

Heartbeat
A periodic "I am alive" message used to detect failure.

Lease
A time-limited grant of a right (such as being leader) that expires unless renewed.

Fencing token
A number that increases each time a lock or lease is granted; resources reject operations carrying an older token.

Consensus
Getting a group of nodes to agree on a single value (or a sequence of values) despite failures.

Term / epoch
A number identifying one era of leadership; a higher term supersedes a lower one.

14.1 Why time is hard

On one machine, "which happened first?" is easy. Across machines it is not, because each has its own clock, and clocks differ.

- **Skew and drift.** Quartz clocks gain or lose a few milliseconds a day. Protocols such as NTP (Network Time Protocol) keep clocks within roughly milliseconds to tens of milliseconds over a network, sometimes worse; they can also *jump* a clock backwards or forwards when correcting it.
- **Two kinds of clock.** A **wall-clock (time-of-day)** gives the date and time and may jump. A **monotonic clock** only moves forward and is meant for *measuring durations* (timeouts, elapsed time) on one machine; its absolute value is meaningless across machines.
- **No global "now".** Message delays are variable and unbounded, so you cannot know how old a timestamp is.

Consequences: never use wall-clock timestamps from different machines to decide *which write came last* without understanding the risk (last-write-wins can silently discard the "later" write if its machine's clock was behind). Never measure a timeout with the wall clock. And never assume a lease granted for 10 seconds is still valid after your process paused for 15 (a long garbage-collection pause or a virtual-machine freeze).

Ordering without clocks

What we usually need is not the exact time but the **order of causally related events**. Leslie Lamport's insight: if event A could have influenced event B, then A must be ordered before B; events with no such connection are *concurrent*, and any order is acceptable.

Lamport clocks give every event a counter. Each node keeps a counter, increments it on every event, attaches it to every message it sends, and on receiving a message sets its counter to max(own, received) + 1. If A causally precedes B then A's number is smaller, but the reverse does not hold (a smaller number does not prove causality). **Vector clocks** keep one counter *per node*; comparing two vectors tells you whether one event happened before the other or whether they are concurrent, which is what a store needs to detect conflicting writes (Chapter 11).

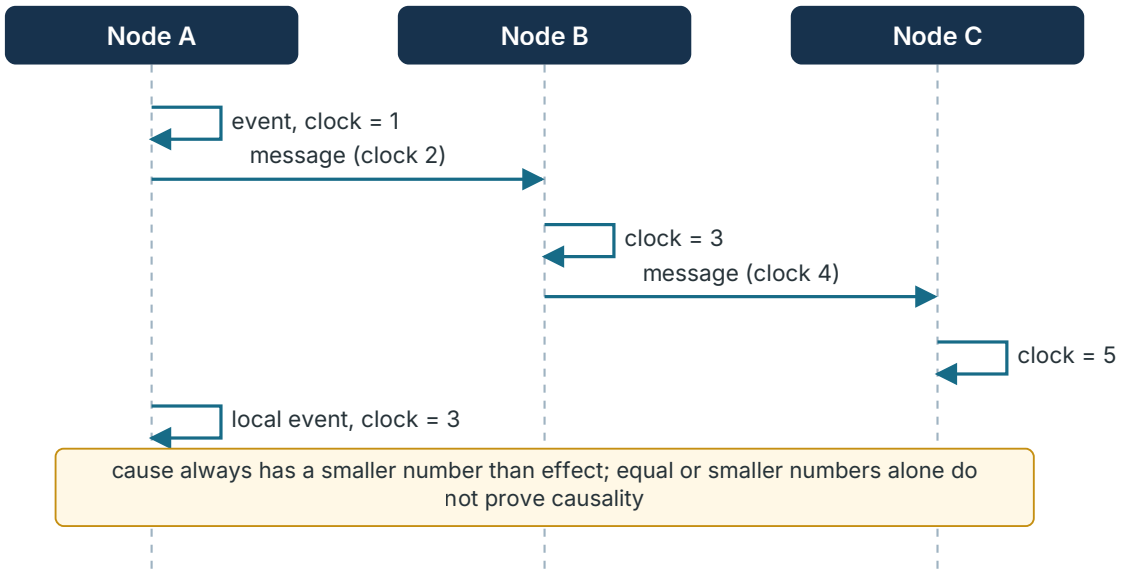


Figure 14.1 Lamport clocks. Each node increments its counter on every event; a received message pushes the receiver's counter beyond the sender's value, so cause always has a smaller number than effect.

SUPPLEMENTAL MATERIAL

Bounding uncertainty instead. Google's Spanner database uses special hardware (GPS and atomic clocks) to expose a clock API that returns an *interval* of uncertainty (typically a few milliseconds) rather than a single instant, and it **waits out** the uncertainty before committing so that transaction order matches real time. **Hybrid logical clocks** combine a physical timestamp with a logical counter, so timestamps stay close to real time yet respect causality; several distributed databases use them.

14.2 Failure detection and the uncertain timeout

How do you know a node has failed? You cannot, you can only *suspect*. The usual method is a **heartbeat with a timeout**: if no heartbeat arrives within T seconds, declare the node dead. But a missing heartbeat has several explanations: the node crashed; it is overloaded or paused; the network between you dropped the message; the network dropped the reply. From the outside these look **identical**.

This leads to a foundational lesson: **when a request times out, you do not know whether it was executed.** The server may never have received it, may be still working on it, or may have finished and had its response lost. A client that retries blindly may duplicate the action; a client that gives up may have lost it. The honest options are:

1. Make the operation **idempotent** (or protect it with an idempotency key, Chapter 6) so a retry is harmless.
2. **Query the outcome** before retrying ("did payment 123 happen?").
3. Treat the result as **unknown** and reconcile later, with a background job that compares systems.

Choosing the timeout is a trade-off: too short, and slow-but-alive nodes are wrongly removed, causing needless failovers and cascading load; too long, and real failures take a long time to notice. Adaptive detectors (the *phi accrual* detector) measure the heartbeat distribution and output a suspicion level instead of a hard yes or no.

14.3 Leader election, leases and fencing

Many designs need exactly **one** node in charge at a time: the database leader, the scheduler that must run a job once, the owner of a shard. Two problems arise: *choosing* the leader, and *making sure the old leader stops*.

The naive approach (a node that stops hearing the leader declares itself leader) produces **split brain** whenever the network partitions: both sides elect a leader and both accept writes. The remedy is the **majority rule**: a node may act as leader only if it has the support of a *majority* of a fixed group. Two disjoint groups cannot both be a majority, so at most one leader exists.

Leadership is granted as a **lease**: a time-limited right that must be renewed. If the leader stops renewing (it crashed, or is partitioned), the lease expires and another node can be elected. But leases rely on clocks and timing, and a paused leader can *wake up believing its lease is still valid*. This is where **fencing tokens** come in: each grant of the lock or lease carries a monotonically increasing number, and the *protected resource* (the storage service or database) remembers the highest number it has seen and **rejects any request carrying a lower one**.

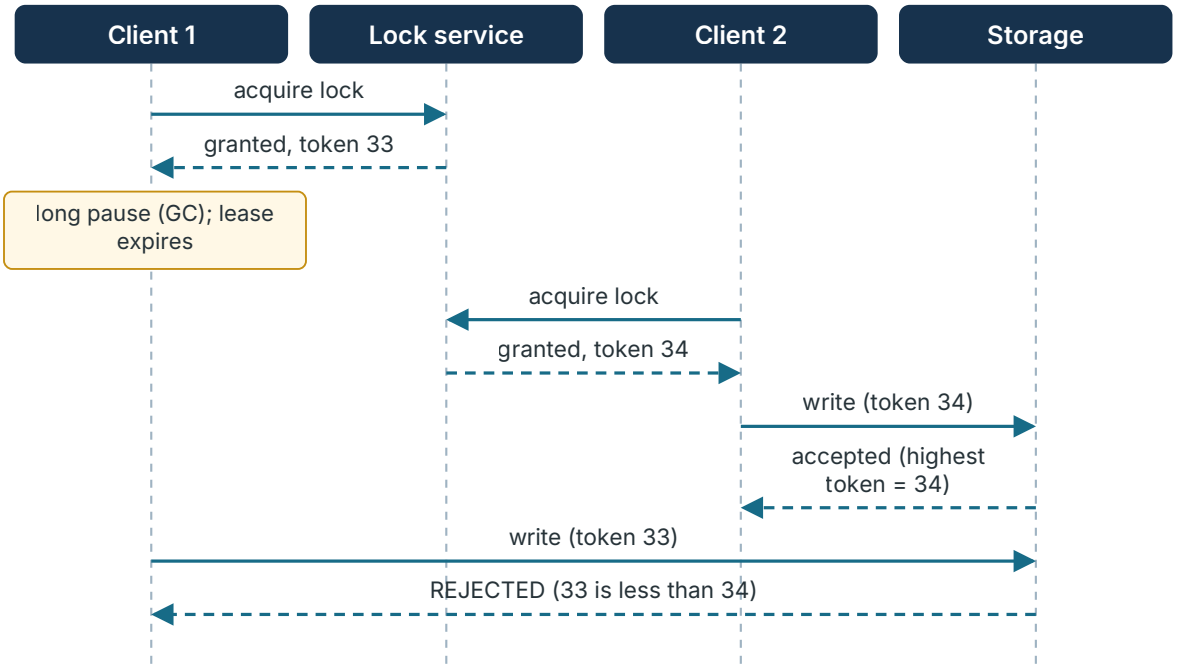


Figure 14.2 Fencing token. Client 1 gets a lock with token 33 and then stalls. The lease expires and client 2 gets token 34 and writes. When client 1 wakes and tries to write with token 33, the storage service rejects it.

Without fencing, correctness depends on timing assumptions that real systems violate. With it, even a zombie leader cannot corrupt data. The lock service alone is not enough; **the resource must check the token**.

14.4 Consensus

Consensus is the problem of getting several nodes to agree on a value (or on an ordered log of values) even if some fail and messages are delayed. It is the foundation for reliable leader election, configuration stores, distributed locks, and strongly consistent replicated databases. A famous result (**FLP impossibility**) shows that no deterministic algorithm can guarantee consensus in a fully asynchronous system with even one possible failure; practical algorithms (Paxos, Raft, Zab, Viewstamped Replication) get around it with timeouts and randomness, giving up *liveness* briefly rather than *safety* when the network misbehaves: they never agree on two different values, but they may pause.

How Raft works (intuition)

Raft is designed to be understandable. A cluster of an odd number of nodes (3, 5 or 7) has at most one **leader** per **term**.

1. **Election.** Nodes start as followers. A follower that hears nothing from a leader for a random timeout becomes a **candidate**, increments the term, and asks the others for votes. A node votes once per term, for a candidate whose log is at least as up to date as its own. A candidate with **a majority of votes** becomes leader. Random timeouts make split votes rare and quickly resolved.
2. **Log replication.** Clients send commands to the leader, which appends them to its log and sends them to followers. When a **majority** has stored an entry, the leader **commits** it, applies it, and replies to the client. Followers apply committed entries in order, so all nodes run the same sequence of commands: a **replicated state machine**.
3. **Safety.** A new leader must have all committed entries (the vote rule guarantees this), so committed data is never lost; a deposed leader's uncommitted entries are overwritten.

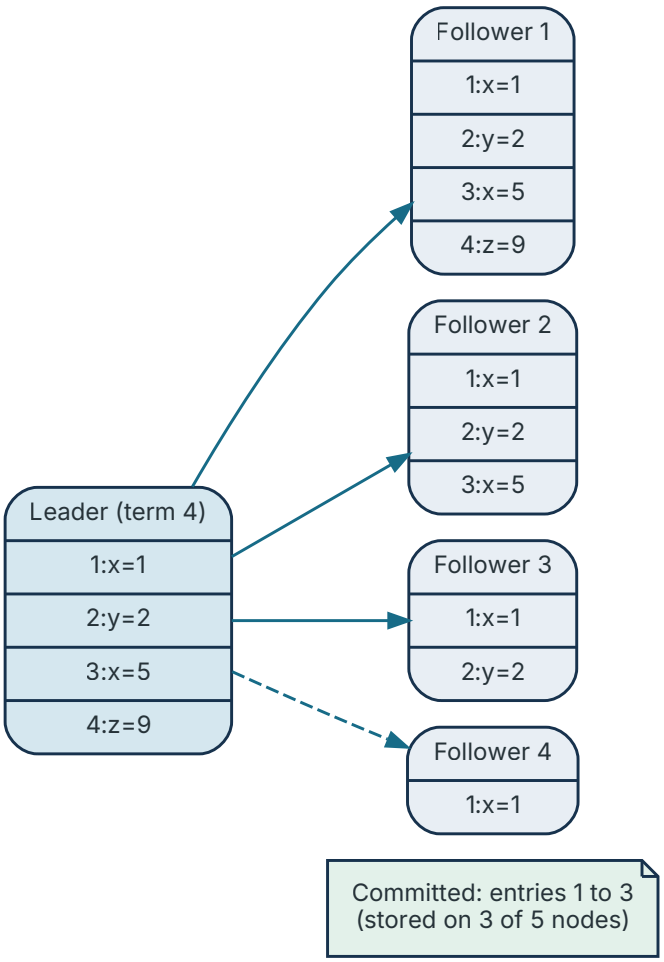


Figure 14.3 Raft in one picture. The leader replicates log entries to followers. An entry is committed once a majority has it (here 3 of 5 including the leader), so the cluster survives the loss of 2 nodes.

Table 14.1 Cluster size and fault tolerance for majority-based consensus.

Nodes	Majority needed	Failures tolerated	Note
1	1	0	No fault tolerance
2	2	0	Worse than 1 in availability: either failing blocks progress
3	2	1	Common minimum
5	3	2	Common for critical metadata
7	4	3	Rarely needed; more nodes make each commit slower

Odd sizes are used because 4 nodes tolerate only 1 failure (majority is 3), just like 3 nodes, but cost more. Spread nodes across **failure domains** (racks, availability zones); three zones allow the loss of one entire zone.

What consensus costs, and when to use it

Every commit needs a round trip to a majority, so writes are **slower** than on a single node, throughput is bounded by the leader, and a minority partition **cannot make progress** (it chooses consistency over availability, a CP system, Chapter 11). Therefore: use consensus

for **small, critical coordination data** (who is leader, configuration, locks, shard maps, schema versions), through an existing service such as **etcd**, **ZooKeeper** or **Consul**; do *not* put bulk application data through a hand-written consensus loop. Writing your own consensus protocol is almost always a mistake; use a proven library or service.

COMMON MISTAKE

Common mistakes. Implementing leader election with a database row and wall-clock timestamps and no fencing. Running a consensus cluster with an even number of nodes, or all nodes in one zone. Putting large data through the consensus log. Treating a lock as a guarantee without fencing. Forgetting that a majority is needed to *read* linearizably too (a stale leader could serve old reads unless it confirms leadership).

14.5 Atomic commit across systems

What if one logical operation must update **two databases** atomically? **Two-phase commit (2PC)** uses a coordinator. *Phase 1 (prepare):* the coordinator asks every participant "can you commit?"; each makes its changes durable and replies yes or no, promising to commit if asked. *Phase 2 (commit):* if all said yes, the coordinator tells everyone to commit; otherwise to abort.

The trouble is that a participant that voted yes **cannot decide alone**; it must wait for the coordinator. If the coordinator crashes after prepare, participants hold locks and are **blocked** until it returns. 2PC therefore trades availability for atomicity, is slow, and is avoided across independent services. The alternative is a **saga** with compensations (Chapter 13), which gives eventual rather than atomic consistency. When you truly need atomicity across data, put it in **one** database (or one shard).

Table 14.2 Two-phase commit versus saga.

	Two-phase commit	Saga
Atomicity	Yes (all or none, with locks held)	Eventual, via compensations
Isolation	Yes (locks held across prepare)	No; intermediate states are visible
Availability	Blocks if the coordinator fails	Continues; compensates on failure
Latency	Several round trips, locks held	Each step local and fast
Best for	Few tightly coupled databases	Long-running, cross-service workflows

14.6 Duplicate and out-of-order events

Distributed systems deliver messages more than once, late, and out of order. Defences: **sequence numbers or versions** attached to entities so a consumer ignores events older than the latest applied; **idempotent handlers** (Chapter 13); **event-time versus processing-time** distinction in stream processing; and **conditional writes** (`apply if version = n`). The goal is to make the *effect* of processing independent of delivery accidents.

FAILURE MODES

Failure modes. Split brain from naive election; a zombie leader writing after its lease expired; clock jumps breaking timeouts or lease checks; false failure detection under load causing failover storms; a minority partition unable to elect a leader (by design); consensus cluster losing a majority and freezing (requires careful manual recovery); 2PC coordinator crash blocking participants; correlated failures (all nodes in one zone); configuration changes (adding nodes) done unsafely, producing two majorities.

COMMON MISCONCEPTION

- "*Timestamps tell me which write happened last.*" Only with synchronised clocks and known error bounds; otherwise use versions or logical clocks.
- "*A timeout means the operation failed.*" It means you do not know.
- "*A distributed lock guarantees mutual exclusion.*" Not without fencing at the resource.
- "*Paxos/Raft make a system fast and always available.*" They make it correct and fault tolerant, at the cost of latency and unavailability of a minority.

INTERVIEW QUESTIONS

Q1. Why can't you rely on timestamps to order events across servers? Clocks skew and jump; message delays vary. Use logical or hybrid clocks, versions, or a single sequencer.

Q2. What is a fencing token? A monotonically increasing number issued with each lock or lease; the protected resource rejects requests with a smaller token, so a paused former lock holder cannot corrupt data.

Q3. How does Raft elect a leader and why does it need a majority? Followers time out, become candidates and ask for votes; a majority makes a leader. Two disjoint groups cannot both be majorities, so at most one leader per term; committed entries exist on a majority, so any new leader has them.

Q4. 2PC versus sagas? 2PC gives atomic commit but blocks and is slow; sagas are available and cheap but only eventually consistent with compensations.

EXERCISES

1. A node has Lamport counter 5 and receives a message stamped 9. What is its counter after receiving? After sending a reply?
2. A five-node cluster is split 3 and 2 by a partition. Which side can elect a leader and commit? What happens to clients of the other side?
3. Process A holds a lease for 10 s, pauses for 20 s, then writes to storage. Explain what goes wrong without fencing and how tokens fix it.
4. Why is a 4-node consensus cluster not better than 3 nodes?

Answers.

1. After receiving: max(5, 9) + 1 = 10. After sending a reply (a new event): 11 (the message carries 11).
2. The side with 3 nodes (a majority) can elect and commit. The 2-node side cannot elect a leader or commit; its clients get errors or timeouts (consistency over availability).

3. During the pause the lease expired and B became leader and wrote; A wakes, still believes it holds the lease and overwrites B's data. With tokens, B's token (higher) was recorded by storage; A's write with the old token is rejected.
4. The majority of 4 is 3, so it tolerates only 1 failure, the same as 3 nodes, but needs more votes per commit and has more nodes to fail.

CHAPTER SUMMARY

- Clocks disagree; use monotonic clocks for durations and logical or hybrid clocks (or versions) for ordering.
- A timeout means "unknown": design for idempotency, outcome queries and reconciliation.
- Elect leaders by majority; use leases with fencing tokens checked at the resource.
- Consensus (Raft, Paxos) replicates a log through a majority: safe, slower, minority unavailable; use it via etcd, ZooKeeper or Consul for small critical data.
- 2PC blocks; sagas are available but eventually consistent.

REVISION CHECKLIST

- [ ] I can explain skew, drift and wall versus monotonic clocks.
- [ ] I can update Lamport clocks on a message exchange.
- [ ] I can draw the fencing-token timeline.
- [ ] I can explain Raft election and commit, and quorum sizes.
- [ ] I can contrast 2PC and sagas.

SUPPLEMENTAL MATERIAL

Further reading. Leslie Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System" (1978); Diego Ongaro and John Ousterhout, "In Search of an Understandable Consensus Algorithm" (Raft, 2014) and the raft.github.io visualisation; Martin Kleppmann, "How to do distributed locking" (2016); Kleppmann, *Designing Data-Intensive Applications*, chapters 8 and 9; the Spanner paper (2012).

CHAPTER 15

Resilience Patterns

Derived from your notes: Notebook chapters 2 and 9 (availability patterns, back pressure) and the "symptom to first fix" revision sheet. **Added in this book:** the whole chapter: timeouts, retry budgets and jitter, circuit breakers, bulkheads, load shedding, graceful degradation, failover design, chaos testing.

LEARNING OBJECTIVES

After this chapter you can:

- explain how a failure in one component spreads, and why slow is worse than down;
- set timeouts and retry policies (backoff, jitter, budgets) that heal failures instead of amplifying them;
- apply circuit breakers, bulkheads, load shedding and graceful degradation;
- design failover that has actually been exercised, and reason about blast radius.

PREREQUISITE KNOWLEDGE

Chapter 5 (timeouts, TCP), Chapter 13 (retries, back pressure), Chapter 14 (uncertain outcomes).

KEY TERMINOLOGY

Resilience
The ability to keep providing acceptable service despite failures, and to recover quickly.

Cascading failure
One component's failure causing others to fail, often via overload.

Blast radius
How much of the system (or how many users) a single failure can affect.

Circuit breaker
A guard that stops calling a failing dependency for a while, failing fast instead.

Bulkhead
Isolating resources so that one failing part cannot exhaust the capacity of the others.

Load shedding
Deliberately rejecting some requests to protect the rest.

Graceful degradation
Offering reduced functionality instead of failing completely.

15.1 How failures spread

Components do not fail one at a time and politely. A typical outage chain: a database slows down; application threads wait on it; the thread pool fills; the web tier can no longer accept requests; the load balancer's health checks fail; healthy servers are removed; the remaining ones receive more traffic and fail faster. A *local* slowdown became a *global* outage. Three lessons:

1. **Slow is worse than down.** A dead dependency fails instantly (connection refused); a slow one ties up a thread, a connection and memory for every request until a timeout, so the failure *propagates upstream*.
2. **Retries multiply load.** If every client retries a failing call three times, a struggling service suddenly receives four times its normal traffic: a **retry storm** that stops recovery.
3. **Tail latency compounds across calls.** A request needing 100 internal calls at p99 = 1 s is slow for most users (Chapter 1).

Resilience engineering is the set of habits that stop these chains: bound every wait, limit retries, isolate resources, shed load, and degrade gracefully.

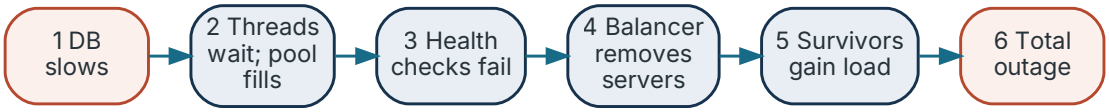


Figure 15.1 A cascading failure. A slow database makes application threads wait; the pool fills; the balancer sees failing health checks and removes servers; load on the survivors increases and the whole tier collapses.

15.2 Timeouts

Every network call needs a timeout. The default in many libraries is infinite or minutes. Without a timeout, a hung dependency holds a thread forever, and the pool drains.

Choosing values: base them on measured latency of the dependency, for example a little above its p99.9, not on hope. Use separate **connect** and **read** timeouts. Propagate a **deadline** through the call chain: if the user's request has 1 s total, the first service gives its downstream at most 800 ms, which gives *its* downstream 600 ms, so no work is done after the client has already given up. Remember from Chapter 14: a timeout means *outcome unknown*, so timeouts on non-idempotent calls require idempotency keys.

WORKED EXAMPLE

A deadline budget. User request: 1,000 ms. Frontend processing: 50 ms. It calls the order service with a deadline of 900 ms. The order service spends 100 ms locally and calls inventory with 700 ms and pricing with 700 ms *in parallel* (not sequentially, which would need 1,400 ms). If inventory is slow and the call exceeds 700 ms, it is cancelled; the order service returns a degraded answer ("availability being checked") instead of waiting. Without deadlines each layer would use its own 5 s timeout and the user would wait 5 s on top of 5 s on top of 5 s.

15.3 Retries done right

Retries are the right response to **transient** failures (a dropped packet, a restarting server), and a dangerous one to **overload**. Rules:

- **Retry only safe operations**: idempotent calls or those with idempotency keys.
- **Retry only on retryable errors**: connection resets, timeouts, 503; never on 400-class errors that will fail again.
- **Exponential backoff**: wait 100 ms, 200, 400, 800, ... to give the dependency time to recover.
- **Jitter**: randomise each delay (for example, a random value between 0 and the backoff, called *full jitter*) so clients de-synchronise instead of retrying in lockstep.
- **Limit attempts** (2 to 3), and use a **retry budget**: retries may be at most, say, 10% of recent requests. When the budget is exhausted, fail fast. This caps the amplification during an outage.
- **Retry at one layer only.** If a client, the gateway and the service each retry three times, one failure becomes 27 attempts.

```
def call_with_retry(fn, attempts=3, base=0.1, cap=2.0):
    for i in range(attempts):
        try:
            return fn(timeout=0.5)
        except RetryableError:
            if i == attempts - 1 or not retry_budget.try_acquire():
                raise
            sleep(random.uniform(0, min(cap, base * 2 ** i)))   # full jitter
```

Hedged requests (sending a second request after a short delay if the first is slow, taking whichever answers first) cut tail latency for idempotent reads, at the price of extra load; use them sparingly and only with a budget.

15.4 Circuit breakers

A **circuit breaker** wraps calls to a dependency and tracks recent failures. It has three states:

- **Closed**: calls pass through; failures are counted.

- **Open**: after too many failures (for example, 50% errors over 20 requests), calls **fail immediately** without touching the dependency, for a cool-down period. This protects both sides: the caller does not waste threads, and the struggling dependency gets breathing room to recover.
- **Half-open**: after the cool-down a few **trial** calls are allowed; if they succeed, the breaker closes; if they fail, it opens again.

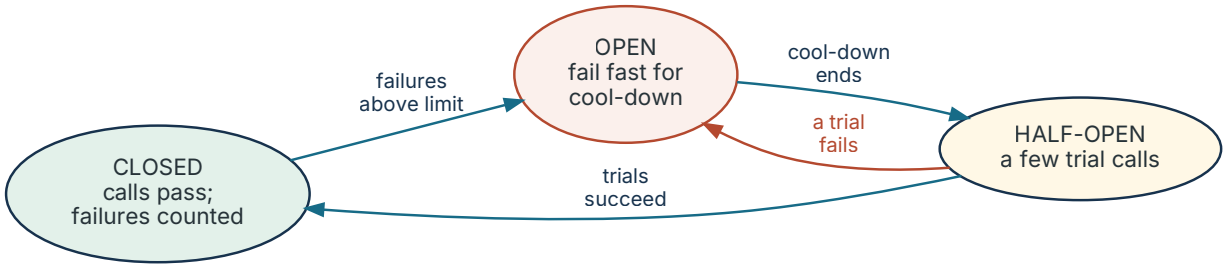


Figure 15.2 The three states of a circuit breaker. Failures open it; after a cool-down a few trial requests decide whether to close it again.

When the breaker is open, the caller needs a **fallback**: a cached value, a default, a queued request, or a clear error. Tune thresholds to the dependency's normal error rate; too sensitive and the breaker flaps, too lax and it never helps.

15.5 Bulkheads and isolation

Ships are divided into watertight compartments so one hole does not sink the vessel. In software, a **bulkhead** gives each dependency, tenant or request class its **own** pool of threads, connections or capacity. If the recommendation service hangs and has its own 10-thread pool, it can exhaust only those 10 threads; checkout, with its own pool, keeps working. Without it, a shared pool is drained by the slowest dependency. Bulkheads also appear as **separate deployments** for critical and non-critical traffic, **per-tenant quotas**, **separate queues**, and **cell-based architecture**, in which the whole system is replicated into independent *cells*, each serving a subset of users, so a bad deployment or a poisoned request affects one cell's users only. That limits the **blast radius**.

15.6 Load shedding and graceful degradation

When demand exceeds capacity, doing everything slowly means *everyone* times out. Doing *less*, quickly, keeps the system healthy.

- **Load shedding**: reject excess requests early and cheaply (HTTP 503 with `Retry-After`), preferably the **least important first** (analytics before checkout, anonymous before logged-in), and based on a cheap signal such as concurrency or queue age. Shed at the edge, before the work is started. Connection limits and the rate limiters of Chapter 7 are forms of shedding.

- **Graceful degradation**: provide reduced but useful function. If personalised recommendations are down, show popular items; if the search ranking service is slow, return unranked results; serve slightly stale cached data; disable expensive features (autocomplete) under load. Decide in advance, per feature, **what is critical** and **what can degrade**, and make that a feature flag operators can flip.
- **Fail open versus fail closed**: if the authorisation service is unreachable, deny access (fail closed, safe for security); if the recommendation service is unreachable, show something anyway (fail open, safe for experience). Make the choice deliberately for each dependency.

15.7 Designing failover

Redundancy is worthless unless the switch-over **works**. Principles:

- **Eliminate single points of failure** at every layer: balancers, application servers, caches, databases, DNS, power, network, and *humans* (the one engineer who knows how).
- **Spread across failure domains**: servers in several racks, **availability zones** (separate data centres within a region), and, for the highest requirements, **regions**. Size capacity so that after losing one zone the rest can carry the load (N+1 or "N+ a zone"): three zones, each at ≤ 66% utilisation, can lose one.
- **Automate detection and promotion**, but protect against false triggers and split brain (Chapter 14).
- **Practise.** Untested failover fails. Run game days, kill instances in production under controlled conditions (**chaos engineering**: inject faults deliberately and observe), and rehearse regional evacuation. Time every recovery to know your real RTO.
- **Make recovery safe**: when a failed component returns, avoid *thundering herds* (ramp traffic up gradually; use slow start at balancers) and re-synchronise data before it serves.

Table 15.1 Patterns and what each defends against.

Pattern	Defends against	Cost or risk
Timeout and deadline	Hung dependencies, wasted work	Wrong values cause false failures
Retry with backoff, jitter and budget	Transient errors	Amplifies overload if unbounded
Circuit breaker	Persistent dependency failure	Needs fallbacks; tuning
Bulkhead	Resource exhaustion spreading	Lower utilisation (partitioned pools)
Load shedding	Overload	Some users are refused
Graceful degradation	Partial outages	Must be designed and tested per feature
Redundancy across zones	Hardware and zone failure	Cost; replication complexity

Pattern	Defends against	Cost or risk
Chaos testing	Untested assumptions	Needs maturity and safeguards

FAILURE MODES

Failure modes of the resilience mechanisms themselves. Timeouts shorter than normal latency causing self-inflicted errors; retries synchronised into storms; a circuit breaker that stays open forever or flaps; fallbacks that are untested and fail when first used; bulkheads sized too small, throttling normal traffic; health checks that remove all servers at once; failover that promotes a stale replica; "metastable" failures, in which a trigger (a brief overload) leaves the system in a bad state (queues full, caches cold, retries high) that persists *after* the trigger is gone, requiring shedding or a restart to recover.

COMMON MISTAKE

Common mistakes. No timeouts. Retrying on every error, at every layer, with no jitter. Treating redundancy as availability without testing failover. Hiding failures: a fallback that silently returns wrong data. Health checks that depend on a shared dependency.

COMMON MISCONCEPTION

- "*If we retry, we are more reliable.*" Not when the failure is overload; unlimited retries destroy recovery.
- "*We have three replicas, so we are safe.*" Only if failures are independent and failover is exercised.
- "*A circuit breaker fixes the failing service.*" It protects callers and gives time; the service still needs repair.
- "*Graceful degradation is a nice-to-have.*" For large systems it is the difference between a blip and an outage.

INTERVIEW QUESTIONS

Q1. A dependency becomes slow. What happens to your service and how do you protect it? Threads and connections pile up waiting; the pool drains and failure cascades upstream. Use timeouts and deadlines, a circuit breaker with a fallback, a bulkhead so other features keep their own pools, and load shedding.

Q2. How should retries be configured? Only for idempotent or keyed operations and retryable errors; exponential backoff with jitter; two or three attempts with a retry budget; retry at a single layer; stop when the circuit is open.

Q3. What is a circuit breaker? A state machine (closed, open, half-open) that fails fast after repeated errors to protect the caller and give the dependency time to recover, probing periodically.

Q4. What is graceful degradation? Give an example. Serving reduced functionality instead of failing: if the recommendation service is down, show popular items from a cache.

EXERCISES

1. A service calls three dependencies sequentially, each with p99 = 400 ms and a 2 s timeout. What are the worst-case and typical-bad latencies, and how would you improve it?
2. A client library retries 3 times, the gateway retries 3 times, and the service retries 3 times against its database. How many database attempts can one user action cause?
3. Design a bulkhead scheme for a web app that calls payments (critical), recommendations (nice to have) and analytics (can be dropped).
4. Three zones each run at 60% utilisation. After losing one zone, what utilisation do the others face?

Answers.

1. Worst case ≈ 3 × 2 s = 6 s; typical bad ≈ 1.2 s. Improve: call them in parallel (about the slowest, not the sum), tighter timeouts near p99.9, a deadline for the whole request, fallbacks for the optional ones.
2. 3 × 3 × 3 = 27 attempts. Retry at a single layer.
3. Separate thread or connection pools: payments gets a protected pool and highest priority; recommendations a small pool with a short timeout and fallback to popular items; analytics a tiny pool or an asynchronous queue where events can be dropped under load.
4. Total load = 3 × 60% = 180% of one zone's capacity; spread over two zones that is 90% each, which leaves almost no headroom. Planning for about 50% per zone gives 150% / 2 = 75% after losing a zone.

CHAPTER SUMMARY

- Failures cascade through waiting and retries; slow is worse than down.
- Timeouts with deadlines everywhere; retries only for safe, retryable errors, with backoff, jitter, a small budget and a single retrying layer.
- Circuit breakers fail fast; bulkheads isolate resources; load shedding and graceful degradation keep the core working.
- Redundancy needs independent failure domains, spare capacity, automation and regular practice.

REVISION CHECKLIST

- [ ] I can draw a cascade and say where each defence cuts it.
- [ ] I can write a retry loop with jitter and a budget.
- [ ] I can describe the three breaker states.
- [ ] I can decide fail-open versus fail-closed for a dependency.

SUPPLEMENTAL MATERIAL

Further reading. Michael Nygard, *Release It!* (circuit breakers, bulkheads); Google's *Site Reliability Engineering* chapters on cascading failures and handling overload; Marc Brooker, "Timeouts, retries, and backoff with jitter" (AWS Builders' Library); Bronson et al., "Metastable Failures in Distributed Systems" (2021).

CHAPTER 16

Application Architecture and Microservices

Derived from your notes: Notebook chapter 5 (separating web and application layers, microservices, service discovery, the comparison table and the drawbacks of each). **Added in this book:** monolith versus modular monolith, bounded contexts and data ownership, service communication, the strangler migration, architecture diagrams, architecture decision records and API-contract testing.

LEARNING OBJECTIVES

After this chapter you can:

- separate web and application tiers and explain why the application tier is stateless;
- compare a monolith, a modular monolith and microservices, and decide which suits a team and a problem;
- draw service boundaries around business capabilities and assign data ownership;
- explain service discovery and the communication patterns between services;
- migrate gradually with the strangler pattern, and record decisions in architecture decision records.

PREREQUISITE KNOWLEDGE

Chapter 6 (contracts), Chapter 7 (gateway, balancing), Chapter 13 (events, sagas) and Chapter 15.

KEY TERMINOLOGY

Monolith
An application built and deployed as a single unit.

Microservice
A small, independently deployable service owning one business capability and its data.

Bounded context
A part of the business with its own vocabulary and model; a natural service boundary.

Service discovery
The mechanism by which services find the current network address of other services.

Strangler pattern
Replacing a legacy system piece by piece by routing more and more traffic to new components.

Architecture decision record (ADR)
A short document capturing one decision, its context, alternatives and consequences.

16.1 Layers and tiers

A web application usually has three logical layers: **presentation** (what the user sees), **application or business logic** (the rules), and **data** (storage). Putting web handling and business logic into **separate tiers** has benefits (from the notebook): they scale **independently** (many web servers for thin request handling, fewer heavy application servers); each can be **deployed and secured** separately (the application tier has no public address); and **teams** can own them separately. A **platform layer** can offer shared services (authentication, notifications) used by all.

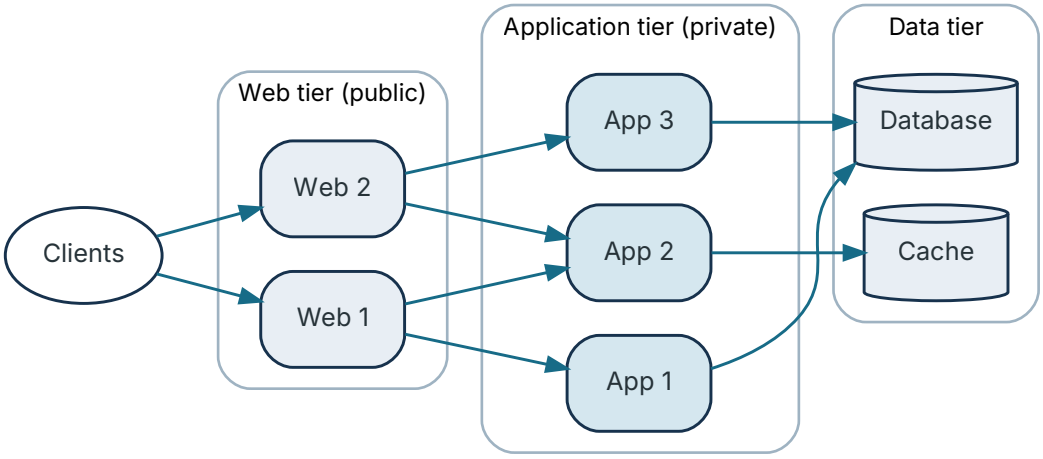


Figure 16.1 Separating tiers. A web tier handles HTTP and delegates to an application tier of workers, which talk to the data stores. Each tier scales on its own.

The key property of the application tier is that it is **stateless**: nothing user-specific is kept in a server's memory, so any instance can serve any request (Chapter 1). State lives in databases, caches and queues. That is what lets you add instances during a spike and replace them when they fail.

16.2 Monolith, modular monolith, microservices

A **monolith** is one deployable application containing all features. It is simple to build, test, run and debug (one process, one database, local function calls, real transactions), and is often the *right* starting point. It strains as teams and code grow: every change redeploys everything, a bug in one module can crash all, one language and framework for all, and scaling means scaling the whole thing.

Microservices split the system into many small services, each owning one business capability and its data, communicating over the network, deployed independently. Benefits: independent deployment and scaling; teams that own services end to end; technology choice per service; fault isolation if designed well. Costs (the notebook's "drawbacks" and more):

- **Distributed system complexity**: every call can be slow, fail or time out (Chapter 15, Chapter 14).
- **Data consistency**: no cross-service transactions; use events, outbox and sagas (Chapter 13).
- **Operational overhead**: many deployments, pipelines, dashboards, on-call rotations; need strong automation, observability (Chapter 18) and platform tooling.
- **Network latency and chattiness**: a screen needing 20 service calls is slow and fragile.
- **Testing and debugging** across services is harder; versions must stay compatible.
- **Organisational demands**: small autonomous teams and clear ownership.

A middle path is the **modular monolith**: one deployable unit, but strictly separated modules with explicit interfaces and private data, so it can be split later if needed. Many successful products stay here for years.

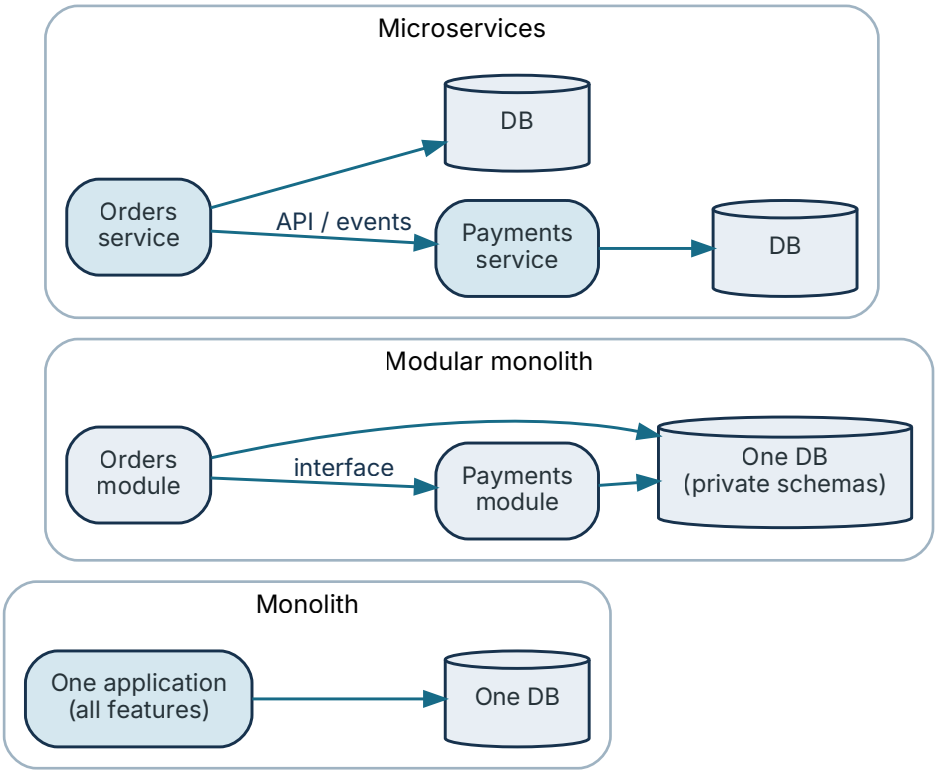


Figure 16.2 Three architectures. The monolith is one unit; the modular monolith adds enforced module boundaries inside one deployable; microservices split into independently deployed services, each with its own data.

Table 16.1 Choosing an architecture.

Factor	Monolith / modular monolith	Microservices
Team size	Up to a few dozen engineers	Many teams needing autonomy
Early-stage product	Strong fit: change is fast, boundaries unknown	Premature: boundaries guessed wrongly are costly
Deployment	Single, simple	Many, needs automation
Scaling needs	Uniform	Very different per component
Transactions	Easy (one database)	Hard (sagas, eventual consistency)
Operational maturity needed	Low	High

CLARIFICATION

Clarification: microservices are an organisational tool as much as a technical one. The notebook lists microservices as a way to scale and to let teams work independently. The honest summary is that they *trade technical simplicity for organisational independence*. If you do not have the organisational problem (many teams blocking each other), you pay the cost without the benefit. A well-known rule of thumb, *Conway's law*, says systems tend to mirror the communication structure of the organisation that builds them; design the teams and the architecture together.

16.3 Drawing the boundaries

Bad boundaries cause more pain than bad technology. A **distributed monolith** (services so tightly coupled that they must be deployed together and call each other constantly) has all the costs and none of the benefits.

Guidelines:

- **Follow business capabilities** (orders, payments, catalogue, shipping), not technical layers ("the database service"). Domain-driven design calls these **bounded contexts**: each has its own model of, say, a "customer" (billing cares about invoices, support about tickets).
- **High cohesion inside, loose coupling between.** Things that change together live together. If two services always change in the same release, they are one service.
- **Each service owns its data.** No other service reads its tables directly. Others use its API or consume its events. This is *the* rule that makes independent change possible; shared databases recreate tight coupling.
- **Prefer coarse over fine** at first. It is easier to split a service later than to merge many tiny ones.
- **Design for failure at every boundary**: timeouts, retries, idempotency, fallbacks.

Data ownership and duplication

Owning data means other services cannot join across it. They need *copies*: for example, the order service stores the product name and price **as of purchase** (a snapshot, which is *correct* behaviour for an order), and the search service keeps a read-optimised copy built from product events. Duplication is accepted; the rules are that **one service is the authority** for each fact, others hold derived copies that are updated by events, and everyone tolerates short staleness.

16.4 Communicating between services

Table 16.2 Ways services talk.

Style	How	Use for	Risk
Synchronous request/response	REST or gRPC call	Queries needing an immediate answer	Latency adds up; failures propagate; availability multiplies down
Asynchronous messaging	Queue or topic	Commands that can wait, notifications	Eventual consistency, duplicates, debugging
Event-driven	Publish facts ("OrderPlaced") that others react to	Decoupling, many consumers	Hard to see the whole flow; event schema evolution
API composition / BFF	A backend-for-frontend aggregates several calls	Screens needing data from many services	The aggregator becomes a bottleneck or hides logic

Chains of synchronous calls multiply failure probability and latency: five services in a chain, each 99.9% available, give about 99.5% overall. Prefer **fewer hops**, **parallel calls**, and **asynchronous events** for things that need not be immediate. A **service mesh** (sidecar proxies beside each service) can move retries, timeouts, mutual TLS and telemetry out of application code; it is useful at scale and adds its own complexity.

16.5 Service discovery

Instances come and go (autoscaling, deploys, crashes), so addresses cannot be hard-coded. **Service discovery** lets a caller find healthy instances by *name*.

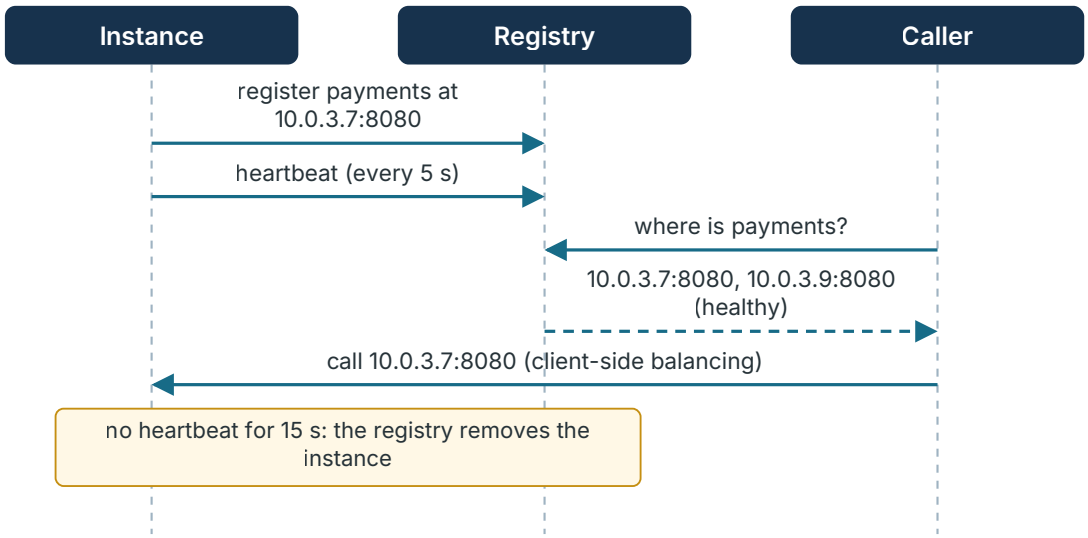


Figure 16.3 Service discovery. Instances register with a registry and send heartbeats; a caller asks the registry for the healthy instances of a service and calls one directly.

How it works: when an instance starts it **registers** (name, address, port, metadata) with a **registry**; it renews with **heartbeats** or the registry actively runs **health checks**; unhealthy or silent instances are removed. Callers either **look up** addresses and cache them (client-side discovery, with client-side load balancing) or send requests to a **router or load balancer** that consults the registry (server-side discovery). Tools: **Consul**, **etcd**, **ZooKeeper** (the notebook's examples, built on consensus, Chapter 14), Kubernetes' built-in service objects and DNS, and cloud-provider equivalents. Plain **DNS** can serve as a simple registry, with the caveat of TTL caching (Chapter 5).

Drawbacks (notebook): the registry is itself critical infrastructure that must be highly available, and clients must tolerate stale results and register or deregister correctly.

16.6 Migrating: the strangler pattern

Rewriting a working system from scratch is notoriously risky. The **strangler fig** pattern grows new functionality around the old system: put a **routing layer** (a gateway or proxy) in front, build one capability at a time as a new service, route that capability's traffic to it, and retire the old code when nothing uses it.

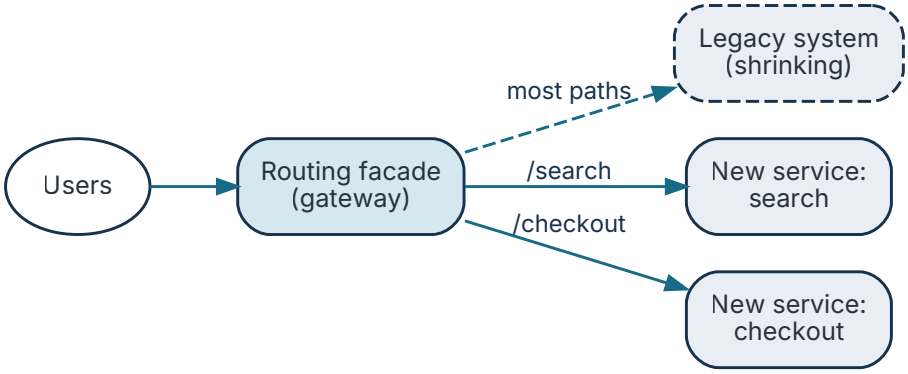


Figure 16.4 The strangler pattern. A routing facade sends some paths to new services and the rest to the legacy system; over time more routes move until the legacy system can be removed.

Practical steps: pick a capability with clear boundaries and modest risk; build the new service behind the facade; **migrate data carefully** (dual-write or change-data-capture, with reconciliation); run both and **compare results** (shadow traffic); move traffic gradually (1%, 10%, 50%, 100%) with instant rollback; delete the old path. Each step is reversible, and value is delivered continuously.

16.7 Recording architecture

Decisions must be understandable to the people who inherit them. Two lightweight tools:

Architecture diagrams. The *C4 model* uses four zoom levels: **context** (the system among users and other systems), **containers** (deployable units: web app, services, databases), **components** (inside one container), **code**. Draw at the right level, label every arrow with *what flows and how* (HTTP, event), and keep one purpose per diagram.

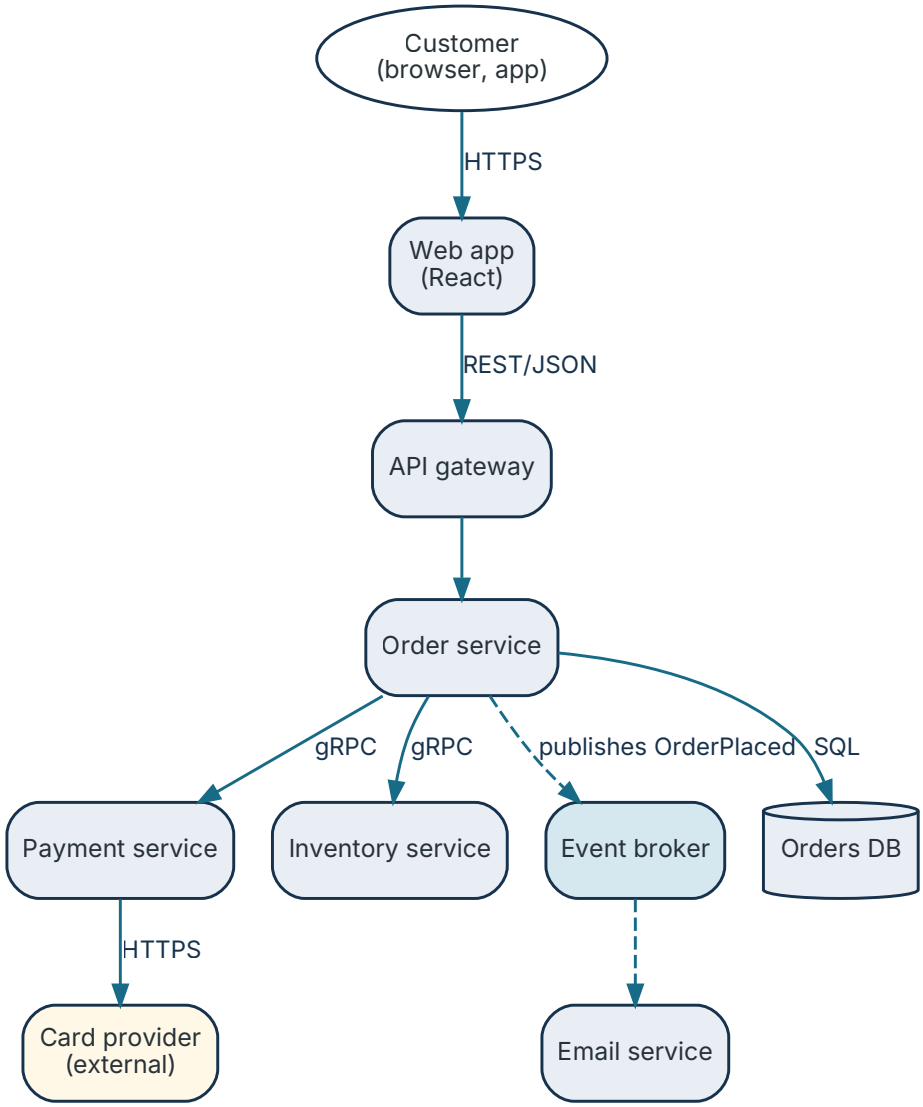


Figure 16.5 A container-level diagram for a shop. Each box is deployable; each arrow is labelled with the protocol and purpose.

Architecture decision records (ADRs). A short, dated file per decision, stored with the code:

```
ADR-014: Use a transactional outbox for order events
Status:   Accepted (2026-10-02)
Context:  Orders must be saved and OrderPlaced published atomically. We run
          PostgreSQL and a managed message broker; no distributed transactions.
Options:  (1) publish after commit  - can lose events on a crash
          (2) 2PC across DB and broker - unsupported, blocking
          (3) outbox table + relay    - at-least-once, needs idempotent consumers
Decision: Option 3.
Consequences: + no lost events  - consumers must deduplicate; relay is a new
          component to monitor. Revisit if event volume exceeds 5,000/s.
```

An ADR records *why*, the alternatives rejected, and when to revisit; it turns "who decided this and why?" from folklore into documentation. Update the status (Superseded by ADR-031) rather than rewriting history.

FAILURE MODES

Failure modes. Distributed monolith (changes ripple across services); a "chatty" service graph with latency and failure amplification; a shared database coupling services; the registry down or stale; version skew between services during deployment (old caller, new callee); a service becoming a bottleneck because everything depends on it; inconsistent data between copies; no one owning a service when it breaks at 3 a.m.; migration stalled half-way with two systems to maintain.

COMMON MISTAKE

Common mistakes. Starting with microservices before understanding the domain. Splitting by technical layer. Letting services read each other's databases. Synchronous call chains everywhere. No tracing, so nobody can see a request's path. Creating a service per entity instead of per capability. Skipping contract tests.

SUPPLEMENTAL MATERIAL

Contract testing. When services deploy independently, you need confidence that a change to a provider will not break its consumers. **Consumer-driven contract tests** let each consumer record the requests it makes and the responses it relies on; the provider's pipeline replays them and fails the build if a change breaks any. This gives fast, reliable feedback without standing up the whole system for end-to-end tests. Combine with additive-only API changes (Chapter 6) and gradual rollouts.

COMMON MISCONCEPTION

- "*Microservices are more scalable.*" They allow independent scaling of parts; a monolith can scale horizontally too.
- "*Every company should use microservices.*" They solve organisational scaling problems; small teams usually should not.
- "*Services are small, so each can use its own database engine freely.*" Possible, but every extra technology is an operational cost.
- "*We can always split later, so design does not matter now.*" Good module boundaries make splitting possible; a tangled monolith is hard to split.

INTERVIEW QUESTIONS

Q1. Monolith or microservices for a new startup? Start with a (modular) monolith: boundaries are unknown, the team is small, transactions are simple. Extract services when a clear organisational or scaling need appears.

Q2. How do you decide a service boundary? By business capability and bounded context, with data owned by one service, high cohesion inside and low coupling outside; things that change together stay together.

Q3. How do services find each other? Service discovery: instances register with a registry (Consul, etcd, ZooKeeper, Kubernetes) and heartbeat; callers resolve healthy instances by name, client-side or through a load balancer.

Q4. How would you migrate a legacy monolith? Strangler pattern behind a routing facade, one capability at a time, with careful data migration, shadow comparisons, gradual traffic shifts and rollback.

EXERCISES

1. A shop has Orders, Payments, Inventory and Search. Propose service boundaries, name each service's data, and say which interactions are synchronous and which asynchronous.
2. Five services in a synchronous chain have availability 99.9% each. Compute the chain's availability and propose two ways to improve it.
3. Write an ADR for "use a message queue between the order service and the email service".
4. Two services always change together and share tables. What is your recommendation?

Answers.

1. Orders (orders, order lines), Payments (transactions), Inventory (stock levels), Search (read-optimised index built from product events). Synchronous: Orders asks Inventory to reserve stock and Payments to charge (fast, needs an immediate answer); asynchronous: OrderPlaced event to Search indexing and to Email; saga for the order flow with compensations.
2. $0.999^5 \approx 0.995$ (99.5%, about 44 hours per year). Improve by removing hops (merge or parallelise), making some calls asynchronous, adding fallbacks, caching, and redundancy.
3. Context: email is slow and may fail; orders should not wait. Options: synchronous call, queue, event topic. Decision: queue with retries and a dead-letter queue. Consequences: delayed delivery; duplicates need an idempotent email sender; queue age to be monitored.
4. Merge them into one service: the boundary is wrong (a distributed monolith); split only along a real seam with separate data.

CHAPTER SUMMARY

- Separate web and application tiers; keep the application tier stateless so it scales out.
- Monoliths are simple; modular monoliths keep options open; microservices buy independence at the price of distributed-system and operational costs.
- Draw boundaries by business capability; each service owns its data; share by API and events, tolerate duplication and staleness.
- Service discovery lets callers find healthy instances; the registry must itself be highly available.
- Migrate with the strangler pattern; record decisions in ADRs and diagrams.

REVISION CHECKLIST

- [ ] I can argue monolith versus microservices for a given team and product.
- [ ] I can define a service boundary and assign data ownership.
- [ ] I can draw service registration and lookup.
- [ ] I can describe a strangler migration and write an ADR.

SUPPLEMENTAL MATERIAL

Further reading. Sam Newman, *Building Microservices* and *Monolith to Microservices*; Eric Evans, *Domain-Driven Design*; Simon Brown, the C4 model (c4model.com); Michael Nygard, "Documenting Architecture Decisions" (2011); Martin Fowler's articles on the strangler fig and on microservice trade-offs.

PART VI

Security and Operations

Protecting systems and their users, then running them, observing them and paying for them.

CHAPTER 17

Security and Privacy

Derived from your notes: Notebook chapter 9 (security basics: encrypt in transit and at rest, sanitise input, parameterised queries, least privilege). **Added in this book:** threat modelling, authentication versus authorization in depth, passwords and tokens, OAuth 2.0 flow, injection and web attacks, secrets and key management, abuse prevention, privacy engineering, audit logging.

LEARNING OBJECTIVES

After this chapter you can:

- do a small threat model for a design and name the main attack classes;
- distinguish authentication from authorization and describe how passwords, sessions, tokens and OAuth 2.0 fit together;
- prevent injection, cross-site scripting, request forgery and broken access control;
- explain encryption in transit and at rest, key management and secrets handling;
- apply least privilege, abuse prevention and privacy-by-design practices, including data retention and deletion.

PREREQUISITE KNOWLEDGE

Chapter 5 (TLS, HTTP), Chapter 6 (status codes, API design), Chapter 7 (gateway, rate limiting) and Chapter 8.

KEY TERMINOLOGY

Authentication (AuthN)
Proving *who* you are.

Authorization (AuthZ)
Deciding *what* an identified party may do.

Least privilege
Giving each person and program only the access it needs, no more.

Encryption at rest / in transit
Protecting stored data, and data moving across a network.

Secret
A credential (password, API key, private key, token) that must stay confidential.

PII (personally identifiable information)
Data that can identify a person: name, email, phone, address, device identifiers.

17.1 Thinking like an attacker

Security is not a feature to bolt on; it is a property to design for. The cheapest technique is a **threat model**: before building, ask four questions: *What are we protecting?* (user data, money, availability, reputation) *Who might attack, and how?* (anonymous internet users, malicious customers, insiders, stolen laptops, compromised dependencies) *What are we doing about it? How will we know if it failed?*

A handy checklist is **STRIDE**: **S**poofing (pretending to be someone), **T**ampering (altering data), **R**epudiation (denying an action with no proof), **I**nformation disclosure (leaks), **D**enial of service, **E**levation of privilege. Walk each arrow in your architecture diagram and ask which of the six applies. A **trust boundary** is any line where data crosses from a less trusted to a more trusted zone (internet to gateway, gateway to service, service to database): every crossing needs authentication, validation and logging.

Two principles carry most of security design. **Defence in depth**: assume any single control can fail, and layer several (network rules, authentication, authorisation, validation, encryption, monitoring). **Least privilege**: the web server's database account cannot drop tables; the image resizer cannot read user passwords; an engineer's access expires.

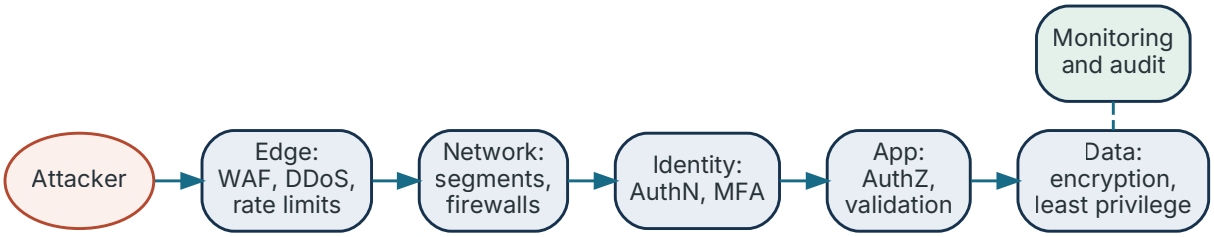


Figure 17.1 Defence in depth. A request passes several independent controls; an attacker must defeat all of them, and each layer limits the damage if another fails.

17.2 Authentication

Authentication establishes identity using something you **know** (password), **have** (phone, hardware key) or **are** (biometric). Use **multi-factor authentication (MFA)** for anything valuable: stolen passwords alone then do not suffice.

Passwords must never be stored in plain text or with fast hashes. Store a **salted, slow, memory-hard hash** (Argon2id, bcrypt or scrypt): a random per-user *salt* ensures identical passwords have different hashes and defeats precomputed tables; deliberate slowness makes guessing billions of passwords expensive. On login, hash the submitted password with the stored salt and compare in constant time. Add rate limiting and lockout or delay after failures (Chapter 7), check against lists of breached passwords, and never roll your own cryptography.

Sessions and tokens

After login the server must recognise the user on later requests (HTTP is stateless, Chapter 5). Two common designs:

- **Server-side session**: the server stores session data and gives the browser a random, unguessable **session id** in a cookie (flagged `HttpOnly`, `Secure`, and `SameSite`). Easy to revoke (delete the session) but needs a shared session store (Chapter 12).
- **Signed token (for example a JWT, JSON Web Token)**: the server signs a token containing the user id, roles and an expiry; any server can verify the signature without a lookup. Good for stateless services and service-to-service calls, but a token **cannot be revoked** before expiry unless you keep a deny-list, so use *short lifetimes* (minutes) with a longer-lived **refresh token** stored and checkable on the server. Never put secrets in a JWT (it is encoded, not encrypted), always verify the signature *and* the expiry, audience and issuer, and reject the `none` algorithm.

OAuth 2.0 and OpenID Connect

Often users should log in through another provider, or an app should act on a user's behalf *without* getting their password. **OAuth 2.0** is a framework for **delegated authorisation**: the user grants an app limited access (a *scope*, such as "read your calendar") to resources on another service, and the app receives an **access token** instead of a password. **OpenID Connect (OIDC)** adds an **ID token** on top, so that OAuth can also be used for **login** ("Sign in with..."). The standard browser flow is the **authorization code flow with PKCE**.

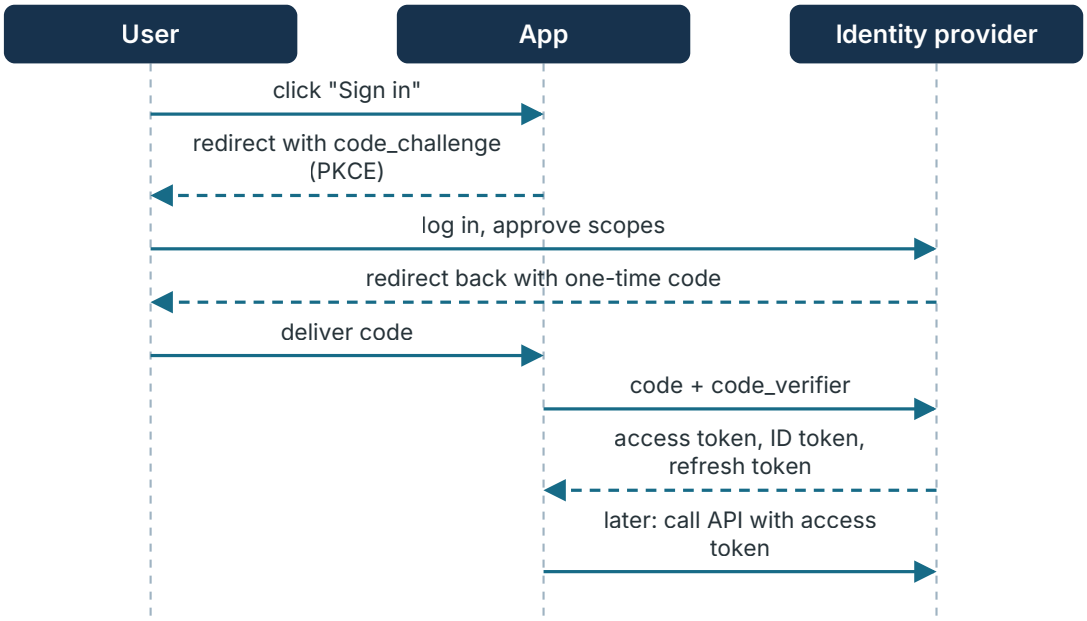


Figure 17.2 Authorisation code flow with PKCE. The user authenticates at the identity provider, never at the app; the app receives a short-lived code and exchanges it, with proof that it started the flow, for tokens.

17.3 Authorization

Authentication answers "who are you?"; authorization answers "may you do this?" Confusing them causes the most common serious vulnerability, **broken access control**: the user is logged in (authenticated) and then fetches `/orders/1234` for *someone else's* order (an **insecure direct object reference**, IDOR) because the server only checked that a session exists.

Rules:

- **Authorise on every request, on the server**, using the identity from the verified session or token, never from client-supplied fields (a `userId` parameter in the request body is untrusted).
- **Check the specific resource**: "does this user own order 1234?", not merely "is this user logged in?"
- **Deny by default**; allow explicitly.
- Use a clear model: **RBAC** (role-based: *admin*, *editor*, *viewer*), **ABAC** (attribute-based: "doctors may see records of patients in their ward during their shift") or **ACLs** (per-resource lists). Keep policy in one place (a library or policy service) rather than scattered across handlers.
- Apply least privilege to *services too*: each service has its own credentials and permissions; a compromised one cannot reach everything.

> CLARIFICATION
>
> **Clarification: server-side validation is authoritative.** Validation in the browser (a disabled button, a form pattern) is only a convenience for honest users; an attacker bypasses it with a script. Every rule that matters (price, permissions, limits, field formats) must be **enforced again on the server**, using server-side data: never trust a price, role, user id, or "is_admin" flag sent by the client.

17.4 Injection and common web attacks

Injection happens when untrusted input is *interpreted as code or commands*. The notebook's two rules, **sanitise user input** and **use parameterised queries**, address the most famous case, SQL injection.

```
# VULNERABLE: input becomes part of the SQL text
db.execute("SELECT * FROM users WHERE email = '" + email + "'")
#   email = "x' OR '1'='1"   -> returns every user
#   email = "x'; DROP TABLE users; --"  -> destroys data

# SAFE: the query shape is fixed; input is only ever a value
db.execute("SELECT * FROM users WHERE email = %s", (email,))
```

Parameterised (prepared) queries send the SQL and the values separately, so the database never mistakes data for commands. The same principle applies to shell commands, LDAP, template engines and more: **never build an interpreter's input by concatenating untrusted strings.**

Table 17.1 Common attacks and their defences.

Attack	What happens	Defence
SQL / command injection	Input executed as code	Parameterised queries; avoid shell calls; least-privilege database account
Cross-site scripting (XSS)	Attacker's script runs in a victim's browser, stealing sessions or acting as them	Escape output for its context; Content Security Policy; `HttpOnly` cookies
Cross-site request forgery (CSRF)	A malicious site makes the victim's browser send an authenticated request	`SameSite` cookies; anti-CSRF tokens; require re-authentication for sensitive actions
Server-side request forgery (SSRF)	The server is tricked into calling internal addresses (such as cloud metadata)	Allow-list outbound destinations; block internal ranges; do not follow redirects blindly
Broken access control / IDOR	Access to other users' data	Authorise per resource, per request, on the server
Credential stuffing	Reused leaked passwords tried at scale	MFA, rate limiting, breached-password checks, bot detection
Denial of service (DoS, DDoS)	Resource exhaustion	CDN or scrubbing service, rate limiting, autoscaling, load shedding
Insecure deserialisation, unsafe file upload	Code execution or storage abuse	Validate and scan; never deserialise untrusted data with unsafe formats; store uploads outside the web root with generated names
Supply-chain compromise	Malicious dependency or build step	Pin and verify dependencies; scan; minimal base images; signed builds

COMMON MISTAKE

Common mistakes. Rolling your own crypto or password storage. Logging secrets, tokens or personal data. Verbose error messages exposing stack traces and database details. Trusting the client for authorisation data. Leaving default credentials, public storage buckets and debug endpoints enabled. Disabling TLS verification "temporarily".

17.5 Encryption and key management

In transit: use TLS (Chapter 5) on *every* hop, including internal service-to-service links; for service identity, **mutual TLS (mTLS)** lets both sides prove who they are. **At rest**: encrypt disks, databases and backups (the notebook's second encryption rule), so a stolen disk or backup reveals nothing; for highly sensitive fields (card numbers, government ids) add **application-level encryption** so even database administrators cannot read them.

Encryption is only as strong as **key management**. Keep keys in a **key management service (KMS)** or hardware security module, *not* in code or config files; use **envelope encryption** (a data key encrypts the data; a master key in the KMS encrypts the data key) so master keys never leave the KMS and can be **rotated** without re-encrypting everything; give access by role and audit every use; plan for rotation and for compromise (how do you replace a leaked key quickly?).

Secrets management. Passwords, API keys and certificates belong in a **secrets manager** (cloud vault services, HashiCorp Vault, Kubernetes secrets with encryption), injected at runtime, **never committed to source control**. Prefer short-lived, automatically rotated credentials, scan repositories for accidental secrets, and revoke immediately if one leaks. Assume anything ever pushed to a repository is public.

17.6 Abuse prevention

Not all attackers break in; many simply *use* the service in unintended ways: spam signups, scraping, credential stuffing, fake reviews, free-tier farming. Defences are layered: **rate limits** per account, IP and device (Chapter 7); **CAPTCHA** or proof-of-work for suspicious flows; **bot detection** and anomaly scoring; **email and phone verification**; **quotas and cost caps**; **honeypots**; a **reporting and takedown** process for user-generated content; and **monitoring** for patterns (many accounts from one device). Balance friction against usability: apply stronger checks only when risk signals are high.

17.7 Privacy by design

Privacy concerns what personal data you collect, why, who sees it and how long you keep it. Regulations such as the GDPR (European Union), the CCPA (California) and India's DPDP Act impose duties; details vary by jurisdiction and change, so involve legal counsel for compliance. Engineering practices that make compliance feasible:

- **Data minimisation**: do not collect what you do not need; the safest data is data you never stored.
- **Purpose limitation and consent**: record why data was collected and what the user agreed to.
- **Classify data** (public, internal, confidential, regulated) and apply controls accordingly; know **where PII lives** (databases, logs, caches, analytics, backups, search indexes).
- **Retention limits**: delete or anonymise data when no longer needed, automatically.
- **User rights**: be able to *export* and *delete* a person's data on request. Deletion across replicas, caches, search indexes and backups is hard: design **data maps** early, use **per-user encryption keys** so deleting the key "crypto-shreds" all copies (including backups), and document how long backups retain data.
- **Pseudonymisation and anonymisation**: replace identifiers with tokens; aggregate data; remember that data thought anonymous can often be re-identified when combined with other data.

- **Keep PII out of logs and analytics**, or mask it.
- **Audit logs**: record who accessed or changed sensitive data and when, in append-only, tamper-evident storage separate from the application.

SUPPLEMENTAL MATERIAL

Security in the design process. Include security in design reviews (a threat model per feature), automate checks in the build pipeline (dependency scanning, static analysis, secret scanning, container image scanning), run regular penetration tests, keep a **vulnerability disclosure policy**, practise **incident response** (who decides, who communicates, how to contain, how to preserve evidence), and patch quickly: most breaches use known, already-fixed flaws.

FAILURE MODES

Failure modes. Leaked credentials in a public repository; an expired or mis-issued certificate; broken access control exposing other users' data; a compromised dependency; session tokens stolen via XSS; an unprotected admin interface; backups unencrypted or accessible to everyone; keys stored beside the data they protect; log files containing passwords or card numbers; a successful DDoS exhausting capacity; insider misuse with no audit trail; encryption turned on but keys never rotated.

COMMON MISCONCEPTION

- "*We use HTTPS, so we are secure*." TLS protects data in transit only; it does nothing against injection, broken access control or weak passwords.
- "*JWTs are encrypted*." They are signed, not encrypted; the contents are readable.
- "*Client-side validation is enough*." Attackers do not use your client.
- "*Security through obscurity (hidden URLs, secret formats) protects us*." It only delays attackers; assume the design is known.
- "*Hashing and encryption are the same*." Hashing is one-way (passwords); encryption is reversible with a key (data).

INTERVIEW QUESTIONS

Q1. Authentication versus authorization? Authentication verifies identity (who you are); authorization decides permissions (what you may do). A logged-in user must still be checked against the specific resource on every request.

Q2. How do you store passwords? Never plain text or fast hashes; use a unique salt and a slow, memory-hard hash (Argon2id, bcrypt, scrypt); add MFA, rate limiting and breached-password checks.

Q3. How do you prevent SQL injection? Parameterised queries (prepared statements), least-privilege database accounts, input validation; never concatenate input into SQL.

Q4. Sessions or JWTs? Sessions: easy revocation, need a shared store. JWTs: stateless verification, hard to revoke; use short expiry plus refresh tokens. Choose by revocation needs and architecture.

EXERCISES

1. List three STRIDE threats for the paste service of Chapter 1 and one control for each.
2. An endpoint `GET /invoices/{id}` returns any invoice to any logged-in user. Describe the vulnerability and the fix, in code or pseudocode.
3. Design secret handling for a service that needs a database password and a payment API key, covering storage, rotation and a leak response.
4. A user asks to be forgotten. List the places their data may live in the architecture of Chapter 1 and a plan to delete it.

Answers.

1. Spoofing: attacker posts as another user, so require authentication and MFA. Information disclosure: private pastes guessable by sequential ids, so use long random identifiers and authorisation checks. Denial of service: mass paste creation, so use rate limits and size limits. (Others: tampering with content, so make pastes immutable; elevation, so least privilege.)
2. Broken access control (IDOR). Fix: `invoice = db.get(id); if invoice.owner_id != current_user.id and not current_user.is_admin: return 404`, deriving `current_user` from the verified session; return 404 rather than 403 to avoid confirming existence.
3. Store in a secrets manager; inject at start-up through the runtime environment or a mounted file; use short-lived database credentials or rotate at least quarterly; separate credentials per service; on a leak, revoke immediately, rotate, audit use, and search logs for abuse.
4. Primary database, replicas, caches, search index, object store, logs, analytics warehouse, backups, and third-party processors. Plan: data map; delete or anonymise in each store; purge caches; propagate by event; for backups, document retention or use per-user keys; confirm and log completion.

CHAPTER SUMMARY

- Threat-model first; layer defences; apply least privilege.
- Authenticate with MFA and slow salted password hashes; use sessions or short-lived signed tokens; OAuth 2.0 plus OIDC for delegated access and login.
- Authorise on the server for every request and resource; validation on the server is authoritative.
- Prevent injection with parameterised queries; defend against XSS, CSRF, SSRF.
- Encrypt in transit and at rest; manage keys and secrets properly; plan for abuse; build privacy in (minimise, retain briefly, make deletion possible); keep audit logs.

REVISION CHECKLIST

- [ ] I can apply STRIDE to an architecture diagram.
- [ ] I can explain salted slow password hashing.
- [ ] I can explain the OAuth 2.0 code flow and PKCE.
- [ ] I can spot and fix an IDOR.
- [ ] I can explain envelope encryption and secrets management.

CHAPTER 18

Observability and Reliability Engineering

Derived from your notes: Notebook chapter 2 (availability in numbers) and chapter 9 (performance bottleneck hunting) and the "symptom to first fix" revision sheet. **Added in this book:** logs, metrics, traces, alerting, SLIs, SLOs, SLAs and error budgets, incident response, postmortems, RPO and RTO, backup restoration drills and disaster-recovery strategies.

LEARNING OBJECTIVES

After this chapter you can:

- explain the three pillars (logs, metrics, traces) and what each is best for;
- choose useful metrics (RED, USE, the golden signals) and avoid high-cardinality traps;
- define SLIs, SLOs and SLAs, and compute an error budget;
- design alerts that page for user-visible symptoms and link to runbooks;
- run an incident and write a blameless postmortem;
- define RPO and RTO and choose a disaster-recovery strategy.

PREREQUISITE KNOWLEDGE

Chapter 4 (availability arithmetic), Chapter 15 and Chapter 16.

KEY TERMINOLOGY

Observability
The ability to understand a system's internal state from the data it emits (logs, metrics, traces) without shipping new code.

SLI (service level indicator)
A measured quantity describing service quality, such as the fraction of requests answered successfully within 300 ms.

SLO (service level objective)
A target for an SLI over a period, for example 99.9% over 30 days.

SLA (service level agreement)
A contract with consequences (credits, penalties) if a promised level is missed.

Error budget
The allowed amount of unreliability: 100% minus the SLO.

RPO / RTO
Recovery point objective (how much data loss is tolerable, measured in time) and recovery time objective (how long recovery may take).

18.1 Seeing inside a running system

With one server, you can log in and look. With a hundred containers across zones, calling dozens of services, you cannot. You must design the system to **emit** the evidence you will need during an outage you cannot yet predict. **Monitoring** answers known questions ("is CPU above 90%?"); **observability** lets you ask *new* questions ("why are requests from this region slow only for premium users?") by slicing rich data after the fact. Three kinds of telemetry cooperate.

Table 18.1 The three pillars.

Pillar	What it is	Best for	Cost and limits
Logs	Timestamped records of discrete events	Details of one event, errors, audits	Volume; expensive to store and search
Metrics	Numbers aggregated over time	Trends, dashboards, alerting, capacity	Lose per-request detail; cardinality limits
Traces	The path and timing of one request across services	Finding where latency or errors arise	Needs instrumentation; sampling loses some requests

18.2 Logs

Write **structured logs** (JSON key-value pairs), not free text: `{"ts":"...","level":"error","service":"orders","order_id":981,"msg":"payment timeout","trace_id":"ab12"}`. Machines can then filter and aggregate. Practices:

- Use **levels** (debug, info, warn, error) and set production to info or warn.
- Include a **correlation (request or trace) id** in every line and propagate it through every service call, so you can gather everything one request did.
- Log **events and decisions**, not secrets or personal data (Chapter 17).
- Ship logs off the machine to central storage (a crashing machine takes its local logs with it); control volume (sampling, retention) because logs are often the biggest observability cost.
- Do not rely on logs for counting; count with metrics.

18.3 Metrics

Metric types: a **counter** only increases (requests served, errors; you chart its *rate*); a **gauge** goes up and down (queue depth, memory in use); a **histogram** records the distribution of values (request latency) so you can compute percentiles (Chapter 1). Never alert on average latency; use p95 or p99.

What to measure? Do not invent from scratch; use proven checklists.

- **RED** (for request-driven services): **R**ate (requests per second), **E**rrors (failed fraction), **D**uration (latency distribution).

- **USE** (for resources such as CPU, disk, network, connection pools): **U**tilisation (busy fraction), **S**aturation (queued or waiting work), **E**rrors.
- Google's **four golden signals**: latency, traffic, errors, saturation.

Add **business metrics** (signups, orders, payments succeeded) because an outage that technical metrics miss still shows up there. Keep metric **cardinality** under control: each distinct combination of label values is a separate time series; labelling by `user_id` or `url` with unlimited values creates millions of series and breaks the monitoring system. Use bounded labels (service, endpoint pattern, status class) and put high-cardinality detail in logs and traces.

18.4 Traces

A **distributed trace** follows one request through many services. Each unit of work is a **span** with a start, duration, and a parent; the whole tree shares a **trace id** passed along in request headers (**context propagation**, standardised by W3C Trace Context and implemented by OpenTelemetry). The waterfall view shows exactly where time goes and which call failed.

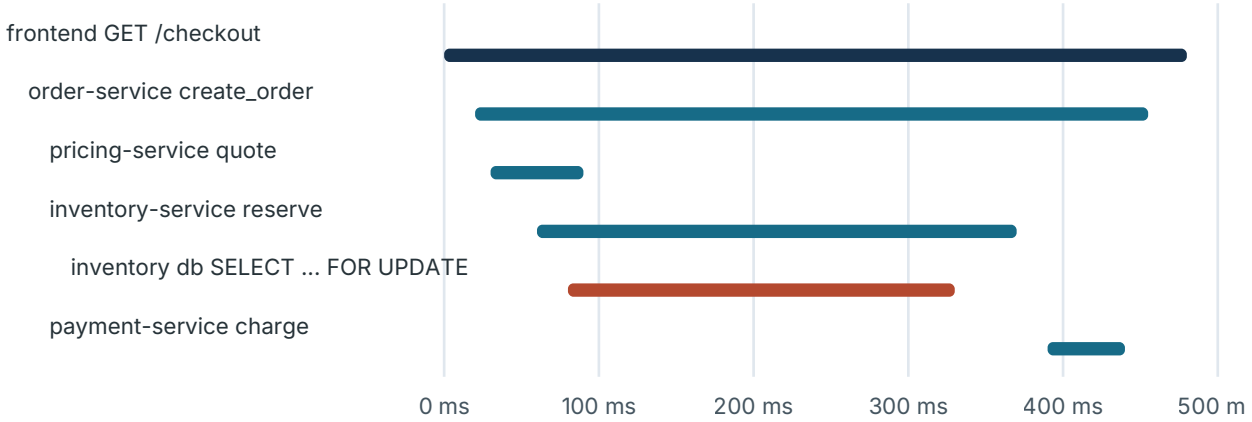


Figure 18.1 A trace waterfall for one request. The total of 480 ms is dominated by one slow inventory query inside the order service; no single service's metrics would show that so clearly.

Tracing every request is costly, so systems **sample**: keep a fixed fraction (say 1%), plus **all errors and slow requests** (tail-based sampling). Instrument at framework level (HTTP servers and clients, database drivers) so most spans appear automatically.

18.5 Alerting

An alert should mean "*a human must act now*". Too many alerts cause **alert fatigue**: people stop reacting, and the one real page is ignored. Principles:

- **Alert on symptoms users feel** (error rate, latency, availability against the SLO), not on every cause (CPU 85%). Use cause-based signals for *dashboards and tickets*.

- **Page only for urgent, actionable** conditions; send the rest to a ticket queue or a chat channel.
- Use **burn-rate alerts** (below): they fire when the error budget is being consumed too fast, which is both sensitive and robust against noise.
- Every alert links to a **runbook**: what it means, how to confirm, likely causes, first mitigations, who to escalate to.
- Review alerts regularly: delete the noisy, add missing ones after incidents.

18.6 SLIs, SLOs, SLAs and error budgets

Engineering reliability means deciding *how reliable is reliable enough*, because 100% is impossible and absurdly expensive.

- Choose **SLIs** that match user experience: *availability* (successful requests / valid requests), *latency* (fraction of requests faster than 300 ms), *freshness*, *correctness*, *durability*.
- Set an **SLO** for each: for example "99.9% of requests succeed, measured over a rolling 30 days".
- The **error budget** is the unreliability you may spend: 0.1% of requests (or about 43.8 minutes in 30 days). As long as budget remains, ship features and take risks; when it is exhausted, freeze risky changes and invest in reliability. This turns the argument between "move fast" and "be stable" into arithmetic.
- An **SLA** is a *contract* (with refunds) and is deliberately **looser** than the internal SLO, so you notice and fix problems before you owe money.

> **WORKED EXAMPLE**
>
> **Error budget and burn rate.** SLO: 99.9% success over 30 days. A service handles 100 requests per second on average, so about 259 million requests in 30 days; the budget is 0.1% of them, about 259,000 failed requests. Today an incident causes 5% errors for 20 minutes: failed = 0.05 × 100 × 1,200 = 6,000 requests, so about 2.3% of the monthly budget is gone. The *burn rate* is how fast the budget is being spent relative to plan: 5% errors against an allowed 0.1% is a burn rate of 50, meaning the entire month's budget would vanish in 30 days / 50 = 14.4 hours. A common policy pages when the burn rate exceeds about 14 over one hour (the budget would be consumed in about two days), and raises a ticket for slow burns (rate 1 to 3 over days). This ties alerts directly to what users and the business care about.

18.7 Incident response

Incidents are inevitable; chaos during them is not.

1. **Detect and declare.** An alert or a report; declare an incident early, with a severity level (Sev1: major outage).

2. **Assign roles.** An **incident commander** coordinates (does not debug); **operations** investigates and changes things; a **communications lead** updates stakeholders and status pages; a **scribe** keeps the timeline. One voice, clear ownership.
3. **Mitigate first, diagnose later.** The goal is to stop user pain: roll back the last change, shift traffic, disable a feature flag, add capacity, shed load. Root causes can wait until the system is stable.
4. **Communicate** regularly, honestly, in plain language.
5. **Resolve and hand over**, then confirm recovery with the SLIs.
6. **Learn** with a postmortem.

A **blameless postmortem** examines how the system and process allowed the failure, not who erred (people act reasonably on what they know; fix the conditions). Include a timeline, impact (users, duration, budget spent), root causes and contributing factors, what went well, what went badly, where luck helped, and **owned, dated action items** (detection, mitigation, prevention). Share it widely. The best postmortems change systems, not just documents.

> COMMON MISTAKE
>
> **Common mistakes.** Alerting on causes instead of symptoms. Unbounded metric labels. No correlation ids. Debugging before mitigating. Postmortems that name a culprit and list no actions. SLOs set to 100%, or set and never reviewed. Dashboards nobody looks at because they do not answer "are users happy?".

18.8 Disaster recovery: RPO, RTO and strategies

Some failures take out more than a node: a region outage, ransomware, a destructive migration, an operator deleting production. **Disaster recovery (DR)** plans for them with two targets (introduced in Chapter 9):

- **RPO**: the maximum tolerable *data loss* measured in time ("we can lose at most 5 minutes of data").
- **RTO**: the maximum tolerable *downtime* ("we must be back within 1 hour").

The tighter the targets, the more it costs. Typical strategies:

Table 18.2 Disaster-recovery strategies, from cheapest to most expensive.

Strategy	What runs in the second site	Typical RTO	Typical RPO	Cost
Backup and restore	Nothing; backups stored elsewhere	Hours to a day	Hours (since last backup; minutes with log archiving)	Lowest
Pilot light	Core data replicated; compute switched off	Tens of minutes to hours	Minutes	Low
Warm standby	A scaled-down full copy, running	Minutes	Seconds to minutes	Medium

Strategy	What runs in the second site	Typical RTO	Typical RPO	Cost
Active-active multi-region	Full capacity in several regions	Seconds to none	Near zero (with conflict handling)	Highest

Whatever the strategy: **backups are not real until restored.** Schedule **restore drills**: restore a recent backup into an isolated environment, run integrity checks, measure the time, and record the result against the RTO. Test **point-in-time recovery**, test restoring in the DR region, and keep backups in a separate account with deletion protection to survive ransomware or a compromised administrator. Document the **runbook**, practise it (game days), and update it when the architecture changes.

FAILURE MODES

Failure modes. Monitoring that fails together with the system it monitors (host it separately); alert floods hiding the real signal; metrics outage during the incident; logs lost when nodes die; sampling that drops the one bad request; dashboards lagging by minutes; an untested DR plan that fails in practice (missing credentials, missing capacity quota in the DR region, an undocumented dependency); a runbook stale after a re-architecture; observability costs growing faster than the business.

COMMON MISCONCEPTION

- "*More dashboards and alerts mean more reliability.*" Focused symptoms and clear runbooks do.
- "*Monitoring and observability are the same.*" Monitoring checks known failure modes; observability helps explore unknown ones.
- "*We have backups, so we can recover.*" Only restores you have rehearsed count.
- "*A 99.99% SLO is always better than 99.9%.*" It costs vastly more and may not match what users need; each extra nine reduces your budget tenfold.

INTERVIEW QUESTIONS

Q1. How would you monitor a web service? RED metrics per service and endpoint, USE for resources, business metrics, structured logs with correlation ids, distributed traces, and symptom-based alerts tied to SLOs with runbooks.

Q2. Define SLI, SLO and SLA, and error budget. SLI: measured quality. SLO: internal target for an SLI. SLA: contractual promise with penalties, looser than the SLO. Error budget: 100% minus SLO, the allowed unreliability that governs risk-taking.

Q3. What do you do in the first minutes of a major incident? Declare it, assign roles, mitigate (roll back, shift traffic, disable features) before diagnosing, communicate, and record a timeline.

Q4. Explain RPO and RTO and how they influence design. RPO is tolerable data loss, RTO tolerable downtime. Tight RPO needs synchronous or very frequent replication; tight RTO needs warm or active standby and automated failover; both raise cost.

EXERCISES

1. An API has an SLO of 99.95% over 30 days. How many minutes of total outage does that allow? If it sees 10 minutes of complete outage in a week, how much budget remains (assume 30-day budget, only this incident)?
2. Design a metric set (RED plus two business metrics) for the paste service.
3. A page fires at 3 a.m. because CPU is 91% on one server, and by morning users were not affected. How would you fix the alert?
4. Choose a DR strategy and RPO/RTO for (a) an internal wiki, (b) a payments system.

Answers.

1. 0.05% of 43,200 minutes = 21.6 minutes. After 10 minutes of full outage: 11.6 minutes remain (about 54%).
2. Rate: `pastes_created_total`, `paste_views_total`; Errors: share of 5xx responses by endpoint class; Duration: histogram of read and create latency; business: pastes created per minute, share of views served from cache (and expired pastes accessed).
3. Remove the cause-based page; alert on a user-visible symptom (error rate or p99 latency burning the SLO budget). Keep CPU as a dashboard metric and a ticket-level alert if sustained for hours.
4. (a) Backup and restore: RPO about 24 h, RTO about a day. (b) Warm standby or active-active across zones and regions: RPO near zero via synchronous replication within a region and fast asynchronous across regions; RTO minutes, automated failover, regularly drilled.

CHAPTER SUMMARY

- Emit logs (structured, correlated), metrics (RED, USE, golden signals; watch cardinality) and traces (spans with propagated context).
- Alert on user-visible symptoms and burn rates, with runbooks.
- SLOs target reliability that users need; error budgets govern speed versus stability; SLAs are looser contracts.
- Run incidents with roles, mitigate first, learn through blameless postmortems with owned actions.
- RPO and RTO drive DR strategy and cost; backups count only when restored; rehearse.

REVISION CHECKLIST

- [ ] I can state RED, USE and the golden signals.
- [ ] I can compute an error budget and a burn rate.
- [ ] I can list incident roles and the first steps.
- [ ] I can match a DR strategy to RPO and RTO.

SUPPLEMENTAL MATERIAL

Further reading. Google, *Site Reliability Engineering* and *The Site Reliability Workbook* (SLOs, alerting on burn rate, postmortems); Charity Majors et al., *Observability Engineering*; the OpenTelemetry documentation; Brendan Gregg on the USE method; Tom Wilkie on the RED method.

CHAPTER 19

Deployment, Cloud Infrastructure, Cost and Capacity

Derived from your notes: Notebook chapter 12.8 (scaling to millions of users on AWS: the stage-by-stage table) and chapter 10 (capacity estimation). **Added in this book:** containers and orchestration, infrastructure as code, deployment strategies (rolling, blue-green, canary, feature flags), autoscaling, managed building blocks, cost estimation and capacity planning, operational ownership.

LEARNING OBJECTIVES

After this chapter you can:

- explain virtual machines, containers and orchestration, and why infrastructure is defined as code;
- choose a deployment strategy (rolling, blue-green, canary) and use feature flags to separate deployment from release;
- design autoscaling that avoids oscillation and respects slow-starting capacity;
- estimate cloud cost for a design and spot the usual cost surprises;
- run a capacity-planning cycle with load tests and headroom, and describe operational ownership.

PREREQUISITE KNOWLEDGE

Chapter 4, Chapter 7, Chapter 15, Chapter 16 and Chapter 18.

KEY TERMINOLOGY

Virtual machine (VM)
: A software-emulated computer running its own operating system on shared hardware.

Container
: A packaged process with its dependencies, isolated by the host operating system and sharing its kernel; lighter than a VM.

Orchestrator (Kubernetes)
: A system that schedules containers onto machines, restarts them, scales them and wires up networking.

Infrastructure as code (IaC)
: Describing servers, networks and services in versioned files that tools apply automatically.

CI/CD
: Continuous integration (automatically build and test every change) and continuous delivery or deployment (automatically release it).

Region / availability zone (AZ)
A geographic cloud area, and an isolated data centre within it.

19.1 From servers to platforms

Running software requires machines. The history is a series of trades between control and effort:

- **Physical servers** you own: maximum control, long lead times, you pay for peak all year.
- **Virtual machines** (cloud instances): rented by the hour, resizable in minutes, still full operating systems you patch.
- **Containers**: package the application and its libraries into an image that runs the same everywhere; start in seconds; many per machine. An **orchestrator** such as **Kubernetes** decides where containers run, restarts failures, rolls out versions, and scales them.
- **Serverless / functions**: you supply code, the provider runs it on demand and scales to zero; you pay per invocation. Excellent for spiky, event-driven work; limits include cold starts, execution time caps and less control.
- **Managed services**: the provider runs the database, queue or cache for you (backups, patching, failover included), trading money and some control for far less operational work.

The right level is the one where your team's scarce attention is spent on the product, not the plumbing. A small team should lean hard toward managed services; a very large one may build its own for cost or control.

Infrastructure as code. Define networks, machines, databases and permissions in files (Terraform, CloudFormation, Pulumi, Kubernetes manifests) kept in version control and applied by a pipeline. Benefits: reviewable changes, reproducible environments (staging resembles production), disaster recovery by re-creating from code, and no undocumented manual changes ("configuration drift"). Treat secrets and state files with care (Chapter 17).

19.2 Shipping changes safely

Most outages follow a change. The pipeline's job is to make changes **small, frequent, tested and reversible**.

CI/CD pipeline: commit; build; unit and integration tests; static analysis and security scans; package an immutable artifact (a container image); deploy to staging; automated tests; deploy to production in stages; verify with metrics; roll back automatically on regressions. **Immutable artifacts** (never patch a running server; replace it) make rollback simply "deploy the previous image".

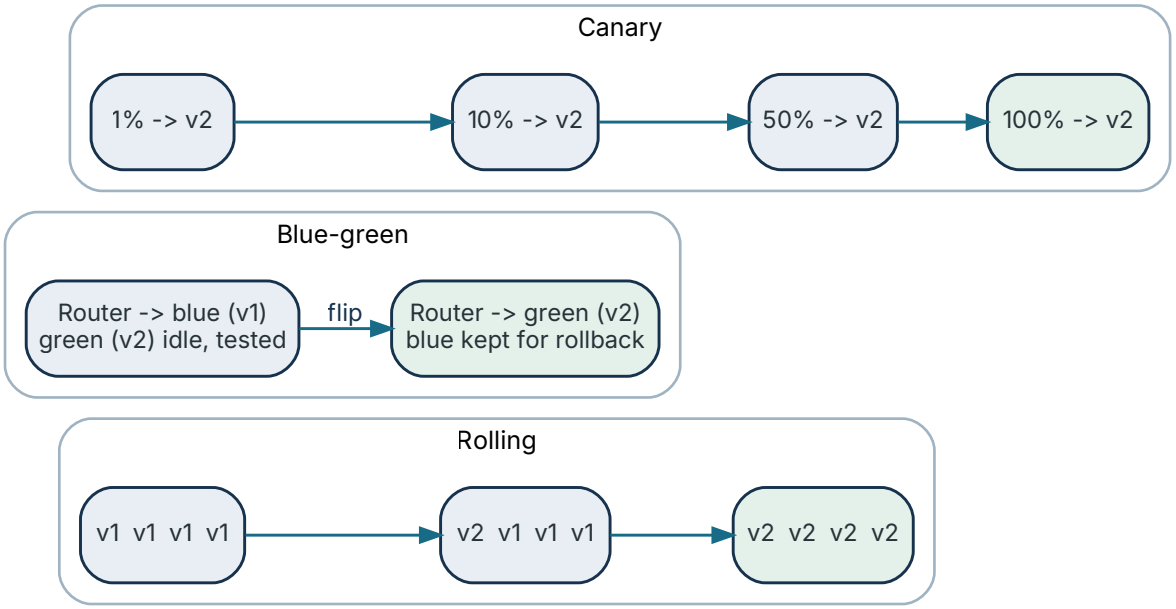


Figure 19.1 Rolling, blue-green and canary deployments. Rolling replaces instances gradually; blue-green switches all traffic between two full environments; canary sends a small share of traffic to the new version first.

Table 19.1 Deployment strategies compared.

Strategy	How	Rollback	Extra capacity	Risk
Recreate	Stop old, start new	Redeploy old	None	Downtime
Rolling	Replace a few instances at a time	Roll forward or back gradually	Small	Mixed versions run together
Blue-green	Two identical environments; flip the router	Flip back instantly	A full second environment	Database changes must work for both versions
Canary	Send 1%, 5%, 25%, 100% to the new version, watching metrics	Route back to 0%	Small	Needs good automated metrics and enough traffic

Because old and new versions **run side by side** during any gradual rollout, every change must be **backward and forward compatible**: schema migrations follow expand and contract (Chapter 8); APIs change additively (Chapter 6); messages tolerate unknown fields.

Feature flags separate *deployment* (code reaches production, switched off) from *release* (the feature is enabled for users). They allow gradual exposure by percentage or user group, instant kill switches (also the degradation controls of Chapter 15), and testing in production safely. Costs: flag debt (remove old flags), combinatorial testing, and the flag service becoming a dependency (default to a safe value if it is unreachable).

19.3 Autoscaling

Autoscaling adds and removes instances automatically as load changes, so you pay for what you use and survive spikes. Three approaches: **reactive** (scale when a metric crosses a threshold: CPU, requests per instance, queue age), **scheduled** (known patterns: scale up before 9 a.m.), and **predictive** (forecast from history).

Pitfalls and rules:

- **Scale on a metric that tracks real load**, ideally one that *leads* trouble: requests per instance or queue age beat CPU for I/O-bound services.
- **Slow starts matter.** If a new instance needs two minutes to boot, warm caches and pass health checks, reactive scaling lags the spike. Scale earlier (a lower threshold), keep a **buffer of spare capacity**, use pre-warmed pools, and use faster-starting containers.
- **Avoid flapping**: use **cooldowns** and hysteresis (scale out at 60% utilisation, scale in only below 30%), and scale in slowly.
- **Set maximums** to cap cost and to protect downstream systems: scaling the web tier tenfold means ten times the database connections (Chapter 9); autoscaling a stateless tier does nothing for a saturated database.
- **State does not autoscale easily**: databases, caches and queues are scaled deliberately (replicas, shards), not by a threshold rule.

19.4 A reference path: scaling a system up

The notebook's final design problem walks one system from a handful of users to millions. Condensed here as a decision table; the full worked version is in Chapter 28.

Table 19.2 A typical growth path and the next bottleneck at each stage.

Stage (users)	Typical architecture	Next bottleneck	Next step
1 to 100s	One server runs web, app and database	Everything shares one machine	Separate database; use a managed service
~10,000	Web servers behind a load balancer; managed database with a replica; object storage for files; CDN	Database reads	Cache; read replicas
~100,000	Stateless app tier autoscaled; cache; queue and workers for slow jobs	Database writes and size	Tune; federate; consider sharding
~1,000,000	Multiple AZs; sharded or federated data; search and analytics stores; observability and deploy automation	Operational complexity, cost	Platform teams; cost management
10,000,000 and up	Multi-region; custom data stores; cells; extensive automation	Global latency, blast radius, cost	Regional isolation, data residency, specialisation

19.5 Cost: estimation and habits

Cloud makes cost *variable and visible*, and also easy to waste. A cost estimate follows the same pattern as capacity estimation (Chapter 4): resources × unit price × time. Price lists change frequently, so use the provider's **current pricing calculator** and treat the figures below as illustrative arithmetic only.

WORKED EXAMPLE

A monthly estimate (illustrative prices, check current ones). Assume: 10 application instances at $0.10 per hour; a database at $1.00 per hour with one replica at the same price; 5 TB of object storage at $0.023 per GB-month; 20 TB of outbound transfer at $0.09 per GB; a managed cache at $0.50 per hour. A month is about 730 hours.

- Compute: 10 × 0.10 × 730 = $730.
- Database: 2 × 1.00 × 730 = $1,460.
- Cache: 0.50 × 730 = $365.
- Storage: 5,000 GB × 0.023 = $115.
- Transfer out: 20,000 GB × 0.09 = $1,800.
- Total ≈ $4,470 per month, and **the single largest line is data transfer**, which many designs forget. A CDN (often cheaper per GB than origin egress), compression and keeping chatty traffic inside one zone each reduce it. Also notice what is *not* here: observability ingestion, load balancers, NAT gateways, cross-zone traffic and support plans, which routinely add 20 to 40 per cent.

Habits that keep costs sane: **tag** every resource with owner and project so cost is attributable; budget alerts; **right-size** instances using real utilisation; use **committed-use or reserved discounts** for the steady base load and **spot (interruptible) instances** for fault-tolerant batch work; **delete** orphans (unattached disks, old snapshots, idle load balancers); **tier storage** (hot, infrequent, archive) with lifecycle rules; watch **egress** and **cross-AZ/region traffic**; log and trace selectively. And treat cost as a **design requirement**: "cost per thousand requests" is a metric worth tracking beside latency.

19.6 Capacity planning

Capacity planning ensures the system can meet next quarter's demand without over-buying. It is a cycle:

1. **Forecast demand**: growth trends, marketing events, seasonality (Chapter 4).
2. **Measure capacity**: **load test** realistic traffic against a production-like environment, increasing load until latency or errors break the SLO; record the maximum sustainable throughput *per instance* and the first bottleneck. Include **soak tests** (hours at load, to find leaks) and **spike tests**.
3. **Compute needs** with headroom: peak forecast ÷ per-instance capacity ÷ target utilisation (50% to 70%), plus **N+1 or N+AZ** spare capacity for failures and deployments.

4. **Provision** (or configure autoscaling limits), and ensure **quotas** (cloud account limits) are raised in advance, including in the DR region.
5. **Monitor utilisation and saturation**, compare with the forecast, revisit.

WORKED EXAMPLE

Per-instance capacity and fleet size. A load test shows one instance sustains 800 requests per second before p99 exceeds the SLO. Forecast peak next quarter: 12,000 requests per second. Target utilisation 60%: usable per instance = 480. Instances = 12,000 / 480 = 25. To survive the loss of one of three zones, size so that two zones carry the load: 25 × 3 / 2 ≈ 38 instances, about 13 per zone. If each costs $0.10 per hour, the fleet costs about 38 × 0.10 × 730 = $2,774 per month; the zone-failure tolerance costs 13 extra instances, about $949 per month, the price of the availability target.

19.7 Operational ownership

Software that nobody owns breaks in the night with nobody responsible. Mature organisations follow **"you build it, you run it"**: the team that writes a service is on call for it, which aligns incentives (a service that pages its authors gets more reliable). Supporting practices: a **service catalogue** (owner, on-call, dashboards, runbooks, dependencies, tier), **production-readiness reviews** before launch (monitoring, alerts, capacity, rollback, security, DR), healthy **on-call** (rotation, handover, a cap on pages per shift, compensation), **runbooks** that are tested, and a **platform team** providing paved roads (templates, pipelines, observability) so product teams do not each reinvent infrastructure.

FAILURE MODES

Failure modes. A bad deploy to all instances at once; incompatible schema and code versions during a rollout; autoscaler oscillating, or scaling the app tier until the database falls over; configuration drift between environments; a flag service outage disabling features; a cost spike from a runaway job, an unbounded log, or egress; quota exhaustion during a spike or in the DR region; deployment pipeline credentials compromised; capacity plans based on averages and failing at peak; nobody knowing who owns a service.

COMMON MISTAKE

Common mistakes. Deploying on Friday afternoon without a rollback plan. Testing in a staging environment that differs from production. Treating autoscaling as capacity planning. Ignoring egress and cross-zone costs. Running one environment manually "just this once". Starting with Kubernetes for a team of three because it is fashionable.

COMMON MISCONCEPTION

- "*The cloud scales automatically.*" Only the parts you configure (and only within quotas, and not your database).
- "*Containers make applications portable and therefore simple.*" They simplify packaging; orchestration adds its own complexity.
- "*Blue-green deployment makes releases risk-free.*" Shared databases and state still have to work for both versions.
- "*Cloud is always cheaper.*" It is cheaper for variable load and small teams; steady, huge workloads can cost more than owned hardware.

INTERVIEW QUESTIONS

Q1. Rolling, blue-green and canary: when would you use each? Rolling: default, low extra capacity. Blue-green: instant switch and rollback when you can afford two environments. Canary: risky changes at scale, checking real metrics on a small share first.

Q2. How do you deploy a database change without downtime? Expand and contract: backward-compatible schema first, dual-write, backfill, switch reads, remove the old structure later; deploy code compatible with both versions throughout.

Q3. How would you design autoscaling for a web tier? Scale on requests per instance or queue age with a lower threshold than the danger point, cooldowns and hysteresis, max limits to protect the database, and warm capacity for slow starts.

Q4. How would you estimate and reduce cloud cost? List resources by unit price and usage; find the dominant lines (often egress, storage, database); tag, right-size, use commitments for base load and spot for batch, tier storage, use a CDN, and delete orphans.

EXERCISES

1. Load test: one instance sustains 500 requests/s. Forecast peak is 9,000 requests/s; target utilisation 70%; two zones, must survive losing one. How many instances in total?
2. A canary at 5% shows an error rate of 0.9% versus 0.2% on the stable version, over 2,000 requests. Should you continue? What additional information do you want?
3. Estimate the monthly cost of transferring 50 TB out at $0.08 per GB, then show how a CDN with a 90% offload at $0.04 per GB changes it (ignore request fees).
4. Explain how a feature flag helps with a risky release and name two costs of flags.

Answers.

1. Per instance usable = 500 × 0.7 = 350. Needed = 9,000 / 350 ≈ 26 instances. Surviving the loss of one of two zones means each zone alone must carry everything: 26 per zone, 52 in total.
2. Probably no: 0.9% is more than four times 0.2%; with 100 canary requests per error the difference is likely real (about 18 errors vs 4 expected from 2,000 requests). Check statistical significance, error types, latency and whether the canary traffic is comparable; halt or roll back.
3. Direct: 50,000 GB × 0.08 = $4,000. With CDN: origin egress = 10% × 50,000 × 0.08 = $400 plus CDN delivery 50,000 × 0.04 = $2,000 (CDN delivery applies to all bytes), total $2,400: a saving of about 40% here; the saving grows if CDN prices drop with volume or caching removes more origin traffic.

4. A flag lets code ship off, then be enabled for 1%, 10%, then all users, and be turned off instantly if metrics degrade. Costs: stale flags accumulate (technical debt) and the number of untested combinations grows; the flag service becomes a dependency.

CHAPTER SUMMARY

- Choose the level of platform (VMs, containers, serverless, managed) that frees your team's attention; define infrastructure as code.
- Ship small, tested, reversible changes; use rolling, blue-green or canary with compatible versions; feature flags separate deploy from release.
- Autoscale stateless tiers on leading metrics with cooldowns and limits; protect state tiers.
- Estimate cost from unit prices and watch egress; plan capacity from load tests with headroom and zone-failure margin.
- Teams own what they run; production-readiness reviews and a platform team make that sustainable.

REVISION CHECKLIST

- [ ] I can compare rolling, blue-green and canary.
- [ ] I can size a fleet from a load test, headroom and zone tolerance.
- [ ] I can estimate a monthly cost and find the dominant line.
- [ ] I can design autoscaling thresholds without flapping.

SUPPLEMENTAL MATERIAL

Further reading. The AWS Well-Architected Framework (and the equivalents from other providers) for current guidance; Jez Humble and David Farley, *Continuous Delivery*; Kief Morris, *Infrastructure as Code*; the FinOps Foundation's framework for cloud cost management; Google's SRE Workbook on canarying releases.

PART VII

Design Interviews and Worked Systems

A repeatable method for open-ended design problems, then eight complete worked designs and two end-to-end exercises.

CHAPTER 20

The System Design Interview Method

Derived from your notes: Notebook chapter 11 (the four steps, the 20-point checklist, the reusable question bank, common mistakes) and the "how to use" study plan. **Added in this book:** the 45-minute timeline, communication techniques, level expectations, a scoring rubric, the standard answer template used by every case study, and a practice routine.

LEARNING OBJECTIVES

After this chapter you can:

- lead an open-ended design conversation using the four steps within a 45-minute plan;
- apply the 20-point checklist so no important area is skipped;
- communicate trade-offs, numbers and failure modes in a structured way;
- recognise what interviewers look for at different levels, and avoid the common mistakes;
- practise effectively using the answer template used by every case study in this part.

PREREQUISITE KNOWLEDGE

Chapter 1 to Chapter 19. This chapter organises what you already know into a method; it adds almost no new technology.

KEY TERMINOLOGY

Open-ended problem
A task with no single correct answer, judged by the quality of reasoning and trade-offs.

Whiteboard (or shared canvas)
The drawing surface used to sketch the design while talking.

Deep dive
Going into detail on one or two components where the problem is hardest.

Trade-off narration
Saying aloud what each choice gains, costs and when you would choose differently.

20.1 What the interview is testing

A system design interview is **not** a quiz with a hidden answer. It is a simulation of the first hour of a design meeting with a colleague. The interviewer wants evidence of several skills at once:

- **Problem framing**: do you ask questions and pin down requirements before designing?

- **Quantitative judgement**: do you use numbers to choose between designs (Chapter 4)?
- **Breadth**: do you know the standard components and when each applies?
- **Depth**: can you go deep on one hard part, such as a data model, a consistency rule, or a failure scenario?
- **Trade-off thinking**: can you say what you give up?
- **Communication**: can the interviewer follow your reasoning? Do you adapt when they redirect you?
- **Pragmatism**: do you start simple and scale only when pressure justifies it?

The problem is deliberately **under-specified**, and the interview is deliberately **interactive**: hints from the interviewer are signals to follow, not traps.

20.2 The four steps and the 45-minute plan

Figure 20.1 A 45-minute plan. Spend the first ten minutes on requirements and numbers, then shape the design, then go deep where the problem is hard, and keep time to scale and discuss failure.

Step 1: use cases, constraints and assumptions (about 10 minutes)

Gather requirements and **scope** the problem. Ask who will use it and how, how many users, what the inputs and outputs are, how much data, how many requests per second, the read-to-write ratio, and which qualities matter (latency, availability, consistency, durability). State **assumptions** explicitly and get agreement ("I will assume 100 million pastes per month, ten times more reads than writes; stop me if that is wrong"). Then **estimate** (Chapter 4): requests per second, storage, bandwidth, memory.

Step 2: high-level design (about 10 minutes)

Sketch the main components and connections: clients, entry layer, services, data stores, caches, queues. **Justify each component** by tying it to a requirement or number ("a cache because reads are 100 times writes"). Define the **API** and the **data model** briefly. **Trace one request** from user to storage and back, and one write path. Resist adding components for their own sake.

Step 3: design the core components (about 15 minutes)

Go **deep where the problem is hard**. For a URL shortener: key generation and collisions, the schema, the lookup path. For a feed: fan-out strategy and the celebrity problem. For a crawler: the frontier, politeness and duplicate detection. Choose one or two such areas, ideally those the interviewer shows interest in, and think aloud through alternatives. Include data models, indexes, consistency requirements and what happens on failure.

Step 4: scale the design (about 10 minutes)

Find bottlenecks and fix them with the tools of the previous chapters: load balancing, horizontal scaling, caching, replication, sharding, queues. Discuss **availability and failure** (what if a node, zone or dependency dies?), **security**, **monitoring**, and **alternatives with trade-offs**. Close by re-walking a request and listing two or three improvements for "more time".

20.3 The 20-point checklist

The four steps expand into a fuller list. You do not have to cover every row in depth, but you should *touch* each one, spending time where the problem is hard.

Table 20.1 The 20-point checklist.

#	Item	Typical content
1	Clarify the problem	Goal, users, what is in and out of scope
2	Functional requirements	The key features (verbs)
3	Non-functional requirements	Latency, availability, consistency, durability, security
4	Scale	Users, daily active users, growth
5	Traffic estimate	Reads and writes per second, peak
6	Storage estimate	Bytes per item, retention, replication
7	API design	Endpoints, request and response shapes, errors
8	Data model	Entities, keys, indexes, SQL or NoSQL and why
9	High-level architecture	Boxes and arrows with justification
10	Request flow	Read and write paths end to end
11	Core components	The hard part in depth
12	Bottlenecks	What saturates first, and why
13	Scaling	Cache, replicas, shards, queues, CDN
14	Availability	Redundancy, failover, zones
15	Consistency	What may be stale, for how long, why acceptable
16	Failure handling	Timeouts, retries, degradation, recovery
17	Security and privacy	AuthN, AuthZ, abuse, data protection
18	Monitoring and operations	Metrics, alerts, deployment, cost
19	Trade-offs and alternatives	What you gave up, other options
20	Summary and next steps	Two-minute recap, improvements

20.4 Communicating well

Technique matters as much as content.

- **Think aloud, structure out loud.** "First I will clarify requirements, then estimate, then sketch. I will go deep on key generation." Signposting keeps the interviewer oriented.
- **Draw as you talk**, keep the diagram clean, label arrows with *what flows* (HTTP, event, SQL), and number the steps of a flow.
- **Anchor on numbers.** Replace "a lot of traffic" with "about 4,000 reads per second, peak 10,000".
- **Narrate every trade-off** in a repeatable form: "*We could use A, which gains X but costs Y. Given that our requirement is Z, I choose A, and I would revisit if W.*"
- **Start simple, then evolve.** "The simplest design is one server and one database. At our numbers the first thing to break is the database reads, so I add a cache."
- **Check in** at transitions ("is this the area you want me to go deeper on?") and **follow hints**.
- **Admit gaps** honestly ("I have not used that database; I would expect X; I would verify Y"). Fabricating certainty is worse than a measured "I don't know, here is how I would find out".
- **Own the time**: if a topic is running long, say so and move on.

> **WORKED EXAMPLE**
>
> **Narrating a trade-off well.** *Question:* "How will you generate the short codes?" *Weak answer:* "I'll use a hash." *Strong answer:* "Two options. One: hash the long URL with MD5 and take the first seven Base62 characters. It is stateless and the same URL gives the same code, but different URLs can collide, so I must check and retry, and 7 characters only gives $62^7 \approx 3.5$ trillion codes. Two: allocate numeric ids from a counter and encode them in Base62: no collisions, but a central counter is a bottleneck and ids are guessable, which may matter for private pastes. At roughly 40 writes per second either works; I would pick the pre-allocated id ranges per server, with a random component if guessability matters, and I would add a unique constraint as a safety net." It names options, numbers, costs and a decision.

20.5 What interviewers expect at different levels

Expectations rise with seniority; calibrate your emphasis.

Table 20.2 Typical expectations by level (indicative, not a rule).

Level	Expected	Differentiators
Early career	A working design for moderate scale; correct use of core components (load balancer, cache, SQL); clear communication	Asking good questions; understanding why components exist

Level	Expected	Differentiators
Mid-level	Sound estimates; sensible data models; scaling plan; some failure handling	Making and defending trade-offs; going deep on one component
Senior	Leads the discussion; chooses depth wisely; consistency, availability and operations; cost and migration awareness	Anticipating bottlenecks and failure modes; evolution over time
Staff and above	Frames the problem and success criteria; cross-system impact; organisational and long-term considerations	Simplifying; risk management; saying no to unnecessary complexity

20.6 Mistakes to avoid

COMMON MISTAKE

Common mistakes. Jumping straight into a database schema or a favourite technology before requirements. Drawing a perfect "final" architecture from memory without reasoning (reciting a company's diagram instead of deriving from the load). Ignoring numbers. Never naming a failure mode. Over-engineering (microservices and five databases for a hundred requests per second). Going silent while thinking. Defending a choice instead of re-examining it when the interviewer hints at a problem. Spending thirty minutes on one component and never scaling. Talking only about the happy path.

20.7 A reusable answer template

Every case study in this part follows the same skeleton so you can internalise it: **problem and requirements; scale estimate; API; data model; high-level design; core flows; deep dive on the hard part; scaling and bottlenecks; failure handling, security and operations; trade-offs and alternatives; lessons and follow-ups.** In practice, memorise the *questions* rather than a script.

Table 20.3 A reusable question bank to ask yourself at every box.

Question	What it exposes
Why does this component exist? What breaks without it?	Unjustified boxes
What happens at ten times the traffic? Which resource saturates first?	Bottlenecks
What happens if this server, database, zone fails? How do we detect and recover?	Availability gaps
What may be stale here, and for how long?	Consistency decisions
What are we giving up with this choice?	Trade-offs
What would I do differently with twice the budget, or half?	Cost awareness
How would a malicious user abuse this?	Security gaps
How will we know it is broken?	Observability

20.8 Practising effectively

Reading designs is not the same as producing them. A productive routine:

1. **Pick a problem** (Chapter 21 to Chapter 28 and the practice bank in Chapter 31).
2. **Attempt before reading**: cover the solution, set a 45-minute timer, and sketch your own design on paper, talking aloud or recording yourself.
3. **Compare** with the worked solution: what did you miss in requirements, numbers, data model, failures?
4. **Redo the weakest part** in 10 minutes.
5. **Practise with a person** if possible (a mock interview), asking them to interrupt and challenge assumptions.
6. **Keep a trade-off notebook**: for each technology, one line each for "gains", "costs", "use when".

SUPPLEMENTAL MATERIAL

A simple scoring rubric for mock interviews. Score each dimension 1 (weak) to 4 (strong): *requirements and scoping*; *estimation*; *architecture and component choice*; *data model and APIs*; *depth on the hard part*; *scaling and bottlenecks*; *reliability and failure handling*; *trade-off articulation*; *communication and time management*. Total out of 36. Practise until no dimension is below 3. Note that *consistency of method* matters more than perfection on any one problem.

FAILURE MODES

Failure modes of the interviewee. Running out of time with the design half-finished; being unable to recover after a wrong assumption; getting stuck on a detail the interviewer does not care about; contradicting earlier statements (a strongly consistent store claimed, then a cache without invalidation drawn); forgetting to ask about scale and designing the wrong system; treating the interview as a performance rather than a conversation.

COMMON MISCONCEPTION

- "*There is one right answer.*" Different designs can be right for different assumptions; the reasoning is what is graded.
- "*More components impress.*" Justified simplicity impresses; unjustified complexity does not.
- "*Memorising famous architectures is enough.*" Interviewers vary the problem; derivation from numbers generalises.
- "*Interviewers want me to know their tools.*" They want the underlying concepts; name a technology as an example, but argue from properties.

INTERVIEW QUESTIONS

Q1. How do you start a design interview? By clarifying requirements and scope, stating assumptions and estimating scale, before drawing anything.

Q2. How do you decide where to go deep? Pick the component that is hardest or most distinctive for the problem and where the interviewer shows interest (for a feed, fan-out; for a crawler, the frontier; for a shortener, key generation).

Q3. How do you handle an interviewer's challenge to your design? Treat it as information: restate the concern, evaluate it with numbers or failure scenarios, and either adjust the design or explain the trade-off and the conditions under which you would change it.

Q4. What do you do if you do not know a technology? Say so, describe the properties you would need, reason from principles, and indicate how you would verify.

EXERCISES

1. Write the questions you would ask in the first five minutes for "design a ride-hailing service".
2. Spend ten minutes producing the 20-point checklist's items 1 to 6 for "design a photo-sharing service": requirements, scale and estimates.
3. Rehearse narrating one trade-off aloud in the form "We could ... which gains ... but costs ...; given ..., I choose ...".
4. Review a previous design you made: which of the 20 points did you skip?

Answers (model for 1). Who are the users (riders, drivers, both)? Which features are in scope (request ride, matching, pricing, payments, ratings, scheduled rides)? Which regions and how many daily rides? What is the latency target for matching? How accurate and how frequent are location updates (every few seconds)? What are availability and consistency needs (a driver must not be double-booked)? Out-of-scope items (pooling, food delivery)? Then estimate: rides per second, location updates per second (usually dominant), storage of trip history.

CHAPTER SUMMARY

- The interview tests framing, numbers, breadth, depth, trade-offs and communication, in an open-ended conversation you lead.
- Four steps: requirements and estimates; high-level design; core components; scale and failure; budget roughly 10, 10, 15 and 10 minutes.
- The 20-point checklist ensures coverage; the question bank exposes gaps.
- Narrate trade-offs in a standard form, anchor on numbers, start simple, signpost, check in.
- Practise by attempting first, comparing, redoing, and rehearsing aloud.

REVISION CHECKLIST

- [ ] I can recite the four steps and the 45-minute plan.
- [ ] I can list the 20 checklist items.
- [ ] I can narrate a trade-off in the standard form.
- [ ] I can name five common mistakes.

CHAPTER 21

Case Study: Pastebin and URL Shortener

Derived from your notes: Notebook chapter 12.1 (Pastebin and URL shortener: requirements, the scale table, Base62 and MD5 key generation, SQL schema, read and write flows, analytics and expiry). **Added in this book:** API contract, key-generation comparison with collision mathematics, object-store split, abuse and security, failure handling, cost, alternatives, follow-up questions.

LEARNING OBJECTIVES

After this chapter you can:

- scope a design problem and derive its scale from stated assumptions;
- design an API, data model and key-generation scheme for short links, comparing hashing, counters and pre-generated keys;
- trace write and read paths and apply caching, expiry and cleanup;
- find the bottlenecks and failure modes of the design and explain what you would change as traffic grows.

PREREQUISITE KNOWLEDGE

Chapter 4, Chapter 6, Chapter 8, Chapter 12 and Chapter 20.

21.1 Problem and requirements

The problem. Design a service where a user pastes text (or submits a long URL) and receives a short link; anyone with the link can view the content (or be redirected). This is the classic first design problem because the idea is simple while the interesting decisions (unique keys, read-heavy traffic, expiry, abuse) are real.

Functional requirements. Create a paste and get a short URL. Open a short URL and see the text. Optionally set an **expiration** (default: never, or a bounded default). Optional: private pastes. Basic **analytics** (monthly visit counts per link).

Non-functional requirements. High availability for reads: a broken link is a visible failure. **Low latency** for reads (target p95 under 200 ms). **Durability**: acknowledged pastes must not be lost. Links should be **hard to guess** for private content. Reads are far more frequent than writes.

Scope and assumptions. Out of scope: accounts, editing, syntax highlighting, full search. The service is **read-heavy** with a **10 : 1 read-to-write ratio**. Text pastes are small (about 1 KB on average, with an upper limit such as 512 KB).

21.2 Scale estimate

Table 21.1 Scale assumptions and derived quantities (from the notebook; all are assumptions, not measurements).

Quantity	Assumption	Derived
Writes	10 million pastes per month	10^7 / 2.5×10^6 s ≈ 4 per second (peak ~10×: ~40/s)
Reads	100 million views per month (10 : 1)	≈ 40 per second (peak ~400/s)
Size per paste	1 KB text + ≈ 270 B metadata	≈ 1.27 KB
Storage growth	10 M × 1.27 KB	≈ 12.7 GB per month; ≈ 450 GB in 3 years
Number of links	10 M per month for 36 months	360 million links
Key space needed	360 M links, with margin	7 Base62 characters = 62^7 ≈ 3.5 trillion

What the numbers say. 40 reads per second is *tiny*: a single database server and one application server could handle it. The scale is not the challenge. The interesting problems are (1) **generating unique, compact keys**, (2) **durable, cheap storage of content**, (3) **cleaning up expired data**, and (4) **abuse**. Conveying this early ("this is a small-scale problem; the care goes into keys and operations") shows judgement. We still design for growth: ten times and a hundred times these numbers (400 and 4,000 reads per second) remain modest, so the design should scale by adding standard components, not by re-architecting.

21.3 API

```
POST /v1/pastes
{"content": "hello world", "expires_in_seconds": 86400, "visibility": "unlisted"}
-> 201 Created   Location: /v1/pastes/7Hq2Zx
   {"id": "7Hq2Zx", "url": "https://paste.example/7Hq2Zx", "expires_at":
"2026-10-13T09:30:00Z"}

GET /{id}               -> 200 text/plain  (or 302 redirect for a URL shortener)
                           404 Not Found | 410 Gone (expired)
DELETE /v1/pastes/{id}  -> 204   (owner token required)
GET /v1/pastes/{id}/stats -> 200 {"month": "2026-10", "views": 1832}
```

Design notes: creation accepts an `Idempotency-Key` header so a retried create does not make two pastes (Chapter 6); the **delete token** is returned at creation, so anonymous users can still delete their own pastes; limit content size and enforce it on the server; return **410 Gone** for expired links to distinguish them from never-existed ones; for a URL shortener `GET /{id}` responds with a `301` or `302` redirect (a **301** is permanent and cached by browsers, so analytics miss repeat visits and later changes are ignored; a **302** keeps every click visible to the service).

21.4 Data model

Separate **metadata** (small, queried, indexed) from **content** (larger, opaque, only fetched by key).

```
CREATE TABLE pastes (
  short_code   CHAR(7)      PRIMARY KEY,          -- the Base62 key
  content_key  VARCHAR(255) NOT NULL,             -- object-store key for the text
  owner_id     BIGINT       NULL,
  visibility   SMALLINT     NOT NULL DEFAULT 0,
  created_at   TIMESTAMP    NOT NULL,
  expires_at   TIMESTAMP    NULL,
  delete_token_hash CHAR(64) NOT NULL
);
CREATE INDEX idx_pastes_expires ON pastes (expires_at) WHERE expires_at IS NOT NULL;
```

Why this split? Pastes are up to hundreds of KB; storing them in database rows makes the database large and slows backups and replication. An **object store** is cheaper per gigabyte, durable, and scales effortlessly, and the database row stays small and cache-friendly. For pastes under a few KB, storing the text *inline* in the row is also a reasonable simplification; decide with the size distribution. **Primary key on** `short_code` makes the dominant lookup (by code) a single index seek. The index on `expires_at` supports the cleanup job. If the data outgrows one database, shard by `short_code` (hash), which is present in every read.

SQL or NoSQL? At 360 million small rows (about 100 GB of metadata) a relational database is comfortable, and its transactions help with uniqueness. A key-value or document store also fits (access is by key only) and scales further with less effort; choose on team skill and operational preference. The point to state is the *access pattern*: lookup by key, no joins.

21.5 Generating keys

The short code must be **unique**, **compact** (seven characters), and, for private pastes, **unguessable**. Three approaches:

1. Hash and truncate. Compute `MD5(content + timestamp + random)`, encode in Base62 (characters `a-z A-Z 0-9`), and take the first seven characters. Stateless and simple, but **collisions are possible** after truncation, so insert with a unique constraint and *retry with a new salt* on conflict. With n existing keys in a space of size $N = 62^7 \approx 3.5 \times 10^{12}$, the chance a new random key collides is about n / N. At $n = 360$ million: $3.6 \times 10^8 / 3.5 \times 10^{12} \approx 1$ in 10,000. So roughly one in ten thousand creates retries once; the second attempt almost always succeeds. That is cheap, but the *check* requires a database round trip anyway.

2. Counter plus Base62. Keep a global counter, give each new paste the next integer, and Base62-encode it (counter 1,000,000 becomes `4c92`). **No collisions**, minimal length (seven characters last until 3.5 trillion). Costs: a central counter is a **bottleneck and a single point of failure** (mitigated by handing each application server a **block of ids** at a

time, say 10,000), and sequential codes are **guessable and enumerable**, which leaks private pastes and lets competitors scrape. Fix by passing the number through a reversible scrambling step (a format-preserving permutation) so codes look random.

3. Pre-generated key service. A background job generates many random unused keys into a "key database" with two tables (*unused*, *used*). Application servers fetch a batch of keys into memory and hand them out; keys are moved to *used* when taken. Creation never hits collisions and never computes anything on the hot path, and the codes are unguessable. Costs: another component; keys fetched but unused when a server crashes are simply lost (acceptable in a 3.5-trillion space); the key database must be replicated.

Table 21.2 Key-generation approaches.

Approach	Collisions	Guessable	Hot-path cost	Failure risk	Best when
Hash + retry	Rare, handled by retry	No	One unique-insert, maybe a retry	Low	Small systems; stateless
Counter + Base62	None	Yes (unless scrambled)	Minimal with id blocks	Counter service	Public links, simplicity
Pre-generated keys	None	No	Fetch from in-memory batch	Key service	Large scale, private links

WORKED EXAMPLE

How long must the code be? For 360 million links, can we use 6 characters? $62^6 \approx 56.8$ billion. The *fill ratio* is 360 M / 56.8 B ≈ 0.63%, so a random 6-character code collides about 1 in 160 times: workable but with more retries and less margin for growth. With 7 characters the key space is 3.5 trillion (fill ratio 0.01%), giving a collision rate of about 1 in 10,000 and headroom for a hundred times growth. Seven characters it is. If custom aliases are allowed ("paste.example/my-notes"), reserve a separate namespace and always check availability.

21.6 Architecture

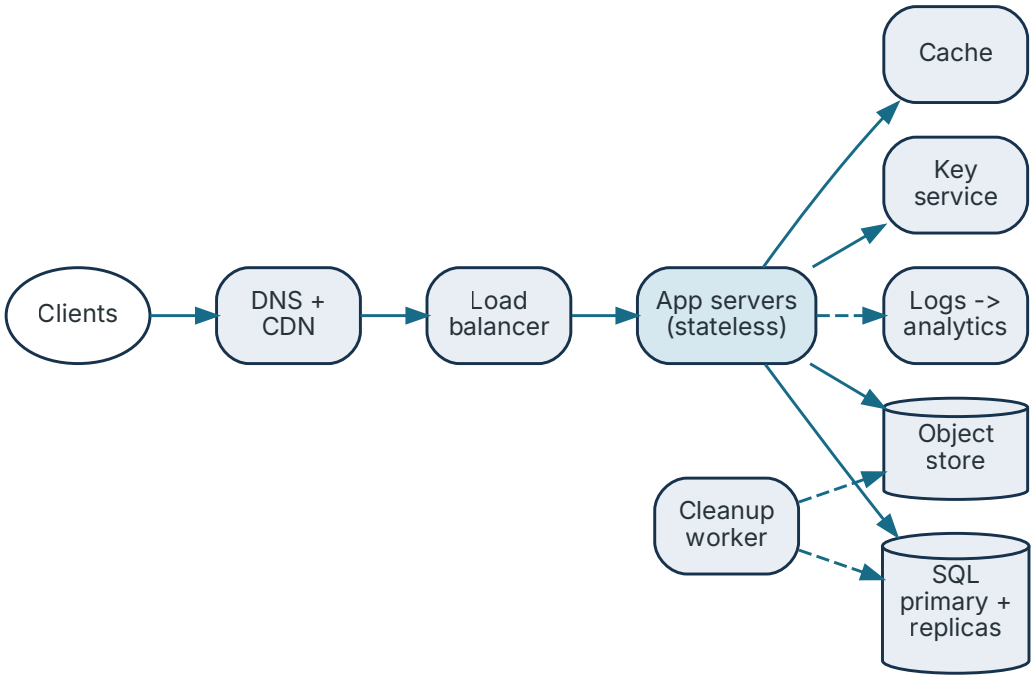


Figure 21.1 High-level architecture. A CDN and cache absorb most reads; a small application tier handles API calls; metadata lives in a replicated SQL database and text in an object store; a background job cleans up expired pastes and a log pipeline computes analytics.

Walk through Figure 21.1. **Clients** reach the service via **DNS** and a **CDN**. Because pastes are immutable once created, they are excellent candidates for caching, even at the CDN, with a TTL no longer than their remaining lifetime. Requests that miss go through a **load balancer** to **stateless application servers**. A **distributed cache** holds hot `short_code → (content or metadata)` entries. On a miss, the server reads **metadata from the SQL database** (a primary with read replicas) and **content from the object store**. A **key service** supplies codes on writes. A **cleanup worker** removes expired pastes. **Access logs** feed an **analytics job**.

Write flow

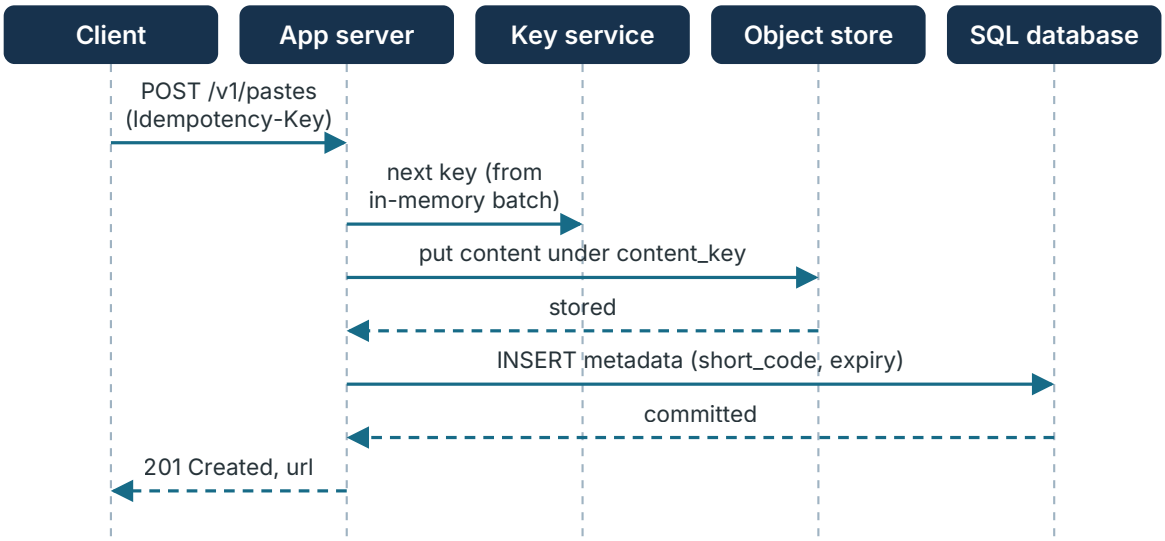


Figure 21.2 Creating a paste. The server obtains a key, stores the text in the object store, inserts the metadata row, and returns the link. Content is written first so a metadata row never points at missing content.

Ordering matters. The server writes the **content first**, then the **metadata row**. If it crashes between the two, the result is an orphaned object (harmless garbage collected later), never a metadata row pointing to nothing. The create call is made **idempotent** with the client's idempotency key stored beside the row.

Read flow

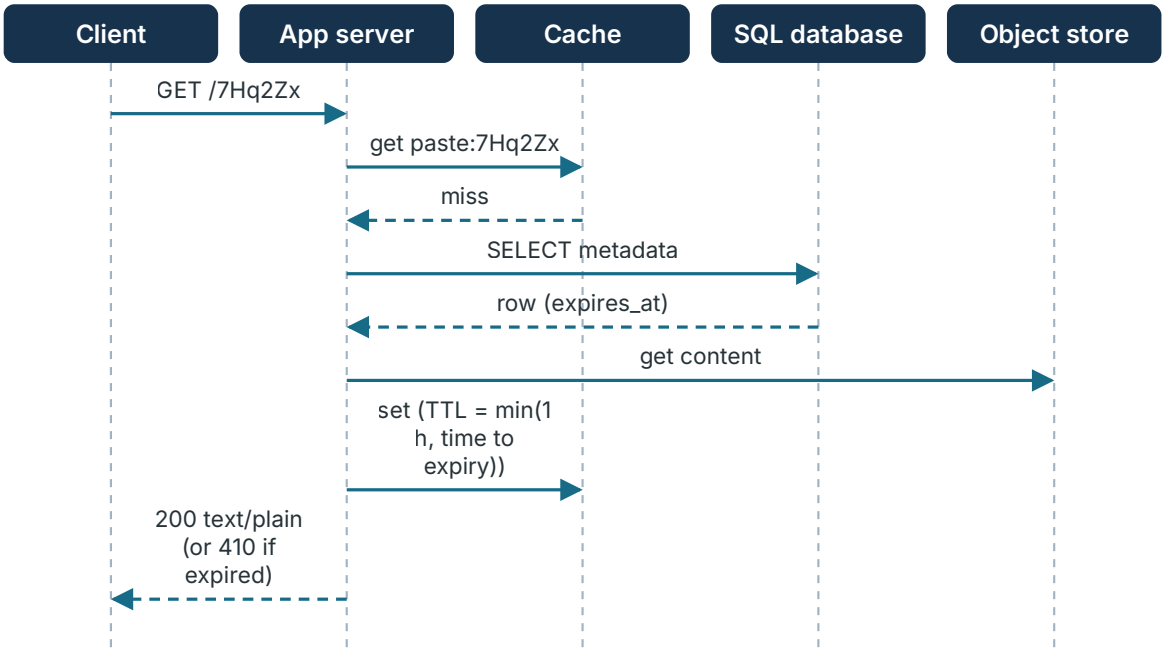


Figure 21.3 Reading a paste. The cache answers most requests. On a miss the server reads metadata and content, fills the cache with a TTL not exceeding the expiry, and returns the text. Expired pastes return 410.

21.7 Caching, expiry and cleanup

Caching. Reads are 10× writes, and popularity is skewed (a paste shared on social media gets thousands of views). Use **cache-aside** (Chapter 12): key `paste:{code}`, value metadata plus content for small pastes, TTL = min(1 hour, time until expiry). Pastes are **immutable**, so staleness is a non-issue except for deletion and expiry, which are handled by explicit invalidation and by checking `expires_at` on the cached object. At 40 reads per second the cache is a convenience; at 4,000 it is essential. A **CDN** in front can take most traffic if responses carry `Cache-Control`.

Expiry. Two mechanisms together. **Lazy expiry**: on each read, check `expires_at`; if past, return 410 and (optionally) enqueue a delete. **Batch cleanup**: a scheduled job scans the `expires_at` index for rows older than now, deletes the object-store content and the row, in small batches to avoid long locks. Lazy expiry gives correctness; the batch job reclaims space. Retention of deleted data for privacy deadlines (Chapter 17) must also cover backups and caches.

Analytics (monthly visits per link). Do not update a counter on every read (a hot row that would serialise writers). Instead, **log each hit** (short code, timestamp) to a stream, and aggregate with a batch or streaming job (a MapReduce-style job in the notebook): *map* each log line to `(code, 1)`, *reduce* by summing per code and month, then write results to an analytics table. Reads of the stats endpoint come from that table. The cost is that stats lag by minutes to hours, which is acceptable.

```
# map: emit one count per hit    reduce: sum per (code, month)
def map_line(line):
    ts, code = parse(line)
    yield ((code, month_of(ts)), 1)
def reduce_key(key, counts):
    yield (key, sum(counts))
```

21.8 Scaling and bottlenecks

At ten times the numbers (400 reads per second, 40 writes) almost nothing changes: add application servers and cache nodes. At a hundred times (4,000 reads, 400 writes per second):

1. **Database reads**: handled by the cache (a 95% hit ratio leaves 200 reads per second) and read replicas.
2. **Database writes**: 400 per second is within one primary's capacity; if it is not, shard by `short_code` hash. The **key service** is sharded the same way.
3. **Storage**: 450 GB grows to 45 TB over three years at 100×, which belongs in an object store with lifecycle rules (move old pastes to cheaper tiers).
4. **Hot pastes**: a viral link is served from the CDN and cache; popular keys can also be replicated across cache nodes.

5. **Multi-region**: replicate the read path (CDN, cache, read replicas) near users; keep writes in one region unless latency demands otherwise.

21.9 Failure handling, security and operations

Failure handling. Cache down: reads fall through to the database (protect it with rate limiting and request coalescing). Database primary fails: promote a replica; reads continue from the cache and replicas meanwhile, so *reads degrade less than writes*. Object store unavailable: serve cached content, return 503 for the rest. Key service unavailable: application servers have a batch in memory; also fall back to the hash-and-retry method. Duplicate create due to retries: idempotency key.

Security and abuse. Pastebin-style sites are heavily abused: spam, phishing links hidden behind shorteners, malware distribution, leaked credentials, illegal content. Controls: **rate limit** creates per IP and per account (Chapter 7); size limits; **scan** content and destination URLs against reputation services, and allow takedown by report; **preview pages** for suspicious redirects; **long, random codes** for private pastes (never sequential); **authorisation** for private pastes and deletion; do not execute or render user content as HTML (serve as `text/plain` with `X-Content-Type-Options: nosniff` and a restrictive Content Security Policy to prevent cross-site scripting); and log access for audit.

Monitoring and cost. Track create and read rates, error rates, cache hit ratio, p95 latency, expiry backlog, key-service reserve, storage growth, and the abuse reports. Cost at 100×: object storage dominates (45 TB × about $0.02 per GB-month ≈ $900 per month), then egress, then a small database and cache fleet; a CDN lowers egress cost. Alert on key pool running low and on the ratio of 5xx errors.

21.10 Alternatives and trade-offs

Table 21.3 Alternatives considered.

Decision	Chosen	Alternative	Why chosen	When to switch
Metadata store	SQL with a primary key on the code	Key-value or document store	Simple, familiar, transactions	Very high scale or low operational budget
Key generation	Pre-generated random keys	Hash + retry; counter + Base62	Unguessable, no hot-path collisions	Small system: hash + retry
Content storage	Object store	Inline in the database	Cheap, scalable, small rows	Tiny pastes: inline
Analytics	Log and batch aggregate	Per-read counter	No hot row	Need real-time counts: streaming aggregation
Redirect code	302	301	Counts every click	Static links where SEO and caching matter

Follow-up questions to expect. *Custom aliases?* Separate namespace with a uniqueness check. *Delete a link permanently, including from caches and CDN?* Purge calls plus short TTL for private pastes. *User accounts and history?* Add a `users` table and an index by `(owner_id, created_at)`. *Multi-region active-active writes?* Region-prefixed keys avoid collisions; handle deletes with a region-aware protocol. *What if reads are 1,000× writes?* Lean entirely on the CDN. *Reduce guessability of public links?* Longer codes (8+ characters) or scrambling.

FAILURE MODES

Failure modes (summary). Hash collisions causing silent overwrites if the unique constraint is missing; key service outage; hot paste overloading one cache node; expiry job running behind and filling storage; orphaned objects after partial failures; abuse flooding creates; redirect to a malicious destination; cached content outliving a takedown.

COMMON MISCONCEPTION

- "*The hard part is scale.*" Here the load is small; the hard parts are keys, expiry, abuse and operations.
- "*MD5 is fine for the key because it is a hash.*" Truncation makes collisions certain at scale; MD5 is also not for security uses.
- "*A 301 is always better because it is cacheable.*" It hides repeat visits from analytics and cannot be changed once cached.
- "*Store everything in the database for simplicity.*" Large blobs bloat the database; split content out when sizes grow.

INTERVIEW QUESTIONS

Q1. How do you generate unique short codes? Compare hashing with retry, counter plus Base62 (with id blocks and scrambling), and pre-generated keys; justify by collisions, guessability, hot-path cost and failure risk; give the key-space arithmetic.

Q2. How would you handle expired pastes? Lazy check on read plus a batch cleanup job over an index on expiry; return 410; remove content from the object store and caches; mind backups for privacy.

Q3. How do you count visits without a hot row? Log hits to a stream and aggregate in batch or streaming jobs; serve statistics from the aggregate table.

Q4. What breaks first at 100× scale? Not the database (cache and replicas), but operations: storage lifecycle, abuse control and the key service, then global latency; add a CDN and sharding as needed.

EXERCISES

1. Recompute the key-space needed if the service must hold 5 billion links with a collision probability under 1 in 100,000 per new key. How many Base62 characters are required?
2. Describe what happens if the application server crashes after writing content but before the metadata row, and after the metadata row but before returning the response.
3. Design the cleanup job: query, batch size, and how you avoid overloading the database.

4. A paste goes viral: 20,000 requests per second for one code. List the layers that absorb it and what could still fail.

Answers.

1. Collision probability $\approx n / N < 10^{-5}$ with $n = 5 \times 10^9$ requires $N > 5 \times 10^{14}$. $62^8 \approx 2.18 \times 10^{14}$ is just short; $62^9 \approx 1.35 \times 10^{16}$ suffices, so 9 characters (or use pre-generated keys that never collide, where 8 characters would still do).
2. After content but before metadata: an orphaned object, collected later by a sweeper that compares keys. After metadata but before the response: the paste exists; the client times out and retries with the same idempotency key, receiving the existing link.
3. `SELECT short_code, content_key FROM pastes WHERE expires_at < now() ORDER BY expires_at LIMIT 500;` delete objects then rows, commit, sleep briefly, repeat; run off-peak, throttle on database load, and make deletes idempotent so reruns are safe.
4. The CDN (cached, immutable), then the distributed cache, then in-process caches; the hot key could still saturate one cache node, so replicate or add a local cache; the object store handles reads on misses but at a cost; protect the origin with request coalescing.

CHAPTER SUMMARY

- Scale here is small; the interesting design is in key generation, storage separation, expiry, analytics and abuse.
- Separate metadata (SQL, key = short code) from content (object store); write content first.
- Choose among hash-and-retry, counter, and pre-generated keys by collision rate, guessability and failure modes.
- Cache aggressively (immutable data), expire lazily and in batches, aggregate analytics from logs.
- Plan for abuse, failure and growth, and state the trade-offs.

REVISION CHECKLIST

- [ ] I can derive writes per second, storage and key length from the assumptions.
- [ ] I can compare three key-generation methods.
- [ ] I can trace create and read paths and the failure of each step.
- [ ] I can explain expiry, analytics and abuse controls.

SUPPLEMENTAL MATERIAL

Further reading. The original System Design Primer's solution for this problem; Alex Xu, *System Design Interview*, chapter on URL shorteners; engineering write-ups on key generation services (for example, Twitter's Snowflake id scheme for time-ordered unique ids).

CHAPTER 22

Case Study: A Social Timeline and Search

Derived from your notes: Notebook chapter 12.2 (Twitter timeline and search: the scale table, fan-out on write, the User Graph service, search service, notification push). **Added in this book:** API and data model, fan-out on read and the hybrid for celebrity accounts, timeline cache sizing, media handling, ranking, failure modes, abuse, cost and alternatives.

LEARNING OBJECTIVES

After this chapter you can:

- derive the dominant loads of a read-heavy social feed and explain why they dictate the architecture;
- compare fan-out on write, fan-out on read and the hybrid approach, and size a precomputed timeline cache;
- design the supporting services: user graph, search, notifications, media;
- identify and handle the celebrity problem, ordering, duplicate delivery and failures.

PREREQUISITE KNOWLEDGE

Chapter 10 (query-first modelling), Chapter 12, Chapter 13 (queues, fan-out), Chapter 11 (eventual consistency, hot keys) and Chapter 20.

22.1 Problem and requirements

The problem. Design the core of a microblogging service where users post short messages ("tweets"), follow other users, see a **home timeline** of recent posts by the people they follow, view a **user timeline** (one person's posts), **search** posts by keyword, and receive **push notifications**.

Functional requirements. Post a tweet (text and optional media). Follow and unfollow. View the home timeline and a user timeline. Search tweets. Notifications for mentions and replies. Out of scope (stated explicitly): ranking algorithms beyond reverse-chronological, direct messages, ads, trends, the full complexity of retweets and quote tweets, moderation tooling (mentioned under abuse).

Non-functional requirements. Reads vastly exceed writes; the home timeline must load fast (p95 under 200 ms). The service must be **highly available**: the timeline should degrade rather than fail. **Eventual consistency is acceptable**: a tweet may take a few seconds to appear in followers' timelines. A user must always see their *own* tweet immediately (read-your-writes, Chapter 9). Durability: posted tweets are not lost.

22.2 Scale estimate

Table 22.1 Assumptions and derived quantities (from the notebook; all are assumptions).

Quantity	Assumption	Derived
Tweets	500 million per day	5×10^8 / 86,400 ≈ 5,800, call it ~6,000 per second
Deliveries	each tweet reaches ~10 followers' timelines on average	~60,000 fan-out writes per second
Timeline reads	250 billion per month	2.5×10^{11} / 2.5×10^6 ≈ 100,000 per second
Search	10 billion per month	≈ 4,000 per second
Storage	≈ 10 KB per tweet including media references and metadata	5 TB/day ≈ 150 TB/month ≈ 5.4 PB in 3 years

What the numbers say. Reads (100,000/s) outnumber tweet creation (6,000/s) by about 17 to 1, and every read needs the *combined* posts of many followed accounts. Computing that at read time would be far too expensive, so we **precompute** timelines: doing more work on write (60,000 fan-out writes per second) to make reads a single cache lookup. Storage at petabyte scale rules out one database: we need sharding and an object store for media. Note that the *average* of 10 followers hides a heavy-tailed distribution: most users have few followers; a few have tens of millions. That tail drives the hardest part of the design.

22.3 Data model and API

```
POST /v1/tweets               {"text": "...", "media_ids": [..]}  -> 201 {"id": "1741...",
"created_at": "..."}
GET  /v1/timeline/home?limit=50&cursor=...   -> 200 {"tweets": [..], "next_cursor": "..."}
GET  /v1/users/{id}/tweets?limit=50&cursor=... -> 200
POST /v1/users/{id}/follow    DELETE same path to unfollow
GET  /v1/search?q=keyword&limit=20&cursor=...
```

Timelines use **cursor pagination** (Chapter 6): clients send the id of the last tweet seen; new tweets appearing at the top never shift pages.

Tweet ids must be unique and **roughly time-ordered** so a timeline can be sorted by id: use a *Snowflake-style* id generator: a 64-bit number made of a millisecond timestamp, a machine id and a per-millisecond sequence number. It needs no central coordination and sorts by creation time.

Table 22.2 Main data and where it lives.

Data	Access pattern	Store	Key
Tweets	By id; by author in time order	Sharded wide-column or SQL	`(author_id, tweet_id)` for user timeline; `tweet_id` lookup
Follow graph	Who does X follow? Who follows X?	Sharded store with two views (follows and followers)	`(user_id) -> list` in both directions
Home timeline cache	Latest ~800 tweet ids per user	In-memory list store (Redis-style)	`home:{user_id}`
Media	Fetch by id	Object store plus CDN	`media_id`
Search index	Keyword to tweet ids	Search cluster (inverted index)	term
Notifications	Per-user inbox	Wide-column or queue-backed store	`(user_id, time)`

22.4 Architecture

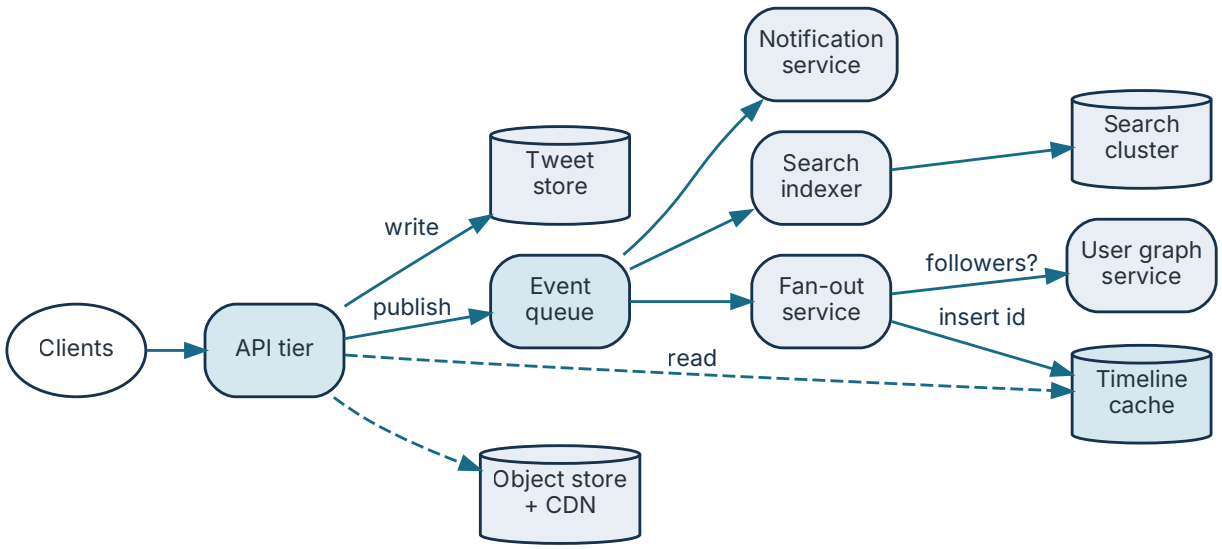


Figure 22.1 High-level architecture. Writes create a tweet and trigger an asynchronous fan-out job; reads fetch a precomputed timeline from the cache and hydrate tweets from the tweet store. Search, notifications and media are separate services fed by events.

Follow Figure 22.1. The **write path** (blue in your mind): the client posts to the **API tier**; the tweet is stored durably; an event is published to a **queue**; a **fan-out service** reads the follower list from the **user graph service** and inserts the tweet id into each follower's **home timeline cache**; the **search indexer** and **notification service** consume the same

event. The **read path**: the API tier reads the timeline's list of tweet ids from the cache, **hydrates** them (fetches tweet bodies and author profiles, from a tweet cache and then the store), and returns the page.

22.5 The core decision: fan-out strategy

How does a tweet get into the timelines of everyone who should see it?

Fan-out on write (push). At post time, insert the tweet id into every follower's timeline list. Reads are trivial (one list lookup), at the cost of write amplification: one tweet becomes F writes, where F is the follower count.

Fan-out on read (pull). Store tweets only by author. At read time, fetch the list of accounts followed, get each one's recent tweets, and merge by time. Writes are cheap; reads are expensive (one lookup *per followed account*) and slow at 100,000 reads per second.

Hybrid. Use push for ordinary accounts and pull for **very popular accounts**. A celebrity with 50 million followers would cause 50 million writes per tweet (and delay delivery by minutes). Instead, store the celebrity's tweets by author only. At read time, the timeline service takes the precomputed list (from push) and **merges in recent tweets from the few celebrities that user follows**.

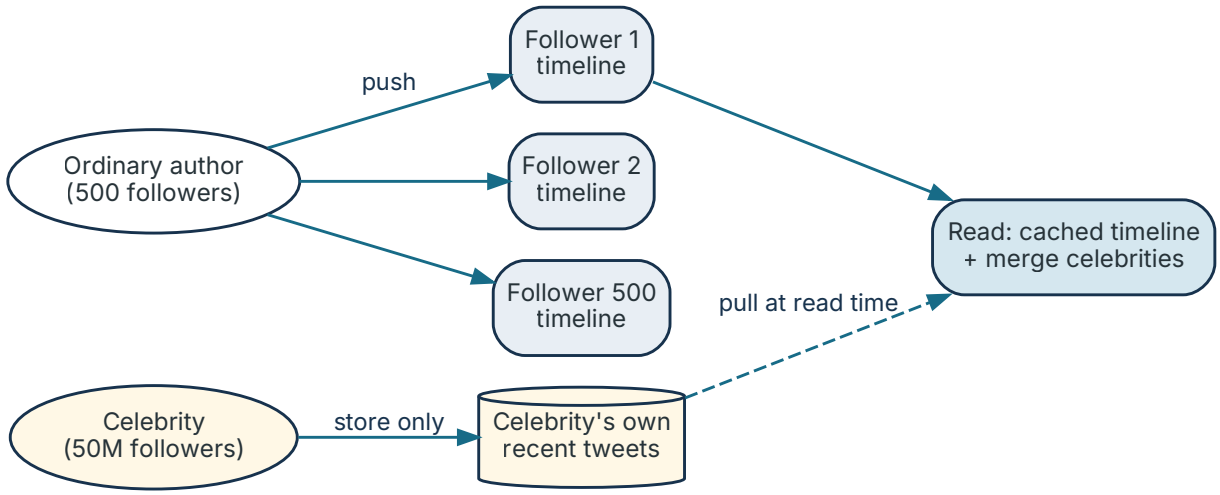


Figure 22.2 Hybrid fan-out. Ordinary authors push tweet ids into followers' cached timelines at write time; celebrity tweets are not pushed and are merged in at read time from the author's own recent-tweets list.

Table 22.3 Fan-out strategies compared.

	Push (on write)	Pull (on read)	Hybrid
Write cost	High, proportional to followers	Low	High for normal users, low for celebrities
Read cost	Very low	High, proportional to follows	Low plus a small merge
Timeliness	Delayed for huge accounts	Immediate	Immediate for celebrities

	Push (on write)	Pull (on read)	Hybrid
Wasted work	On inactive followers	None	Limited
Complexity	Medium	Medium	High

Refinements. Push only to **active** users (those who logged in recently); inactive users' timelines are rebuilt on demand when they return. Drop the oldest ids so each timeline is bounded (for example the latest 800). For a **brand-new follow**, backfill recent posts of the followed account into the timeline asynchronously.

WORKED EXAMPLE

Sizing the timeline cache. Each timeline stores about 800 tweet ids at 8 bytes each = 6.4 KB; add overhead (list structure, keys), say 10 KB per user. If 300 million users are *active enough to cache*, that is 3 TB of memory, or about 30 cache nodes of 100 GB (plus replicas: about 60). Only caching timelines for recently active users (say 100 million) cuts it to 1 TB. The fan-out throughput: 60,000 inserts per second spread over those nodes is 1,000 to 2,000 each, easy. The cost: memory is the dominant expense; hence tight bounds, inactive-user eviction, and rebuild-on-demand.

Write flow and read flow

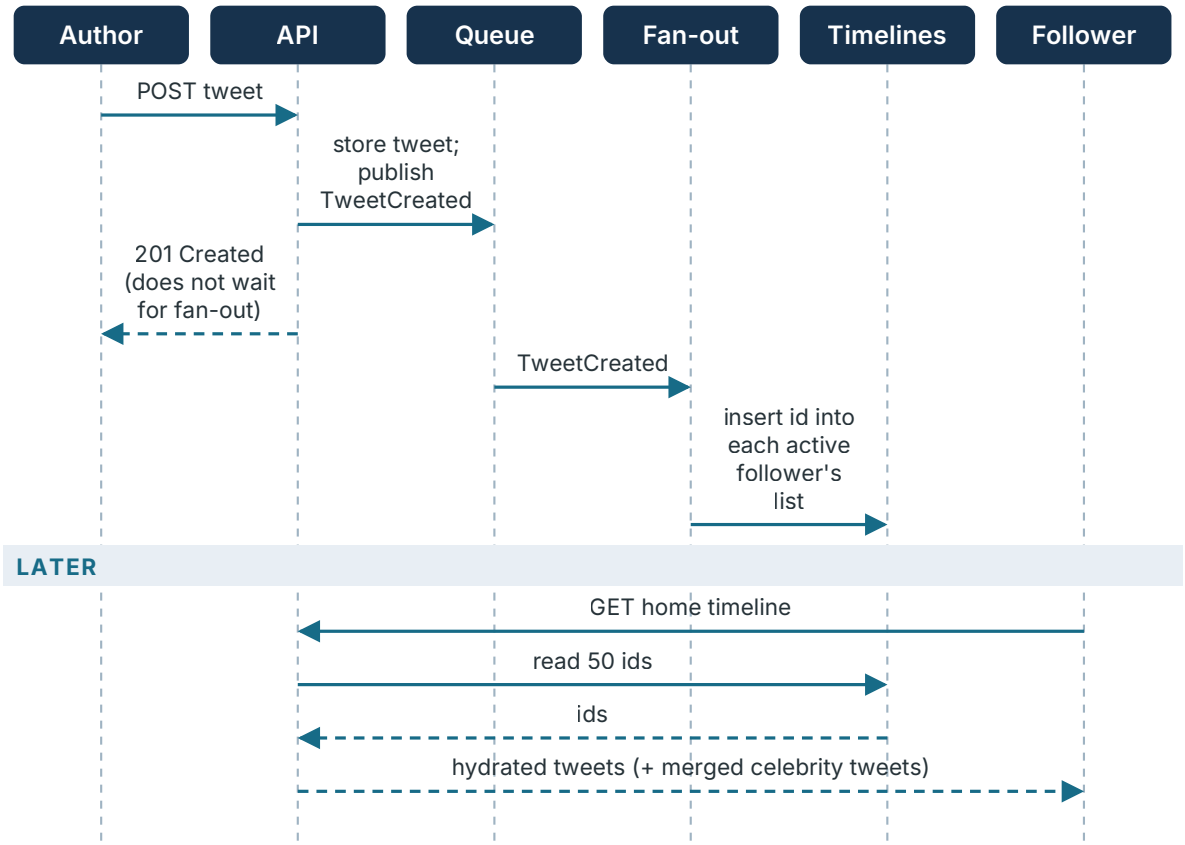


Figure 22.3 Posting and reading. The post returns once the tweet is durable and queued; the fan-out happens asynchronously. A read fetches ids from the timeline cache and hydrates them.

Read-your-writes. The poster's own client inserts the new tweet into its local view at once, and the server also writes it synchronously into the poster's own timeline cache, so the author never sees their tweet missing even if fan-out to followers is still in progress.

22.6 Supporting services

User graph service. Stores who follows whom in two directions (`following(u)` and `followers(u)`), sharded by user id. It must answer "give me the followers of u in pages" for fan-out and "does a follow b?" for authorisation. Hot celebrity follower lists are paged and cached. The notebook calls this the *User Graph Service*; it can be backed by a sharded database or a graph-capable store.

Search service. Tweets are indexed into an **inverted index** (term → list of tweet ids, Chapter 10). Indexing is asynchronous: the indexer consumes tweet events, tokenises (lower-case, strip punctuation, handle hashtags and mentions), and updates the index within seconds ("near-real-time search"). The index is **partitioned by time** (recent tweets in hot, small partitions on fast storage; older ones in larger, colder partitions) and each partition by tweet-id hash. A query is a **scatter-gather**: the query router sends the query to relevant partitions, each returns its top-k by score, and the router merges. At 4,000 queries per second, results for popular queries can be cached briefly.

Notifications. Mentions and replies publish an event; the notification service determines recipients, applies user preferences, stores an in-app notification, and pushes through a mobile push provider. Rate-limit and batch ("12 people liked your tweet") to avoid floods. Delivery is at-least-once, so the client deduplicates by notification id (Chapter 13).

Media. Images and video go to an **object store** via **pre-signed upload URLs** (the client uploads directly, not through the API servers). A worker pipeline generates thumbnails and transcodes video into several resolutions; the tweet stores media ids; delivery goes through a **CDN**. Large storage drives the bulk of the 150 TB per month.

22.7 Scaling, bottlenecks and failure handling

Bottlenecks. (1) *Fan-out volume and celebrity spikes*: handled by the hybrid model, queue-based buffering and worker autoscaling on queue age. (2) *Timeline cache memory*: bounded lists, active users only. (3) *Hot keys*: a celebrity's tweet is read by millions; cache the tweet object with local in-process caches and replicate hot keys (Chapter 11). (4) *Tweet storage growth*: shard by `author_id` or tweet id, tier old data to cheaper storage. (5) *Graph reads*: cache follower pages.

Failure handling. *Cache lost*: rebuild timelines from the tweet store using the pull method (slow but correct), throttle rebuilds to protect the database, and serve a degraded, partial timeline rather than an error. *Fan-out worker lag*: tweets appear late, but nothing is lost since the queue is durable; alert on lag. *Duplicate fan-out* (a worker retries): inserting an id already present in a timeline list is idempotent when the list is a sorted set keyed by tweet id. *Out-of-order*: sort by Snowflake id. *Search index down*: serve cached results or a

clear message; timelines unaffected. *Region failure*: replicate tweets and graph to a second region (asynchronous) and fail over reads, accepting loss of the last seconds of writes; this is a business decision about RPO.

Security, privacy and abuse. Authorise every call; **protected accounts** restrict who may see tweets (enforce at read time as well as fan-out time, since follow status can change); block, mute and report features; rate-limit posting, following and search (Chapter 7); detect bot networks; handle deletion (a deleted tweet must disappear from caches, timelines and search; propagate a delete event and tolerate brief lag); keep personal data out of logs. Content moderation pipelines are a large separate system.

Operations and cost. Metrics: tweet create latency, fan-out lag (p99 seconds from post to last follower), timeline cache hit ratio, search p99, queue age, error rates, hot-key alerts. Deploy gradually (canary) because this system is always busy. Cost is dominated by cache memory, storage (5.4 PB), egress for media (mitigated by CDN) and search index infrastructure.

22.8 Alternatives and trade-offs

Table 22.4 Decisions and alternatives.

Decision	Chosen	Alternative	Reason	Revisit if
Fan-out	Hybrid push/pull	Pure push or pure pull	Balances read speed and celebrity cost	Follower distribution changes
Timeline store	In-memory lists of ids	Precomputed in a database	Fast reads; ids are small	Memory too costly: store on SSD with a cache
Consistency	Eventual, read-your-writes for author	Strong	Availability and speed	Regulatory needs
Search	Separate inverted-index cluster	SQL full-text	Scale and relevance features	Tiny scale
Ids	Snowflake time-ordered	UUIDs	Sortable, compact	Clock concerns: add safeguards
Media	Object store + CDN	Database blobs	Cost, throughput	None

Follow-up questions to expect. *How do you rank rather than sort by time?* Add a scoring service that re-ranks candidates from the timeline cache (blending recency, engagement, relationship), trading transparency for engagement. *How do retweets work?* Store a pointer to the original; fan out the retweet as an event. *How do you delete a user and all their tweets?* Delete events plus a background sweep (Chapter 17). *How would you support trending topics?* Stream processing with sliding windows and count-min sketches (Chapter 3). *How do you shard tweets if one user is extremely prolific?* Key by tweet id and index by author separately.

FAILURE MODES

Failure modes (summary). Celebrity fan-out overwhelming workers; timeline cache eviction storm; unbounded lists; fan-out lag; stale protected-account visibility; delete not propagating; search index drift; hot-key cache node saturated; retry storms on the graph service.

COMMON MISCONCEPTION

- "*Push is always better because reads are cheap.*" Push breaks for accounts with huge audiences.
- "*The timeline must be perfectly consistent.*" A few seconds of delay is acceptable and enables a far cheaper design.
- "*Search can use the primary database.*" At 4,000 queries per second over petabytes it needs a dedicated index.
- "*The average follower count describes the load.*" The tail dominates cost and risk.

INTERVIEW QUESTIONS

Q1. How do you build the home timeline? Precompute: on post, asynchronously fan out the tweet id to each follower's cached timeline list; reads fetch ids and hydrate. For celebrities skip push and merge at read time.

Q2. What is the celebrity problem and how do you solve it? One tweet needing millions of writes delays delivery and overloads workers; use a hybrid model with pull for accounts above a follower threshold.

Q3. How is search implemented? Asynchronous indexing into a time-partitioned inverted index; scatter-gather queries merged by score; short-lived result caching.

Q4. How do you ensure the author sees their own tweet immediately? Write it synchronously to the author's own timeline and show it locally in the client; the rest of the fan-out may lag.

EXERCISES

1. If the average follower count rises from 10 to 50, what is the new fan-out write rate? What changes in the design?
2. Choose a follower threshold for switching to pull, given workers can do 100,000 inserts/s total and 6,000 tweets/s arrive. Argue the trade-off.
3. Estimate memory for timelines of 200 million cached users with 500 ids each at 8 bytes plus 40% overhead.
4. A tweet is deleted. List the places it must be removed and how.

Answers.

1. 6,000 × 50 = 300,000 inserts per second: three times the worker budget. Respond with more workers, a lower celebrity threshold, pushing only to active users, batching inserts, and relying on cache bounds.
2. If a tweet by a user with F followers costs F inserts, a threshold such as 100,000 followers bounds a single tweet's cost to 100,000 inserts (one second of the total budget); lower thresholds shift cost to reads. Pick it by measuring the follower distribution and

read cost per merged celebrity (usually merges of fewer than ~10 celebrity accounts keep reads cheap).

3. 200 M × 500 × 8 B = 800 GB; with 40% overhead ≈ 1.12 TB.
4. Tweet store (tombstone or delete), tweet cache, timeline caches (lazy filtering at read time because removing from millions of lists is costly), search index (delete event), media (if unreferenced), analytics and backups (retention policy); readers filter out deleted ids during hydration.

CHAPTER SUMMARY

- Reads dominate (≈17:1) and need many accounts' posts; precompute timelines at write time.
- Fan-out on write is fast to read but breaks for celebrities; the hybrid merges celebrity posts at read time.
- Timelines are bounded lists of ids in memory; tweets are hydrated from caches and stores; ids are time-ordered.
- Search is an asynchronously updated, partitioned inverted index with scatter-gather queries; media uses an object store and CDN.
- Eventual consistency is acceptable; protect the author's read-your-writes; plan for lag, hot keys, deletes and abuse.

REVISION CHECKLIST

- [ ] I can derive writes, fan-out writes and reads per second.
- [ ] I can compare push, pull and hybrid and size the timeline cache.
- [ ] I can describe the search pipeline and scatter-gather.
- [ ] I can list failure modes and degradations.

SUPPLEMENTAL MATERIAL

Further reading. Raffi Krikorian, "Timelines at Scale" (QCon 2012) on Twitter's fan-out architecture; the Snowflake id generator announcement; Earlybird, the real-time search system described in Twitter engineering publications; Kleppmann, *Designing Data-Intensive Applications*, the Twitter example in chapter 1.

CHAPTER 23

Case Study: A Web Crawler and Search Engine

Derived from your notes: Notebook chapter 12.3 (web crawler and search: the scale table, links_to_crawl and crawled_links, the crawl loop, duplicate detection by page signature, reverse index service, document service, query API). **Added in this book:** the frontier with politeness queues, robots.txt, URL normalisation, spider traps, freshness scheduling, simhash near-duplicate detection, DNS caching, partitioning workers by host, failure handling and ethics.

LEARNING OBJECTIVES

After this chapter you can:

- estimate crawl rate, bandwidth and storage for a web-scale crawler;
- design the URL frontier with priority, politeness and de-duplication;
- detect exact and near-duplicate pages and avoid spider traps;
- connect crawling to indexing and query serving with a reverse (inverted) index;
- identify failure modes and ethical constraints (robots.txt, rate limits).

PREREQUISITE KNOWLEDGE

Chapter 3 (bloom filters, heaps, tries), Chapter 10, Chapter 13 (queues), Chapter 11 (partitioning) and Chapter 20.

23.1 Problem and requirements

The problem. Design a system that **crawls the web**: starting from seed URLs, it downloads pages, extracts links, and keeps following them; it stores what it finds and builds a **search index** so users can type a query and receive a ranked list of matching pages, each with a **title and snippet**.

Functional requirements. Crawl pages from seed URLs, discovering new ones. Never crawl the same page repeatedly without reason (**avoid loops and duplicates**). Re-crawl pages periodically to stay fresh (more often for pages that change often). Build an index from page text. Serve queries returning a ranked list with titles and snippets.

Non-functional requirements. *Scalable* (billions of pages). *Polite* (do not overload sites; obey robots.txt). *Robust* (the web is full of malformed pages, traps, slow servers). *Extensible* (new content types). Search results with **low latency** (tens to hundreds of milliseconds) and high availability; the crawl itself is a throughput-oriented batch-like pipeline that can tolerate delays and restarts.

Scope. Out of scope: sophisticated ranking (learning to rank, link analysis in detail), personalisation, handling JavaScript-rendered pages (mention as an extension), image and video search, and legal and indexing-policy details. Assume classic components of Chapter 3 and Chapter 10.

23.2 Scale estimate

Table 23.1 Assumptions and derived quantities (from the notebook; all are assumptions).

Quantity	Assumption	Derived
Links	1 billion pages, refreshed about weekly	≈ 4 billion crawls per month
Crawl rate	4×10^9 / 2.5×10^6 s	≈ 1,600 page fetches per second
Page size	≈ 500 KB per page	1,600 × 500 KB = 800 MB/s ≈ 6.4 Gbps of download
Storage	4×10^9 crawls × 500 KB per month (raw, if all stored)	≈ 2 PB per month if every crawl is stored; far less if only changed, compressed or text-only
Search	100 billion queries per month	≈ 40,000 queries per second
Index	Text only, compressed (assume ≈ 5% of raw)	tens of terabytes to petabytes, partitioned

What the numbers say. 1,600 fetches per second is sustained work for a *fleet*, not one machine: if one crawler worker manages 20 concurrent downloads at 500 ms average, it does about 40 pages per second, so you need about 40 workers (more with headroom). The bandwidth (6.4 Gbps) and the storage (petabytes) are the real constraints, so store **compressed text**, keep **only changed pages**, and retain raw HTML only briefly. The query side (40,000 per second) is a separate scaling problem: a partitioned, cached index serving system.

23.3 Architecture overview

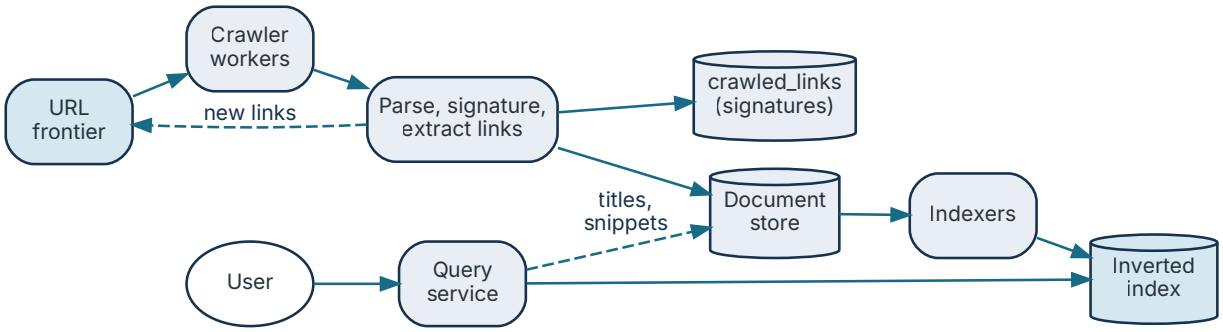


Figure 23.1 The crawl and search pipeline. The frontier feeds crawler workers; fetched pages are checked for duplicates, parsed, stored, and their links are fed back. Indexers build the inverted index from stored documents; the query path uses the index and the document store.

The system is a loop with a branch. In Figure 23.1: the **URL frontier** hands out URLs to crawl. **Crawler workers** fetch them (respecting politeness and robots.txt), pass pages to a **content processor** that extracts text and links, computes a **signature** for duplicate detection, and stores the result in the **document store**. New links go back to the frontier (after URL normalisation and filtering). In parallel, **indexers** read new documents and update the **inverted index**; a **query service** answers searches by consulting the index and the document store.

23.4 The URL frontier

The frontier is the heart of a crawler. It must decide **what to fetch next** while meeting three goals: **priority** (important and fast-changing pages first), **politeness** (never hammer one site), and **freshness** (revisit on a sensible schedule).

The notebook's version uses two stores: `links_to_crawl`, a ranked list ordered by priority (for instance a sorted set in an in-memory store), and `crawled_links`, a record of what has been fetched (URL and page signature) in a NoSQL store. A scalable design extends this into the classic two-layer queue structure:

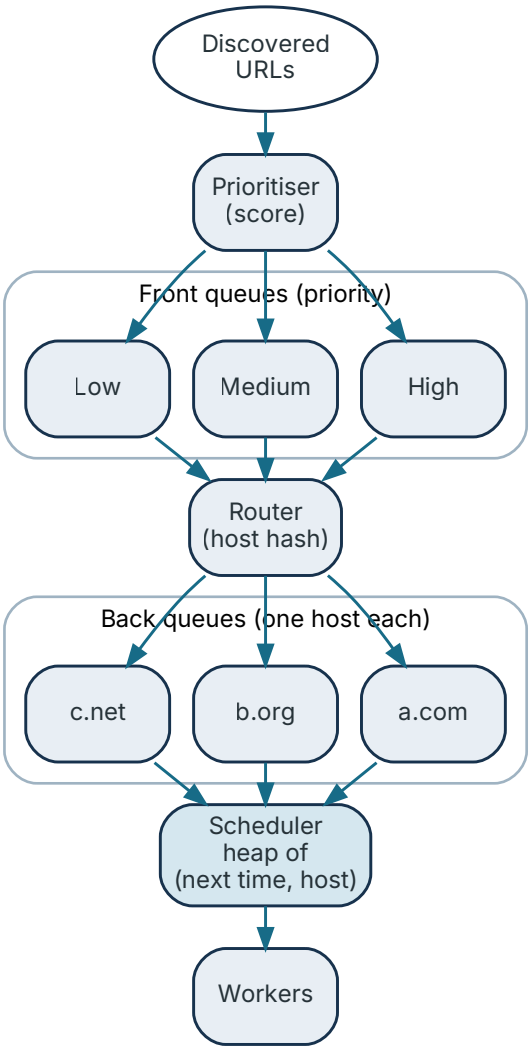


Figure 23.2 A two-layer frontier. Front queues order URLs by priority; a router maps each URL to a back queue per host; a scheduler releases at most one URL per host per politeness interval.

- **Front queues (priority)**: URLs are scored (page importance, expected change rate, depth, domain quality) and placed into several priority queues; a selector draws from higher-priority queues more often.
- **Back queues (politeness)**: each back queue contains URLs for **one host only**. A scheduler keeps a **heap** of (next-allowed-time, host) so that after fetching from host *h* it will not return to *h* until, say, 1 to 10 seconds later (or the `Crawl-delay` the site asks for). Workers are assigned **by host hash**, so one worker owns a host's rate limiting, keeps its connections alive and caches its robots.txt and DNS answers.
- **Persistence**: the frontier is far too big for memory alone; keep the hot portions in memory and the rest on disk (log-structured, Chapter 3) so crashes do not lose the queue.

URL normalisation and filtering before enqueueing: lower-case the host, remove default ports and fragments (`#section`), resolve relative links, sort or drop tracking query parameters, collapse `/a/../b`, and reject unwanted schemes and file types. Without this, `HTTP://Example.com:80/a` and `http://example.com/a` count as two pages.

Seen-URL detection. Before adding a URL, ask "have I seen it?" With billions of URLs a hash set in memory is too big; use a **bloom filter** (Chapter 3) for the fast path (definitely-new URLs skip any lookup; a false positive merely skips a rare page), backed by the `crawled_links` store for exact answers on "probably seen".

> **WORKED EXAMPLE**
>
> **Memory for the seen set.** 10 billion distinct URLs discovered (more than the 1 billion we keep, because links point at pages we reject). Exact set at 16 bytes per URL hash = 160 GB: shardable but heavy. A bloom filter with 10 bits per entry needs $10^{10} \times 10$ bits = 100 Gbit = 12.5 GB at about 1% false-positive rate: it fits in memory on a few machines, shared by host-hash partition. The 1% of URLs wrongly considered seen are skipped *this round*; they are likely rediscovered through other links.

23.5 Fetching politely and robustly

- **robots.txt**: fetch and cache each host's `/robots.txt`; obey `Disallow` rules and crawl delays. Re-fetch periodically (for example daily). A crawler that ignores it risks being blocked and violates web norms.
- **Identify yourself** with a descriptive `User-Agent` and contact information, so site owners can reach you.
- **DNS caching**: DNS lookups are slow and a naive crawler spends much of its time on them (Chapter 5); run a local caching resolver and prefetch.
- **Timeouts and limits**: connect and read timeouts, a maximum page size (stop after a few MB), maximum redirects, and content-type checks. A slow server must not tie up a worker (Chapter 15).
- **Retry** transient failures a few times with backoff; mark hosts that keep failing as temporarily blocked (a per-host circuit breaker).
- **Conditional requests**: send `If-Modified-Since` or `If-None-Match` with the stored ETag; a `304 Not Modified` saves bandwidth and storage.

23.6 Duplicates, traps and freshness

Exact duplicates. Compute a **signature** of the page content (a hash of the normalised text). If the signature already exists in `crawled_links`, the page is a duplicate: record the URL as an alias and do not store or re-index it. The notebook also suggests detecting duplicates *in bulk* with a MapReduce job that groups URLs by signature and keeps one canonical copy.

Near-duplicates. Many pages differ only by a timestamp or advertisement. **SimHash** (a locality-sensitive hash) produces a short fingerprint in which similar documents have fingerprints differing in only a few bits, so near-duplicates are found by comparing finger-

prints within a small Hamming distance. Alternatively compare sets of *shingles* (overlapping word sequences) using MinHash. Use these to avoid indexing thousands of nearly identical pages.

Spider traps. A trap generates an unbounded number of URLs (an infinite calendar, session ids in URLs, `/a/a/a/a/...`). Defences: limit **crawl depth** and **URLs per host**, limit URL length, detect repeated path segments, normalise away session parameters, and use priority scoring so low-value, deep or unusual URLs sink.

Freshness. Pages change at different rates; a news front page changes hourly, a manual yearly. Estimate each page's **change rate** from the history of crawls (did the signature change?) and schedule the next visit adaptively: increase the interval when unchanged, decrease it when changed, weighted by importance. A uniform weekly refresh (our estimate) wastes bandwidth on static pages and misses fast-changing ones.

23.7 From documents to search results

After a page is stored, the **indexing** stage extracts terms and updates the **inverted (reverse) index**: a map from each term to the list of documents containing it (a *postings list*), usually with positions and weights. A query for "system design" fetches the postings list for each term and intersects them, ranks the candidates and returns the top results.

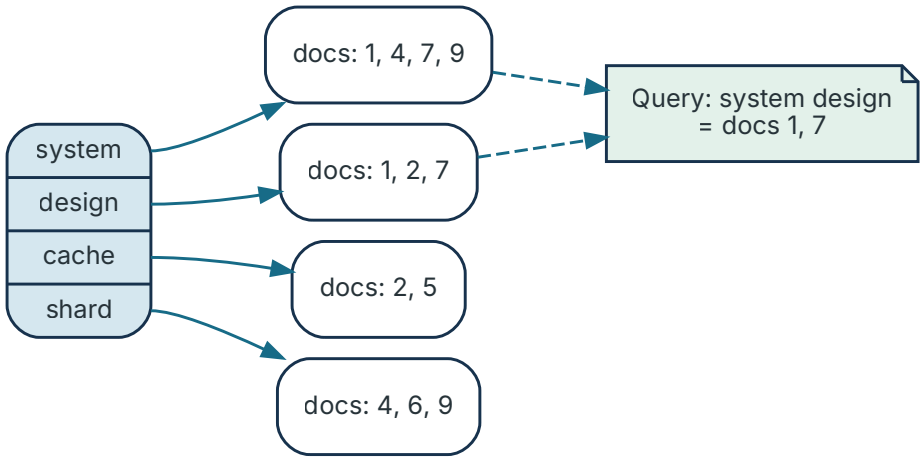


Figure 23.3 A tiny inverted index. Each term points to the documents that contain it; a multi-word query intersects the lists.

The notebook splits the query side into services: a **Reverse Index Service** (given terms, return matching document ids ranked by a relevance score), a **Document Service** (given ids, return title and snippet), and a **Query API** (accepts the query, parses and normalises it, calls both services, and returns the results). Normalisation includes lower-casing, tokenising, removing stop words, stemming ("designing" to "design"), spelling correction and synonyms.

Scale of the query side. The index is far too large for one machine, so it is **partitioned** by document (each shard indexes a subset of documents) and each shard is **replicated**. A query goes to *every* shard (scatter), each returns its local top-k, and the query service

merges (gather). Latency is set by the *slowest* shard, so tail latency matters (Chapter 1); hedged requests to replicas help. Cache popular queries (the head of the query distribution repeats heavily), and keep the index of the most important pages on fast storage (a tiered index). Ranking combines text relevance (term weights such as TF-IDF or BM25), link-based authority (PageRank-style computation done offline), freshness and other signals; a two-stage pipeline (cheap candidate retrieval, then costlier re-ranking of the top few hundred) keeps it fast.

> **SUPPLEMENTAL MATERIAL**
>
> **Computing link-based authority offline.** The web graph (pages and links) is processed in large batch jobs: PageRank starts with equal scores and repeatedly passes each page's score along its outgoing links, damped by a factor, until scores converge. At web scale this is a distributed graph computation (MapReduce or a graph engine) that runs periodically; its output is stored as a per-page score used by ranking. This also illustrates why the crawler stores *link structure* and not only text.

23.8 Scaling, failures and operations

Scaling. Partition by **host hash**: frontier shards, crawler workers and per-host state align, so politeness stays local. Add workers to raise throughput; the limiting resources are bandwidth, DNS, and politeness (a few huge sites hold many pages, so per-host rate limits cap how fast you can crawl them regardless of fleet size). Use **batching** and **append-only logs** for storage writes (Chapter 3); store raw content compressed in an object store or distributed file system, and structured metadata in a wide-column store.

Failure handling. *Worker crash*: URLs it held are returned to the frontier after a lease timeout (a visibility timeout as in Chapter 13); fetching is idempotent so a repeat is harmless. *Frontier lost*: rebuild from persisted logs; the crawl is restartable. *Site down or slow*: back off per host. *Poison pages* (huge, malformed, decompression bombs): size limits, sandboxed parsers. *Index shard failure*: replicas answer; degrade by omitting a shard's results rather than failing the query, and report partial results. *Clock or ordering issues*: do not rely on wall clock for dedupe.

Security and ethics. Parse untrusted content in sandboxes; do not execute scripts unless intentionally rendering; avoid fetching internal addresses via redirects (SSRF-style, Chapter 17); respect robots.txt, rate limits and copyright and data-protection rules; honour removal requests; avoid crawling behind logins.

Operations and cost. Metrics: pages fetched per second, error and timeout rates, politeness violations (should be zero), frontier size and age, duplicate ratio, freshness (age of the average page), index lag (time from crawl to searchable), query p99, cache hit ratio. Cost: bandwidth and storage dominate the crawl, memory and CPU dominate serving.

23.9 Alternatives and trade-offs

Table 23.2 Decisions and alternatives.

Decision	Chosen	Alternative	Reason	Revisit if
Frontier	Two-layer priority + per-host queues	Single FIFO	Politeness and priority	Small crawl
Seen-URL check	Bloom filter + exact store	Exact set only	Memory efficiency	Need exactness
Duplicate detection	Content hash + SimHash	URL equality only	Mirrors and templates	Low duplication
Refresh	Adaptive per page	Uniform weekly	Freshness vs bandwidth	Uniform data
Index partition	By document, scatter-gather	By term	Balanced load, simpler updates	Very rare terms
Raw storage	Compressed, changed pages only	Keep everything	Cost	Compliance or research needs

Follow-up questions to expect. *How do you crawl JavaScript-heavy sites?* A rendering tier with headless browsers (expensive: reserved for important pages). *How would you keep news fresh within minutes?* A separate fast-lane crawler using feeds, sitemaps and push notifications. *How do you handle a site that blocks you?* Respect it. *How do you detect and demote spam?* Link-graph analysis, content classifiers, domain reputation. *How do you make the crawl distributed across regions?* Crawl from regions near the sites to cut latency, with a global frontier coordinating hosts.

FAILURE MODES

Failure modes (summary). Overloading a small site (politeness bug); infinite spider traps filling the frontier; DNS bottleneck; frontier or seen-set loss; duplicate explosion bloating the index; stale index from slow re-crawl; scatter-gather tail latency; a poison page crashing parsers; bandwidth saturation; accidental violation of robots.txt.

COMMON MISCONCEPTION

- "*A crawler is a loop over a list of URLs.*" The frontier's priority, politeness and duplicate control are the design.
- "*More workers always crawl faster.*" Per-host politeness and DNS or bandwidth limits cap throughput.
- "*Store every page forever.*" It is unaffordable and mostly redundant.
- "*Hash equality finds all duplicates.*" Near-duplicates need similarity hashes.

INTERVIEW QUESTIONS

Q1. How do you avoid crawling the same URL twice? Normalise URLs, check a bloom filter plus an exact store, and use page content signatures to catch duplicates under different URLs.

Q2. How do you stay polite? Per-host back queues with a scheduler enforcing delays, robots.txt and crawl-delay compliance, host-aligned workers, identification in the User-Agent.

Q3. How is the search index organised and queried? An inverted index partitioned by document and replicated; queries scatter to all shards, each returns top-k, results are merged; popular queries are cached; ranking in two stages.

Q4. How do you decide how often to re-crawl a page? Adaptive scheduling from observed change rate and page importance; use conditional requests to save bandwidth.

EXERCISES

1. If the average page is 300 KB and the crawl must refresh 2 billion pages every 7 days, compute fetches per second and bandwidth in Gbps.
2. A worker handles 25 concurrent fetches, average 400 ms per fetch. How many workers sustain 2,000 fetches per second with 50% headroom?
3. Propose a signature scheme for catching near-duplicates among 10 billion pages and the approximate storage for fingerprints.
4. Design the schema of `crawled_links` and the index you need for fast "have we seen this signature?" checks.

Answers.

1. 2×10^9 / (7 × 86,400) ≈ 3,300 fetches per second; 3,300 × 300 KB ≈ 1 GB/s ≈ 8 Gbps.
2. One worker: 25 / 0.4 = 62.5 fetches per second. Need 2,000 / 62.5 = 32 workers; with 50% headroom, 48.
3. A 64-bit SimHash per page, with lookups by permuted prefixes to find fingerprints within Hamming distance 3. 10^{10} × 8 B = 80 GB (plus index structures), shardable by prefix.
4. `crawled_links(url_hash PK, url, signature, last_crawl_time, etag, change_score, status)`; a secondary index (or separate table) keyed by `signature` to map signature to canonical URL; partition by `url_hash`.

CHAPTER SUMMARY

- The frontier (priority, per-host politeness, persistence) is the core of the crawler.
- Normalise URLs, detect seen URLs with a bloom filter plus exact store, catch duplicates with content signatures and SimHash, and avoid traps.
- Schedule re-crawls adaptively; use conditional requests; store compressed text and changed pages only.
- Search uses a partitioned, replicated inverted index with scatter-gather queries, caching and two-stage ranking.
- Be polite and ethical; handle failures at every stage; measure freshness and index lag.

REVISION CHECKLIST

- [ ] I can estimate fetch rate, bandwidth and storage.
- [ ] I can draw the two-layer frontier.
- [ ] I can explain duplicate and near-duplicate detection.
- [ ] I can explain the inverted index and scatter-gather query.

CHAPTER 24

Case Study: A Personal Finance Aggregator

Derived from your notes: Notebook chapter 12.4 (personal finance aggregator: accounts linked via an Accounts API, transaction extraction as an async job, category assignment by seller, budgets and notifications, scale table). **Added in this book:** idempotent ingestion, deduplication and pending-versus-posted transactions, money as integers, credential security and aggregator tokens, category rules and overrides, budget evaluation as a stream, storage layout and partitioning, failure handling and privacy.

LEARNING OBJECTIVES

After this chapter you can:

- scope a finance-aggregation product and estimate its write and read load;
- design an asynchronous, idempotent transaction-ingestion pipeline;
- model money, accounts, transactions, categories and budgets correctly;
- explain category assignment, budget alerts and why a read-heavy dashboard needs summaries;
- protect credentials and sensitive financial data and plan for failure.

PREREQUISITE KNOWLEDGE

Chapter 6 (idempotency), Chapter 8, Chapter 13 (queues, outbox), Chapter 17 and Chapter 20.

24.1 Problem and requirements

The problem. Users connect their bank and card accounts. The service pulls their **transactions**, assigns each to a **spending category** (groceries, rent, travel), and shows **spending by category and month**. Users set **budgets** per category and are **notified** when they approach or exceed them.

Functional requirements. Link and unlink accounts. Fetch transactions periodically and when users ask. Categorise by seller (with user overrides). Show per-category monthly summaries and transaction lists. Create budgets (a default may be suggested from history) and send alerts.

Non-functional requirements. *Correctness of money* matters more than speed: no lost, duplicated or altered amounts. *Security and privacy*: financial data is highly sensitive. *Availability*: the dashboard should work even if a bank is down (show last known data, clearly marked). *Freshness*: a daily update is acceptable; the notebook assumes transactions are pulled **daily** for active users. *Scalability* to tens of millions of users.

Scope. Out of scope: moving money (payments), investments, credit scoring, and the details of each bank's interface. We treat the connection to banks as a third-party **aggregator API** that returns transaction data through tokens (we should never store raw bank passwords if a token flow exists).

24.2 Scale estimate

Table 24.1 Assumptions from the notebook, with derived quantities.

Quantity	Assumption	Derived
Users	10 million	
Linked accounts	5 per user	50 million accounts
Transactions	30 per account per month	1.5 billion per month
Write rate	1.5×10^9 / 2.5×10^6 s	≈ 600 transaction writes per second average
Reads	10 dashboard views per user per month, a few queries each	≈ 40 reads per second average, strongly bursty
Record size	≈ 200 bytes per transaction	1.5×10^9 × 200 B = 300 GB per month, about 3.6 TB per year

What the numbers say. The workload is **write-heavy and batch-like**: ingestion is a steady stream (600 per second average, perhaps 10 times that in daily batch windows), while reads are rare per user but need to be fast. Storage grows to tens of terabytes over several years, which is beyond one SQL server comfortably, so partition by user. Peak ingestion is set by the daily refresh window: if all 50 million accounts refresh in a 4-hour window, that is 50×10^6 / 14,400 ≈ 3,500 account fetches per second, each returning about 1 new transaction per day on average, and far more on first link.

24.3 Architecture overview

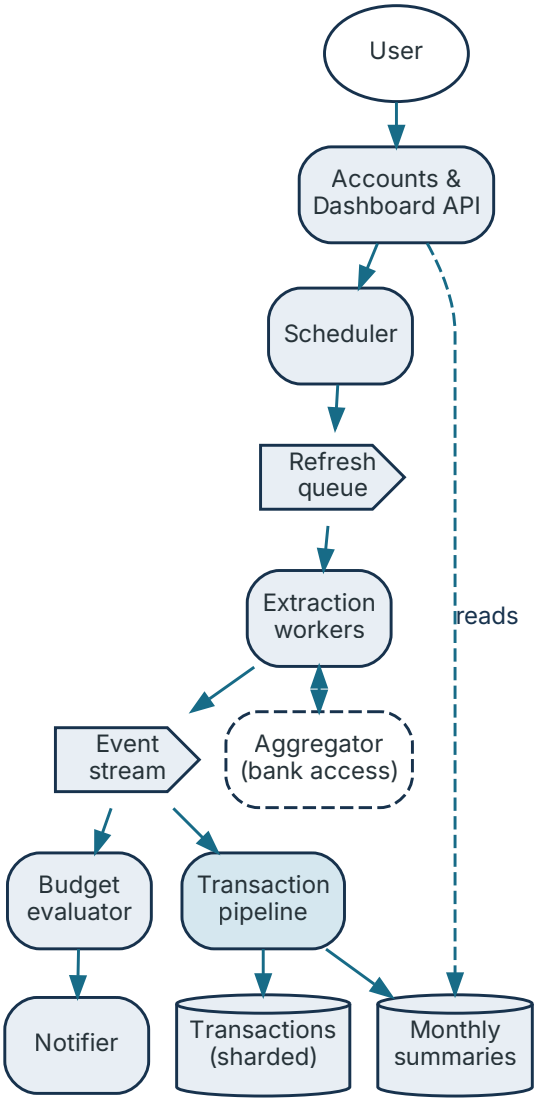


Figure 24.1 Mint architecture. A scheduler enqueues refresh jobs; extraction workers call the aggregator; a transaction pipeline normalises, deduplicates, categorises and stores; a budget evaluator and notifier consume the same stream; the dashboard reads summaries.

In Figure 24.1, the **Accounts API** receives link requests and stores account metadata (never raw credentials). A **scheduler** places a refresh job per account on a queue. **Extraction workers** call the aggregator with the stored token and publish raw transactions to a stream. The **transaction pipeline** normalises amounts and dates, deduplicates, assigns categories and writes to the transaction store; the same events update **monthly summaries** and feed the **budget evaluator**, which emits notifications. The **dashboard API** reads summaries and transaction lists.

24.4 API contracts

Method and path	Purpose	Notes
`POST /v1/accounts/link`	start linking an institution	returns a redirect or link token; secrets stay with the aggregator
`DELETE /v1/accounts/{id}`	unlink and schedule data deletion	idempotent
`POST /v1/accounts/{id}/refresh`	request refresh now	returns `202` with a job id; rate-limited per account
`GET /v1/transactions?from=&to=&category=&cursor=`	list transactions	cursor pagination (Chapter 6)
`PUT /v1/transactions/{id}/category`	override category	optional "apply to all from this seller"
`GET /v1/summary?month=2026-10`	totals by category	served from precomputed summaries
`PUT /v1/budgets/{category}`	create or update budget	amount in minor units plus currency

Refresh returns **202 Accepted** because the work is asynchronous: the client polls the job or receives a notification, rather than holding a connection while a bank is slow.

24.5 Data model

Money is an integer. Store amounts as integer **minor units** (cents) with an ISO currency code. Floating point cannot represent 0.10 exactly and accumulates error; decimal types are acceptable, but integers are simplest. Never sum across currencies without conversion.

Table 24.2 Core tables (partitioned by `user_id`).

Table	Key columns	Notes
`accounts`	`account_id PK, user_id, institution, token_ref, status, last_sync`	`token_ref` points to a secrets store, not the secret itself
`transactions`	`(user_id, account_id, external_txn_id) UNIQUE, txn_id, posted_date, amount_minor, currency, seller_raw, seller_id, category_id, state`	`state` is pending or posted; unique key gives dedupe
`sellers`	`seller_id, normalised_name, default_category_id`	maps messy descriptions to a canonical seller
`category_overrides`	`(user_id, seller_id) PK, category_id`	user's own mapping wins
`monthly_summary`	`(user_id, month, category_id) PK, total_minor, count`	updated incrementally
`budgets`	`(user_id, category_id) PK, limit_minor, currency, alert_levels`	

Indexes. `transactions(user_id, posted_date DESC)` serves "recent transactions"; `(user_id, category_id, posted_date)` serves category views. Partitioning by `user_id` keeps one user's data together so dashboard queries touch one shard.

Relational or NoSQL? The notebook uses a SQL database for user and account data and sketches a wider choice for the transaction volume. A reasonable answer: keep accounts, budgets and overrides in a relational database (relationships, transactions, small); keep the large, append-mostly transaction history in a sharded relational store or a wide-column store keyed by user and time (Chapter 10), with the unique external id enforced per partition.

24.6 Ingestion, step by step

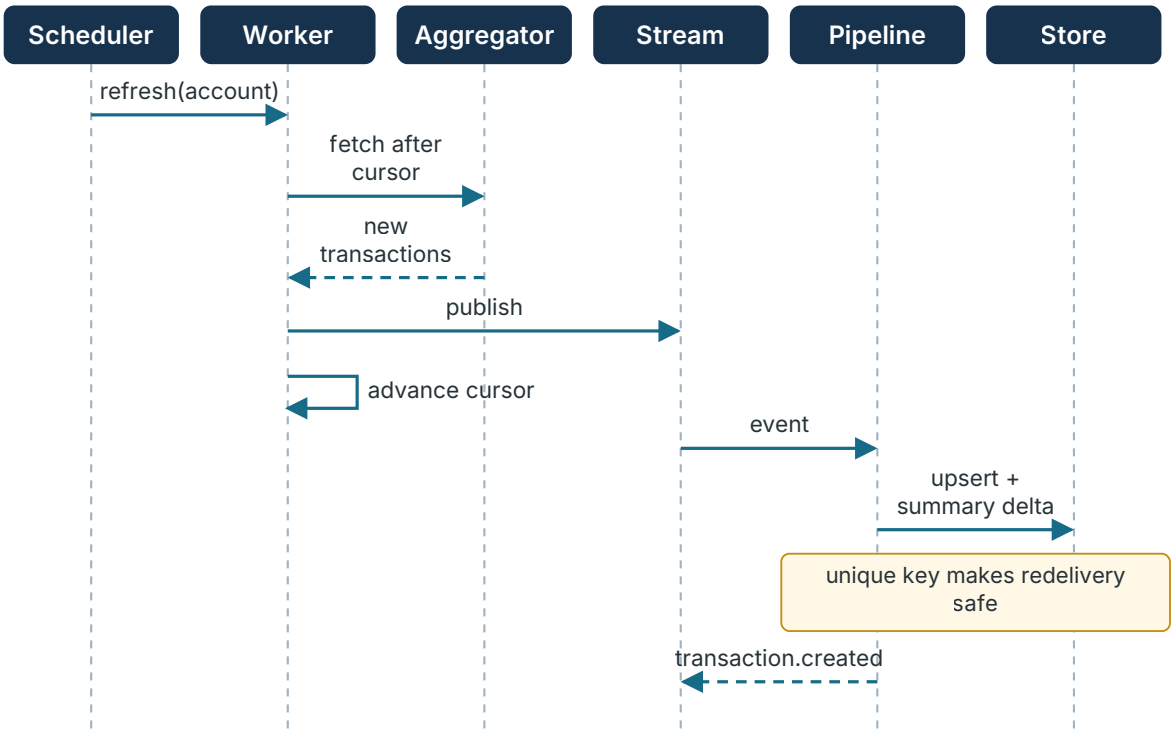


Figure 24.2 One refresh. The scheduler enqueues a job; the worker fetches with a cursor, publishes events, and the pipeline writes transactions idempotently and updates summaries and budgets.

1. The scheduler enqueues `refresh(account_id)`; the job carries an idempotency key (account and date).
2. A worker reads the account's **sync cursor** (the last position returned by the aggregator) and requests new transactions after it.
3. It publishes raw transactions to the stream and **advances the cursor only after publishing succeeds**.
4. The pipeline normalises each record (amount to minor units, date to UTC date, seller name cleaned) and upserts it using the unique key `(user, account, external id)`, so redelivery causes no duplicates (at-least-once delivery plus idempotent writes, Chapter 13).

5. It assigns a category (next section) and, in the **same transaction** or through an outbox, updates `monthly_summary` and emits a `transaction.created` event.
6. The budget evaluator reads the event, adds the amount to the month's running total, and notifies when thresholds (for example 80% and 100%) are crossed, once per threshold per month.

Pending versus posted. Banks first report *pending* transactions, whose amount or id can change when they post. Treat `state` as part of the record: update a pending entry in place when the posted version arrives (matching by external id, or by amount, merchant and date heuristics when ids change), adjusting summaries by the **difference**, never counting both.

Corrections and reversals. Refunds are new negative transactions; never edit history silently. If the aggregator revises a past transaction, record the change and adjust the summary by the delta.

24.7 Categories, summaries and budgets

Category assignment. The notebook assigns a category by **seller** using a lookup (a `Category` abstraction with a `seller_category_map`). Pipeline: clean the raw description ("AMZN MKTP US*2K4 SEATTLE" *becomes "Amazon"), look up the canonical seller, apply the user override if present, else the seller default, else "Uncategorised" and queue the item for a classifier or review. Heap-based top-k lookups for "recently used categories" and "most common seller categories" are cheap with in-memory structures (Chapter 3). Override rules should apply to* future* transactions and optionally to history (a background re-categorisation job that adjusts summaries by delta).

Summaries. Dashboard reads must not scan millions of transactions. Maintain `monthly_summary` incrementally on write (add the amount to its row), and recompute from source periodically as a reconciliation check. Because writes arrive at-least-once, the summary update must be **idempotent**: apply it only when the transaction insert actually created or changed a row, in the same transaction.

Budgets. The notebook computes a default budget from spending, for example a percentage above the average of recent months. The evaluator keeps a per-user, per-category running total and compares with limits; it must handle month boundaries (by the user's time zone), refunds (totals can drop), and late-arriving transactions (dated earlier, so affect a past month).

> **WORKED EXAMPLE**
>
> **Budget alert.** Groceries budget 500.00 (50,000 cents). Posted so far this month: 38,000. A new transaction of 2,500 arrives: total 40,500 = 81%, crossing the 80% threshold: send one alert and record `alerted_80 = true` for this month. Later a refund of −1,000 lowers the total to 39,500 (79%): do not re-alert when it crosses 80% again unless the product rule says so; document whichever rule you choose.

24.8 Security, privacy and compliance

- **Credentials.** Prefer OAuth-style tokens from an aggregator; never store bank passwords in plain form. Keep tokens in a secrets manager or encrypted column with a key held in a KMS, with access limited to extraction workers (Chapter 17).
- **Encryption** in transit (TLS) and at rest; field-level encryption for account numbers (store only the last four digits for display).
- **Least privilege and audit**: log who accessed what; support staff see masked data.
- **Data rights**: export and delete on request; deleting an account must remove transactions and derived summaries and tokens, and revoke the token at the aggregator.
- **Regional and financial regulations** apply (for example data protection and open-banking rules); mention them and defer to specialists.
- **Fraud and abuse**: rate-limit refresh and linking per user; protect against credential stuffing on the login.

24.9 Scaling, failures and operations

Scaling. Ingestion scales horizontally: more workers per queue partition; throttle per institution because banks rate-limit the aggregator. Spread the daily refresh over hours with **jitter** to avoid a thundering herd. Partition transactions by `user_id`. Cache summaries for the dashboard (a short TTL is fine; invalidate on write).

Failure handling. *Bank or aggregator down*: retry with backoff and a circuit breaker per institution; show "last updated" and a stale marker. *Expired or revoked token*: mark the account `needs_reauth` and notify the user. *Worker crash mid-job*: the queue redelivers; idempotent upserts make repeat safe. *Poison transaction* (unparseable): send to a dead-letter queue and count it. *Pipeline lag*: back pressure and autoscaling on queue age. *Summary drift*: nightly reconciliation compares summaries with sums from the transaction store and repairs differences.

Operations. Track sync success rate per institution, freshness (age of last sync), queue age, duplicate rate, uncategorised percentage, budget alert latency, and reconciliation mismatches. Cost is dominated by transaction storage and aggregator fees per linked account, so stop refreshing dormant users.

24.10 Alternatives and trade-offs

Table 24.3 Decisions and alternatives.

Decision	Chosen	Alternative	Reason	Revisit if
Ingestion	Queue plus idempotent upsert	Synchronous fetch on dashboard open	Isolation from slow banks	Interactive refresh is the main feature
Summaries	Incremental table plus reconciliation	Compute on read	Fast dashboard	Small data

Decision	Chosen	Alternative	Reason	Revisit if
Storage	Relational for core, partitioned transactions	Single NoSQL store	Integrity for accounts	Extreme volume
Money type	Integer minor units	Floating point	Exactness	Never use floats
Category source	Seller map plus overrides	ML-only	Predictable, explainable	Many unknown sellers

Follow-ups. *Multiple currencies?* Store original currency and amount; convert for display with dated rates. *Shared household accounts?* Introduce a household entity and permissions. *Real-time notifications?* Use aggregator webhooks to trigger refresh. *Recurring bills detection?* A batch job over history grouping by seller and periodic gaps.

FAILURE MODES

Failure modes (summary). Duplicate transactions from redelivery; pending and posted both counted; summary drift; missed alert or repeated alerts; expired tokens silently stopping sync; thundering herd at refresh time; floating-point rounding errors; credential leak; deleted users' data left in summaries.

COMMON MISCONCEPTION

- "*Just store the bank password.*" Use token flows and a secrets store; passwords are a liability.
- "*Floats are fine for dollars.*" They are not; use integers.
- "*The dashboard can sum transactions on every view.*" Maintain summaries.
- "*Retries cause no harm.*" Only if writes are idempotent.

INTERVIEW QUESTIONS

Q1. How do you avoid duplicate transactions? Unique key on user, account and the bank's transaction id; upsert; cursor advanced after publish; idempotent summary updates.

Q2. How do you handle pending transactions? Keep state, update in place when posted, apply summary deltas.

Q3. How do you keep the dashboard fast? Incrementally maintained monthly summaries, partitioning by user, short-TTL cache.

Q4. What if the bank is down? Show stale data with timestamp; retry with backoff; circuit breaker per institution.

EXERCISES

1. With 20 million users, 4 accounts each and 25 transactions per account per month at 250 bytes, compute transactions per second (average) and yearly storage.
2. Show how a pending 40.00 charge that posts as 42.50 changes a category total of 300.00.
3. Design the idempotency of the summary update when the same event is delivered twice.
4. A user's budget alert fired twice. List three possible causes.

Answers.

1. $20\times10^6 \times 4 \times 25 = 2\times10^9$ per month; $\div 2.5\times10^6$ s = 800 per second. Storage $2\times10^9 \times 250$ B = 500 GB/month, 6 TB/year.
2. Pending adds 4,000 cents: total 30,000 + 4,000 = 34,000. On posting, apply the delta +250 (42.50 − 40.00 = 2.50): total 34,250 cents = 342.50. It must not add 4,250 again.
3. Insert the transaction with `ON CONFLICT DO NOTHING` (or compare version); update the summary only if a row was inserted or its amount changed, in the same database transaction; store processed event ids otherwise.
4. The alert flag was not stored atomically with the total; redelivery of the same event; refund and re-crossing with no rule; evaluator replicas without partitioning by user.

CHAPTER SUMMARY

- Treat ingestion as an asynchronous, idempotent pipeline: queue, cursor, upsert, delta-based summaries.
- Money is integers; pending and posted states and reversals need explicit handling.
- Summaries and partitioning by user make a read dashboard fast on a write-heavy store.
- Protect credentials with tokens, secrets management and least privilege; support deletion.
- Plan for bank outages, token expiry, drift and thundering herds.

REVISION CHECKLIST

- [ ] I can compute write rate and storage.
- [ ] I can explain idempotent ingestion.
- [ ] I can describe the schema and indexes.
- [ ] I can explain budgets and alerts without duplicates.

CHAPTER 25

Case Study: Social Graph and Shortest Path

Derived from your notes: Notebook chapter 12.5 (social network data structures: Person, Person Server, Lookup Service, Graph Service, BFS for shortest path, sharding persons across servers, scale table). **Added in this book:** bidirectional BFS and its cost analysis, adjacency-list storage, graph partitioning trade-offs, caching hot neighbourhoods, privacy filtering, degrees-of-separation limits, and graph databases as an alternative.

LEARNING OBJECTIVES

After this chapter you can:

- model a social network as a graph and estimate its storage and query load;
- explain why a single-machine breadth-first search fails at scale;
- design a sharded person store with a lookup service;
- run and cost a bidirectional BFS across shards, with caching and limits;
- discuss privacy, hot nodes and graph-database alternatives.

PREREQUISITE KNOWLEDGE

Chapter 3, Chapter 4, Chapter 11 (sharding) and Chapter 12.

25.1 Problem and requirements

The problem. Given two users, find the **shortest path** of friendships connecting them ("how many degrees of separation?"), for a network with hundreds of millions of users, so that the data does not fit on one machine.

Functional requirements. Given user A and user B, return a shortest path (or its length) if one exists within a limit. Friend relations are **undirected** (mutual). Friends can be added and removed.

Non-functional requirements. Low latency (hundreds of milliseconds to a few seconds), high availability for reads, and scalability across many machines. **Privacy** applies: only traverse friendships the requester may see.

Scope. We assume an unweighted, undirected graph, an approximate or length-limited answer is acceptable (say up to 6 hops), and the graph store is already sharded. Weighted paths, recommendations ("people you may know") and real-time graph updates in search results are extensions.

25.2 Scale estimate

Table 25.1 Assumptions and derived quantities (the notebook gives the style of calculation; the figures are assumptions).

Quantity	Assumption	Derived
Users	400 million	
Friends per user	300 average (heavy tail: some have 5,000+)	1.2×10^{11} adjacency entries (each friendship stored twice)
Entry size	8-byte id	≈ 1 TB of adjacency data (more with metadata)
Queries	100 million shortest-path requests per month	≈ 40 per second average
Branching	average degree d = 300	level k holds about 300^k users

What the numbers say. About a terabyte of adjacency lists does not fit in one machine's memory comfortably but fits in a cluster of 10 to 50 machines with replication. The read rate is modest; the *cost per query* is the problem. With d = 300, level 1 has 300 users, level 2 has about 90,000, level 3 about 27 million (so most of a country), and a path of length 6 would need to touch nearly the whole network. Plain BFS from A to depth 6 is hopeless; we need to be clever.

25.3 Why naive BFS fails

Breadth-first search explores all neighbours of the start, then their neighbours, level by level, and finds a shortest path in an unweighted graph. On one machine the notebook's `shortest_path` uses a queue of nodes to visit and a `visited` set, following `friend_ids` to neighbours. On a sharded graph each step "fetch this person's friend list" is a **network call** to whichever server holds the person, found through a **Lookup Service** (person id to Person Server). Exploring millions of nodes means millions of calls.

Bidirectional BFS is the key optimisation. Search from A and from B simultaneously, expanding the smaller frontier each time, and stop when the frontiers meet.

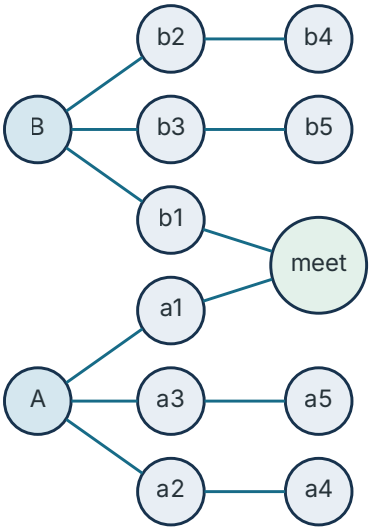


Figure 25.1 Bidirectional BFS. Each side expands only a few levels, so the explored region is two small balls instead of one huge one.

Cost. If the shortest distance is L and the branching factor is d, one-sided BFS visits about d^L nodes, bidirectional visits about 2·d^(L/2). For d = 300 and L = 4: one-sided $300^4 = 8.1\times10^9$ versus bidirectional $2 \times 300^2 = 180{,}000$. That difference (five orders of magnitude) turns an impossible query into a feasible one. For L = 6 the numbers are 7×10^{14} versus 5.4×10^7, still heavy, which is why production systems cap the depth, use caching, or precompute.

25.4 Architecture

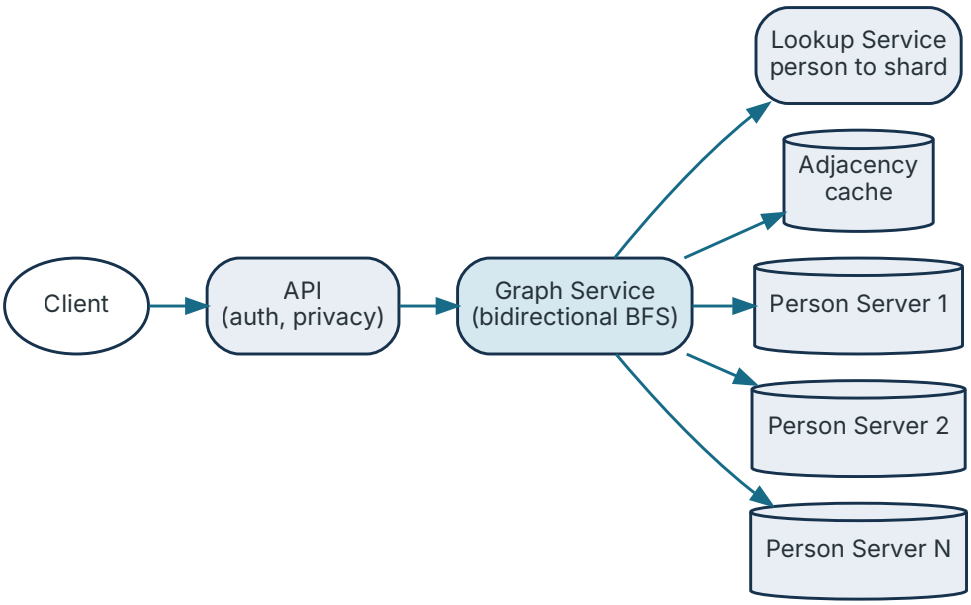


Figure 25.2 Components. The Graph Service runs the search; the Lookup Service finds the shard of each person; Person Servers hold adjacency lists.

The notebook's components, with their roles:

- **Person Server**: holds a partition of persons, each with an id, name and list of friend ids (the adjacency list).
- **Lookup Service**: maps `person_id` to the Person Server that stores it, as an in-memory map or by a fixed hash function.
- **Graph Service (User Graph Service)**: receives (A, B), performs bidirectional BFS, batching lookups and friend-list fetches, and returns the path.
- **Client/API**: authenticates the requester and applies privacy rules.

Sharding the graph. Partition persons across servers by **hash of id** (balanced, trivial lookup, but neighbours are scattered, so a traversal touches many servers), or by **community/locality** (people who are friends tend to share a shard, reducing cross-server calls, but needs a graph-partitioning algorithm and can become unbalanced and hard to maintain). Start with hash partitioning plus caching; consider locality-aware partitioning when the cross-shard traffic dominates cost.

Batching. Instead of fetching one friend list at a time, expand a whole frontier level: group the frontier ids by shard and send one batched request per shard. A frontier of 90,000 ids across 50 shards becomes 50 requests, not 90,000, turning latency from "number of nodes" into "number of levels times one round trip".

25.5 Algorithm in detail

```
function shortest_path(a, b, max_depth):
    if a == b: return [a]
    seen_a = {a: None}; seen_b = {b: None}
    frontier_a = {a};   frontier_b = {b}
    for depth in 1..max_depth:
        # expand the smaller frontier
        if len(frontier_a) <= len(frontier_b):
            frontier_a = expand(frontier_a, seen_a)
        else:
            frontier_b = expand(frontier_b, seen_b)
        meet = frontier_a ∩ frontier_b   (or seen_a ∩ seen_b)
        if meet not empty:
            return build_path(meet.any, seen_a, seen_b)
    return NOT_FOUND

function expand(frontier, seen):
    new = {}
    for shard, ids in group_by_shard(frontier):      # batched, parallel
        for person, friends in shard.get_friends(ids):
            for f in friends:
                if f not in seen:
                    seen[f] = person                  # parent pointer
                    new.add(f)
    return new
```

Why correct. Each expansion adds one level around its source; the first time the two balls share a node, the sum of their radii is the shortest distance (BFS levels are by increasing distance). To get exactly the shortest path when frontiers are expanded unevenly, finish the level in progress before returning. The parent pointers in `seen` rebuild the path from both halves.

Complexity. Time is proportional to nodes and edges examined (bounded by the two balls); memory is the size of both seen sets. Both are controlled by `max_depth` and by the **frontier size cap**.

> **WORKED EXAMPLE**
>
> **Dry run.** Friendships: A–C, C–D, D–E, E–B, A–X, X–Y. Search A to B; seen sets start as {A} and {B}. Step 1: frontiers tie in size, so expand A: frontier {C, X}. Step 2: B's frontier (1) is smaller, expand it: {E}; no overlap. Step 3: expand B's side again: {D}; no overlap. Step 4: expand D, reaching C, which is already in A's seen set: the frontiers meet at C. Parent pointers give A, C on one side and C, D, E, B on the other: path A, C, D, E, B (length 4). The branch X, Y was never explored beyond X.

25.6 Caching, hot nodes and limits

- **Cache adjacency lists** of popular users in memory; a celebrity with millions of friends appears in many paths. Use an LRU cache (Chapter 12) in front of Person Servers.
- **Hot nodes** can poison the search: expanding a user with 5 million friends explodes the frontier. Strategies: cap the number of friends expanded per node, skip "super-nodes" unless the path needs them (and prefer paths through ordinary users), or precompute hub-to-hub distances.
- **Limits**: maximum depth (6), maximum nodes visited, and a deadline. Return "not found within limit" rather than run unbounded: this is an *approximate* answer, which must be stated.
- **Result caching**: cache (A, B) answers briefly, since the graph changes slowly and popular pairs repeat.
- **Precomputation**: offline jobs can compute landmarks (distances from some hub nodes) giving bounds that prune the search; or maintain 2-hop neighbour sets for active users at a storage cost.

25.7 Privacy, consistency and operations

Privacy. Do not reveal connections that the requester should not see. Each edge may carry visibility (public, friends only). The Graph Service must filter edges by what the requester can see *during* traversal, or a path can leak existence of a private friendship. This can make the "shortest" path depend on the viewer.

Consistency. Friendship changes are written to the owning shard(s); because each friendship is stored on both endpoints, which may live on different shards, an update touches two shards and can be briefly inconsistent (one side present). Writes use an idempotent two-step update with repair jobs (or a durable queue) to converge; reads tolerate brief staleness (Chapter 11).

Failures. A Person Server down: replicas serve reads; if a shard is unavailable return a partial or failed result rather than a wrong "no path". *Distinguish "no path exists" from "could not search".*

Operations. Metrics: p50/p99 latency, nodes visited per query, cache hit ratio, batched request fan-out, percentage of limited or failed searches, hot-node events. Cost is dominated by memory for the adjacency data and cache.

25.8 Alternatives and trade-offs

Table 25.2 Decisions and alternatives.

Decision	Chosen	Alternative	Reason	Revisit if
Search	Bidirectional BFS, depth-limited	One-sided BFS	Orders of magnitude fewer nodes	Tiny graph
Partition	Hash of person id	Community-aware	Simplicity and balance	Cross-shard cost dominates
Storage	Adjacency lists in sharded KV or memory	Graph database	Predictable scale	Rich graph queries needed
Expansion	Batched per shard	Node-by-node	Latency	Single shard
Hubs	Cap or skip super-nodes	Expand all	Bounded cost	Exact answers required

Follow-ups. *Mutual friends count?* Intersect two adjacency lists (sorted lists, merge, or a bitmap). *Friend suggestions?* Count friends-of-friends not already friends, ranked by number of mutual friends, precomputed offline. *Weighted edges?* Use Dijkstra or A *with a heuristic.* Billions of users?* Locality-aware sharding, in-memory graph, precomputed landmarks.

FAILURE MODES

Failure modes (summary). Exploding frontier at a hub; cross-shard call storms; stale or half-written friendships; wrong "no path" during shard outage; privacy leak through paths; memory exhaustion from large seen sets.

COMMON MISCONCEPTION

- "*BFS is cheap.*" It is exponential in depth with a high branching factor.
- "*Sharding makes the graph query parallel for free.*" Cross-shard traversals add latency unless batched.
- "*Not found means no connection.*" It may mean the limit was reached.

INTERVIEW QUESTIONS

Q1. Why bidirectional BFS? It explores two balls of radius L/2 instead of one of radius L: about 2·d^(L/2) rather than d^L nodes.

Q2. How do you traverse a sharded graph efficiently? Lookup service plus batching of each level by shard, with caching of hot lists.

Q3. How do you handle celebrities? Cap or skip super-nodes, cache their lists, precompute hubs.

Q4. How do you keep privacy? Filter edges by viewer permission during traversal.

EXERCISES

1. With average degree 200, compute nodes explored by one-sided BFS and by bidirectional BFS for distance 4 and for distance 6.
2. Suppose 60 shards and a frontier of 120,000 ids. How many requests with batching versus without?
3. Show that a friendship stored on two shards needs an idempotent update. What breaks otherwise?
4. Give a limit policy that yields a bounded response time.

Answers.

1. L = 4: $200^4 = 1.6\times10^9$ vs $2\times200^2 = 80{,}000$. L = 6: $200^6 = 6.4\times10^{13}$ vs $2\times200^3 = 1.6\times10^7$.
2. Batched: at most 60 requests per level (per-shard); unbatched: 120,000.
3. A retry after a partial failure could add a duplicate edge or leave one direction missing; an idempotent "add friend (a,b)" on both sides plus repair makes retries safe.
4. Maximum depth 6, a cap on nodes (say 5 million) and a deadline (say 1 second); return not-found-within-limit with the reason.

CHAPTER SUMMARY

- Social graphs have high branching, so one-sided BFS is infeasible beyond a few hops.
- Bidirectional BFS reduces the work from d^L to about 2·d^(L/2).
- Shard by person id; use a lookup service, batched level expansion and caches.
- Limit depth and handle hubs; make approximate answers explicit.
- Respect privacy during traversal; keep friendship writes idempotent.

REVISION CHECKLIST

- [ ] I can explain why bidirectional BFS is cheaper and compute the gain.
- [ ] I can describe the roles of Person Server, Lookup Service and Graph Service.
- [ ] I can explain batching per shard and hub handling.

CHAPTER 26

Case Study: A Query Cache with LRU

Derived from your notes: Notebook chapter 12.6 (key-value cache for search queries: Query API server, Reverse Index Service, Document Service, Memory Cache with LRU using a linked list and hash table, cache update, scale table). **Added in this book:** distributed cache sharding with consistent hashing, hot-key and stampede protection, TTL and invalidation, negative caching, thread-safe LRU, approximations such as sampled LRU and TinyLFU, and measuring hit ratio.

LEARNING OBJECTIVES

After this chapter you can:

- explain why a search service caches whole queries and estimate the hit ratio and memory required;
- implement an LRU cache with a hash map and doubly linked list in O(1);
- design a distributed, sharded cache and keep it consistent enough;
- prevent stampedes and hot-key overload, and choose TTLs and invalidation;
- test and monitor a cache.

PREREQUISITE KNOWLEDGE

Chapter 3 (hash tables, linked lists), Chapter 12, Chapter 11 (consistent hashing).

26.1 Problem and requirements

The problem. A search service receives about 40,000 queries per second (the notebook's scale). Each query runs a costly pipeline: parse and normalise, look up postings in the reverse index, rank, fetch titles and snippets. Many queries repeat ("weather", a trending news item). Design a **cache of query results** so repeated queries are served quickly and cheaply.

Functional requirements. Given a query string, return cached results if present, else compute, store and return. Evict old entries when full. Allow fast lookups.

Non-functional requirements. Very low latency (cache hit well under 10 ms), high hit ratio, high availability (a cache failure must degrade performance, not correctness), reasonable freshness (results may be minutes old).

Scope. We cache results for the *query string after normalisation*. Out of scope: personalised results (they fragment the key space; mention as an extension) and partial result caching.

26.2 Scale and sizing

Table 26.1 Assumptions and derived quantities.

Quantity	Assumption	Derived
Queries	40,000 per second peak	100 billion per month
Result size	10 results × (title + snippet + url) ≈ 4 KB	
Distribution	Zipf-like: top 1% of queries produce about 50% of traffic	caching the head is very effective
Cache size	100 million entries × 4 KB	≈ 400 GB across the cache cluster
Hit ratio target	60 to 80%	misses cut from 40,000 to 8,000 to 16,000 per second

Why a cache helps so much. Query popularity is heavily skewed: a small head of queries is asked over and over, with a long tail asked once. A modest cache captures the head. If the hit ratio is 70%, the backend sees 12,000 queries per second instead of 40,000, and average latency falls from (say) 200 ms to $0.7 \times 5 + 0.3 \times 200 = 63.5$ ms. The long tail never hits; do not expect 100%.

26.3 LRU: the data structure

Least Recently Used evicts the entry unused for the longest time, assuming recent use predicts near-future use. To do `get` and `put` in O(1) combine two structures:

- a **hash map** from key to node (O(1) lookup), and
- a **doubly linked list** ordered from most recently used (head) to least (tail), so moving a node to the head and removing the tail are O(1).

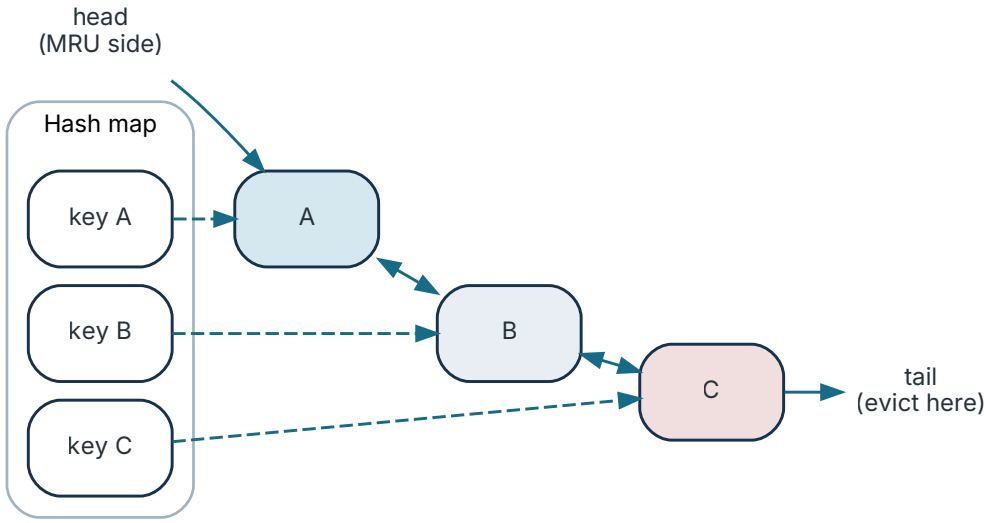


Figure 26.1 LRU cache internals: a hash map points into a doubly linked list ordered by recency. A hit moves the node to the front; an insertion at capacity evicts the tail.

```
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}              # key -> node
        self.head = Node(); self.tail = Node()   # sentinels
        self.head.next = self.tail; self.tail.prev = self.head

    def _remove(self, n):  n.prev.next = n.next; n.next.prev = n.prev
    def _push_front(self, n):
        n.next = self.head.next; n.prev = self.head
        self.head.next.prev = n; self.head.next = n

    def get(self, key):
        n = self.map.get(key)
        if n is None: return None          # miss
        self._remove(n); self._push_front(n)
        return n.value

    def put(self, key, value):
        n = self.map.get(key)
        if n:                              # update existing
            n.value = value; self._remove(n); self._push_front(n); return
        if len(self.map) >= self.cap:      # evict least recently used
            lru = self.tail.prev
            self._remove(lru); del self.map[lru.key]
        n = Node(key, value); self.map[key] = n; self._push_front(n)
```

Invariants. (1) `len(map)` equals the number of real nodes and never exceeds capacity; (2) every node in the list is in the map under its own key; (3) the list order is exactly recency order. **Tests:** get on empty returns miss; put up to capacity keeps all; adding one more evicts the oldest; a `get` refreshes recency so a different key is evicted; updating a key does not change size; capacity 1 works; concurrent get/put keeps the invariants (needs a lock, or one lock per shard).

Concurrency. A shared LRU needs a mutex because even `get` mutates the list. Under heavy contention shard the cache into many small LRUs by key hash (each with its own lock), or approximate LRU: sampled eviction (pick a few random entries and evict the oldest) and CLOCK, which avoid the global list.

Beyond LRU. One-time scans can flush valuable entries; policies such as **LFU**, **segmented LRU** and **TinyLFU** admit only items likely to be reused. Name the trade-off: LRU is simple and adapts quickly; frequency-aware policies resist scans.

26.4 The query path

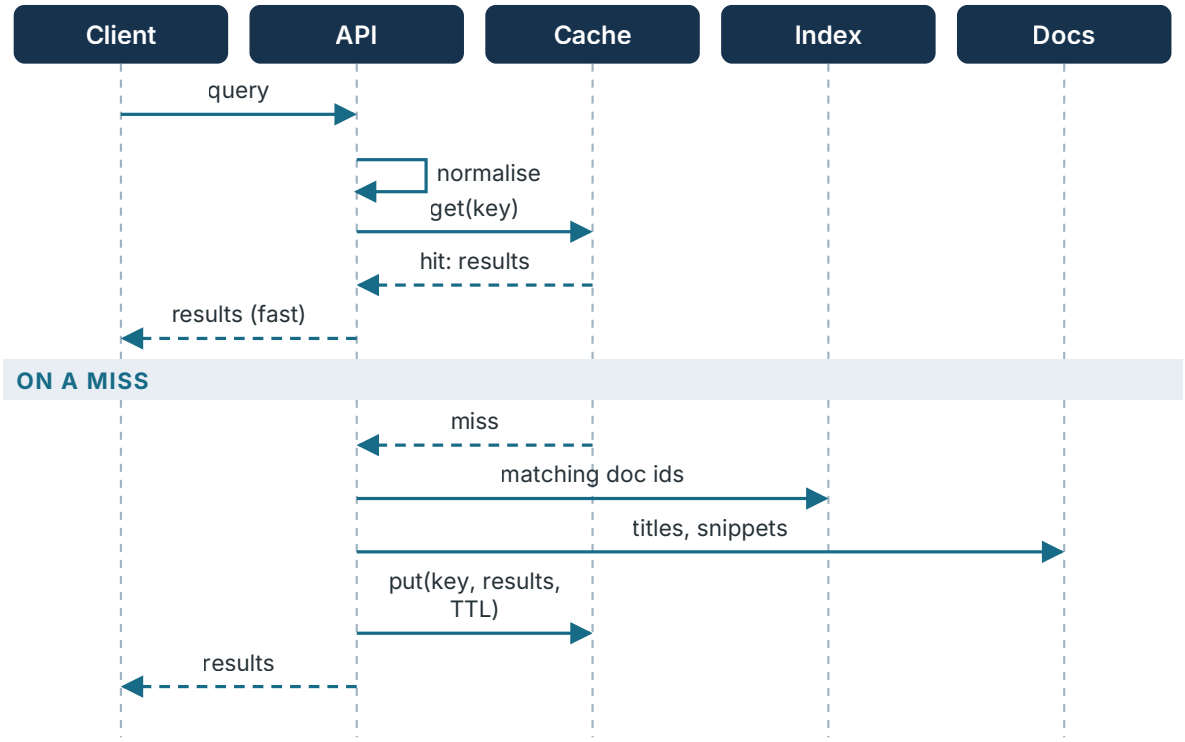


Figure 26.2 Query flow with a cache: normalise, hash to a cache shard, hit returns at once; a miss runs the index and document services, then stores the result.

The notebook's pieces: the **Query API server** parses and normalises the query (trimming, lower-casing, removing stop words, correcting spelling), then checks the **Memory Cache**. On a **hit** it returns the stored results and refreshes recency. On a **miss** it calls the **Reverse Index Service** to find matching document ids and the **Document Service** for titles and snippets, writes the assembled result into the cache, and returns it.

Cache key. The normalised query plus anything that changes the answer (language, region, safe-search flag, result page). Forgetting a dimension returns the wrong results to the wrong user; including too many fragments the cache and kills the hit ratio.

26.5 Distributed cache design

One machine cannot hold 400 GB at 40,000 operations per second, so spread the cache over many nodes.

- **Sharding.** Choose the node by **consistent hashing** of the key (Chapter 11), so adding or removing a node moves only a fraction of keys rather than reshuffling everything (a naive `hash mod N` would invalidate most of the cache on resize and send a flood to the backend).
- **Replication.** Replicate hot shards or accept loss: a cache node failure only costs hit ratio. Replicas add memory but lift availability and spread read load.

- **Client-side or proxy routing.** Clients or a proxy tier know the ring; the Query API servers talk directly to the owning node.
- **Local plus shared caches.** A tiny in-process cache (hundreds of the very hottest keys, TTL of seconds) in front of the shared cache removes network hops and absorbs hot keys.

26.6 Freshness, stampedes and hot keys

Freshness. Results go stale as the index changes. Use a **TTL** (for instance 5 minutes for typical queries, 30 seconds for news-like queries) and optionally **invalidation** when a document that is in a cached result is updated (hard to do precisely: you would need a reverse map from documents to queries; most systems rely on TTL alone). State plainly that the cache serves **possibly stale** results.

Cache stampede (thundering herd). When a hot entry expires, thousands of requests miss at once and all hit the backend. Defences: **request coalescing** (single flight: one request recomputes, others wait for it), **jittered TTLs** so entries do not expire together, **stale-while-revalidate** (serve the old value while one request refreshes), and early probabilistic refresh. See Chapter 12.

Hot keys. One extremely popular query can overload a single cache node. Replicate the hot key across nodes, split it into several copies (`key#1..key#k`) and pick one at random, or serve from the local in-process tier.

Negative caching. Cache "no results" for a short time so repeated bad queries do not hammer the index; keep that TTL short.

Cache poisoning and safety. Never cache an error or a partial result as if it were complete (or cache it with a very short TTL and a flag). Validate input to avoid unbounded key sizes; cap the value size.

26.7 Operations and failure modes

Metrics. Hit ratio (overall and per shard), miss latency, eviction rate, memory usage, key and value size distributions, top keys, stampede events (concurrent recomputations per key), p99 latency. A falling hit ratio after a deploy often means a changed key format or normalisation.

Failures. *Cache node down*: ring routes its keys elsewhere; hit ratio dips; protect the backend with a **rate limit and load shedding** (Chapter 15), since losing the cache can send 40,000 queries per second into a backend sized for 12,000. *Cold start*: warm the cache gradually (replay popular queries) before taking full traffic. *Network partition*: treat timeouts as misses, but with a circuit breaker so slow cache calls do not slow queries.

WORKED EXAMPLE

Hit-ratio and capacity. Suppose the head of 10 million distinct queries accounts for 70% of traffic and each result is 4 KB. Caching that head needs $10^7 \times 4$ KB = 40 GB. A 100 GB cluster (with headroom for overhead, say 1.5×) therefore covers the whole head; going to 400 GB adds only the thin tail, perhaps raising the hit ratio from 70% to 75%: diminishing returns. Plot hit ratio against cache size before buying memory.

26.8 Alternatives and trade-offs

Table 26.2 Decisions and alternatives.

Decision	Chosen	Alternative	Reason	Revisit if
Eviction	LRU (sharded)	LFU / TinyLFU	Simple, adaptive	Scan-heavy traffic
Placement	Consistent hashing	Mod-N hashing	Minimal reshuffle on resize	Fixed size cluster
Freshness	TTL	Event-based invalidation	Simplicity	Strict freshness
Write path	Cache-aside	Write-through	Results are derived, not source	Pre-computed answers
Hot keys	Local tier plus key splitting	Do nothing	Protect a node	Uniform traffic

FAILURE MODES

Failure modes (summary). Stampede on expiry; hot key melting one node; mod-N resize flushing the cache; stale results served too long; wrong key (missing locale) serving the wrong answer; caching errors; backend overload after cache loss; memory fragmentation or giant values.

COMMON MISCONCEPTION

- "*Caching removes the need for a correct backend.*" It must still cope with misses and cold starts.
- "*Higher TTL is always better.*" It raises hit ratio but serves older data.
- "*LRU is always the best policy.*" Scans and frequency effects can beat it.
- "*A cache node failure is harmless.*" The backend must survive the miss storm.

INTERVIEW QUESTIONS

Q1. Implement an LRU cache. Hash map plus doubly linked list; get and put in O(1); evict the tail.

Q2. How do you scale it? Shard with consistent hashing; replicate hot data; local tier; thread safety by per-shard locks.

Q3. How do you stop a stampede? Single flight, jittered TTL, stale-while-revalidate.

Q4. How do you keep results fresh? TTL by query type; invalidation where feasible; accept bounded staleness.

EXERCISES

1. A cache with capacity 3 receives: put A, put B, put C, get A, put D, get B, put E. Which keys remain and in what order (most recent first)?
2. Backend latency 150 ms, cache hit 4 ms, hit ratio 80%. Compute mean latency, and at 40,000 queries per second the backend load.
3. A cluster of 10 nodes uses mod-N hashing and gains an 11th node. Roughly what fraction of keys moves? How about with consistent hashing?
4. Write three unit tests that would catch a broken LRU.

Answers.

1. After A,B,C: order (recent first) C,B,A. get A: A,C,B. put D evicts B (tail): D,A,C. get B: miss (B was evicted). put E evicts C (tail): E,D,A. Remaining: E, D, A.
2. $0.8 \times 4 + 0.2 \times 150 = 3.2 + 30 = 33.2$ ms. Backend: $0.2 \times 40{,}000 = 8{,}000$ per second.
3. Mod-N: a key stays only if hash mod 10 = hash mod 11, about 1/11 of keys stay in the same place, so roughly $10/11 \approx 91\%$ move. Consistent hashing: about $1/11 \approx 9\%$ (those taken over by the new node).
4. (a) Fill to capacity, add one, assert the first key is gone and the rest remain; (b) `get` the oldest key before adding, assert a different key is evicted; (c) update an existing key and assert size does not grow and value changes.

CHAPTER SUMMARY

- Query popularity is skewed, so a modest cache removes most backend load.
- LRU = hash map + doubly linked list, O(1) get and put, with clear invariants and tests.
- Distribute with consistent hashing; add local tiers; protect against stampedes and hot keys.
- Freshness is bounded by TTL; design the backend to survive cache loss.

REVISION CHECKLIST

- [ ] I can write LRU get and put from memory.
- [ ] I can compute hit-ratio effects on latency and load.
- [ ] I can explain stampede and hot-key defences.

CHAPTER 27

Case Study: Sales Rank by Category

Derived from your notes: Notebook chapter 12.7 (sales rank by category: Sales API, Analytics, Transaction logs, MapReduce jobs, results in an SQL analytics database, scale table, read cache). **Added in this book:** streaming aggregation as an alternative to batch, exactly-once effects through idempotent aggregation, time windows and late data, approximate top-k with heavy hitters, ranking ties and consistency, backfills and reconciliation.

LEARNING OBJECTIVES

After this chapter you can:

- scope a "best sellers by category" feature and estimate its volume;
- design a batch pipeline (log, MapReduce, results store) and a streaming alternative;
- handle windows, late events, duplicates and ties correctly;
- serve ranks with a read cache and decide how fresh they must be;
- discuss approximate top-k and reconciliation.

PREREQUISITE KNOWLEDGE

Chapter 4, Chapter 13, Chapter 12, Chapter 8 and Chapter 3 (heaps).

27.1 Problem and requirements

The problem. An e-commerce site wants to show the **top products by sales in each category** over the past week, such as "Best Sellers in Books", and to let the product page show a **rank** like "#12 in Kitchen".

Functional requirements. Compute, for each category, the products ordered by units sold (or revenue) in the last 7 days. Show the top N per category and a product's rank. Update periodically.

Non-functional requirements. The read path is very read-heavy and must be fast. The numbers need not be real time: **hourly** refresh is fine for the notebook design. Counts must be accurate enough that the ranking is trustworthy (no double counting). The system must tolerate worker failures and be reprocessable.

Scope. Rankings are by category and by one time window; sub-categories can be handled by rolling up. Out of scope: personalised rankings, fraud detection in sales and inventory.

27.2 Scale estimate

Table 27.1 Assumptions and derived quantities from the notebook's style of estimate.

Quantity	Assumption	Derived
Transactions	10 billion per month	≈ 4,000 per second on average
Log record	≈ 70 bytes (timestamp, product id, category id, seller, quantity, price)	10^{10} × 70 B = 700 GB per month
Retention	Raw logs one year	≈ 8.4 TB, compressed far less
Products	50 million	
Rank reads	40,000 per second (page views)	cached
Results table	(category, week, product, units) per hour	small compared with the log

What the numbers say. The write rate (4,000 per second) is easy for a log. The expensive part is the **aggregation over 700 GB per month**; weekly windows touch about 160 GB each time. That is a good fit for a distributed batch job, and the output (top lists per category) is tiny, which suits caching. The mismatch between huge input and tiny output is the key insight of this chapter.

27.3 Architecture overview



Figure 27.1 Batch sales-rank pipeline. Sales events are appended to a durable log; MapReduce-style jobs aggregate and rank; results go to an analytics store; a cache serves reads.

The notebook's design: the **Sales API** records each sale in **transaction logs** (an object store or distributed log); **Analytics** jobs (MapReduce) read the logs, count units per product per category, sort, and write results to a **SQL analytics database** (the notebook notes that such a table might otherwise grow large, so prune old windows); the **Sales Rank service** reads results through a cache and serves product pages and category pages.

27.4 Batch computation step by step

The notebook describes the MapReduce job in two steps (and a third for sorting):

1. **Map**: for each log line emit `((category, product_id), quantity)`.
2. **Reduce**: sum values by key, giving units per product per category for the window: `(category, product_id) -> total_units`.

3. **Sort/rank**: a second job (or a secondary sort) orders products within each category by total units and emits `(category, rank, product_id, units)`.

```
# job 1
def map(line):               # line: ts, product, category, qty
    emit((line.category, line.product), line.qty)
def reduce(key, quantities):
    emit(key, sum(quantities))

# job 2 (per category: partition by category, sort by -units)
def map2(((category, product), units)):
    emit(category, (units, product))
def reduce2(category, pairs):
    top = sorted(pairs, reverse=True)[:N]    # or a bounded heap of size N
    for rank, (units, product) in enumerate(top, 1):
        write(category, rank, product, units)
```

A **bounded min-heap** of size N keeps memory small for huge categories (Chapter 3): push each product and pop the smallest when the heap exceeds N; this is O(M log N) for M products. Reducers are partitioned by category, so a single category with enormous traffic can become a **skewed** reducer; mitigate with a two-level aggregation (partial top-N per shard, then merge).

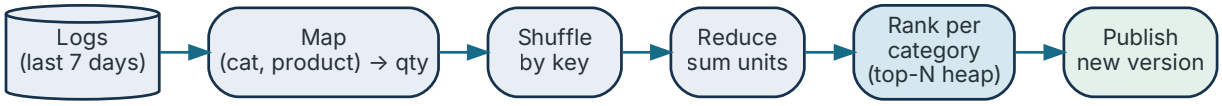


Figure 27.2 Data flow for one hour. Logs of the past 7 days are mapped, shuffled by (category, product), summed, ranked, and published atomically.

Windows. "Last 7 days" can be computed as a **sliding window** recomputed each hour from raw logs (simple, expensive), or **incrementally**: keep hourly partial sums and add the newest hour while subtracting the hour that left the window (168 partials). Incremental aggregation reads far less data and is the practical choice at scale.

Atomic publish. Write results into a new versioned table or partition and swap a pointer when complete, so readers never see a half-written ranking.

27.5 Correctness: duplicates, late data, ties

- **Duplicates.** At-least-once logging can write the same sale twice. Give every sale a unique `sale_id` and de-duplicate in the job (distinct by id within the window), or make log writes idempotent. This is the same lesson as Chapter 13: *exactly-once effects* come from idempotent processing, not magic delivery.
- **Late events.** A sale may reach the log hours late (a mobile client offline). Assign it to its **event time** hour and recompute affected partials; accept that last hour's numbers are provisional and settle after a lateness allowance (say 6 hours).
- **Refunds and cancellations.** Record as negative events or tombstones so units decrease; decide whether rank counts net or gross sales.

- **Ties.** Break ties deterministically (units, then revenue, then product id) so the ranking does not flicker between runs.
- **Category changes.** If a product moves category, decide whether history moves with it; record the category *as of the sale* for historical accuracy.
- **Reconciliation.** Periodically compare the pipeline's totals with the orders database (a trusted source) and alert on drift beyond a tolerance, such as 0.1%.

> **WORKED EXAMPLE**
>
> **Incremental window.** Hourly partial for Kitchen, product P: hours 1..168 hold units $u_1..u_{168}$; weekly total $T = \Sigma u_i$. When hour 169 arrives with u_{169}, new total = $T + u_{169} - u_1$. Cost: two updates per product per hour instead of re-reading 168 hours of logs, a reduction of about 84 times in data read. If hour 169 later receives a late sale of 3 units, add 3 to u_{169} and to T.

27.6 Streaming alternative

If ranks must be fresh within seconds or minutes, replace the batch job with a **stream processor** consuming the sales log (Chapter 13): it keeps per-key counters in windows, emits updated counts, and a sink maintains top-N per category (a sorted set in an in-memory store, scored by units). Event-time windows, watermarks for lateness and checkpointed state give accuracy comparable to batch. A common hybrid is the **lambda** approach: a streaming layer for fresh approximate values plus a nightly batch recomputation that is authoritative; the **kappa** variation reprocesses the same log through the same streaming code to backfill. State the cost: streaming is operationally harder than a daily batch, so choose it only when freshness is a requirement.

Approximate top-k. At extreme volume, use a **heavy-hitters** structure (count-min sketch plus a small heap, Chapter 3) to track the top products in bounded memory with a small, controlled error: acceptable for "popular" lists but not for revenue accounting.

27.7 Serving ranks

Reads are 40,000 per second but the data changes hourly, so a **cache** (Chapter 12) in front of the analytics store is very effective: key `rank:{category}:{window}` holds the top-N list (a few kilobytes), with a TTL near the refresh period or invalidated on publish. A product's own rank is stored as a field of its product data, or looked up from the category list (the top 100 only; beyond that show "Top 10,000" style buckets). Because publishes are atomic and versioned, readers see either the old or the new list, never a mix. Use a CDN for public category pages.

API. `GET /v1/categories/{id}/bestsellers?window=7d&limit=50` returns an ordered list with `rank`, `product_id`, `units` (or a rounded band), and `as_of` timestamp so users and clients know how fresh it is.

27.8 Failure handling and operations

Job fails midway: it is restartable; output is written to a new version and published only on success, so the previous ranking stays live. *Skewed category*: two-level aggregation. *Late or missing logs*: monitor the log's completeness per hour (event counts compared with the Sales API's counter) before publishing. *Analytics store down*: serve the cached or last published list with a stale marker. *Data privacy*: aggregated ranks reveal no individual purchase; keep buyer ids out of the analytics logs.

Metrics. Job duration and success, freshness (`now − as_of`), reconciliation drift, duplicate rate, late-event rate, reducer skew, cache hit ratio. **Cost.** Compute for batch jobs scales with the window size if recomputed from raw logs, which is why the incremental design matters; storage is dominated by raw logs, moved to cheaper tiers after 30 days.

27.9 Alternatives and trade-offs

Table 27.2 Decisions and alternatives.

Decision	Chosen	Alternative	Reason	Revisit if
Compute	Hourly incremental batch	Streaming	Simplicity, accuracy	Need minute-level freshness
Window	Hourly partials, add/subtract	Recompute weekly from raw	80+ times less data	Tiny data
Duplicates	De-dup by sale id	Trust the log	Exactness	Idempotent log guaranteed
Results store	SQL analytics table (versioned)	NoSQL or sorted set	Ad-hoc queries	Very high write churn
Serving	Cache plus CDN	Query DB on each view	40,000 reads per second	Low traffic
Top-k	Exact	Count-min plus heap	Accuracy	Extreme cardinality

FAILURE MODES

Failure modes (summary). Double counting; late sales missed; reducer skew on a huge category; half-published results; rank flicker from ties; stale data presented as fresh; drift versus the orders database; unbounded growth of the results table.

COMMON MISCONCEPTION

- "*Real-time is always better.*" Hourly ranking is enough for most shoppers and far cheaper.
- "*Exactly-once delivery solves duplicates.*" Make processing idempotent.
- "*Sort all products to get the top N.*" Use a bounded heap or partial sort.
- "*A cache makes ranks wrong.*" Staleness is bounded and labelled.

INTERVIEW QUESTIONS

Q1. How do you compute the weekly top sellers? Log sales, aggregate per (category, product) per hour, maintain a rolling 168-hour sum, rank per category, publish atomically.

Q2. How would you make it near real time? Streaming windows with watermarks plus a sorted-set sink, with batch reconciliation.

Q3. How do you avoid counting a sale twice? Unique sale ids and idempotent aggregation.

Q4. How do you handle one gigantic category? Two-level aggregation and a bounded heap.

EXERCISES

1. With 6 billion sales per month at 80 bytes each, compute log volume per month and for a 7-day window, and the average sales per second.
2. A product sold [5, 3, 0, 4, 2, 6, 1] units on seven days; on day 8 it sells 7. Show the update to the weekly total with the add/subtract method.
3. Two products tie at 120 units. Define a deterministic tiebreak and explain why it matters.
4. Estimate memory for a bounded heap of N = 100 over 5 million products versus sorting all.

Answers.

1. $6\times10^9 \times 80$ B = 480 GB per month; 7 days ≈ 480 × 7/30 = 112 GB; $6\times10^9 / 2.592\times10^6$ s ≈ 2,300 per second (30-day month).
2. Initial total = 21. Day 8: 21 + 7 − 5 (day 1 leaves) = 23.
3. Order by units desc, then revenue desc, then product id asc; without it, results flip between runs and users see rank changes that mean nothing.
4. Heap of 100 entries × 16 B = 1.6 KB; sorting all 5 million × 16 B = 80 MB (and O(M log M) time) per category-job instance.

CHAPTER SUMMARY

- Huge input, tiny output: aggregate in batch, serve from a cache.
- Incremental windows (add the new hour, subtract the old) save most of the work.
- Correctness needs de-duplication by id, event-time handling, deterministic ties and reconciliation.
- Publish atomically; label freshness; consider streaming only when freshness demands it.

REVISION CHECKLIST

- [ ] I can sketch the log → MapReduce → store → cache pipeline.
- [ ] I can explain the incremental weekly window.
- [ ] I can list duplicate, late-data and tie handling.

CHAPTER 28

Case Study: Scaling from One User to Millions

Derived from your notes: Notebook chapter 12.8 (scaling on AWS from one user to millions: the staged evolution from a single box, to separate web and database tiers, load balancing, read replicas, CDN, caches, auto scaling, asynchronism, SQL scaling patterns, NoSQL, and monitoring). **Added in this book:** the reasoning that triggers each stage, cost and reliability trade-offs, multi-zone design, stateless tier rules, a capacity table, a migration order, and vendor-neutral equivalents.

LEARNING OBJECTIVES

After this chapter you can:

- describe the staged evolution of a web system from one server to a multi-zone architecture;
- name the *trigger* (the metric or failure) that justifies each next step;
- map each stage to managed cloud building blocks and vendor-neutral equivalents;
- estimate capacity and cost at each stage and order the work sensibly;
- avoid premature complexity.

PREREQUISITE KNOWLEDGE

Chapter 1, Chapter 7 (load balancers, CDN), Chapter 9, Chapter 12, Chapter 13 and Chapter 19.

28.1 Problem and approach

The problem. You start with a simple application and one user. Over time the user base grows to millions. How should the architecture evolve, and when should each change be made?

This chapter is a *narrative* case study rather than a single design: the lesson is the **order of decisions**. Each stage is introduced only when a measurable problem appears (high CPU, a saturated database, a single point of failure, slow pages for far-away users). Doing everything at once wastes money and adds failure modes; doing nothing until an outage wastes users' trust. Names of services follow the notebook's use of Amazon Web Services; every component has an equivalent elsewhere (Table 28.2).

28.2 Stage 1: one box (users: 1 to 1,000)

Everything runs on one virtual machine: web server, application code and database, with a static public address from DNS. **Benefits**: cheap, simple to build and debug. **Risks**: one failure takes down everything; web and database compete for CPU, memory and disk; no isolation. Do keep **backups** and **infrastructure as code** even here (Chapter 19), because rebuilding a hand-configured box is slow and error-prone.

Trigger to move on: resource contention, a need for safe deploys, or the first outage that mattered.

28.3 Stage 2: separate tiers and vertical scaling (to about 10,000)

Move the database to a **managed relational database** (backups, patching, failover handled by the provider) and keep the web tier separate. Scale **vertically** first: a larger instance is the quickest fix and needs no code change (Chapter 1). Put **static assets** (images, scripts, downloads) in **object storage** served by a **CDN**, taking the largest bytes off the web servers and reducing latency worldwide (Chapter 7).

Choice of database. Start with SQL unless the access pattern clearly demands otherwise (Chapter 8); use NoSQL for specific high-volume, simple-access data such as sessions or event logs (Chapter 10).

Trigger: the largest instance is still saturated, or a single instance failure is unacceptable (vertical scaling has a ceiling and a single point of failure).

28.4 Stage 3: load balancing and a stateless web tier (to about 100,000)

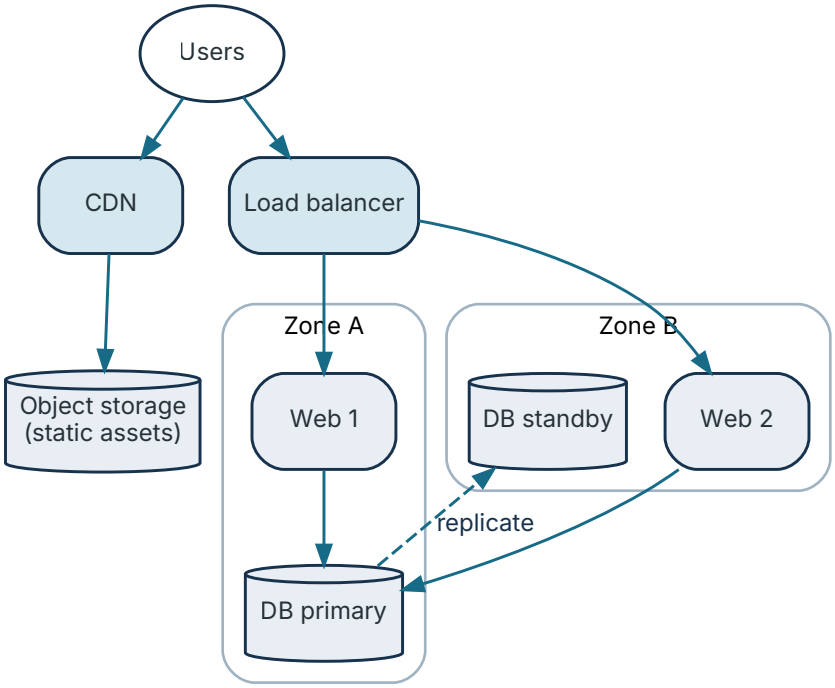


Figure 28.1 Stage 3: multiple stateless web servers behind a load balancer across two zones, a primary database with a standby replica, object storage and a CDN for static content.

Add a **load balancer** and **several web servers** across at least **two availability zones**, so that losing a zone does not take the site down (see Figure 28.1). This requires the web tier to be **stateless**:

- user sessions move out of server memory into a shared store (a cache or database) or become signed tokens;
- uploaded files go to object storage, not local disks;
- configuration comes from the environment, not hand-edited files;
- health checks tell the load balancer which servers to use, and **graceful shutdown** drains connections.

Run the database as a **primary with a synchronous or asynchronous standby** in another zone for failover (Chapter 9; remember replication lag and the split-brain risk). Introduce **auto scaling** on a metric such as CPU or request rate, with a minimum of two instances.

Trigger: database CPU or connections saturate; read queries dominate; p99 latency grows.

28.5 Stage 4: caching, read replicas and asynchronism (to about 1 million)

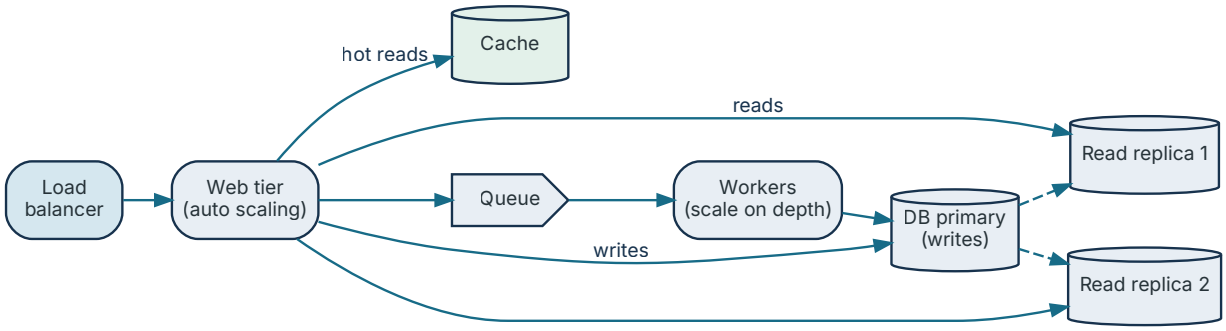


Figure 28.2 Stage 4: add an in-memory cache, read replicas, and a queue with background workers so slow work leaves the request path.

Because most traffic is reads, three moves relieve the database in order of effort:

1. **Cache** hot reads in an in-memory store using cache-aside (Chapter 12): usually the biggest single improvement.
2. **Read replicas**: send read-only queries that tolerate small staleness to replicas, writes to the primary. Remember read-your-writes (Chapter 11).
3. **Asynchronism**: move slow or non-urgent work (emails, image resizing, reports) onto a queue processed by workers (Chapter 13); pre-compute expensive pages in the background; respond to users with a "we're working on it" status where appropriate. Workers auto scale on queue depth.

Add **monitoring and alerts** now (Chapter 18): without metrics you will not know which stage to fix next.

Trigger: write volume or data size outgrows one primary; a single table is huge; replicas lag.

28.6 Stage 5: scaling the data tier (millions of users)

When one primary cannot take the writes or hold the data, apply the SQL scaling patterns (Chapter 9) in this order of preference, each adding operational cost:

1. **Tune and index** queries; archive old data; denormalise carefully for hot reads.
2. **Federation**: split databases by function (users, orders, products).
3. **Sharding**: partition large tables by a key such as user id (and pay the cross-shard query and rebalancing costs).
4. **NoSQL** for data whose access pattern is simple and enormous (sessions, feeds, events), with query-first modelling (Chapter 10).

At the same time: multiple **regions** if latency to far users or regional disaster tolerance requires it (with the consistency complications of multi-region writes, Chapter 11); **service decomposition** where team or scaling boundaries justify it (Chapter 16); and **automation** (deployment pipelines, autoscaling, runbooks).

28.7 Capacity and cost along the way

Table 28.1 An illustrative capacity ladder (all numbers are assumptions to practise estimation, not benchmarks).

Users (daily active)	Requests/s (avg, peak)	Typical architecture	Main risk
1,000	0.1, 1	One box	Single failure
10,000	1, 10	Web + managed DB, CDN	DB and web contend
100,000	10, 100	LB, 2+ web servers, DB standby	DB saturation
1,000,000	100, 1,000	+ cache, read replicas, queue	Write volume
10,000,000	1,000, 10,000	+ sharding or NoSQL, multi-zone, autoscale	Operational complexity

Worked arithmetic. 1 million daily users, 30 requests each per day: 3×10^7 requests/day ÷ 86,400 ≈ 350 per second average; with a peak factor of 5, about 1,750 per second. If one web instance serves 200 per second at 50% target utilisation (100 per second), you need 1,750 / 100 ≈ 18 instances at peak, plus one zone's worth of spare (a third more) for zone failure: about 24. If 80% of reads hit the cache, the database sees roughly 20% of read traffic.

> **WORKED EXAMPLE**
>
> **Which step first?** Symptoms: p99 latency doubled; the database CPU is 90%; web servers at 30% CPU; 85% of the DB load is the same 20 queries. Candidate actions: (a) more web servers: no effect, the web tier is not the bottleneck; (b) bigger database: works short term, expensive; (c) cache the hot queries: attacks 85% of the load directly and cheaply; (d) shard: heavy and premature. Choose (c) first, then (b) if necessary, and schedule (d) only when writes demand it. The method: *measure, find the bottleneck, apply the cheapest effective fix.*

28.8 Vendor-neutral map

Table 28.2 Building blocks and equivalents.

Role	AWS example	Generic equivalent
Compute	EC2, containers, functions	Virtual machines, container orchestrator, functions
Load balancer	Elastic Load Balancing	NGINX, HAProxy, cloud LB
Relational DB	RDS, Aurora	Managed PostgreSQL or MySQL
NoSQL	DynamoDB	Cassandra, MongoDB, other key-value stores

Role	AWS example	Generic equivalent
Cache	ElastiCache	Redis, Memcached
Object storage and CDN	S3, CloudFront	Any object store and CDN
Queue	SQS, SNS	RabbitMQ, Kafka, managed brokers
DNS	Route 53	Any authoritative DNS with health checks
Monitoring	CloudWatch	Prometheus, Grafana, OpenTelemetry

28.9 Failure thinking at every stage

Ask at each stage "what is the single point of failure now?" Stage 1: everything. Stage 2: the web server and the one database instance. Stage 3: the load balancer is managed and redundant; the database primary remains (mitigated by standby). Stage 4: the cache (a cold cache must not crush the database: Chapter 12). Stage 5: shard placement metadata, cross-region links. Practise failure injection ("game days", Chapter 15). Backups are not replicas: test restores (Chapter 9).

FAILURE MODES

Failure modes (summary). Stateful web servers blocking horizontal scaling; autoscaling without limits causing a cost or database connection storm; replica lag causing stale reads; cache loss sending a miss storm to the database; premature sharding; missing alerts; deployment without rollback; single-zone design.

COMMON MISCONCEPTION

- "*Scale-out from day one.*" Start simple; add each stage when a metric demands it.
- "*Autoscaling fixes everything.*" It scales stateless tiers; databases need other strategies, and scaling the web tier can overload the database.
- "*Multi-AZ means backups.*" It protects against zone loss, not against a bad delete.

INTERVIEW QUESTIONS

Q1. Your site is slow with growing traffic; what do you do? Measure first; find the bottleneck; apply cache, replicas, queue or scaling accordingly, cheapest first.

Q2. Why must web servers be stateless? So any server can handle any request, allowing load balancing, autoscaling and replacement without losing sessions.

Q3. When would you shard? When a single primary cannot handle write volume or data size after tuning, caching, replicas and federation.

Q4. What is the first step to improve availability? Remove single points of failure: multiple servers across zones behind a load balancer, a database standby, and tested backups.

EXERCISES

1. 5 million daily users, 20 requests each, peak factor 4. Compute average and peak requests per second, and the number of instances if each handles 150 requests per second at 60% target utilisation, with one-third spare for a zone loss.
2. Your database has 70% reads and a 90% cache hit ratio is achieved on reads. What fraction of the original database operations remain?
3. List what must change in an application before it can run on multiple web servers.
4. Which stage addresses "a deploy takes down the site for 10 minutes"? Describe how.

Answers.

1. 10^8 requests/day / 86,400 ≈ 1,160 per second average; peak ≈ 4,630. Each instance: 150 × 0.6 = 90. 4,630/90 ≈ 52; add one third ≈ 69 instances.
2. Reads become 0.7 × 0.1 = 0.07; writes remain 0.30; total 0.37, so 37% of the original operations remain.
3. Externalise sessions, store uploads in shared object storage, use environment configuration, ensure no local state or scheduled jobs run on every node, add health checks and graceful shutdown.
4. Stage 3: multiple servers behind a load balancer allow rolling or blue-green deployments with health checks, so servers are updated a few at a time.

CHAPTER SUMMARY

- Evolve in stages, each triggered by a measured problem.
- A stateless web tier plus load balancer across zones is the foundation of horizontal scaling.
- Cache, read replicas and queues relieve the database before sharding.
- Always identify the next single point of failure and the next bottleneck.
- Managed services map to open equivalents; the principles are portable.

REVISION CHECKLIST

- [ ] I can list the stages and their triggers.
- [ ] I can do the capacity arithmetic.
- [ ] I can explain why statelessness matters.

CHAPTER 29

End-to-End Exercises: Payments and Notifications

Added in this book: Two end-to-end design exercises that are not in the notebook, built to combine the book's patterns: a payment service (idempotency, state machine, ledger, reconciliation) and a notification system (fan-out, preferences, rate limiting, retries, provider failover).

SUPPLEMENTAL MATERIAL

This whole chapter is supplemental. The notebook has no payments or notification case study. They are included because together they exercise nearly every pattern in the book: idempotency, state machines, queues, outbox, retries, rate limiting and reconciliation. Work each one yourself using the method of Chapter 20 before reading the discussion.

LEARNING OBJECTIVES

After this chapter you can:

- design a payment flow that never charges twice and survives timeouts and crashes;
- model money movement with a double-entry ledger and reconcile with an external provider;
- design a multi-channel notification system with preferences, rate limits and provider failover;
- apply outbox, retries, dead-letter queues and idempotency in realistic combination.

PREREQUISITE KNOWLEDGE

Chapter 6 (idempotency keys), Chapter 8 (transactions), Chapter 13 (outbox, sagas, DLQ), Chapter 15 and Chapter 17.

29.1 Exercise 1: A payment service

Requirements. A merchant's checkout calls our service to **charge a card** through an external payment provider. Customers must never be charged twice and never charged without the merchant learning of it. **Scope:** card payments through one provider, refunds, webhooks; out of scope: storing raw card numbers (use the provider's tokens to stay out of the heaviest compliance scope), fraud scoring, currency conversion.

Estimate (assumptions). 20 million payments per month ≈ 8 per second average, peak 10 times that (80 per second). Each payment creates about 5 ledger rows and 6 events: about 40 rows and 50 events per second at peak. Tiny by throughput; the difficulty is **correctness**.

The core problem: timeouts leave the outcome unknown

If we call the provider and the connection times out, the charge may have succeeded or not (Chapter 15). Blind retry risks a double charge; giving up risks a lost sale. The solution is **idempotency end to end** plus a **state machine** that records intent *before* acting.

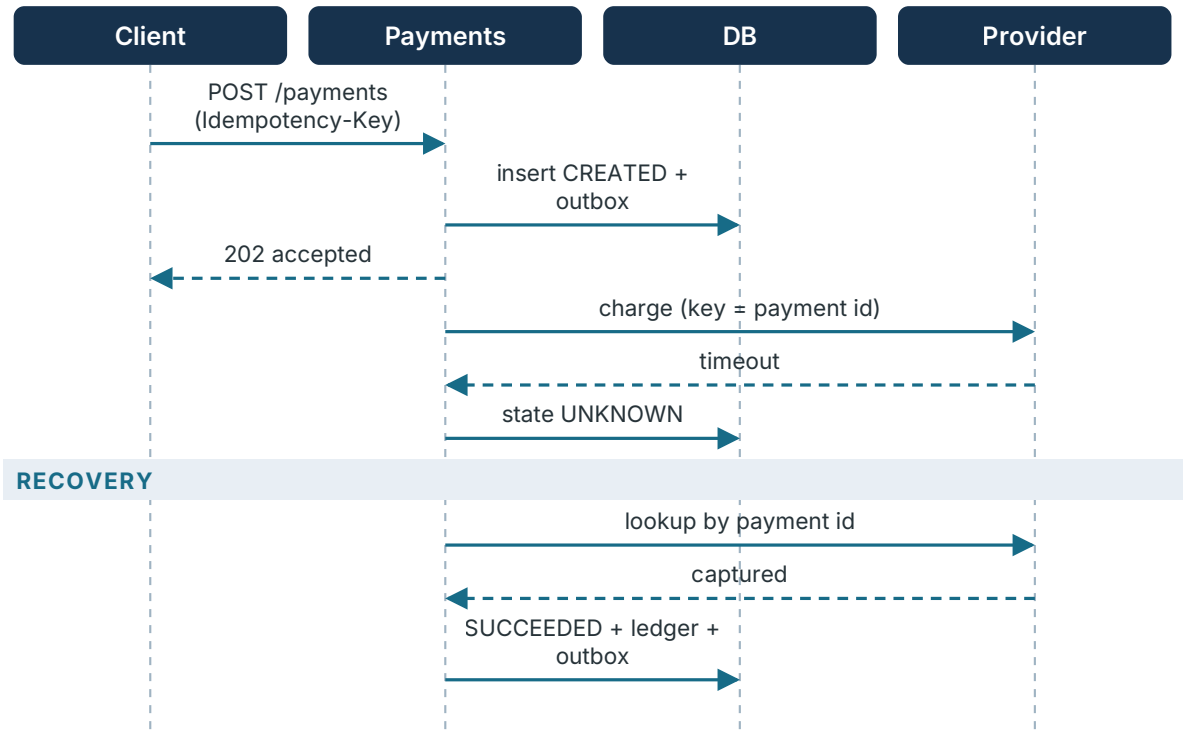


Figure 29.1 One payment. The intent is stored first; the provider call carries the same idempotency key; an unknown outcome is resolved by querying or by a webhook.

Table 29.1 Payment state machine.

State	Meaning	Allowed next states
CREATED	Intent stored, nothing sent	PROCESSING, CANCELLED
PROCESSING	Request sent to provider, outcome not yet known	SUCCEEDED, FAILED, UNKNOWN
UNKNOWN	Timeout or ambiguous response	SUCCEEDED, FAILED (after lookup or webhook)
SUCCEEDED	Provider confirmed capture	REFUND_PENDING
FAILED	Provider declined or error confirmed	none (a new attempt is a new payment or a retry with the same key)
REFUND_PENDING	Refund requested	REFUNDED, SUCCEEDED (refund failed)

Step by step

1. The client sends `POST /v1/payments` with an `Idempotency-Key`, amount in minor units, currency and a payment-method token. The service looks up the key: if found with the same body, return the stored result; with a different body, return `409`.
2. In **one database transaction**, insert the payment as CREATED with the key (unique constraint) and an **outbox** row "charge requested".
3. A worker reads the outbox, moves the payment to PROCESSING and calls the provider **passing our payment id as the provider's idempotency key**, so a retry cannot create a second charge.
4. On a clear success or decline, write SUCCEEDED or FAILED and a ledger entry in one transaction; emit `payment.succeeded` through the outbox for the merchant's webhook.
5. On timeout, mark UNKNOWN and schedule a **lookup** by payment id with backoff; also accept the provider's **webhook**, verifying its signature. Webhooks may arrive twice or out of order: apply them only if they move the state forward, idempotently.
6. A nightly **reconciliation** job compares our ledger with the provider's settlement file; mismatches go to a human queue.

Data model

`payments(id PK, merchant_id, idempotency_key UNIQUE per merchant, amount_minor, currency, state, provider_ref, created_at, updated_at)`; `ledger_entries(id, payment_id, account, debit_minor, credit_minor, created_at)` (append-only, double entry: every movement debits one account and credits another, so the entries always sum to zero); `outbox(id, topic, payload, published_at)`; `webhook_deliveries(id, merchant_id, event_id, attempts, next_attempt_at, status)`.

Why a ledger? Balances are *derived* from immutable entries, so history is auditable and errors are corrected by new entries (a reversal), never by editing rows.

Consistency, failure and security

- **Within our database** use ACID transactions (Chapter 8). **Across the provider and us** there is no transaction: this is a **saga** (Chapter 13): the steps are charge, record, notify, with compensation (refund or void) when a later step cannot complete.
- **Merchant webhooks** use at-least-once delivery with retries (exponential backoff, dead-letter after N tries), signed payloads and event ids so merchants de-duplicate.
- **Security:** never log card data; only tokens cross our boundary; TLS everywhere; secrets in a vault; per-merchant API keys with scopes; audit logs; rate limits against card-testing attacks.
- **Failure matrix:** crash after step 2 (the outbox row exists, so a worker resumes); crash after the provider call but before recording (the lookup by payment id resolves it); duplicate client request (the idempotency key returns the stored result); provider down (circuit breaker, queue payments as PENDING, tell the merchant honestly); clock skew (rely on state, not timestamps).

WORKED EXAMPLE

Timeout walk-through. t0: payment P1 CREATED. t1: worker calls provider with key `P1`; the connection drops at 10 s. t2: state UNKNOWN. t3: lookup for `P1` at the provider returns "captured": state SUCCEEDED, ledger entries written (debit customer funds in transit, credit merchant payable), `payment.succeeded` emitted. If the lookup had returned "not found", it is safe to retry the charge **with the same key** because a charge for `P1` does not exist. At no point could two charges exist.

29.2 Exercise 2: A notification system

Requirements. Other services request notifications (order shipped, password reset, marketing) to users over **push, SMS and email**. Respect **user preferences** (channels, quiet hours, opt-outs), **rate limits**, and deliver **important messages reliably** and **promotional ones best-effort**. **Scope:** templating and delivery; out of scope: building the email or SMS carriers, analytics UI.

Estimate (assumptions). 50 million users, 5 notifications per user per day: 2.5×10^8 per day ≈ 2,900 per second average. A marketing blast to 10 million users in 10 minutes needs 10^7 / 600 s ≈ 16,700 per second at peak. Providers rate-limit us, so the **queue** absorbs the burst.

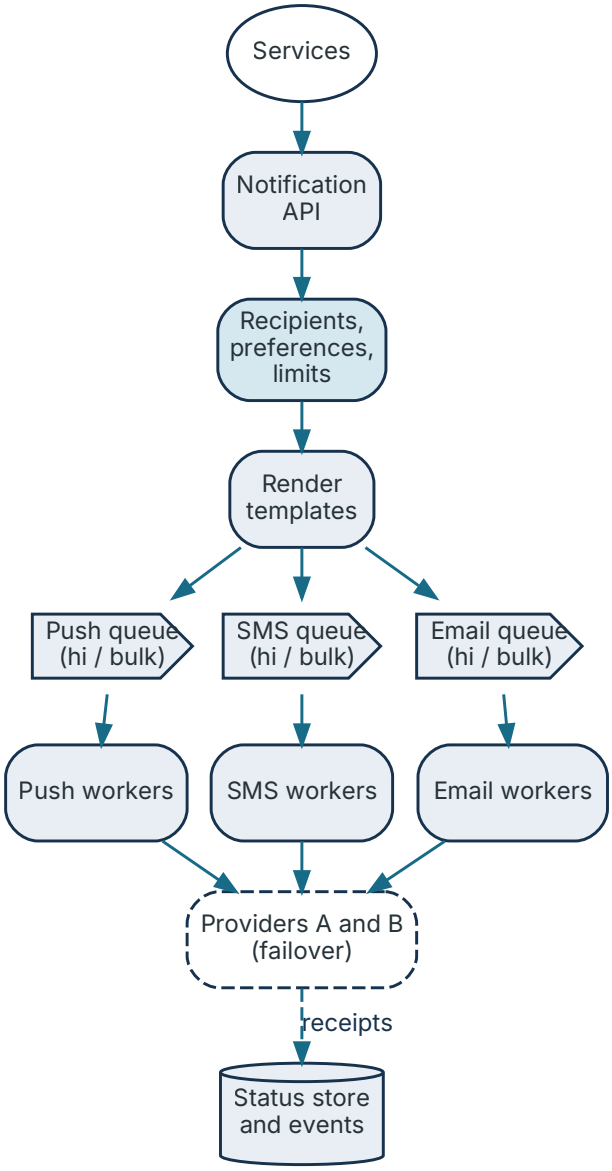


Figure 29.2 Notification pipeline. Requests are validated, checked against preferences and limits, rendered, and placed on per-channel priority queues; channel workers call providers with failover; delivery receipts flow back.

Design

- **API:** `POST /v1/notifications` with `type`, `user_id` (or a segment), `template_id`, parameters, `priority` and an **idempotency key**. Return `202` with an id.
- **Pipeline:** validate; **resolve recipients** (a segment expands into a stream of user ids by a batch job, never inside the request); **filter** by preferences, opt-outs, quiet hours (in the user's time zone) and frequency caps; **render** a localised template; **enqueue** on the right channel queue.
- **Priority lanes:** separate queues (or priorities) for transactional (password reset: seconds) and bulk marketing, so a blast never delays a reset code. Reserve worker capacity for the high lane.

- **Channel workers** call providers with timeouts and retries; use **two providers per channel** with a circuit breaker and automatic failover. Devices tokens that a push provider reports invalid are removed.
- **Delivery tracking:** a `notifications` table with state (QUEUED, SENT, DELIVERED, FAILED, SUPPRESSED) updated by provider callbacks; keep events in a log for analytics.
- **De-duplication:** the idempotency key plus a unique `(user, template, dedupe_key)` constraint prevents sending the same alert twice when upstream retries; accept that **at-least-once** delivery can still repeat a message after a worker crash, and make messages safe to see twice.
- **Rate limits:** per user (no more than N marketing messages a day), per provider (token bucket matching the contract) and per tenant, as in Chapter 7.
- **Scheduling:** delayed and recurring notifications use a scheduler table keyed by send time, scanned by workers or a delay queue.

Table 29.2 Design choices.

Decision	Chosen	Alternative	Why
Fan-out	Batch expansion of segments to a queue	Expand inside the API call	Large audiences would time out
Queues	Per channel and priority	One shared queue	Isolation of urgent traffic
Delivery semantics	At-least-once plus de-dup keys	Exactly-once claims	Not achievable end to end
Providers	Two per channel with failover	Single provider	Provider outages are common
Preferences	Checked at send time	Checked at request time	Users change settings after requests queue

Failure modes. Provider outage (failover, backlog drains later; expire stale messages such as a "your code is 123456" after minutes); retry storm to a struggling provider (back-off with jitter, circuit breaker); duplicate sends after redelivery; time-zone bugs sending marketing at 3 a.m.; spam complaints (honour opt-outs instantly; keep a suppression list); privacy (do not put sensitive details in push or SMS text).

WORKED EXAMPLE

Blast sizing. 10 million emails; provider allows 3,000 per second per account and we have two accounts: 6,000 per second, so the blast takes 10^7 / 6,000 ≈ 1,667 s ≈ 28 minutes, not 10. To meet a 10-minute target either add provider capacity (about 17,000 per second) or tell the sender the real duration. Meanwhile the transactional lane stays unaffected because it has reserved workers and its own provider quota.

FAILURE MODES

Failure modes (summary). Payments: double charge on retry, lost outcome after timeout, webhook replay, ledger imbalance, provider mismatch. Notifications: blast starving urgent messages, duplicate sends, stale OTPs, provider outage, preference races.

COMMON MISCONCEPTION

- "*A timeout means the payment failed.*" The outcome is unknown until resolved.
- "*Exactly-once delivery makes retries safe.*" Idempotency keys make effects safe.
- "*One queue is simpler and fine.*" A single queue lets bulk traffic delay critical messages.
- "*Webhooks arrive once and in order.*" Treat them as duplicated and reordered.

INTERVIEW QUESTIONS

Q1. How do you prevent a double charge? Idempotency key at our API, the same key to the provider, a unique constraint, and a state machine that resolves unknown outcomes by lookup.

Q2. Why a double-entry ledger? Immutable, balanced entries give an audit trail; corrections are reversals.

Q3. How do you keep urgent notifications fast during a blast? Separate priority lanes with reserved capacity and provider quota.

Q4. How do you handle a provider outage? Circuit breaker, failover provider, backoff, expiry of stale messages, dead-letter queue.

EXERCISES

1. Payments: 25 million per month, peak factor 12. Compute average and peak payments per second and ledger rows per second at 5 rows each.
2. Draw the sequence when the provider's webhook arrives *before* our worker records the response.
3. Notifications: 40 million users, 6 per day, 15% of messages sent in a one-hour evening peak. Compute the peak per second.
4. Propose a unique constraint that stops duplicate "order shipped" emails for the same order.

Answers.

1. $2.5\times10^7 / 2.592\times10^6 \approx 9.6$ per second average; peak ≈ 116 per second; ledger rows ≈ 580 per second at peak.
2. Webhook handler loads the payment (PROCESSING), applies SUCCEEDED idempotently and writes the ledger; the worker then finds the state already SUCCEEDED and skips (state only moves forward). Both orders end in the same state.
3. $40\times10^6 \times 6 = 2.4\times10^8$ per day; 15% $= 3.6\times10^7$ in 3,600 s $= 10{,}000$ per second.
4. `UNIQUE(user_id, template_id, dedupe_key)` where the dedupe key is the order id; an insert conflict means "already sent or queued".

CHAPTER SUMMARY

- Payments are an idempotency and state-machine problem; record intent first, pass keys end to end, resolve unknowns, reconcile.
- A double-entry ledger gives an auditable, balanced record.
- Notification systems separate priority lanes, filter by preferences, rate-limit and fail over across providers.
- Both rely on at-least-once delivery made safe by idempotency, not on exactly-once promises.

REVISION CHECKLIST

- [ ] I can draw the payment state machine and timeout recovery.
- [ ] I can explain the outbox and ledger roles.
- [ ] I can size a notification blast and design priority lanes.

PART VIII

Object-Oriented Design

Turning requirements into classes, state machines and tested code.

CHAPTER 30

Object-Oriented Design Exercises

Derived from your notes: Notebook chapter 13 (object-oriented design: method, then hash map, LRU cache, call center, deck of cards and blackjack, parking lot, online chat, and the open circular-array exercise). **Added in this book:** explicit responsibilities, invariants, state transitions, runnable reference code and unit tests for every design; resize in the hash map; corrections to the skeleton code in the original exercises.

LEARNING OBJECTIVES

After this chapter you can:

- follow a repeatable method from vague requirement to classes, relationships and behaviour;
- state each class's responsibility and the invariants it protects;
- model state with enums and transitions instead of flags;
- implement six classic designs and the circular buffer, and test them;
- spot common design flaws such as missing back-pointers, wrong super calls and hidden state.

PREREQUISITE KNOWLEDGE

Chapter 3 (hash tables, linked lists, queues) and Chapter 2 for the thread-safety remarks.

30.1 Method

1. **Clarify** scope, valid inputs and constraints (fits in memory? thread safety? scale?). Write assumptions down.
2. **Find nouns and verbs.** Nouns become candidate classes, verbs become methods. Decide *is-a* (inheritance) versus *has-a* (composition); prefer composition unless subtypes truly behave differently.
3. **Assign responsibilities.** Each class has one reason to change and protects its own **invariants** (facts that must always be true).
4. **Model state explicitly** with enums and a transition table, rather than booleans scattered around.
5. **Choose data structures** that make the key operation cheap.
6. **Walk through scenarios**, including edge cases, then **test**: one test per invariant and per transition, plus a randomised comparison against a trusted reference where one exists.

All code in this chapter is Python 3, was run, and passes its 13 tests (the test file is shipped with the book sources). Code is a reference for the interview whiteboard: it favours clarity over production hardening, and thread safety is discussed rather than implemented unless stated.

30.2 Design 1: Hash map

Assumptions. Integer keys, collisions resolved by **chaining**, resize when the load factor passes 0.75, valid input, all in memory.

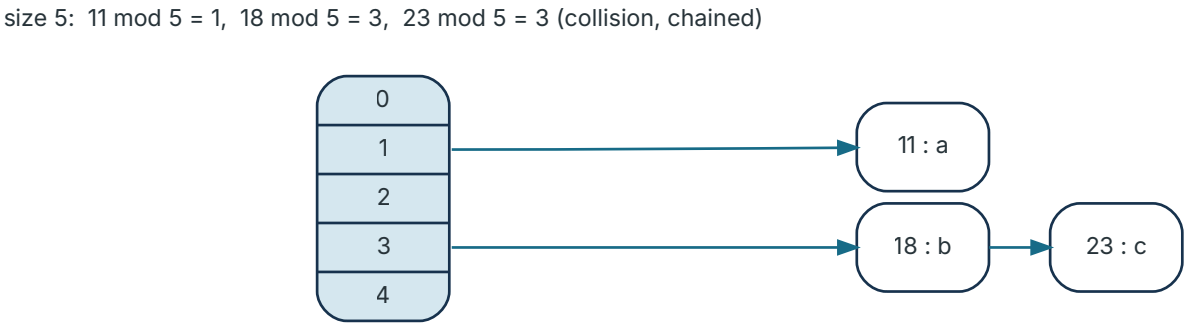


Figure 30.1 Hash map with chaining: key mod size selects a bucket; colliding keys share a chain.

Responsibilities. `HashMap` owns the bucket array and the item count. `set` updates or inserts; `get` and `remove` scan one bucket and raise `KeyError` if absent; `_resize` rehashes everything.

Invariants. (1) every item is in bucket `key % size`; (2) no key appears twice; (3) `count` equals the total number of items; (4) load factor stays at or below the limit after `set`.

```
class HashMap:
    """Chained hash map for integer keys. Invariant: every item sits in bucket key % size."""
    def __init__(self, size=8, max_load=0.75):
        self._buckets = [[] for _ in range(size)]
        self._count = 0
        self._max_load = max_load

    def _bucket(self, key):
        return self._buckets[key % len(self._buckets)]

    def set(self, key, value):
        bucket = self._bucket(key)
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)       # update in place
                return
        bucket.append((key, value))
        self._count += 1
        if self._count / len(self._buckets) > self._max_load:
            self._resize(2 * len(self._buckets))

    def get(self, key):
        for k, v in self._bucket(key):
            if k == key:
                return v
        raise KeyError(key)

    def remove(self, key):
        bucket = self._bucket(key)
        for i, (k, _) in enumerate(bucket):
            if k == key:
                del bucket[i]
                self._count -= 1
                return
        raise KeyError(key)

    def _resize(self, new_size):
        items = [kv for b in self._buckets for kv in b]
        self._buckets = [[] for _ in range(new_size)]
        for k, v in items:                      # rehash every item
            self._bucket(k).append((k, v))

    def __len__(self):
        return self._count
```

Complexity. Average O(1) per operation for well-spread keys; O(n) worst case when all keys land in one bucket. Resizing is O(n) but amortised O(1) per insert because capacity doubles. **Tests** cover collision, update-not-duplicate, missing keys, forced resizes, and a randomised comparison with Python's `dict`. **Extensions:** string keys with a better hash, open addressing, shrinking, thread safety by lock striping.

30.3 Design 2: LRU cache

The same structure as the query cache (Chapter 26), now seen as classes: `LRUCache` composes a dictionary and a doubly linked list of nodes.

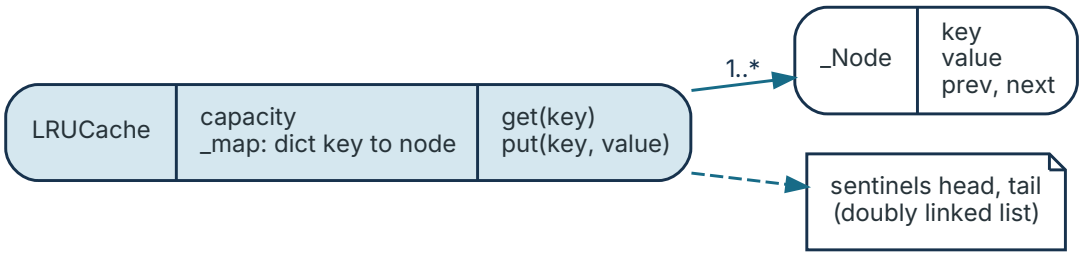


Figure 30.2 LRU cache classes. The cache owns a map and the list; nodes store the key so eviction can also delete the map entry.

Design note (a flaw in the skeleton we started from). The notebook's node stores only the results. Eviction needs the evicted node's **key** (to delete it from the map) and unlinking from the middle needs a **previous pointer**. The reference node below stores both.

```python
class _Node:
    __slots__ = ("key", "value", "prev", "next")
    def __init__(self, key=None, value=None):
        self.key, self.value, self.prev, self.next = key, value, None, None

class LRUCache:
    """O(1) get/put. Invariants: len(map) <= capacity; list order == recency order."""
    def __init__(self, capacity):
        if capacity < 1:
            raise ValueError("capacity must be >= 1")
        self.capacity = capacity
        self._map = {}
        self._head, self._tail = _Node(), _Node()      # sentinels
        self._head.next, self._tail.prev = self._tail, self._head

    def _unlink(self, n):
        n.prev.next, n.next.prev = n.next, n.prev

    def _push_front(self, n):
        n.prev, n.next = self._head, self._head.next
        self._head.next.prev = n
        self._head.next = n

    def get(self, key):
        n = self._map.get(key)
        if n is None:
            return None
        self._unlink(n); self._push_front(n)
        return n.value

    def put(self, key, value):
        n = self._map.get(key)
        if n is not None:
            n.value = value
            self._unlink(n); self._push_front(n)
            return
        if len(self._map) >= self.capacity:
            lru = self._tail.prev
            self._unlink(lru)
            del self._map[lru.key]                     # evict from list AND map
        n = _Node(key, value)
        self._map[key] = n
        self._push_front(n)

    def keys_by_recency(self):
        out, n = [], self._head.next
        while n is not self._tail:
            out.append(n.key); n = n.next
        return out
```

Invariants. `len(map) <= capacity`; every list node is in the map under its own key; list order equals recency order (head newest). **State transition** per operation: hit moves the node to the front; insert at capacity removes the tail first. **Thread safety:** wrap each public method in a lock (even `get` mutates the list), or shard keys over several caches.

30.4 Design 3: Call center

Assumptions. Three ranks (operator, supervisor, director). A call goes to a free operator first; if none is free it goes to a supervisor, then a director. An employee may **escalate** a call, which raises the call's minimum rank and re-dispatches it. If nobody eligible is free the call is **queued** and assigned when someone frees up. Directors cannot escalate.

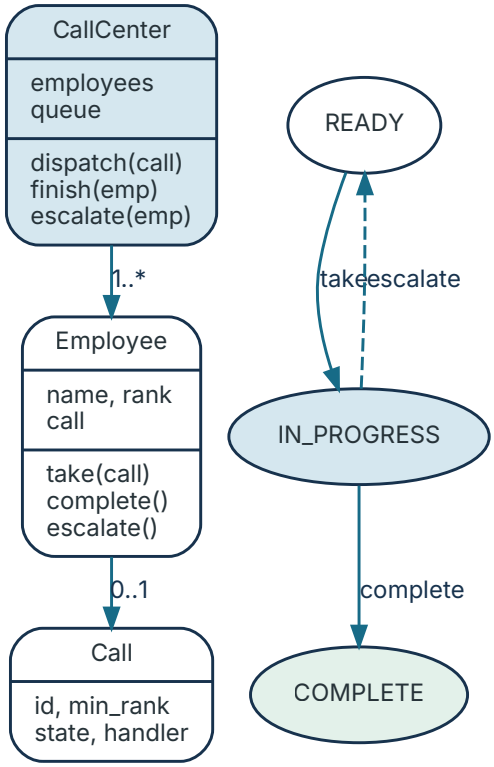


Figure 30.3 Call center classes and call state machine (READY, IN_PROGRESS, COMPLETE).

Table 30.1 Call state transitions.

From	Event	To	Guard
READY	employee takes call	IN_PROGRESS	employee free and rank at least `min_rank`
IN_PROGRESS	employee completes	COMPLETE	none
IN_PROGRESS	employee escalates	READY (rank raised)	handler is not a director
READY	no eligible employee free	READY (queued)	none

Invariants. An employee has at most one call; a call has at most one handler; `IN_PROGRESS` implies a handler, other states imply none; an employee never holds a call above their rank.

```
from collections import deque
from enum import Enum

class Rank(Enum):
    OPERATOR = 0
    SUPERVISOR = 1
    DIRECTOR = 2

class CallState(Enum):
    READY = "ready"
    IN_PROGRESS = "in_progress"
    COMPLETE = "complete"

class Call:
    def __init__(self, call_id, min_rank=Rank.OPERATOR):
        self.id, self.min_rank = call_id, min_rank
        self.state, self.handler = CallState.READY, None

class Employee:
    def __init__(self, name, rank):
        self.name, self.rank, self.call = name, rank, None

    @property
    def free(self):
        return self.call is None

    def take(self, call):
        assert self.free and call.state is CallState.READY
        assert call.min_rank.value <= self.rank.value, "employee too junior"
        self.call, call.handler, call.state = call, self, CallState.IN_PROGRESS

    def complete(self):
        call, self.call = self.call, None
        call.state, call.handler = CallState.COMPLETE, None
        return call

    def escalate(self):
        """Return the call to READY at the next rank; directors cannot escalate."""
        if self.rank is Rank.DIRECTOR:
            raise ValueError("directors cannot escalate")
        call, self.call = self.call, None
        call.state, call.handler = CallState.READY, None
        call.min_rank = Rank(self.rank.value + 1)
        return call

class CallCenter:
    def __init__(self, employees):
        self.employees = list(employees)
        self.queue = deque()

    def dispatch(self, call):
        for rank in Rank:                                  # lowest rank first
            if rank.value < call.min_rank.value:
                continue
            for e in self.employees:
                if e.rank is rank and e.free:
                    e.take(call)
                    return e
        self.queue.append(call)                            # nobody free
        return None

    def finish(self, employee):
```

```
        employee.complete()
        return self._assign_next(employee)

    def escalate(self, employee):
        call = employee.escalate()
        return self.dispatch(call)

    def _assign_next(self, employee):
        for _ in range(len(self.queue)):                    # first queued call this person
may take
            call = self.queue.popleft()
            if call.min_rank.value <= employee.rank.value:
                employee.take(call)
                return call
            self.queue.append(call)
        return None
```

Flaws in the notebook skeleton, fixed here. Its subclasses did not pass the call-center argument to the base constructor and two called `super()` with the wrong class; here ranks are an enum and there is one `Employee` class, so subclasses are unnecessary (behaviour differs only by rank data). Mention that inheritance would be justified only if ranks had different *behaviour*. **Extensions:** VIP calls with priority queues, skill routing, fairness (longest idle first), concurrency control on dispatch.

30.5 Design 4: Deck of cards and blackjack

Assumptions. A generic 52-card deck usable for several games; design the generic layer first then extend for blackjack (face cards count 10; an ace counts 1 or 11).

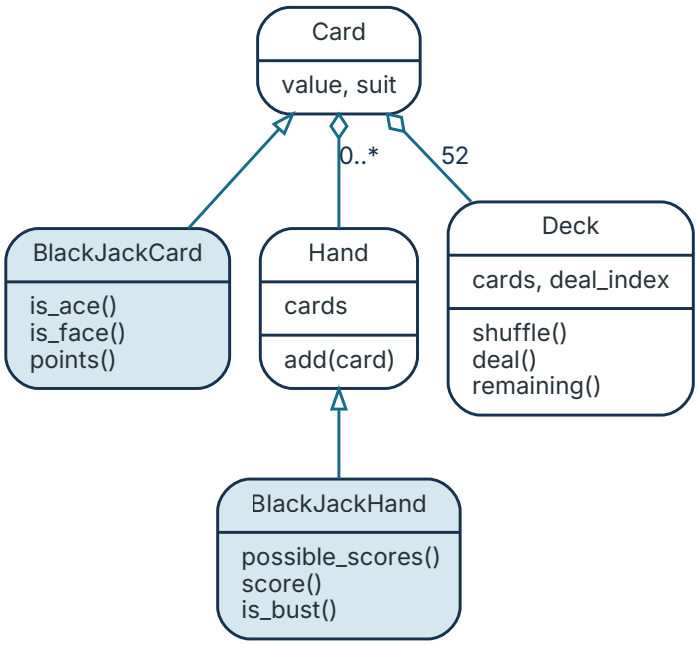


Figure 30.4 Generic Card, Hand and Deck, with blackjack specialisations.

Responsibilities. `Deck` owns the cards and a **deal index** (dealing never removes cards, so `remaining = 52 - deal_index`); `Hand` holds cards; `BlackJackHand` computes all possible scores and picks the best; `BlackJackCard` knows points and aces.

Invariants. A full deck has exactly 52 distinct cards; `0 <= deal_index <= 52`; a hand's score is the largest possible total at most 21, or the smallest total if all exceed 21.

```
import random
from enum import Enum

class Suit(Enum):
    CLUBS, DIAMONDS, HEARTS, SPADES = range(4)

class Card:
    def __init__(self, value, suit):               # value: 1..13 (1 = ace, 11-13 = J, Q, K)
        assert 1 <= value <= 13
        self.value, self.suit = value, suit

class BlackJackCard(Card):
    def is_ace(self):
        return self.value == 1
    def is_face(self):
        return 11 <= self.value <= 13
    def points(self):
        return 10 if self.is_face() else self.value   # ace counted as 1 here

class Hand:
    def __init__(self, cards=None):
        self.cards = list(cards or [])
    def add(self, card):
        self.cards.append(card)

class BlackJackHand(Hand):
    def possible_scores(self):
        base = sum(c.points() for c in self.cards)
        aces = sum(1 for c in self.cards if c.is_ace())
        return sorted({base + 10 * k for k in range(aces + 1)})   # each ace may add 10 more
    def score(self):
        scores = self.possible_scores()
        under = [s for s in scores if s <= 21]
        return max(under) if under else min(scores)
    def is_bust(self):
        return self.score() > 21

class Deck:
    def __init__(self, card_cls=Card, rng=None):
        self.cards = [card_cls(v, s) for s in Suit for v in range(1, 14)]
        self.deal_index = 0
        self._rng = rng or random.Random()
    def remaining(self):
        return len(self.cards) - self.deal_index
    def shuffle(self):
        self._rng.shuffle(self.cards)
        self.deal_index = 0
    def deal(self):
        if self.remaining() == 0:
            raise IndexError("deck is empty")
        card = self.cards[self.deal_index]
        self.deal_index += 1
        return card
```

Why the ace logic works. Count every ace as 1, then each ace may add 10: with k aces there are $k + 1$ totals, not 2^k, because only the **number** of aces counted as 11 matters. Two aces give 2, 12 or 22. **Extensions:** multi-deck shoe with a cut card, split and double-down, dealer rules, secure shuffling (use a cryptographic source for real money games).

30.6 Design 5: Parking lot

Assumptions. Several levels, each a row of spots of size compact or large. A motorcycle fits any spot, a car fits compact or large, a bus needs **five consecutive large spots**.

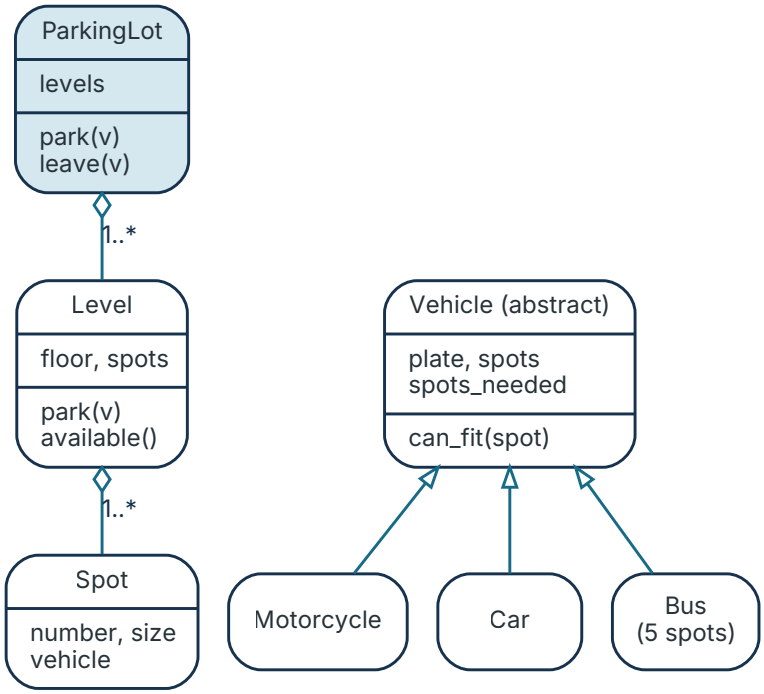


Figure 30.5 Parking lot classes. Vehicle subclasses encode fit rules and how many spots they need.

Responsibilities. `ParkingLot` asks each `Level` to park; a `Level` searches for a window of adjacent free spots that all fit; the `Vehicle` remembers its spots so leaving frees exactly those. Polymorphism (`can_fit`, `spots_needed`) removes any `if bus ... elif car ...` chain.

Invariants. A spot holds at most one vehicle; a parked vehicle's `spots` are exactly the spots that reference it; a vehicle is parked at most once.

```python
from enum import Enum

class Size(Enum):
    COMPACT = 1
    LARGE = 2

class Vehicle:
    spots_needed = 1
    def __init__(self, plate):
        self.plate, self.spots = plate, []
    def can_fit(self, spot):
        raise NotImplementedError

class Motorcycle(Vehicle):
    def can_fit(self, spot): return True

class Car(Vehicle):
    def can_fit(self, spot): return True          # compact or large

class Bus(Vehicle):
    spots_needed = 5
    def can_fit(self, spot): return spot.size is Size.LARGE

class Spot:
    def __init__(self, number, size):
        self.number, self.size, self.vehicle = number, size, None
    def is_free(self):
        return self.vehicle is None

class Level:
    def __init__(self, floor, spots):
        self.floor, self.spots = floor, spots
    def available(self):
        return sum(s.is_free() for s in self.spots)
    def park(self, v):
        need = v.spots_needed
        for i in range(len(self.spots) - need + 1):           # window of `need` adjacent
spots
            window = self.spots[i:i + need]
            if all(s.is_free() and v.can_fit(s) for s in window):
                for s in window:
                    s.vehicle = v
                v.spots = window
                return True
        return False

class ParkingLot:
    def __init__(self, levels):
        self.levels = levels
    def park(self, v):
        if v.spots:
            raise ValueError("already parked")
        return any(level.park(v) for level in self.levels)
    def leave(self, v):
        if not v.spots:
            raise ValueError("not parked")
        for s in v.spots:
            s.vehicle = None
        v.spots = []
```

Complexity. Parking scans each level in O(spots × window). A per-level free counter lets full levels be skipped; for large lots keep free spots indexed by size. **Extensions:** pricing and tickets, nearest-to-entrance search, reservations and electric-vehicle spots, concurrency (two cars choosing the same spot), a state machine for a spot (FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE).

30.7 Design 6: Online chat

Scope. Text only. Users can be added, send and answer **friend requests**, create private chats with friends, and group chats. No scaling concerns yet.

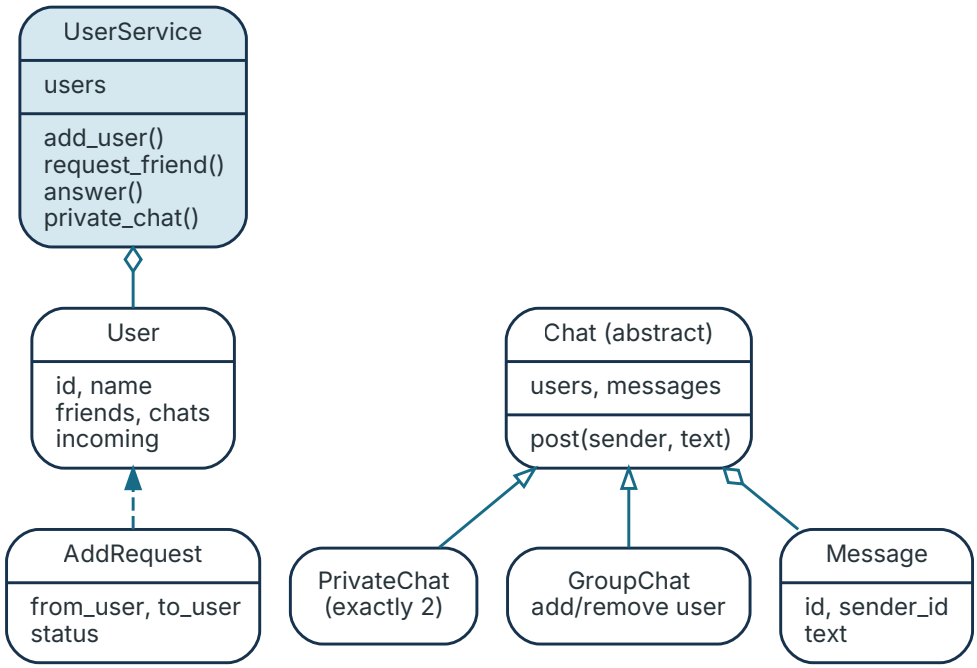


Figure 30.6 Chat classes: UserService as entry point; abstract Chat with PrivateChat and GroupChat; requests have a status.

Table 30.2 Friend request status transitions.

From	Event	To
UNREAD	recipient accepts	ACCEPTED (friendship added to both users)
UNREAD	recipient rejects	REJECTED
ACCEPTED or REJECTED	any answer	error: already answered

Invariants. Friendship is symmetric; a private chat has exactly two distinct users who are friends; only members can post; a request is answered at most once.

```
from enum import Enum
from itertools import count

class RequestStatus(Enum):
    UNREAD = 0
    ACCEPTED = 1
    REJECTED = 2

class Message:
    _ids = count(1)
    def __init__(self, sender_id, text):
        self.id, self.sender_id, self.text = next(Message._ids), sender_id, text

class Chat:
    _ids = count(1)
    def __init__(self, users):
        self.id, self.users, self.messages = next(Chat._ids), list(users), []
    def post(self, sender, text):
        if sender not in self.users:
            raise PermissionError("not a member")
        m = Message(sender.id, text)
        self.messages.append(m)
        return m

class PrivateChat(Chat):
    def __init__(self, a, b):
        if a is b:
            raise ValueError("a private chat needs two different users")
        super().__init__([a, b])

class GroupChat(Chat):
    def add_user(self, user):
        if user not in self.users:
            self.users.append(user)
    def remove_user(self, user):
        self.users.remove(user)

class AddRequest:
    def __init__(self, from_user, to_user):
        self.from_user, self.to_user = from_user, to_user
        self.status = RequestStatus.UNREAD

class User:
    def __init__(self, user_id, name):
        self.id, self.name = user_id, name
        self.friends, self.chats, self.incoming = {}, [], []

class UserService:
    def __init__(self):
        self.users = {}
    def add_user(self, user_id, name):
        if user_id in self.users:
            raise ValueError("duplicate id")
        self.users[user_id] = User(user_id, name)
        return self.users[user_id]
    def request_friend(self, from_id, to_id):
        a, b = self.users[from_id], self.users[to_id]
        if a is b or b.id in a.friends:
            raise ValueError("invalid request")
        req = AddRequest(a, b)
        b.incoming.append(req)
        return req
```

```
    def answer(self, req, accept):
        if req.status is not RequestStatus.UNREAD:
            raise ValueError("already answered")
        if accept:
            req.status = RequestStatus.ACCEPTED
            req.from_user.friends[req.to_user.id] = req.to_user      # friendship is symmet-
ric
            req.to_user.friends[req.from_user.id] = req.from_user
        else:
            req.status = RequestStatus.REJECTED
    def private_chat(self, a_id, b_id):
        a, b = self.users[a_id], self.users[b_id]
        if b.id not in a.friends:
            raise PermissionError("not friends")
        chat = PrivateChat(a, b)
        a.chats.append(chat); b.chats.append(chat)
        return chat
```

Scaling it later (not asked now). Shard users by id, keep persistent connections for push delivery, store history in a wide-column store keyed by chat and time, and use per-user queues for offline delivery (Chapter 10 and Chapter 13). The in-memory objects become rows and the invariants become constraints and idempotent operations.

30.8 Exercise from the notebook: a circular array

The original lists "design a circular array" as an open exercise. A fixed array plus a head index and a size gives a FIFO buffer with O(1) push and pop; indices wrap by modulo. The one decision to state aloud is **what happens when full**: reject (raise) or overwrite the oldest.

```
class CircularBuffer:
    """Fixed-capacity FIFO. Invariants: 0 <= size <= cap; items are at head, head+1, ... mod-
ulo cap."""
    def __init__(self, cap, overwrite=False):
        self._a, self._cap, self._head, self._size = [None] * cap, cap, 0, 0
        self._overwrite = overwrite
    def push(self, x):
        if self._size == self._cap:
            if not self._overwrite:
                raise OverflowError("full")
            self._head = (self._head + 1) % self._cap          # drop the oldest
            self._size -= 1
        self._a[(self._head + self._size) % self._cap] = x
        self._size += 1
    def pop(self):
        if self._size == 0:
            raise IndexError("empty")
        x = self._a[self._head]
        self._a[self._head] = None
        self._head = (self._head + 1) % self._cap
        self._size -= 1
        return x
    def __len__(self):
        return self._size
```

30.9 Tests and what they prove

The test file contains 13 tests: hash map (basic, collision, update, resize, randomised against `dict`), LRU (eviction order, update, capacity one, randomised against an ordered-dictionary reference), call center (dispatch, escalation, queue, junior restriction), blackjack (ace totals, bust, two aces), deck (52 distinct cards, exhaustion), parking (car, bus, leave, contiguity), chat (friend flow, membership, group) and the circular buffer (wrap-around, full, overwrite). **Honest scope:** the tests exercise single-threaded behaviour; none tests concurrency, and the randomised tests use fixed seeds.

COMMON MISCONCEPTION

- "*More classes means better design.*" Create a class when it owns a responsibility or invariant.
- "*Inheritance models every difference.*" Use data (an enum) when behaviour is the same.
- "*Working code means correct design.*" Check invariants and edge cases.

INTERVIEW QUESTIONS

Q1. How do you start an OO design question? Clarify, list nouns and verbs, assign responsibilities, state invariants, choose data structures, walk through scenarios.

Q2. Inheritance or composition? Composition by default; inheritance when subtypes have different behaviour behind one interface (vehicles' fit rules).

Q3. How would you make the call center thread-safe? One lock around dispatch and state changes (or per-employee locks with ordered acquisition) so a call is never assigned twice.

Q4. Why does the LRU node store its key? So eviction can remove the map entry.

EXERCISES

1. Add `resize` shrinking to `HashMap` when the load factor falls below 0.2. State the invariant you must preserve.
2. A bus parks, then a car leaves a compact spot in the middle of the lot. Does the bus's allocation change? Explain.
3. Write the transition table for a parking `Spot` with states FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE.
4. Two operators finish at the same moment while one call is queued. What can go wrong without a lock, and what is the fix?

Answers.

1. Shrink by rebuilding with half the buckets when `count / size < 0.2` and size above a minimum; the invariant is that every item stays in bucket `key % new_size`, achieved by rehashing all items.
2. No: the bus's `spots` are the five spots it was given; freeing a different spot touches only the car's allocation.

3. FREE → RESERVED (reserve); RESERVED → OCCUPIED (arrive), RESERVED → FREE (cancel or timeout); OCCUPIED → FREE (leave); any → OUT_OF_SERVICE (maintenance); OUT_OF_SERVICE → FREE (repaired).
4. Both could pop the same queued call and both take it, violating one handler per call; guard dequeue and assignment with a lock (or use an atomic queue pop that returns a call to only one caller).

CHAPTER SUMMARY

- Method: clarify, nouns and verbs, responsibilities, invariants, state enums, data structures, scenarios, tests.
- Hash map, LRU, call center, deck, parking lot, chat and circular buffer are small designs with clear invariants.
- Fix the flaws in skeletons: LRU nodes need key and previous pointer; avoid needless subclasses.
- Tests per invariant and transition, plus randomised comparisons, give real confidence.

REVISION CHECKLIST

- [ ] I can write the LRU cache and the hash map from memory.
- [ ] I can state invariants for each design.
- [ ] I can draw the call and request state transitions.
- [ ] I can explain when to use inheritance versus data.

PART IX

Practice and Real-World Architectures

Twenty-three further design questions to practise with, and a method for learning from the architectures of real systems.

CHAPTER 31

Practice Bank: Twenty-Three More Design Questions

Derived from your notes: Notebook chapter 14 (the practice bank of additional questions and the lists of real-world and company architectures). **Added in this book:** a structured starting sketch for every question (clarifying questions, core design, the hard part, chapters to revisit), a self-assessment rubric, and a study plan.

LEARNING OBJECTIVES

After this chapter you can:

- start any of the notebook's additional questions with the right clarifying questions and a defensible skeleton;
- identify the single hardest sub-problem in each and the chapter that covers it;
- assess your own answer against a rubric and plan further practice.

PREREQUISITE KNOWLEDGE

All of Part VII (the interview method and the worked designs). Attempt each question for 30 to 40 minutes *before* reading its sketch.

SUPPLEMENTAL MATERIAL

How these sketches were written. The notebook lists the questions (with reading links) but gives no answers. Each entry below is the book's own starting sketch: it names what to ask, the core components, the hard part, and where in this book to look. They are seeds for your design, not complete solutions, and they are not taken from the linked articles.

31.1 How to use the bank

Follow the loop from Chapter 20: requirements, estimates, API, data model, high-level design, deep dive, bottlenecks, failure handling. Time-box yourself, speak your reasoning aloud (or write it), then compare with the sketch and the **rubric** in Table 31.1.

Table 31.1 Self-assessment rubric (score each 0 to 2; 14 is the maximum).

Dimension	0	1	2
Requirements	Started designing at once	Some questions	Functional, non-functional and scope agreed
Estimates	None	Rough	Used to justify choices
API and data	Missing	Partial	Clear contracts, keys, indexes

Dimension	0	1	2
Architecture	Boxes only	Flow explained	Flows, consistency and scaling explained
Deep dive	Skipped	Described	Alternatives and trade-offs weighed
Failure	Not mentioned	A few cases	Systematic, with recovery and monitoring
Communication	Disorganised	Mostly clear	Structured, checks in with the interviewer

31.2 The questions

1. File sync like Dropbox. *Ask:* file sizes, devices per user, conflict policy, sharing. *Core:* split files into content-addressed **chunks** (hash each), upload only chunks the server lacks (deduplication), keep file→chunk-list metadata in a database, chunks in object storage; a notification channel tells other devices of changes; clients keep a local index and sync by diffing versions. *Hard part:* **conflict resolution** when two devices edit offline (keep both copies, or merge). *Revisit:* Chapter 10, Chapter 13, Chapter 11.

2. A search engine like Google. *Ask:* corpus size, freshness, ranking quality. *Core:* crawler, document store, inverted index, ranking, query serving with caches. *Hard part:* index partitioning and tail latency. *Revisit:* Chapter 23, Chapter 26.

3. A scalable web crawler. *Core:* priority and politeness frontier, URL and content de-duplication, shard by host. *Hard part:* traps and politeness at scale. *Revisit:* Chapter 23.

4. Google Docs (collaborative editing). *Ask:* number of simultaneous editors, offline support. *Core:* clients send **operations** to a server that orders them; **operational transformation** or a **CRDT** (conflict-free replicated data type) merges concurrent edits; persist an operation log with periodic snapshots; presence over WebSockets. *Hard part:* convergence: all replicas end with the same text regardless of arrival order. *Revisit:* Chapter 11, Chapter 14 (ordering and clocks).

5. A key-value store like Redis. *Core:* in-memory hash table, single-threaded event loop or sharded threads, durability by snapshots plus an append-only log, primary-replica replication, sharding by consistent hashing. *Hard part:* durability versus latency (fsync policy) and failover. *Revisit:* Chapter 3, Chapter 11, Chapter 26.

6. A cache like Memcached. *Core:* slab-style memory management, LRU per slab class, client-side consistent hashing, no persistence. *Hard part:* memory fragmentation and hot keys. *Revisit:* Chapter 26, Chapter 12.

7. Recommendations like Amazon's "customers also bought". *Core:* offline batch job computing item-to-item co-occurrence (or embeddings), results stored per item, online lookup behind a cache, filters for stock and region. *Hard part:* cold start and freshness versus cost. *Revisit:* Chapter 27, Chapter 12.

8. TinyURL or Bitly. *Core:* unique code generation, key-value store, cache, redirect service. *Hard part:* collision-free code generation across servers. *Revisit:* Chapter 21.

9. A chat service like WhatsApp. *Ask:* group size limits, history retention, end-to-end encryption. *Core:* persistent connections to a gateway, a per-user inbox (queue), delivery receipts, offline storage, group fan-out through a messaging service. *Hard part:* ordering and exactly-once *effects* (message ids, ack and retry), and connection fleet scaling. *Revisit:* Chapter 13, Chapter 30.

10. Photo sharing like Instagram. *Core:* object storage plus CDN for images, asynchronous thumbnail pipeline, sharded metadata, feed generation. *Hard part:* feed fan-out for popular accounts. *Revisit:* Chapter 22, Chapter 7.

11. A news feed (Facebook). *Core:* hybrid fan-out, ranking stage, caches. *Hard part:* celebrities and ranking freshness. *Revisit:* Chapter 22.

12. A timeline (Facebook). *Core:* per-user post lists in cache backed by durable storage; read-heavy. *Hard part:* keeping lists consistent with deletes and privacy changes. *Revisit:* Chapter 22, Chapter 12.

13. Facebook-style chat with presence. *Core:* chat design plus a **presence** service holding short-lived "online" state (heartbeats with TTL), with fan-out limited to active friends. *Hard part:* presence churn volume (batch and rate-limit updates). *Revisit:* Chapter 13, Chapter 12.

14. Graph search. *Core:* social graph storage plus inverted indexes over typed edges and attributes, privacy filters. *Hard part:* query planning and visibility checks. *Revisit:* Chapter 25, Chapter 23.

15. A CDN like Cloudflare. *Core:* edge points of presence, routing by DNS or anycast, tiered cache, origin shield, purge API. *Hard part:* cache invalidation worldwide and origin protection. *Revisit:* Chapter 7, Chapter 5.

16. Trending topics. *Core:* stream processing over sliding windows, approximate counting, top-k heap, time-decay scoring, spam filtering. *Hard part:* distinguishing a surge from a baseline (compare with a historical average). *Revisit:* Chapter 27, Chapter 3.

17. Random ID generation. *Core:* a time-ordered 64-bit id of timestamp, machine id and sequence (a Snowflake-style scheme), or pre-allocated id ranges per server. *Hard part:* clock going backwards and machine-id assignment. *Revisit:* Chapter 14.

18. Top *k* requests in a window. *Core:* hash counts per time bucket plus a min-heap of size k; at scale, partition by key and merge partial top-ks; use a count-min sketch for bounded memory. *Hard part:* accuracy versus memory. *Revisit:* Chapter 3, Chapter 27.

19. Serving from multiple data centers. *Core:* geo-routing, replication (synchronous within a region, asynchronous across), conflict handling, failover drills. *Hard part:* write conflicts and the choice of consistency. *Revisit:* Chapter 11, Chapter 14, Chapter 19.

20. An online multiplayer card game. *Core:* authoritative server holds game state; rooms and turns as a state machine; real-time channel; reconnect handling; anti-cheat (hide other players' cards on the server side). *Hard part:* consistent state with unreliable clients. *Revisit:* Chapter 30.

21. A garbage collection system. *Core:* reference counting versus tracing (mark and sweep), generational collection, stop-the-world pauses versus concurrent marking. *Hard part:* cycles and pause times. *Revisit:* Chapter 2, Chapter 3.

22. An API rate limiter. *Core:* token bucket or sliding-window counters per key in a shared fast store, atomic increments, local approximation plus periodic sync. *Hard part:* accuracy across many gateway nodes. *Revisit:* Chapter 7, Chapter 12.

23. A stock exchange. *Core:* an **order book** per symbol with price-time priority, a single-writer matching engine, a sequencer assigning order, a journal for replay and recovery, market-data fan-out. *Hard part:* determinism and ultra-low latency (single thread per symbol, no locks). *Revisit:* Chapter 14, Chapter 2.

31.3 A study plan

Over four weeks: **week 1**, redo the eight worked designs of Part VII from memory; **week 2**, questions 1 to 8 on paper, scoring yourself with the rubric; **week 3**, questions 9 to 16 with a partner who plays the interviewer; **week 4**, questions 17 to 23 plus two mock interviews, concentrating on the dimensions where your rubric score is lowest. Keep a log of *mistakes* (a missing estimate, an unexamined failure mode): they repeat.

WORKED EXAMPLE

Scoring example. A candidate on question 22 scores: requirements 2, estimates 1, API and data 2, architecture 2, deep dive 1 (did not compare token bucket with sliding window), failure 0 (never discussed a Redis outage), communication 2. Total 10 of 14; the action is to practise failure discussion (what happens to limits if the store is down: fail open or fail closed?) and alternatives.

COMMON MISCONCEPTION

- "*Memorise one answer per question.*" Interviewers vary the constraints; practise the method.
- "*The hint sketches are complete designs.*" They are starting points.

INTERVIEW QUESTIONS

Q1. You are asked a question you have never seen; what do you do? Apply the same loop: clarify, estimate, draw the request path, find the hardest part, and name trade-offs.

Q2. How do you choose which sub-problem to deep dive? The one that is hardest or most specific to the problem (conflict resolution for sync, ordering for the exchange), or the one the interviewer signals.

EXERCISES

1. Pick question 4, 9 or 19 and write a one-page design using the rubric headings.
2. For question 17, show why a clock moving backwards can duplicate ids and give two mitigations.

3. For question 22, decide fail-open versus fail-closed for a login endpoint and for a public read API, with reasons.

Answers.

1. Compare your page with the sketch; a strong page contains numbers, a data model and a failure section.
2. If the timestamp part repeats while the sequence restarts at zero, two ids can collide. Mitigations: refuse to generate ids until the clock passes the last timestamp (wait), or keep a persisted high-water mark and use it as the minimum timestamp.
3. Login: usually fail closed or degrade to a stricter local limit, because unlimited attempts enable password guessing. Public read API: usually fail open with a local limit, because blocking all reads is a bigger harm than temporary over-use.

CHAPTER SUMMARY

- Each question has a recognisable core, a hard part and a chapter to revisit.
- Practise with a time box and a rubric; log mistakes.
- The method transfers; memorised answers do not.

REVISION CHECKLIST

- [ ] I can state the hard part of every question in one sentence.
- [ ] I have scored at least eight practice answers with the rubric.

CHAPTER 32

Reading Real-World Architectures

Derived from your notes: Notebook chapter 15 (real-world architectures: the 17 systems of the primer's table, the 23 company architecture themes, and the list of engineering blogs). **Added in this book:** a method for reading architecture write-ups, a pattern-to-chapter map, and reading questions for each system.

LEARNING OBJECTIVES

After this chapter you can:

- read an architecture write-up by extracting requirements, components, patterns and trade-offs instead of memorising stacks;
- place each well-known system in the patterns taught in this book;
- choose which engineering write-ups to read for a given interview or project.

PREREQUISITE KNOWLEDGE

Parts III to V of this book. Descriptions here are deliberately high-level and reflect public papers and write-ups as the notebook lists them; internals change, so check current sources before relying on a detail.

32.1 A method for reading

The notebook's advice is not to memorise nitty-gritty details but to find the **shared principles**, what problem each component solves, where it works and where it does not, and the lessons learned. Use this routine on any write-up:

1. **Requirements first.** What load, data size, latency and consistency did the authors need? A design is only good *for* a requirement.
2. **Draw the request path** (Chapter 1). Circle every box and write the problem it solves.
3. **Name the pattern:** cache, queue, partitioning, replication, log, consensus, batch or stream.
4. **Find the trade-off:** what did they give up (consistency, flexibility, operational simplicity, cost)?
5. **Ask what changed:** most architectures are scars from earlier failures or growth stages (compare Chapter 28).
6. **Check the date.** A 2010 post may describe a stack since replaced.



Figure 32.1 Most large-company diagrams are this skeleton with different brand names on each box.

32.2 The seventeen systems in the notebook's table

Table 32.1 Systems, what they are, the pattern they teach and the trade-off.

System (origin)	What it is	Pattern it teaches	Trade-off or when to use	Book chapter
MapReduce (Google)	Distributed batch processing: map, shuffle, reduce	Move computation to data; parallel batch	High latency; good for logs and rankings	Chapter 27
Spark	Batch processing keeping working sets in memory	Reuse data in RAM between stages	Needs memory; faster for iterative jobs	Chapter 27
Storm	Stream processing of events as they arrive	Low-latency continuous computation	More complex than batch	Chapter 13
Bigtable (Google)	Wide-column store with sorted keys	Huge sparse tables, range scans	Weak ad-hoc querying	Chapter 10
HBase	Open-source Bigtable-style store on HDFS	Same, in the Hadoop ecosystem	Operational weight	Chapter 10
Cassandra (Facebook)	Wide-column, highly available, tunable consistency	Replication with quorums, AP bias	Model queries up front	Chapter 11
DynamoDB (Amazon)	Managed key-value and document store	Scale without running servers	Access patterns fixed in advance	Chapter 10
MongoDB	Document store	Flexible schema, rich document queries	Limited joins and cross-document transactions	Chapter 10
Spanner (Google)	Globally distributed relational database	Strong consistency across regions using synchronised clocks and consensus	Cost and write latency	Chapter 14
Memcached	Distributed memory cache	Cache-aside, simple and fast	No persistence; weak consistency	Chapter 12
Redis	Cache plus data structures plus persistence options	Sorted sets, queues, counters	Memory-bound; single-node limits	Chapter 12
GFS (Google)	Distributed file system	Large files as replicated chunks	Poor for tiny files and low latency	Chapter 11
HDFS	Open-source GFS-style file system	Same, in Hadoop	Same	Chapter 11

System (origin)	What it is	Pattern it teaches	Trade-off or when to use	Book chapter
Chubby (Google)	Lock and coordination service	Consensus for small critical data, leader election	Becomes a critical dependency	Chapter 14
Dapper (Google)	Distributed tracing	Follow one request across services	Needs instrumentation and sampling	Chapter 18
Kafka (LinkedIn)	Publish-subscribe message log	Durable ordered log decoupling producers and consumers	Operational complexity	Chapter 13
ZooKeeper	Coordination, configuration, naming	Consensus for small shared state	Not a general database	Chapter 14

Reading questions. For MapReduce: what happens if one mapper is slow (straggler)? For Cassandra: what do `R + W > N` settings give you? For Spanner: why does it wait out clock uncertainty before committing? For Kafka: what is a consumer offset and why does it enable replay? For Chubby: why must clients tolerate losing a lock?

32.3 Company architectures: what to look for

The notebook links one or more write-ups per company. Instead of learning each stack, read each with a question in mind. The right column is the pattern *expected* from the notebook's title for that write-up; verify it against the article itself.

Table 32.2 Company themes and the pattern to look for.

Company (theme listed in the notebook)	Pattern to look for	Chapter
Amazon	Service-oriented decomposition with independent teams and APIs	Chapter 16
Cinchcast (1,500 hours of audio per day)	Media pipeline with queues and workers	Chapter 13
DataSift (120,000 tweets per second)	Stream ingest, partitioning and real-time filtering	Chapter 13
Dropbox	Metadata versus bulk storage; sync protocol	Chapter 31
ESPN (100,000 events per second)	Absorbing spiky read traffic with caching and edge	Chapter 12
Google	Commodity clusters with a file system, batch and table store	Chapter 11
Instagram	Photo storage, sharded metadata, caching	Chapter 9
Justin.tv (live video)	Ingest, transcode, distribute	Chapter 7
Facebook (caching, social graph store, photos, live video)	Cache layers over databases and a purpose-built graph store	Chapter 25
Flickr	Sharding and caching for a photo service	Chapter 9
Mailbox (0 to 1 million users in 6 weeks)	Scaling under surprise growth; back pressure and elasticity	Chapter 28
Netflix (the whole stack; pressing play)	Microservices, resilience, CDN for video	Chapter 15

Company (theme listed in the notebook)	Pattern to look for	Chapter
Pinterest (billions of page views)	Growth stages: caching, then sharding	Chapter 28
Playfish (50 million monthly users)	Game back end scaling with stateless servers	Chapter 28
PlentyOfFish	How far a lean setup can go with caching	Chapter 12
Salesforce (1.3 billion transactions per day)	Multi-tenant relational architecture at scale	Chapter 9
Stack Overflow	Vertical scaling plus caching can go far	Chapter 1
TripAdvisor (30 TB, 200 million dynamic views)	Caching dynamic pages; data growth	Chapter 12
Tumblr (15 billion page views per month)	Fan-out of posts to follower dashboards	Chapter 22
Twitter (timelines, 250 million tweets per day, firehose)	Fan-out on write, storage, big and small data	Chapter 22
Uber (real-time marketplace; many services)	Real-time geo-matching; microservice sprawl lessons	Chapter 16
WhatsApp	Connection-heavy messaging run by a small team	Chapter 31
YouTube	Video storage and serving; thumbnails and caching	Chapter 7

The figures in parentheses are the titles the notebook gives for the linked articles; they are historical and are not claims about today's scale.

32.4 Engineering blogs worth following

Interview questions often come from the interviewing company's own domain, so read that company's engineering blog first. The notebook lists, among others: Airbnb, Atlassian, AWS, Bitly, Box, Cloudera, Dropbox, Quora, eBay, Evernote, Etsy, Facebook, Flickr, Foursquare, GitHub, Google Research, Groupon, Heroku, HubSpot, High Scalability, Instagram, Intel Software, Jane Street, LinkedIn, Microsoft, Netflix, PayPal, Pinterest, Reddit, Salesforce, Slack, Spotify, Stripe, Twilio, Twitter, Uber, Yahoo, Yelp and Zynga, and points to a larger community list (the *engineering-blogs* repository by *kilimchoi*). The links were not re-verified for this book; search for the current blog address of each.

> **WORKED EXAMPLE**
>
> **Reading exercise: Kafka.** Requirements: ingest very high volumes of events and let many consumers read them independently and replay. Request path: producers append to a partitioned log; brokers replicate partitions; consumers track an offset per partition. Pattern: durable log decoupling producers and consumers (Chapter 13). Trade-off: ordering is only per partition, and consumers must handle duplicates (at-least-once). Date check: the broker's coordination layer has changed over versions, so verify against current documentation.

COMMON MISCONCEPTION

- "*I must know every company's stack.*" Know the patterns; stacks change.
- "*A famous architecture is the right architecture for me.*" It fits their requirements and scale.

INTERVIEW QUESTIONS

Q1. Which real system would you cite for leader election and why? A coordination service built on consensus (Chubby or ZooKeeper), because small critical state needs strong consistency; also state its cost as a critical dependency.

Q2. What does Cassandra teach about consistency? Tunable quorums: with replication factor N, choosing read and write counts so that $R + W > N$ gives overlapping quorums; lower settings trade consistency for latency and availability.

EXERCISES

1. Choose one write-up from the notebook's list and fill the six steps of the reading method in one page.
2. Map three systems from Table 32.1 to the CAP and PACELC trade-off they exemplify (use Chapter 11).
3. Explain why Spark was created as an improvement on MapReduce for iterative jobs.

Answers.

1. A good page lists requirements with numbers, a labelled request path, patterns, at least one trade-off and a date check.
2. For example (hedged, per public descriptions): Cassandra leans to availability with tunable consistency; Spanner favours consistency, paying latency; DynamoDB offers a choice per read between eventual and strong consistency.
3. MapReduce writes intermediate results to disk between stages; Spark keeps working sets in memory, so iterative algorithms avoid repeated disk reads.

CHAPTER SUMMARY

- Read architectures for requirements, patterns and trade-offs, not for brand names.
- The seventeen systems map onto a small set of patterns taught in this book.
- Company write-ups are dated; verify before relying on them.

REVISION CHECKLIST

- [ ] I can apply the six-step reading method.
- [ ] I can map each listed system to a pattern and a chapter.

BACK MATTER

Appendices

Revision sheets, the coverage matrix, the glossary and the references.

APPENDIX A

Revision Sheets and Formulas

Derived from your notes: Notebook chapter 15 (revision sheets: symptom to first fix, numbers and formulas, one-line trade-offs). **Added in this book:** formulas with units, decision tables, failure-to-remedy map and an extended one-line trade-off list covering the supplemental chapters.

CHAPTER SUMMARY

Use this appendix in the last days before an interview or exam. Everything here is explained in the chapters; the sheets compress it. If a line is unfamiliar, follow the chapter reference.

A.1 Symptom to first fix

Table A.1 Start with the cheapest fix that attacks the measured bottleneck.

Symptom	Likely cause	First fix	Where
Slow page, database CPU high	Hot repeated reads	Cache (cache-aside), add index	Chapter 12, Chapter 8
Slow queries on one table	Missing or wrong index, full scans	Inspect the query plan, add index	Chapter 8
Read load exceeds one DB	Read-heavy traffic	Read replicas (mind lag)	Chapter 9
Write load exceeds one DB	Write-heavy or huge data	Tune, federate, then shard	Chapter 9
Users far away see slow loads	Distance and latency	CDN, regional deployment	Chapter 7
Spiky load crashes the service	No buffering	Queue, rate limit, back pressure	Chapter 13
Slow work blocks requests	Synchronous heavy work	Move to a queue and workers	Chapter 13
One slow dependency freezes everything	No timeouts or isolation	Timeouts, circuit breaker, bulkheads	Chapter 15
Retries make an outage worse	Retry storm	Backoff with jitter, retry budgets	Chapter 15
Duplicate side effects	At-least-once delivery	Idempotency keys, idempotent consumers	Chapter 6, Chapter 13
Stale reads after a write	Replica lag, cache	Read from primary after write, versioning	Chapter 11
Cache miss storm after expiry	Stampede	Single flight, jittered TTL	Chapter 12

Symptom	Likely cause	First fix	Where
Single server failure takes site down	Single point of failure	Redundancy across zones, load balancer	Chapter 28
Two leaders after a network split	Split brain	Consensus, leases with fencing tokens	Chapter 14
One tenant slows others	Shared resources	Quotas, isolation, per-tenant limits	Chapter 7
Deploy causes outage	Big-bang release	Canary, blue-green, feature flags, rollback	Chapter 19
Cannot tell what is wrong	No observability	Metrics, structured logs, traces, SLO alerts	Chapter 18
Credential leak	Secrets in code	Secrets manager, rotation, least privilege	Chapter 17

A.2 Numbers and formulas

Table A.2 Formulas to be able to derive, with units.

Quantity	Formula	Example
Seconds per day	86,400 ≈ 10^5	
Seconds per month (30 days)	$2.592\times10^6 \approx 2.5\times10^6$	
Average request rate	requests per day / 86,400	10^8 per day ≈ 1,160 per second
Peak rate	average × peak factor (2 to 10)	1,160 × 4 ≈ 4,600 per second
Storage	items × bytes per item × retention	10^9 × 1 KB = 1 TB
Bandwidth	requests per second × bytes per response	1,000 × 100 KB = 100 MB/s = 800 Mbps
Little's law	concurrent requests = rate × latency	2,000/s × 0.1 s = 200 in flight
Servers needed	peak rate / (capacity per server × target utilisation)	4,600 / (200 × 0.6) ≈ 39
Availability of serial parts	$A = A_1 \times A_2 \times \ldots$	0.999 × 0.999 = 0.998
Availability of parallel parts	$A = 1 - (1 - A_1)(1 - A_2)$	two 0.99 nodes: 0.9999
Downtime per year	(1 − A) × 525,600 minutes	99.9% → 526 minutes (8.8 h)
Quorum overlap	R + W > N	N = 3, R = 2, W = 2
Majority	⌊N/2⌋ + 1	5 nodes: 3; tolerates 2 failures
Cache average latency	h × t_cache + (1 − h) × t_backend	0.8 × 4 + 0.2 × 150 = 33.2 ms
Bloom filter bits per item	≈ 1.44 $\log_2(1/p)$	p = 1%: about 9.6 bits
Bidirectional BFS cost	≈ 2·d^(L/2) vs d^L	d = 300, L = 4: 1.8×10^5 vs 8.1×10^9

Quantity	Formula	Example
Consistent hashing movement	≈ 1/(N + 1) of keys when adding a node	N = 10: about 9%
Exponential backoff	delay = min(cap, base × 2^n) × random jitter	base 100 ms, n = 3: 800 ms

Latency numbers to remember (orders of magnitude; see Chapter 4): memory read about 100 ns; SSD random read about 100 µs; datacentre round trip about 500 µs; disk seek about 10 ms; cross-continent round trip about 100 to 150 ms.

A.3 One-line trade-offs

- **Cache:** faster reads against possible staleness and invalidation work.
- **Replication:** availability and read scale against lag and failover complexity.
- **Sharding:** write and storage scale against cross-shard queries and rebalancing.
- **Denormalisation:** fast reads against duplicate data to keep consistent.
- **NoSQL:** scale and flexible shape against weaker queries and transactions.
- **Strong consistency:** simple reasoning against latency and lower availability under partition.
- **Eventual consistency:** availability and speed against surprising stale reads.
- **Queue:** smoothing and decoupling against delay and at-least-once duplicates.
- **Microservices:** independent teams and scaling against distributed-system costs.
- **Synchronous call:** simple and immediate against coupling and cascading failure.
- **Retries:** resilience to transient errors against load amplification and duplicates.
- **Two-phase commit:** atomicity across systems against blocking and coordinator dependency.
- **Saga:** availability across services against temporary inconsistency and compensation logic.
- **Fan-out on write:** fast reads against heavy writes for popular authors.
- **Fan-out on read:** cheap writes against slow reads.
- **Bloom filter:** tiny memory against false positives and no deletion.
- **LRU:** simple, adaptive eviction against vulnerability to scans.
- **Batch:** accuracy and throughput against latency; **stream:** freshness against operational complexity.
- **Vertical scaling:** simplicity against a ceiling and a single point of failure; **horizontal:** growth and redundancy against coordination.

REVISION CHECKLIST

- [ ] I can recite the symptom table without looking.
- [ ] I can derive each formula and its units.
- [ ] I can state both sides of every trade-off in one breath.

APPENDIX B

Coverage Matrix: Notebook to Book

Derived from your notes: Notebook chapter 15 coverage map (primer to notebook), extended to map each notebook section to its place in this book. **Added in this book:** a list of the topics this book adds beyond the notebook, with their locations.

This appendix is the audit trail for the promise that **the notebook is the primary source for coverage**. Every section of the 51-page handwritten notebook (the book's source) appears in the left column; the right columns say where this book treats it. A topic is marked *expanded* when the book adds depth beyond the notebook's page.

Table B.1 Notebook section to book section.

Notebook section	Book location	Status
1. The big picture: life of a request	Section 1.1, Section 5.7	Expanded
1. Performance vs scalability	Section 1.3	Covered
1. Latency vs throughput	Section 1.4	Expanded (percentiles)
1. Scaling up vs scaling out	Section 1.5	Covered
2. CAP theorem	Section 11.2	Expanded (PACELC, partitions)
2. Consistency patterns	Section 11.3, Section 11.4	Expanded
2. Availability patterns	Section 11.6, Section 15.7	Expanded
2. Availability in numbers	Section 4.5	Covered
3. Layers in one minute	Section 5.1	Covered
3. HTTP	Section 5.6	Expanded (HTTP/1.1, 2, 3)
3. TCP and UDP	Section 5.3, Section 5.4	Expanded (QUIC, TLS)
3. RPC	Section 6.2	Covered
3. REST	Section 6.3, Section 6.4	Expanded
4. DNS	Section 5.2	Covered
4. CDN	Section 7.2	Covered
4. Load balancers	Section 7.3	Expanded
4. Reverse proxy	Section 7.4	Covered
5. Separating web and application layers	Section 16.1	Covered
5. Microservices	Section 16.2, Section 16.3	Expanded

Notebook section	Book location	Status
5. Service discovery	Section 16.5	Covered
6. RDBMS and ACID	Section 8.3, Section 8.2	Expanded
6. Replication	Section 9.2	Covered
6. Federation	Section 9.3	Covered
6. Sharding	Section 9.4	Covered
6. Denormalization	Section 9.3	Covered
6. SQL tuning	Section 9.5	Covered
7. What is NoSQL?	Section 10.1	Covered
7. The four families	Section 10.2	Covered
7. SQL or NoSQL?	Section 10.4	Covered
8. Why cache?	Section 12.1	Covered
8. When to update the cache	Section 12.3	Expanded
9. Asynchronous workflows	Section 13.1, Section 13.2	Expanded
9. Security basics	Section 17.2, Section 17.3, Section 17.4	Expanded
10. Why estimate?	Section 4.1	Covered
10. Powers of two	Section 4.2	Covered
10. Latency numbers worth knowing	Section 4.3	Covered
10. Turning traffic into numbers	Section 4.4	Covered
11. The four steps	Section 20.2	Expanded
11. The 20-point checklist	Section 20.3	Covered
12.1 Pastebin / URL shortener	Chapter 21	Expanded
12.2 Twitter timeline and search	Chapter 22	Expanded
12.3 Web crawler and search	Chapter 23	Expanded
12.4 Mint	Chapter 24	Expanded
12.5 Social network shortest path	Chapter 25	Expanded
12.6 Query cache (LRU)	Chapter 26	Expanded
12.7 Sales rank by category	Chapter 27	Expanded
12.8 Scaling to millions on AWS	Chapter 28	Expanded
13. OO design method, hash map, LRU, call center, cards, parking lot, chat	Chapter 30	Expanded (code and tests)
13. Circular array (open exercise)	Section 30.8	Completed
14. More questions to practise (23)	Chapter 31	Expanded
14. Real-world architectures (17 systems)	Section 32.2	Covered

Notebook section	Book location	Status
14. Company architectures (23 themes)	Section 32.3	Covered
14. Engineering blogs	Section 32.4	Covered (links not re-verified)
15. Symptom to first fix	Section A.1	Expanded
15. Numbers and formulas	Section A.2	Expanded
15. Self-test: trade-offs	Section A.3	Expanded
15. Coverage map (primer to notebook)	Appendix B	Superseded by this appendix

B.1 Topics added beyond the notebook

Every item below is **supplemental**: it was requested for a professional-level book and is not in the notebook. Each is labelled in its chapter opener as "added in this book".

Table B.2 Supplemental topics and where they are treated.

Topic	Location
Processes, threads, locks, races, deadlock	Chapter 2
Hash tables, B-trees, LSM trees, heaps, tries, Bloom filters, HyperLogLog	Chapter 3
Idempotency keys, pagination, versioning, webhooks	Section 6.5, Section 6.6, Section 6.7, Section 6.8
Rate limiting and token bucket	Section 7.5
Schema migrations (expand and contract)	Section 8.4
PITR, backups versus replicas versus failover	Section 9.6
Quorums, consistent hashing, conflicts and repair	Section 11.4, Section 11.7, Section 11.5
Cache failure patterns (stampede, penetration, hot keys)	Section 12.5
Outbox, sagas, dead-letter queues, back pressure	Section 13.6, Section 13.4, Section 13.5
Clocks, fencing tokens, leader election, consensus, two-phase commit	Chapter 14
Resilience patterns	Chapter 15
Strangler pattern, architecture decision records	Section 16.6, Section 16.7
Threat modelling, OAuth and tokens, secrets, privacy by design	Chapter 17
Logs, metrics, traces, SLOs, incident response, RPO and RTO	Chapter 18
Containers, deployment strategies, autoscaling, cost, capacity planning	Chapter 19
Payments and notification end-to-end exercises	Chapter 29
Practice rubric and study plan	Section 31.1, Section 31.3

B.2 Source content that could not be used

Nothing in the notebook text was unreadable. Two limitations are stated for honesty. First, the notebook's page of *company links* and *blog links* lists titles only; this book does not claim the linked articles say anything beyond those titles. Second, where the notebook's skeleton code contained defects (the LRU node without a key, constructors with wrong base-class calls), the book corrects them and says so (Section 30.3, Section 30.4).

APPENDIX C

Glossary

Added in this book: the glossary, written for this book.

Definitions are short on purpose; the chapter that teaches each term is in the last column.

Table C.1 Glossary of terms.

Term	Meaning	Chapter
ACID	Atomicity, consistency, isolation, durability: the guarantees of a database transaction	Chapter 8
Availability	Fraction of time a system answers correctly; often quoted in "nines"	Chapter 4
Back pressure	Signalling a producer to slow down when a consumer is overloaded	Chapter 13
Bloom filter	Compact set structure with false positives but no false negatives	Chapter 3
Bulkhead	Isolating resources so one failure cannot exhaust all of them	Chapter 15
Cache-aside	Application reads the cache first and fills it from the database on a miss	Chapter 12
Cache stampede	Many requests recompute the same expired entry at once	Chapter 12
CAP theorem	During a network partition a system must choose between consistency and availability	Chapter 11
CDN	Content delivery network: edge caches close to users	Chapter 7
Circuit breaker	Stops calling a failing dependency for a while to let it recover	Chapter 15
Consensus	Agreement of several nodes on one value despite failures (Paxos, Raft)	Chapter 14
Consistent hashing	Mapping keys to nodes so adding or removing a node moves few keys	Chapter 11
CRDT	Replicated data type whose copies merge without conflicts	Chapter 31
Dead-letter queue	Holds messages that repeatedly failed processing	Chapter 13
Denormalisation	Storing redundant data to speed reads	Chapter 9
Eventual consistency	Replicas converge if updates stop, but reads may be stale meanwhile	Chapter 11
Exactly-once	Marketing term; in practice at-least-once delivery plus idempotent processing	Chapter 13
Fan-out	Delivering one event to many recipients	Chapter 22
Federation	Splitting a database by function into separate databases	Chapter 9
Fencing token	Increasing number that lets a resource reject a stale lock holder	Chapter 14
Heartbeat	Periodic signal showing a node is alive	Chapter 14

Term	Meaning	Chapter
Horizontal scaling	Adding more machines	Chapter 1
HTTP/3	HTTP over QUIC, which runs over UDP	Chapter 5
Idempotent	Repeating an operation has the same effect as doing it once	Chapter 6
Index	Auxiliary structure that speeds lookups at the cost of writes and space	Chapter 8
Inverted index	Map from terms to the documents containing them	Chapter 23
Isolation level	How much concurrent transactions can see of each other	Chapter 8
Latency	Time for one operation	Chapter 1
Leader election	Choosing one node to coordinate	Chapter 14
Lease	Time-limited right to act as owner	Chapter 14
Load balancer	Spreads requests across servers	Chapter 7
Load shedding	Rejecting some work deliberately to protect the rest	Chapter 15
LRU	Least recently used eviction policy	Chapter 26
LSM tree	Log-structured merge tree: write-optimised storage	Chapter 3
MVCC	Multi-version concurrency control: readers see snapshots	Chapter 8
Microservice	Small, independently deployable service owning its data	Chapter 16
Observability	Ability to understand internal state from outputs (logs, metrics, traces)	Chapter 18
Outbox pattern	Write an event to a table in the same transaction as the data, publish it later	Chapter 13
Partition (network)	Nodes cannot talk to each other	Chapter 11
Partition (data)	A slice of data placed on one node; see sharding	Chapter 9
PACELC	Extends CAP: else (no partition) trade latency against consistency	Chapter 11
Percentile	Value below which a given share of observations fall (p99)	Chapter 1
PITR	Point-in-time recovery from base backup plus log	Chapter 9
Pub/sub	Publishers send to topics; subscribers receive copies	Chapter 13
QUIC	Encrypted transport over UDP with multiplexed streams	Chapter 5
Quorum	Minimum number of replicas that must respond	Chapter 11
Race condition	Outcome depends on unlucky timing	Chapter 2
Raft	Understandable consensus algorithm with a leader and a log	Chapter 14
Rate limiting	Restricting request frequency	Chapter 7
Read replica	Copy of a database that serves reads	Chapter 9
Replication	Keeping copies of data on several nodes	Chapter 9
Reverse proxy	Server that forwards client requests to backends	Chapter 7
RPC	Remote procedure call	Chapter 6
RPO	Recovery point objective: how much data loss is acceptable	Chapter 18

Term	Meaning	Chapter
RTO	Recovery time objective: how long recovery may take	Chapter 18
Saga	Sequence of local transactions with compensations	Chapter 13
Scalability	Ability to handle growth by adding resources	Chapter 1
Sharding	Splitting one dataset across nodes by key	Chapter 9
SLA / SLO / SLI	Contract / target / measurement of service quality	Chapter 18
Split brain	Two nodes both believe they are the leader	Chapter 14
Stateless	Server keeps no per-user state between requests	Chapter 28
Strangler pattern	Gradually replace a legacy system piece by piece	Chapter 16
Throughput	Operations per unit time	Chapter 1
Token bucket	Rate limiter allowing bursts up to a bucket size	Chapter 7
Trie	Prefix tree	Chapter 3
Two-phase commit	Atomic commit protocol with prepare and commit phases	Chapter 14
Vertical scaling	Using a bigger machine	Chapter 1
WAL	Write-ahead log: changes are logged before being applied	Chapter 8

APPENDIX D

References and Further Reading

Added in this book: the reference list, compiled for this book.

Provenance and licence. The curriculum follows the open-source *System Design Primer* by Donne Martin (https://github.com/donnemartin/system-design-primer, licensed under CC BY 4.0). All explanations, diagrams, tables and code in this book were written for it. The links below were not fetched while writing; they are standard, stable references that the reader should check for current editions.

D.1 Books

- Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly, 2017. The best single book behind Parts III to V.
- Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds). *Site Reliability Engineering*. O'Reilly, 2016 (free online). Monitoring, SLOs, incident response.
- Nygard, M. *Release It!* 2nd ed. Pragmatic Bookshelf, 2018. Resilience patterns.
- Xu, A. *System Design Interview* (volumes 1 and 2). Worked interview problems.
- Manning, C., Raghavan, P., Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008 (free online). Crawling and indexing.
- Tanenbaum, A., van Steen, M. *Distributed Systems*. Background on clocks, consensus and replication.
- Shostack, A. *Threat Modeling: Designing for Security*. Wiley, 2014.
- Anderson, R. *Security Engineering*, 3rd ed. Wiley, 2020 (free online).
- Newman, S. *Building Microservices*, 2nd ed. O'Reilly, 2021.
- Evans, E. *Domain-Driven Design*. Addison-Wesley, 2003.
- Silberschatz, A., Korth, H., Sudarshan, S. *Database System Concepts*. Transactions and storage.

D.2 Papers

- Dean, J., Ghemawat, S. "MapReduce: Simplified Data Processing on Large Clusters." OSDI 2004.
- Ghemawat, S., Gobioff, H., Leung, S.-T. "The Google File System." SOSP 2003.

- Chang, F. et al. "Bigtable: A Distributed Storage System for Structured Data." OSDI 2006.
- DeCandia, G. et al. "Dynamo: Amazon's Highly Available Key-value Store." SOSP 2007.
- Lakshman, A., Malik, P. "Cassandra: A Decentralized Structured Storage System." 2009.
- Corbett, J. et al. "Spanner: Google's Globally-Distributed Database." OSDI 2012.
- Burrows, M. "The Chubby Lock Service for Loosely-Coupled Distributed Systems." OSDI 2006.
- Ongaro, D., Ousterhout, J. "In Search of an Understandable Consensus Algorithm" (Raft). USENIX ATC 2014.
- Lamport, L. "Time, Clocks, and the Ordering of Events in a Distributed System." CACM 1978.
- Brewer, E. "CAP Twelve Years Later: How the 'Rules' Have Changed." IEEE Computer 2012; Gilbert, S., Lynch, N. "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services." 2002.
- Abadi, D. "Consistency Tradeoffs in Modern Distributed Database System Design" (PACELC). IEEE Computer 2012.
- Sigelman, B. et al. "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure." Google Technical Report 2010.
- Kreps, J. "The Log: What every software engineer should know about real-time data's unifying abstraction." 2013.
- Heydon, A., Najork, M. "Mercator: A Scalable, Extensible Web Crawler." 1999.
- Manku, G., Jain, A., Das Sarma, A. "Detecting Near-Duplicates for Web Crawling." WWW 2007.
- Karger, D. et al. "Consistent Hashing and Random Trees." STOC 1997.
- Bloom, B. "Space/Time Trade-offs in Hash Coding with Allowable Errors." CACM 1970.
- Garcia-Molina, H., Salem, K. "Sagas." SIGMOD 1987.

D.3 Standards and specifications

- RFC 9110 (HTTP semantics), RFC 9113 (HTTP/2), RFC 9114 (HTTP/3), RFC 9000 (QUIC), RFC 8446 (TLS 1.3), RFC 1035 (DNS).
- RFC 6749 (OAuth 2.0), RFC 7636 (PKCE), RFC 7519 (JSON Web Token).
- NIST SP 800-63B, Digital Identity Guidelines: Authentication and Lifecycle Management.
- OWASP Top 10 and OWASP Cheat Sheet Series for web security.
- OpenTelemetry specification for traces, metrics and logs.

D.4 Other resources

- The *System Design Primer* repository and its linked articles.
- Company engineering blogs listed in Section 32.4.
- The Common Crawl project (public web crawl data).

CLARIFICATION

Verification status. References were written from the author's knowledge of these works; titles, venues and years should be confirmed before citing them formally. The book makes no claim that any link was reachable at the time of writing.

REFERENCE

Index

A

ACID 74, 88, 98, 256
Availability 13, 36, 66, 84

B

B-tree 30, 76
Back pressure 121, 140, 224
Bloom filter 31, 117, 213
blue-green 174, 253
Bulkhead 143

C

Cache stampede 23, 111, 238
Cache-aside 111, 196, 250
CAP theorem 77, 101
Cassandra 95, 287
CDN 10, 38, 53, 65
Circuit breaker 142, 213, 224, 238
Consensus 25, 87, 103, 131
Consistent hashing 30, 71, 91, 101
CRDT 281

D

Dead-letter queue 125, 156, 224, 260
Denormalisation 76, 83
DNS 10, 45, 64, 103
DynamoDB 94, 289

E

Eventual consistency 85, 94, 101, 122
Exactly-once 121, 243, 260, 282

F

Fan-out 19, 30, 97, 184
feature flag 144, 171, 180
Federation 83, 250
Fencing token 138

G

GraphQL 58

H

hash table 29, 281
heap 32, 212, 243, 282
Heartbeat 133, 155
Horizontal scaling 9, 94, 122, 185
HTTP/3 45
HyperLogLog 33

I

Idempotent 55, 58, 121, 133
Index 13, 29, 42, 61
Inverted index 98, 207, 211, 281
Isolation level 77

K

Kafka 124, 287

L

Latency 9, 36, 45, 61
Leader election 103, 131, 289
Lease 132, 215
linearizability 97, 101
Load balancer 10, 53, 64, 141
Load shedding 17, 117, 140, 238
LRU 34, 111, 231, 234
LSM tree 31

M

MapReduce 213, 241, 287
Memcached 94, 120, 281
Microservice 157
MinHash 214
MVCC 79

O

OAuth 62, 159
Observability 149, 168, 178
Outbox pattern 126

P

PACELC 103, 289
PageRank 215
Partition (data) 89, 94, 101, 118
Partition (network) 89, 94, 101, 118
Paxos 135
Percentile 13
PITR 90
Pub/sub 124

Q

QUIC 45
Quorum 87, 100, 101, 131

R

Race condition 20
Raft 135
Rate limiting 59, 69, 94, 117
read-your-writes 85, 104, 201, 250
Redis 32, 94, 118, 281
Replication 11, 39, 80, 83
REST 56
Reverse proxy 11, 64
RPC 56
RPO 90, 167, 206
RTO 90, 144, 167

S

Saga 126, 137, 156, 256
Scalability 9, 219, 227, 288
Sharding 11, 39, 80, 83
SimHash 213
SLA 170
SLI 172
SLO 169, 178
Snowflake 199, 205
Split brain 86, 131, 144
Stateless 9, 50, 58, 65
Strangler pattern 147

T

TCP 45, 64, 140
Throughput 9, 24, 36, 94
TLS 45, 64, 151, 159
Token bucket 69, 259, 283
Trie 32
Two-phase commit 88, 131

U

UDP 45, 67

V

Vertical scaling 15, 248

W

WAL 77
write-behind 111
write-through 111